



RTL Design Sherpa

RTL Math Library Module Reference — Rev 0.90

July 2026

Table of Contents

1 RTL Math Library.....	35
1.1 Overview.....	36
1.2 Integer Arithmetic.....	36
1.2.1 Addition.....	36
1.2.2 Subtraction.....	37
1.2.3 Multiplication.....	37
1.3 Floating-Point Arithmetic.....	38
1.3.1 Formats.....	38
1.3.2 Core operators (per format).....	39
1.3.3 Division & reciprocal.....	39
1.3.4 Conversion.....	40
1.3.5 Comparison & range.....	40
1.3.6 Activation functions (ML).....	40
1.4 Generation Automation (bin/).....	40
1.4.1 Integer arithmetic — bin/math_generate.py.....	40
1.4.2 Floating-point — bin/rtl_generators/ieee754/generate_all.py.....	41
1.4.3 The emitter framework — bin/rtl_generators/.....	41
1.4.4 Regeneration rule.....	41
1.5 Research References.....	41
1.6 Related Documentation.....	42
1.7 Navigation.....	42
2 Basic Adder Building Blocks.....	42

2.1 Overview.....	42
2.2 Module Declarations.....	43
2.2.1 Half Adder.....	43
2.2.2 Full Adder.....	43
2.2.3 N-bit Full Adder Array.....	43
2.3 Parameters.....	44
2.3.1 Half Adder & Full Adder.....	44
2.3.2 N-bit Full Adder Array.....	44
2.4 Ports.....	44
2.4.1 Half Adder Ports.....	44
2.4.2 Full Adder Ports.....	44
2.4.3 N-bit Full Adder Ports.....	45
2.5 Functionality.....	45
2.5.1 Half Adder.....	45
2.5.2 Full Adder.....	45
2.5.3 N-bit Full Adder Array.....	46
2.6 Usage Examples.....	47
2.6.1 Half Adder: Parity Generator.....	47
2.6.2 Full Adder: Carry-Save Adder Stage.....	47
2.6.3 N-bit Full Adder: BCD Digit Adder.....	48
2.6.4 Building a 2-bit Adder from Scratch.....	48
2.7 Timing Characteristics.....	49
2.7.1 Propagation Delays.....	49
2.8 Performance Characteristics.....	49

2.8.1 Resource Utilization.....	49
2.8.2 When to Use Each Module.....	49
2.9 Design Considerations.....	50
2.9.1 Why Use These Modules.....	50
2.9.2 Behavioral vs Structural.....	50
2.9.3 FPGA-Specific Considerations.....	50
2.10 Common Pitfalls.....	50
2.11 Related Modules.....	52
2.12 References.....	52
2.13 Navigation.....	52
3 Brent-Kung Parallel Prefix Adder.....	52
3.1 Overview.....	52
3.2 Module Hierarchy.....	52
3.3 Top-Level Module Declaration.....	53
3.4 Parameters.....	53
3.4.1 User-Settable Parameters.....	53
3.5 Ports.....	53
3.5.1 Inputs.....	53
3.5.2 Outputs.....	54
3.6 Algorithm Overview.....	54
3.6.1 Parallel Prefix Addition Theory.....	54
3.6.2 Brent-Kung Algorithm Stages.....	54
3.7 Architecture Details.....	55
3.7.1 Three-Stage Pipeline.....	55

3.7.2 Internal Building Blocks.....	56
3.7.3 Prefix Network Structure (32-bit Example).....	56
3.8 Timing Characteristics.....	57
3.8.1 Combinational Delay Analysis.....	57
3.8.2 Critical Paths.....	57
3.9 Usage Examples.....	58
3.9.1 Basic 32-bit Addition.....	58
3.9.2 8-bit Incrementer.....	58
3.9.3 16-bit Subtraction (A - B).....	59
3.9.4 Multi-Precision Addition (64-bit via 2×32-bit).....	59
3.9.5 Overflow Detection (Signed Addition).....	60
3.10 Performance Characteristics.....	60
3.10.1 Resource Utilization.....	60
3.10.2 Speed vs Area Trade-offs.....	61
3.11 Design Considerations.....	61
3.11.1 Width Selection.....	61
3.11.2 Synthesis Considerations.....	62
3.11.3 Verification Strategy.....	62
3.12 Common Pitfalls.....	62
3.13 Related Modules.....	64
3.14 References.....	64
3.15 Navigation.....	64
4 Carry-Save Adder.....	64
4.1 Overview.....	64

4.2 Module Declarations.....	65
4.2.1 Single-bit Carry-Save Adder.....	65
4.2.2 N-bit Carry-Save Adder.....	65
4.3 Parameters.....	65
4.3.1 Single-bit CSA.....	65
4.3.2 N-bit CSA.....	65
4.4 Ports.....	66
4.4.1 Single-bit CSA Ports.....	66
4.4.2 N-bit CSA Ports.....	66
4.5 Functionality.....	66
4.5.1 Single-bit Carry-Save Adder (3:2 Compressor).....	66
4.5.2 N-bit Carry-Save Adder.....	67
4.5.3 Final Addition Step.....	67
4.6 Timing Characteristics.....	68
4.6.1 Propagation Delays.....	68
4.7 Usage Examples.....	68
4.7.1 Adding 3 Numbers (Single CSA Stage).....	68
4.7.2 Adding 4 Numbers (2-Level CSA Tree).....	69
4.7.3 Adding 7 Numbers (Wallace Tree Structure).....	69
4.7.4 8-bit×8-bit Multiplier Partial Product Reduction.....	70
4.8 Performance Characteristics.....	71
4.8.1 Resource Utilization.....	71
4.8.2 Speed Advantages.....	71
4.9 Design Considerations.....	71

4.9.1 When to Use Carry-Save Adders.....	71
4.9.2 When NOT to Use CSA.....	71
4.9.3 CSA Tree Design Guidelines.....	72
4.9.4 Final Adder Selection.....	72
4.10 Common Pitfalls.....	72
4.11 Related Modules.....	73
4.12 References.....	73
4.13 Navigation.....	73
5 math_adder_full.....	74
5.1 Overview.....	74
5.2 Module Declaration.....	74
5.3 Parameters.....	74
5.4 Ports.....	74
5.4.1 Inputs.....	74
5.4.2 Outputs.....	75
5.5 Functionality.....	75
5.5.1 Full Adder Logic.....	75
5.5.2 Truth Table.....	75
5.6 Implementation Details.....	76
5.6.1 Sum Generation.....	76
5.6.2 Carry Generation.....	76
5.7 Logic Gate Implementation.....	77
5.7.1 Optimized Implementation.....	79
5.8 Timing Characteristics.....	79

5.9 Usage Examples.....	79
5.9.1 Basic Full Adder.....	79
5.9.2 Building a 4-Bit Ripple Carry Adder.....	80
5.9.3 Part of a Carry-Save Adder.....	81
5.10 Design Considerations.....	81
5.10.1 Advantages.....	81
5.10.2 Performance Characteristics.....	81
5.11 Synthesis Considerations.....	81
5.11.1 Technology Mapping.....	81
5.11.2 Optimization Notes.....	81
5.12 Verification Examples.....	82
5.12.1 Testbench Structure.....	82
5.13 Related Modules.....	82
5.14 Applications.....	82
5.15 Navigation.....	83
6 Han-Carlson Parallel Prefix Adder.....	83
6.1 Overview.....	83
6.2 Module Hierarchy.....	83
6.3 Module Declaration.....	83
6.3.1 16-bit Han-Carlson Adder.....	83
6.3.2 48-bit Han-Carlson Adder.....	84
6.4 Parameters.....	84
6.5 Ports.....	84
6.6 Algorithm.....	84

6.6.1 Han-Carlson Structure.....	84
6.6.2 Stage-by-Stage Breakdown (16-bit).....	85
6.6.3 Prefix Network Visualization (16-bit simplified).....	85
6.6.4 Why Han-Carlson is Optimal.....	86
6.7 Timing Characteristics.....	86
6.7.1 16-bit Adder.....	86
6.7.2 48-bit Adder.....	87
6.7.3 Depth Formula.....	87
6.8 Usage Examples.....	87
6.8.1 Basic 16-bit Addition.....	87
6.8.2 Subtraction (A - B).....	87
6.8.3 In BF16 Multiplier (Final CPA).....	88
6.8.4 In BF16 FMA (Wide Addition).....	88
6.9 Performance Characteristics.....	88
6.9.1 Resource Utilization.....	88
6.9.2 Comparison with Other Architectures.....	88
6.10 Design Considerations.....	89
6.10.1 Design Optimization Priorities.....	89
6.10.2 When to Use Han-Carlson.....	89
6.10.3 Auto-Generated Code.....	89
6.11 Common Pitfalls.....	89
6.12 Related Modules.....	90
6.13 References.....	90
6.14 Navigation.....	90

7 Kogge-Stone Parallel Prefix Adder.....	90
7.1 Overview.....	91
7.2 Module Declaration.....	91
7.3 Parameters.....	91
7.3.1 User-Settable Parameters.....	91
7.4 Ports.....	91
7.4.1 Inputs.....	91
7.4.2 Outputs.....	91
7.5 Functionality.....	92
7.5.1 Algorithm Overview.....	92
7.5.2 Implementation Details.....	92
7.5.3 Timing Analysis.....	93
7.5.4 Comparison to Other Adders.....	93
7.6 Usage Examples.....	94
7.6.1 Basic 16-bit Addition.....	94
7.6.2 Adding with Carry Input (External Logic).....	94
7.6.3 Arbitrary Width Addition (37-bit example).....	95
7.6.4 Multi-Precision Addition (64-bit via 2×32-bit).....	95
7.6.5 Accumulator.....	96
7.6.6 Adder Array (Multiple Parallel Adders).....	96
7.7 Timing Characteristics.....	96
7.7.1 Combinational Delay Analysis.....	96
7.7.2 Critical Paths.....	97
7.7.3 Performance vs Width.....	97

7.8 Performance Characteristics.....	97
7.8.1 Resource Utilization.....	97
7.8.2 Comparison to Alternatives.....	98
7.9 Design Considerations.....	98
7.9.1 When to Use This Adder.....	98
7.9.2 When to Use Alternatives.....	98
7.9.3 Lack of Carry Input Workarounds.....	99
7.9.4 Synthesis Considerations.....	99
7.10 Verification Strategy.....	99
7.11 Common Pitfalls.....	100
7.12 Related Modules.....	100
7.13 References.....	101
7.14 Navigation.....	101
8 Carry Lookahead Adder.....	101
8.1 Overview.....	101
8.2 Module Declaration.....	101
8.3 Parameters.....	102
8.3.1 User-Settable Parameters.....	102
8.4 Ports.....	102
8.4.1 Inputs.....	102
8.4.2 Outputs.....	102
8.5 Functionality.....	103
8.5.1 Algorithm Overview.....	103
8.5.2 Implementation.....	103

8.5.3 Timing Analysis.....	104
8.5.4 Comparison to Other Adders.....	104
8.6 Usage Examples.....	104
8.6.1 Basic 8-bit Addition.....	104
8.6.2 4-bit Incrementer.....	105
8.6.3 16-bit ALU Component.....	105
8.6.4 Multi-Precision Addition (32-bit via 2×16-bit).....	106
8.6.5 Conditional Increment (For Loop Counters).....	107
8.7 Timing Characteristics.....	108
8.7.1 Combinational Delay Analysis.....	108
8.7.2 Critical Paths.....	108
8.7.3 Performance vs Width.....	108
8.8 Performance Characteristics.....	108
8.8.1 Resource Utilization.....	108
8.8.2 Speed vs Area Trade-offs.....	109
8.9 Design Considerations.....	109
8.9.1 When to Use This Adder.....	109
8.9.2 Width Selection Guidelines.....	109
8.9.3 Synthesis Considerations.....	110
8.9.4 Verilator Lint Note.....	110
8.10 Verification Strategy.....	110
8.11 Common Pitfalls.....	111
8.12 Related Modules.....	112
8.13 References.....	112

8.14 Navigation.....	112
9 Ripple Carry Adder.....	112
9.1 Overview.....	112
9.2 Module Declaration.....	113
9.3 Parameters.....	113
9.3.1 User-Settable Parameters.....	113
9.4 Ports.....	113
9.4.1 Inputs.....	113
9.4.2 Outputs.....	113
9.5 Functionality.....	114
9.5.1 Algorithm: Sequential Carry Propagation.....	114
9.5.2 Full Adder Building Block.....	114
9.5.3 Implementation.....	114
9.5.4 Timing Analysis.....	115
9.5.5 Comparison to Other Adders.....	115
9.6 Usage Examples.....	115
9.6.1 Basic 8-bit Addition.....	115
9.6.2 4-bit Incrementer.....	116
9.6.3 BCD Digit Adder (4-bit).....	116
9.6.4 Multi-Precision Addition (16-bit via 2×8-bit).....	117
9.6.5 Simple ALU (Add/Subtract).....	117
9.7 Timing Characteristics.....	118
9.7.1 Combinational Delay Analysis.....	118
9.7.2 Critical Paths.....	118

9.7.3 Performance vs Width.....	118
9.8 Performance Characteristics.....	119
9.8.1 Resource Utilization.....	119
9.8.2 Comparison: Area vs Speed.....	119
9.9 Design Considerations.....	120
9.9.1 When to Use Ripple Carry Adder.....	120
9.9.2 When to Use Alternatives.....	120
9.9.3 Width Selection Trade-offs.....	120
9.9.4 Synthesis Considerations.....	120
9.10 Verification Strategy.....	121
9.11 Common Pitfalls.....	121
9.12 Related Modules.....	122
9.13 References.....	122
9.14 Navigation.....	122
10 Add/Subtract Unit.....	123
10.1 Overview.....	123
10.2 Module Declaration.....	123
10.3 Parameters.....	123
10.4 Ports.....	123
10.5 Functionality.....	124
10.5.1 Two's Complement Subtraction.....	124
10.5.2 Implementation.....	125
10.5.3 Operation Modes.....	125
10.6 Usage Examples.....	125

10.6.1 Simple Add/Subtract ALU.....	125
10.6.2 Detecting Underflow in Subtraction.....	126
10.6.3 Simple Calculator.....	127
10.6.4 Multi-Function ALU.....	127
10.7 Timing Characteristics.....	128
10.8 Performance Characteristics.....	129
10.8.1 Resource Utilization.....	129
10.9 Design Considerations.....	129
10.9.1 Advantages.....	129
10.9.2 When to Use This Module.....	129
10.9.3 Synthesis Considerations.....	130
10.10 Common Pitfalls.....	130
10.11 Related Modules.....	131
10.12 References.....	131
10.13 Navigation.....	131
11 BF16 Adder.....	131
11.1 Overview.....	131
11.2 Module Declaration.....	131
11.3 Parameters.....	132
11.4 Ports.....	132
11.5 BF16 Format.....	133
11.6 Architecture.....	133
11.6.1 Block Diagram.....	133
11.6.2 Processing Pipeline.....	136

11.7 Functionality.....	136
11.7.1 Exponent Comparison and Swap.....	136
11.7.2 Mantissa Alignment.....	136
11.7.3 Round-to-Nearest-Even.....	137
11.7.4 Special Case Priority.....	137
11.8 Usage Examples.....	138
11.8.1 Basic Addition (Combinational).....	138
11.8.2 Pipelined Configuration.....	138
11.8.3 Subtraction via Sign Manipulation.....	139
11.8.4 With Status Checking.....	139
11.9 Pipeline Latency Configurations.....	139
11.10 Performance Characteristics.....	140
11.10.1 Resource Utilization (Estimated).....	140
11.10.2 Timing Characteristics.....	140
11.11 Design Considerations.....	140
11.11.1 Flush-to-Zero (FTZ).....	140
11.11.2 Invalid Operation (inf - inf).....	140
11.11.3 Sign of Zero.....	141
11.11.4 Rounding Mode.....	141
11.12 Dependencies.....	141
11.13 Common Pitfalls.....	141
11.14 Related Modules.....	141
11.15 References.....	142
11.16 Navigation.....	142

12 BF16 Exponent Adder.....	142
12.1 Overview.....	142
12.2 Module Declaration.....	142
12.3 Ports.....	143
12.3.1 Inputs.....	143
12.3.2 Outputs.....	143
12.4 BF16 Exponent Background.....	144
12.4.1 BF16 Format.....	144
12.4.2 Multiplication Exponent Formula.....	144
12.5 Functionality.....	144
12.5.1 Special Case Detection.....	144
12.5.2 Extended Precision Arithmetic.....	145
12.5.3 Overflow/Underflow Detection.....	145
12.5.4 Result Saturation.....	145
12.6 Usage Examples.....	146
12.6.1 Basic Usage.....	146
12.6.2 In BF16 Multiplier.....	146
12.6.3 Example Calculations.....	147
12.7 Timing Characteristics.....	147
12.8 Performance Characteristics.....	147
12.8.1 Resource Utilization.....	147
12.8.2 Design Optimization Priorities.....	148
12.9 Design Considerations.....	148
12.9.1 Why Override Flags for Special Cases?.....	148

12.9.2 Exponent Ranges.....	148
12.9.3 Order of Checks.....	148
12.10 Auto-Generated Code.....	148
12.11 Related Modules.....	148
12.12 References.....	149
12.13 Navigation.....	149
13 BF16 Extended Modules.....	149
13.1 Overview.....	149
13.2 Module Categories.....	149
13.2.1 Activation Functions.....	149
13.2.2 Mathematical Operations.....	150
13.2.3 Comparison and Selection.....	151
13.2.4 Format Conversions.....	152
13.3 Special Value Handling.....	153
13.4 Implementation Notes.....	154
13.4.1 Piecewise Linear Approximations.....	154
13.4.2 LUT-Based Operations.....	154
13.5 Auto-Generation.....	154
13.6 Related Documentation.....	155
13.7 Navigation.....	155
14 BF16 Fused Multiply-Add (FMA).....	155
14.1 Overview.....	155
14.2 Module Declaration.....	155
14.3 Ports.....	156

14.4 Format Specifications.....	156
14.4.1 BF16 Format.....	156
14.4.2 FP32 Format.....	156
14.5 Architecture.....	157
14.5.1 Block Diagram.....	157
14.5.2 Processing Pipeline.....	159
14.6 Functionality.....	160
14.6.1 Mixed Precision Computation.....	160
14.6.2 Exponent Alignment.....	160
14.6.3 Effective Addition/Subtraction.....	160
14.6.4 Normalization with CLZ.....	161
14.6.5 Special Case Priority.....	161
14.7 Usage Examples.....	162
14.7.1 Basic FMA Operation.....	162
14.7.2 Neural Network Dot Product.....	162
14.7.3 With Pipeline Register.....	163
14.8 Timing Characteristics.....	164
14.9 Performance Characteristics.....	164
14.9.1 Resource Utilization.....	164
14.9.2 Design Optimization Priorities.....	164
14.10 Design Considerations.....	165
14.10.1 Why FP32 Accumulator?.....	165
14.10.2 Fusion Benefits.....	165
14.10.3 Product Underflow Handling.....	165

14.10.4 Invalid Operation Cases.....	165
14.11 Common Pitfalls.....	165
14.12 Auto-Generated Code.....	166
14.13 Related Modules.....	166
14.14 References.....	166
14.15 Navigation.....	167
15 BF16 Mantissa Multiplier.....	167
15.1 Overview.....	167
15.2 Module Declaration.....	167
15.3 Ports.....	167
15.3.1 Inputs.....	167
15.3.2 Outputs.....	168
15.4 BF16 Format Background.....	168
15.4.1 BF16 Structure.....	168
15.4.2 Mantissa Multiplication.....	169
15.5 Functionality.....	169
15.5.1 Mantissa Extension.....	169
15.5.2 8x8 Multiplication.....	169
15.5.3 Normalization Detection.....	169
15.5.4 Mantissa Extraction.....	169
15.5.5 Rounding Bits for RNE.....	170
15.6 Usage Examples.....	170
15.6.1 Basic Usage.....	170
15.6.2 In BF16 Multiplier.....	171

15.7 Timing Characteristics.....	171
15.8 Performance Characteristics.....	172
15.8.1 Resource Utilization.....	172
15.8.2 Design Optimization Priorities.....	172
15.9 Design Considerations.....	172
15.9.1 Flush-to-Zero (FTZ) Mode.....	172
15.9.2 Why Separate from Multiplier?.....	172
15.9.3 Round-to-Nearest-Even.....	172
15.10 Auto-Generated Code.....	172
15.11 Related Modules.....	173
15.12 References.....	173
15.13 Navigation.....	173
16 BF16 Multiplier.....	173
16.1 Overview.....	173
16.2 Module Declaration.....	173
16.3 Ports.....	174
16.4 BF16 Format.....	174
16.5 Architecture.....	175
16.5.1 Processing Pipeline.....	175
16.6 Functionality.....	176
16.6.1 Special Case Detection.....	176
16.6.2 Round-to-Nearest-Even.....	176
16.6.3 Special Case Priority.....	176
16.7 Usage Examples.....	177

16.7.1 Basic Multiplication.....	177
16.7.2 With Status Checking.....	178
16.7.3 In Neural Network Layer.....	178
16.8 Timing Characteristics.....	178
16.9 Performance Characteristics.....	179
16.9.1 Resource Utilization.....	179
16.9.2 Design Optimization Priorities.....	179
16.10 Design Considerations.....	179
16.10.1 Flush-to-Zero (FTZ).....	179
16.10.2 NaN Handling.....	179
16.10.3 Sign of Zero.....	179
16.10.4 Rounding Mode.....	179
16.11 Common Pitfalls.....	179
16.12 Auto-Generated Code.....	180
16.13 Related Modules.....	180
16.14 References.....	180
16.15 Navigation.....	180
17 4:2 Compressor.....	181
17.1 Overview.....	181
17.2 Module Declaration.....	181
17.3 Ports.....	181
17.4 Functionality.....	182
17.4.1 Architecture.....	182
17.4.2 Truth Table (simplified).....	182

17.4.3 Key Property: Cout Independence.....	183
17.5 Timing Characteristics.....	183
17.6 Usage Examples.....	183
17.6.1 Single Compressor.....	183
17.6.2 Chained Compressors (Multiplier Column).....	184
17.6.3 In Dadda Tree Multiplier.....	184
17.7 Performance Characteristics.....	185
17.7.1 Resource Utilization.....	185
17.7.2 Comparison with 3:2 Compressor (Full Adder).....	185
17.7.3 Design Optimization Priorities.....	185
17.8 Design Considerations.....	185
17.8.1 Advantages.....	185
17.8.2 Applications.....	185
17.9 Related Modules.....	186
17.10 References.....	186
17.11 Navigation.....	186
18 FP16 (Half Precision) Modules.....	186
18.1 Overview.....	186
18.2 Module Categories.....	187
18.2.1 Activation Functions.....	187
18.2.2 Comparison and Selection.....	187
18.2.3 Format Conversions.....	188
18.3 Usage Examples.....	188
18.3.1 Mixed Precision Pipeline.....	188

18.3.2 Comparison Operations.....	189
18.4 Special Values.....	189
18.5 Auto-Generation.....	189
18.6 Related Documentation.....	189
18.7 Navigation.....	190
19 FP32 (Single Precision) Modules.....	190
19.1 Overview.....	190
19.2 Module Categories.....	190
19.2.1 Activation Functions.....	190
19.2.2 Comparison and Selection.....	191
19.2.3 Format Conversions (Downconversion).....	191
19.3 Usage Examples.....	192
19.3.1 Training Pipeline (FP32 Master).....	192
19.3.2 Quantization Pipeline.....	192
19.4 Special Values.....	193
19.5 Auto-Generation.....	193
19.6 Related Documentation.....	193
19.7 Navigation.....	193
20 FP8 (8-bit Floating-Point) Modules.....	193
20.1 Overview.....	194
20.2 Format Comparison.....	194
20.3 E4M3 Modules.....	194
20.3.1 Core Arithmetic.....	194
20.3.2 Activation Functions.....	195

20.3.3 Comparison and Selection.....	195
20.3.4 Format Conversions.....	196
20.4 E5M2 Modules.....	196
20.4.1 Core Arithmetic.....	196
20.4.2 Activation Functions.....	196
20.4.3 Comparison and Selection.....	197
20.4.4 Format Conversions.....	197
20.5 Usage Examples.....	197
20.5.1 Inference Pipeline (E4M3).....	197
20.5.2 Mixed Precision Training.....	198
20.6 Special Values.....	198
20.6.1 E4M3 Special Values.....	198
20.6.2 E5M2 Special Values.....	199
20.7 Design Considerations.....	199
20.7.1 E4M3 Overflow Handling.....	199
20.7.2 Subnormal Handling.....	199
20.8 Auto-Generation.....	199
20.9 Related Documentation.....	199
20.10 Navigation.....	200
21 IEEE 754-2008 Compliant Modules.....	200
21.1 Overview.....	200
21.2 Module Summary.....	200
21.2.1 FP16 (Half Precision) IEEE 754.....	200
21.2.2 FP32 (Single Precision) IEEE 754.....	200

21.3 Module Interfaces.....	201
21.3.1 FP16 Adder.....	201
21.3.2 FP32 Multiplier.....	201
21.3.3 FP32 FMA.....	202
21.4 Architecture.....	202
21.4.1 FP32 Multiplier Pipeline.....	202
21.5 IEEE 754 Compliance.....	203
21.5.1 Special Value Handling.....	203
21.5.2 Rounding Modes.....	204
21.5.3 Status Flags.....	204
21.6 Comparison: IEEE 754 vs Simplified Modules.....	204
21.7 Usage Examples.....	205
21.7.1 High-Precision Computation.....	205
21.7.2 Pipelined FP16 Adder.....	205
21.8 Dependencies.....	206
21.9 Auto-Generation.....	206
21.10 Related Documentation.....	206
21.11 Navigation.....	206
22 Basic Multiplier Building Blocks.....	207
22.1 Overview.....	207
22.2 Module Declarations.....	207
22.2.1 Basic Multiplier Cell.....	207
22.2.2 Carry-Save Array Multiplier.....	207
22.3 Parameters.....	208

22.3.1 Basic Multiplier Cell.....	208
22.3.2 Carry-Save Array Multiplier.....	208
22.4 Ports.....	208
22.4.1 Basic Multiplier Cell Ports.....	208
22.4.2 Carry-Save Array Multiplier Ports.....	208
22.5 Functionality.....	209
22.5.1 Basic Multiplier Cell.....	209
22.5.2 Carry-Save Array Multiplier.....	209
22.5.3 Multiplication Example: 5×6 (4-bit).....	211
22.6 Usage Examples.....	211
22.6.1 Basic 4×4 Multiplication.....	211
22.6.2 Generic Width Multiplier.....	212
22.6.3 Building Custom Array from Cells.....	212
22.6.4 Multiplier with Valid/Ready Handshake.....	213
22.7 Timing Characteristics.....	214
22.8 Performance Characteristics.....	215
22.8.1 Resource Utilization.....	215
22.8.2 Comparison to Optimized Multipliers.....	215
22.8.3 When to Use Array Multipliers.....	215
22.9 Design Considerations.....	216
22.9.1 Advantages.....	216
22.9.2 Limitations.....	216
22.9.3 Basic Cell Design Notes.....	216
22.9.4 Synthesis Considerations.....	216

22.9.5 Pipelining Strategy.....	216
22.10 Common Pitfalls.....	217
22.11 Educational Value.....	218
22.11.1 Understanding Multiplication.....	218
22.11.2 Progression to Optimized Multipliers.....	218
22.12 Related Modules.....	218
22.13 References.....	219
22.14 Navigation.....	219
23 Dadda Multiplier with 4:2 Compressors.....	219
23.1 Overview.....	219
23.2 Module Declaration.....	219
23.3 Parameters.....	220
23.4 Ports.....	220
23.5 Architecture.....	220
23.5.1 Three-Stage Pipeline.....	220
23.5.2 Partial Product Matrix.....	220
23.5.3 Dadda Reduction with 4:2 Compressors.....	221
23.5.4 Reduction Example (Column 7).....	221
23.5.5 Component Instantiation.....	221
23.6 Timing Characteristics.....	222
23.6.1 Critical Path.....	222
23.7 Usage Examples.....	223
23.7.1 Basic 8x8 Multiplication.....	223
23.7.2 In BF16 Mantissa Multiplier.....	223

23.7.3 With Pipeline Register.....	223
23.8 Performance Characteristics.....	224
23.8.1 Resource Utilization.....	224
23.8.2 Comparison: 4:2 vs 3:2 Dadda.....	224
23.8.3 Design Optimization Priorities.....	224
23.9 Design Considerations.....	225
23.9.1 Advantages.....	225
23.9.2 Limitations.....	225
23.9.3 Signed Multiplication.....	225
23.10 Auto-Generated Code.....	225
23.11 Related Modules.....	225
23.12 References.....	226
23.13 Navigation.....	226
24 Dadda Tree Multipliers.....	226
24.1 Overview.....	226
24.2 Module Declarations.....	227
24.2.1 8-bit Dadda Tree Multiplier.....	227
24.2.2 16-bit Dadda Tree Multiplier.....	227
24.2.3 32-bit Dadda Tree Multiplier.....	227
24.3 Parameters.....	228
24.4 Ports.....	228
24.5 Functionality.....	228
24.5.1 Dadda Reduction Algorithm.....	228
24.5.2 Dadda Sequence Example.....	229

24.5.3 8-bit Implementation Structure.....	230
24.5.4 Reduction Comparison: Dadda vs Wallace.....	232
24.6 Usage Examples.....	233
24.6.1 Basic 8×8 Multiplication.....	233
24.6.2 16×16 Multiplication with Pipeline.....	233
24.6.3 Multiply-Accumulate (MAC) Unit.....	234
24.6.4 FIR Filter Tap.....	235
24.6.5 Parameterized Multiplier Selector.....	235
24.7 Timing Characteristics.....	236
24.8 Performance Characteristics.....	237
24.8.1 Resource Utilization.....	237
24.8.2 Multiplier Architecture Selection.....	238
24.9 Design Considerations.....	238
24.9.1 Advantages.....	238
24.9.2 Limitations.....	238
24.9.3 When to Use Dadda Tree.....	238
24.9.4 Dadda vs Wallace Decision.....	238
24.9.5 Signed Multiplication.....	239
24.9.6 Pipelining for High Frequency.....	239
24.10 Common Pitfalls.....	240
24.11 Related Modules.....	241
24.12 Algorithm Reference: Dadda Sequence.....	241
24.13 References.....	241
24.14 Navigation.....	242

25 Wallace Tree Multipliers.....	242
25.1 Overview.....	242
25.2 Module Declarations.....	243
25.2.1 8-bit Wallace Tree Multiplier.....	243
25.2.2 16-bit Wallace Tree Multiplier.....	243
25.2.3 32-bit Wallace Tree Multiplier.....	243
25.3 Parameters.....	243
25.4 Ports.....	244
25.5 Functionality.....	244
25.5.1 Wallace Tree Algorithm.....	244
25.5.2 8-bit Example Structure.....	245
25.5.3 Reduction Pattern.....	247
25.6 Usage Examples.....	248
25.6.1 Basic 8×8 Multiplication.....	248
25.6.2 16×16 Multiplication with Pipeline Register.....	248
25.6.3 32×32 Multiplication for DSP.....	248
25.6.4 Multi-Stage Pipelined Multiplier.....	249
25.7 Timing Characteristics.....	250
25.8 Performance Characteristics.....	250
25.8.1 Resource Utilization.....	250
25.8.2 Area-Speed Tradeoffs.....	252
25.9 Design Considerations.....	253
25.9.1 Advantages.....	253
25.9.2 Limitations.....	253

25.9.3 When to Use Wallace Tree.....	253
25.9.4 Signed Multiplication.....	253
25.9.5 Pipelining Strategy.....	254
25.9.6 Synthesis Considerations.....	254
25.10 Common Pitfalls.....	255
25.11 The <code>_csa_</code> Variant.....	256
25.12 Related Modules.....	256
25.13 Wallace Tree vs Dadda Tree.....	257
25.14 References.....	257
25.15 Navigation.....	257
26 Prefix Cell Gray.....	258
26.1 Overview.....	258
26.2 Module Declaration.....	258
26.3 Ports.....	258
26.4 Functionality.....	259
26.4.1 Implementation.....	259
26.4.2 Why Gray Cells Don't Need P Output.....	259
26.5 Timing Characteristics.....	261
26.6 Usage Examples.....	261
26.6.1 In Han-Carlson Final Stage.....	261
26.6.2 In Brent-Kung Reverse Tree.....	261
26.6.3 Computing Final Sum.....	262
26.7 Performance Characteristics.....	262
26.7.1 Resource Utilization.....	262

26.7.2 Comparison with Black Cell.....	262
26.7.3 Area Savings in Adder.....	262
26.8 Design Considerations.....	263
26.8.1 When to Use Gray Cells.....	263
26.8.2 Design Optimization Priorities.....	263
26.8.3 Architectural Trade-offs.....	263
26.9 Related Modules.....	263
26.10 Applications.....	264
26.11 References.....	264
26.12 Navigation.....	264
27 Prefix Cell (Black Cell).....	264
27.1 Overview.....	264
27.2 Module Declaration.....	265
27.3 Ports.....	265
27.4 Functionality.....	265
27.4.1 Parallel Prefix Theory.....	265
27.4.2 Implementation.....	265
27.4.3 Example: Combining Adjacent Pairs.....	266
27.5 Timing Characteristics.....	266
27.6 Usage Examples.....	266
27.6.1 In Kogge-Stone Adder (Stage 1).....	266
27.6.2 In Kogge-Stone Adder (Stage 2).....	267
27.6.3 In Han-Carlson Adder (Even Position).....	267
27.7 Performance Characteristics.....	267

27.7.1 Resource Utilization.....	267
27.7.2 Comparison: Black Cell vs Gray Cell.....	267
27.8 Design Considerations.....	268
27.8.1 When to Use Black vs Gray Cells.....	268
27.8.2 Design Optimization Priorities.....	268
27.8.3 Prefix Network Architectures.....	268
27.9 Related Modules.....	268
27.10 Applications.....	269
27.11 References.....	269
27.12 Navigation.....	269
28 Binary Subtractors.....	269
28.1 Overview.....	269
28.2 Module Declarations.....	270
28.2.1 Half Subtractor.....	270
28.2.2 Full Subtractor.....	270
28.2.3 N-bit Ripple Borrow Subtractor.....	270
28.2.4 N-bit Carry Lookahead Subtractor.....	270
28.3 Parameters.....	270
28.3.1 Single-bit Subtractors.....	270
28.3.2 N-bit Subtractors.....	271
28.4 Ports.....	271
28.4.1 Half Subtractor Ports.....	271
28.4.2 Full Subtractor Ports.....	271
28.4.3 N-bit Subtractor Ports.....	271

28.5 Functionality.....	272
28.5.1 Half Subtractor.....	272
28.5.2 Full Subtractor.....	272
28.5.3 N-bit Ripple Borrow Subtractor.....	273
28.5.4 N-bit Carry Lookahead Subtractor.....	273
28.6 Usage Examples.....	273
28.6.1 Basic Subtraction (Using Subtractor).....	273
28.6.2 Detecting Underflow ($A < B$).....	273
28.6.3 Modern Approach: Subtraction Using Adder.....	274
28.6.4 Multi-Precision Subtraction (16-bit via 2×8-bit).....	274
28.7 Timing Characteristics.....	275
28.8 Performance Characteristics.....	275
28.8.1 Resource Utilization.....	275
28.8.2 Subtractor vs Adder-Based Subtraction.....	275
28.9 Design Considerations.....	276
28.9.1 Why Adders Are Preferred for Subtraction.....	276
28.9.2 Implementing Subtraction in ALU.....	276
28.10 Common Pitfalls.....	277
28.11 Related Modules.....	278
28.12 References.....	278
28.13 Navigation.....	278

1 RTL Math Library

Location: rtl/common/math_*.sv **Generators:** bin/math_generate.py,
bin/math_generate.sh, bin/rtl_generators/ **Status:** Production Ready

1.1 Overview

The `rtl/common/math_*` family is a ~170-module arithmetic library covering integer add/subtract/multiply and IEEE-754-style floating-point (bf16, fp16, fp32, fp8) arithmetic, comparison, conversion, and machine-learning activation functions. The great majority of these modules are **code-generated** by the Python framework under `bin/rtl_generators/`, so the library is best understood by **operation** and, within each operation, by the **methodology** (algorithm) used — rather than as a flat list of 170 files.

This document is the organizing map: each operation lists its methodologies, the research each is based on, the module name pattern, and a link to the detailed per-methodology doc. The floating-point cores are themselves built from the integer methodologies below (Han-Carlson prefix adders for exponents, Dadda trees for mantissa multiplies), so the two halves of the library share a foundation.

Naming convention: width-parameterized generated instances carry the width suffix (`_008`, `_016`, `_032`, ...) and/or the format tag (`bf16`, `fp16`, `fp32`, `fp8_e4m3`, `fp8_e5m2`, `ieee754_2008_*`). One methodology doc covers all of its width/format instances.

1.2 Integer Arithmetic

1.2.1 Addition

Methodology	Modules	Research	Detail
Ripple-carry / full / half	<code>math_adder_full</code> , <code>math_adder_half</code> , <code>math_adder_full_nbit</code>	Classic ripple carry	math_adder_full.md , math_adder_basic.md
Carry-save (CSA)	<code>math_adder_carry_save_nbit</code>	Redundant carry-save form (used in multiplier trees)	math_adder_carry_save.md
Carry-lookahead	(see subtractor / adder-basic)	Weinberger & Smith, “A Logic for High-Speed Addition,” NBS Circular 591 (1958)	math_adder_pg_chain.md

Methodology	Modules	Research	Detail
Brent-Kung (parallel prefix)	<code>math_adder_brent_kung_{008,016,032} + prefix_cells(_black, _gray, _pg, _bitwisepg, _grouppg_*, _sum)</code>	Brent, R.P. & Kung, H.T. (1982), “A Regular Layout for Parallel Adders,” <i>IEEE Trans. Computers</i> C-31(3):260-264	math_adder_brent_kung.md , math_prefix_cell.md
Han-Carlson (parallel prefix)	<code>math_adder_han_carlson_{016,022,032,044,048,072}</code>	Han, T. & Carlson, D.A. (1987), “Fast area-efficient VLSI adders,” <i>Proc. 8th IEEE Symp. Computer Arithmetic (ARITH)</i> :49-56	math_adder_han_carlson.md
Add/subtract	<code>math_addsub_full_nbit</code>	Two’s-complement add/sub select	math_addsub.md

Parallel-prefix trade-off: all three prefix adders compute the carry network in $O(\log n)$ depth but differ in the area/wiring/fan-out balance — Brent-Kung minimizes cells/wiring (more depth), Kogge-Stone minimizes depth (most wiring), and Han-Carlson is the middle ground (a Kogge-Stone/Brent-Kung hybrid). See the [prefix-cell](#) and [gray-cell](#) docs for the shared generate/propagate building blocks.

1.2.2 Subtraction

Methodology	Modules	Detail
Ripple-carry / carry-lookahead / full / half	<code>math_subtractor_{full, half, full_nbit, ripple_carry, carry_lookahead}</code>	math_subtractor.md

1.2.3 Multiplication

Methodology	Modules	Research	Detail
Basic cell / carry-save	<code>math_multiplier_basic_cell</code> , <code>math_multiplier_carry_save</code>	Shift-add partial-product cells	math_multiplier_basic.md

Dadda tree	<code>math_multiplier_dadda_tree_{008,016,032},</code> <code>math_multiplier_dadda_4to2_{008,011,024}</code>	Dadda, L. (1965), “Some schemes for parallel multipliers,” <i>Alta Frequenza</i> 34:349-356	math_multiplier_dadda_tree.md , math_multiplier_dadda_4to2.md
Wallace tree	<code>math_multiplier_wallace_tree_{008,016,032}</code> , <code>math_multiplier_wallace_tree_csa_{008,016,032}</code>	Wallace, C.S. (1964), “A Suggestion for a Fast Multiplier,” <i>IEEE Trans. Electronic Computers</i> EC-13(1):14-17	math_multiplier_wallace_tree.md

Dadda vs Wallace: both reduce the partial-product matrix to two rows in $O(\log n)$ depth before a final carry-propagate add, but Dadda uses the *minimum* number of (3:2) counters that still meets the height schedule (fewer gates, slightly more wiring), while Wallace reduces as early/greedily as possible. The `_csa` Wallace variant reduces with explicit carry-save adders. The 4to2 variants use 4:2 compressors (see [math_compressor_4to2.md](#)) and are the building block the floating-point mantissa multipliers reuse.

1.3 Floating-Point Arithmetic

IEEE-754-2008 single (fp32) and half (fp16) plus the ML-oriented narrow formats bfloat16 (bf16) and 8-bit (fp8 e4m3 / e5m2). Each format’s core operators are assembled from the integer methodologies above: the **exponent path** uses a Han-Carlson prefix adder and the **mantissa multiply** uses a Dadda 4:2 tree.

1.3.1 Formats

Format	Modules tag	Notes
bfloat16	<code>math_bf16_*</code>	1-8-7; truncated fp32 range
IEEE fp16	<code>math_fp16_*</code> , <code>math_ieee754_2008_fp16_*</code>	1-5-10
IEEE fp32	<code>math_fp32_*</code> , <code>math_ieee754_2008_</code>	1-8-23

Format	Modules tag	Notes
	fp32_*	
fp8 E4M3	math_fp8_e4m3_*	1-4-3
fp8 E5M2	math_fp8_e5m2_*	1-5-2

1.3.2 Core operators (per format)

Operation	Modules	Method	Detail
Exponent add	math_*_exponent_adder	Han-Carlson prefix adder	math_bf16_exponent_adder.md
Mantissa multiply	math_*_mantissa_mult	Dadda 4:2 tree	math_bf16_mantissa_mult.md
Add	math_*_adder	Align → add → normalize → round (IEEE-754 §5)	math_bf16_adder.md
Multiply	math_*_multiplier	Exponent add + mantissa mult + normalize/round	math_bf16_multiplier.md
Fused multiply-add	math_*_fma	Single-rounding $a \cdot b + c$	math_bf16_fma.md

Detailed per-format coverage: [math_fp16_modules.md](#), [math_fp32_modules.md](#), [math_fp8_modules.md](#), [math_ieee754_modules.md](#), and the bf16 set ([math_bf16_extended.md](#)).

1.3.3 Division & reciprocal

Methodology	Modules	Research
Goldschmidt (multiplicative convergence)	math_bf16_goldschmidt_div, math_bf16_divider	Goldschmidt, R.E. (1964), “Applications of Division by Convergence,” M.Sc. thesis, MIT
Newton-Raphson reciprocal	math_bf16_newton_raphson_recip, math_bf16_reciproc	Iterative $x_{n+1} = x_n(2 - a \cdot x_n)$

Methodology	Modules	Research
	al, math_bf16_fast_rec iprocal	refinement of a seed

1.3.4 Conversion

`math_{src}_to_{dst}` — all bidirectional conversions among bf16 / fp16 / fp32 / fp8_e4m3 / fp8_e5m2, plus integer bridges (`math_bf16_to_int`, `math_int_to_bf16`, `math_bf16_scale_to_int8`). Each does exponent re-bias + mantissa round/truncate with correct saturation/subnormal handling for the destination format.

1.3.5 Comparison & range

`math_*_comparator`, `math_*_max`, `math_*_min`, `math_*_max_tree_8`, `math_*_min_tree_8`, `math_*_clamp` — magnitude compare (sign/exp/mantissa ordering), pairwise and 8-wide reduction trees, and clamp-to-range.

1.3.6 Activation functions (ML)

`math_*_{relu, leaky_relu, gelu, silu, sigmoid, tanh, softmax_8}` plus the transcendental helpers `math_bf16_{exp2, log2, log2_scale}`. These implement the standard neural-network activations directly in each low-precision format (piecewise / polynomial / LUT approximations sized to the format's mantissa), so an accelerator can keep activations in bf16/fp8 without promoting to fp32.

1.4 Generation Automation (bin/)

Almost every module above is emitted by a Python code-generator, so the RTL is regenerated rather than hand-edited. Two entry points:

1.4.1 Integer arithmetic — `bin/math_generate.py`

```
# one methodology + width per invocation
python bin/math_generate.py --type <brent_kung|dadda|wallace_fa|
wallace_csa> \
                        --path <out_dir> --buswidth <N>
```

`bin/math_generate.sh` is the batch driver that sweeps the standard widths (e.g. Brent-Kung at 8/16/32) into `math_outputs/`. The `--type` values map to the methodologies: `brent_kung` (prefix adder), `dadda / wallace_fa / wallace_csa` (the three multiplier reduction schemes). The emitters live in `bin/rtl_generators/utils` (`write_bk / write_dadda / write_wallace`) and `bin/rtl_generators/{adders, multipliers}/`.

1.4.2 Floating-point — `bin/rtl_generators/ieee754/generate_all.py`

`python bin/rtl_generators/ieee754/generate_all.py` [output_directory]

Generates the full FP library: the shared integer cores (`han_carlson_adder`, `dadda_4to2_multiplier`) followed by each format's {`mantissa_mult`, `exponent_adder`, `multiplier`, `adder`, `fma`} and all cross-format conversions. The `bf16` set has its own `bin/rtl_generators/bf16/generate_all.py`, and the activation / comparison / conversion families are emitted by `fp_activations.py`, `fp_comparisons.py`, and `fp_conversions.py`.

1.4.3 The emitter framework — `bin/rtl_generators/`

Path	Role
<code>verilog/module.py</code> , <code>param.py</code> , <code>signal.py</code> , <code>verilog_parser.py</code>	Structured Verilog emission (ports, params, signals) — the backend every generator writes through
<code>utils/utils.py</code>	Shared helpers (<code>write_bk</code> , <code>write_dadda</code> , <code>write_wallace</code> , formatting)
<code>adders/</code> , <code>multipliers/</code>	Integer methodology generators (Brent-Kung, Dadda, Wallace)
<code>ieee754/</code> , <code>bf16/</code>	Per-format FP operator generators + <code>generate_all.py</code>
<code>ecc/</code> , <code>amba/</code>	Other generated families (not part of <code>math_*</code>)
<code>unittests/</code>	Generator self-tests

1.4.4 Regeneration rule

These are generated files. Per the repo's Critical Rule #0, when a **generator** changes you must delete and regenerate **all** affected outputs (not a single width/format) and re-run the tests, because widths/cells share interfaces and a partial regen creates silent interface mismatches. Do not hand-edit a `math_*.sv` that a generator owns — change the generator and regenerate.

1.5 Research References

- Brent, R.P. & Kung, H.T. (1982). "A Regular Layout for Parallel Adders." *IEEE Transactions on Computers*, C-31(3), 260-264.
- Han, T. & Carlson, D.A. (1987). "Fast area-efficient VLSI adders." *Proc. 8th IEEE Symposium on Computer Arithmetic (ARITH-8)*, 49-56.

- Kogge, P.M. & Stone, H.S. (1973). “A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations.” *IEEE Transactions on Computers*, C-22(8), 786-793.
 - Dadda, L. (1965). “Some schemes for parallel multipliers.” *Alta Frequenza*, 34, 349-356.
 - Wallace, C.S. (1964). “A Suggestion for a Fast Multiplier.” *IEEE Transactions on Electronic Computers*, EC-13(1), 14-17.
 - Weinberger, A. & Smith, J.L. (1958). “A Logic for High-Speed Addition.” *National Bureau of Standards Circular 591*, 3-12.
 - Goldschmidt, R.E. (1964). “Applications of Division by Convergence.” M.Sc. thesis, MIT.
 - IEEE Std 754-2008, *IEEE Standard for Floating-Point Arithmetic*.
-

1.6 Related Documentation

- Prefix cells: [math_prefix_cell.md](#) · [math_prefix_cell_gray.md](#)
 - Multipliers: [dadda_tree](#) · [dadda_4to2](#) · [wallace_tree](#) · [basic](#) · [compressor_4to2](#)
 - Subtraction: [math_subtractor.md](#)
 - Floating-point: [bf16_extended](#) · [fp16](#) · [fp32](#) · [fp8](#) · [ieee754](#)
-

Last Updated: 2026-07-15

1.7 Navigation

- [Back to RTLCommon Index](#)

2 Basic Adder Building Blocks

Fundamental single-bit and multi-bit adder primitives (half adder, full adder, and N-bit full adder array) that serve as building blocks for more complex arithmetic circuits.

2.1 Overview

This document covers the three basic adder modules that form the foundation of all adder architectures: - **math_adder_half** - Single-bit half adder (2 inputs: A, B) - **math_adder_full** -

Single-bit full adder (3 inputs: A, B, Cin) - **math_adder_full_nbit** - N-bit array of full adders (identical to ripple carry adder)

These modules are used as building blocks in ripple carry adders, carry-save adders, multipliers, and other arithmetic circuits.

2.2 Module Declarations

2.2.1 Half Adder

```
module math_adder_half #(
    parameter int N = 1      // Unused parameter (kept for
    consistency)
) (
    input  logic i_a,        // Input A
    input  logic i_b,        // Input B
    output logic ow_sum,     // Sum output
    output logic ow_carry    // Carry output
);
```

2.2.2 Full Adder

```
module math_adder_full #(
    parameter int N = 1      // Unused parameter (kept for
    consistency)
) (
    input  logic i_a,        // Input A
    input  logic i_b,        // Input B
    input  logic i_c,        // Carry input
    output logic ow_sum,     // Sum output
    output logic ow_carry    // Carry output
);
```

2.2.3 N-bit Full Adder Array

```
module math_adder_full_nbit #(
    parameter int N = 4      // Number of bits
) (
    input  logic [N-1:0] i_a,    // Operand A
    input  logic [N-1:0] i_b,    // Operand B
    input  logic          i_c,    // Initial carry input
    output logic [N-1:0] ow_sum,  // Sum output
    output logic          ow_carry // Final carry output
);
```

2.3 Parameters

2.3.1 Half Adder & Full Adder

Parameter	Type	Default	Description
N	int	1	Unused legacy parameter (single-bit modules)

Note: The N parameter exists for consistency but is not used. These are single-bit modules.

2.3.2 N-bit Full Adder Array

Parameter	Type	Default	Description
N	int	4	Number of bits in the adder (range: 1-64)

2.4 Ports

2.4.1 Half Adder Ports

Port	Width	Description
i_a	1	Input A
i_b	1	Input B
ow_sum	1	Sum output ($A \oplus B$)
ow_carry	1	Carry output ($A \& B$)

2.4.2 Full Adder Ports

Port	Width	Description
i_a	1	Input A
i_b	1	Input B
i_c	1	Carry input
ow_sum	1	Sum output ($A \oplus B \oplus C_{in}$)
ow_carry	1	Carry output

2.4.3 N-bit Full Adder Ports

Port	Width	Description
i_a	N	Operand A
i_b	N	Operand B
i_c	1	Initial carry input
ow_sum	N	Sum output (A + B + Cin)
ow_carry	1	Final carry output

2.5 Functionality

2.5.1 Half Adder

The half adder adds two single-bit inputs without carry input:

Logic Equations:

$$\text{Sum} = A \oplus B \quad (\text{XOR})$$

$$\text{Carry} = A \cdot B \quad (\text{AND})$$

Truth Table: | A | B | Sum | Carry | |—|—|—|—| | 0 | 0 | 0 | 0 | | 0 | 1 | 1 | 0 | | 1 | 0 | 1 | 0 | | 1 | 1 | 0 | 1 |

Implementation:

```
assign ow_sum    = i_a ^ i_b;    // XOR for sum
assign ow_carry  = i_a & i_b;    // AND for carry
```

Use Cases: - LSB addition when no carry input exists - Building block for full adders - Parity generation (sum output) - Detecting when both inputs are 1 (carry output)

2.5.2 Full Adder

The full adder adds three single-bit inputs (A, B, Cin):

Logic Equations:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\begin{aligned} \text{Carry} &= (A \cdot B) \mid (\text{Cin} \cdot (A \oplus B)) \\ &= \text{Majority}(A, B, \text{Cin}) \end{aligned}$$

Truth Table: | A | B | Cin | Sum | Cout | |—|—|—|—|—| | 0 | 0 | 0 | 0 | 0 | | 0 | 0 | 1 | 1 | 0 | | 0 | 1 | 0 | 1 | 0 | | 0 | 1 | 1 | 0 | 1 | | 1 | 0 | 0 | 1 | 0 | | 1 | 0 | 1 | 0 | 1 | | 1 | 1 | 0 | 0 | 1 | | 1 | 1 | 1 | 1 | 1 |

Implementation:

```

assign ow_sum    = i_a ^ i_b ^ i_c;           // 3-input XOR
assign ow_carry = (i_a & i_b) | (i_c & (i_a ^ i_b)); // Majority
function

```

Alternative Carry Implementation (equivalent):

```

assign ow_carry = (i_a & i_b) | (i_a & i_c) | (i_b & i_c); // Full
majority

```

Use Cases: - Building block for ripple carry adders - Building block for carry-save adders - Multi-bit addition (chained together) - 3:2 compression in multiplier trees

2.5.3 N-bit Full Adder Array

Chains N full adders to create a multi-bit adder:

Structure:

```

Bit 0:  FA(A[0], B[0], Cin)      → Sum[0], C[0]
Bit 1:  FA(A[1], B[1], C[0])    → Sum[1], C[1]
Bit 2:  FA(A[2], B[2], C[1])    → Sum[2], C[2]
...
Bit N-1: FA(A[N-1], B[N-1], C[N-2]) → Sum[N-1], Cout

```

Implementation:

```

logic [N:0] w_c; // Intermediate carries
assign w_c[0] = i_c; // Initial carry

genvar i;
generate
    for (i = 0; i < N; i++) begin : gen_full_adders
        math_adder_full fa (
            .i_a(i_a[i]),
            .i_b(i_b[i]),
            .i_c(w_c[i]),
            .ow_sum(ow_sum[i]),
            .ow_carry(w_c[i+1])
        );
    end
endgenerate

assign ow_carry = w_c[N]; // Final carry out

```

Note: This module is functionally identical to `math_adder_ripple_carry`. The only difference is internal structure: - `math_adder_ripple_carry`: Instantiates first FA separately, then generates remaining - `math_adder_full_nbit`: Uniform generate loop for all FAs

2.6 Usage Examples

2.6.1 Half Adder: Parity Generator

```
logic [7:0] data;
logic parity;

// Generate even parity bit
genvar i;
generate
    if (i == 0) begin
        assign parity_chain[0] = data[0];
    end else begin
        math_adder_half u_parity (
            .i_a(data[i]),
            .i_b(parity_chain[i-1]),
            .ow_sum(parity_chain[i]), // XOR accumulation
            .ow_carry()               // Unused
        );
    end
endgenerate

assign parity = parity_chain[7];
```

2.6.2 Full Adder: Carry-Save Adder Stage

```
// Add 3 numbers using carry-save adder (1 level)
logic [7:0] a, b, c;
logic [7:0] sum, carry;

genvar i;
generate
    for (i = 0; i < 8; i++) begin : gen_csa
        math_adder_full fa (
            .i_a(a[i]),
            .i_b(b[i]),
            .i_c(c[i]),
            .ow_sum(sum[i]), // Sum bits
            .ow_carry(carry[i]) // Carry bits (shifted left)
        );
    end
endgenerate

// Final result: sum + (carry << 1) using conventional adder
logic [8:0] result;
assign result = {1'b0, sum} + {carry, 1'b0};
```

2.6.3 N-bit Full Adder: BCD Digit Adder

```
// Add two BCD digits (0-9)
logic [3:0] bcd_a, bcd_b, bcd_sum_raw;
logic carry_raw;

math_adder_full_nbit #(.N(4)) u_bcd_add (
    .i_a(bcd_a),
    .i_b(bcd_b),
    .i_c(1'b0),
    .ow_sum(bcd_sum_raw),
    .ow_carry(carry_raw)
);

// BCD correction
logic bcd_overflow;
assign bcd_overflow = carry_raw | (bcd_sum_raw > 4'd9);

logic [3:0] bcd_sum_corrected;
math_adder_full_nbit #(.N(4)) u_bcd_correct (
    .i_a(bcd_sum_raw),
    .i_b(bcd_overflow ? 4'd6 : 4'd0), // Add 6 for correction
    .i_c(1'b0),
    .ow_sum(bcd_sum_corrected),
    .ow_carry()
);
```

2.6.4 Building a 2-bit Adder from Scratch

```
logic [1:0] a, b, sum;
logic cin, cout;
logic c_intermediate;

// Bit 0: Half adder (if no carry input) or full adder
math_adder_full fa0 (
    .i_a(a[0]),
    .i_b(b[0]),
    .i_c(cin),
    .ow_sum(sum[0]),
    .ow_carry(c_intermediate)
);

// Bit 1: Full adder
math_adder_full fa1 (
    .i_a(a[1]),
    .i_b(b[1]),
    .i_c(c_intermediate),
    .ow_sum(sum[1]),
    .ow_carry(cout)
);
```



```
        .ow_carry(cout)
    );
```

2.7 Timing Characteristics

2.7.1 Propagation Delays

Module	Logic Levels	Typical Delay (ns)
Half Adder	1 (XOR/AND parallel)	~0.2
Full Adder	2 (XOR then carry)	~0.4
N-bit Array	2N (ripple chain)	~0.4N

Critical Paths:

Half Adder:

```
i_a/i_b → ow_sum    (1 XOR gate)
i_a/i_b → ow_carry  (1 AND gate)
```

Full Adder:

```
i_a/i_b/i_c → ow_sum    (2 XOR gates)
i_a/i_b/i_c → ow_carry  (2 gates: XOR + OR/AND)
```

N-bit Array:

```
i_c → FA[0].Cout → FA[1].Cout → ... → FA[N-1].Cout → ow_carry
      (2N gate delays)
```

2.8 Performance Characteristics

2.8.1 Resource Utilization

Module	LUTs	Description
Half Adder	2	1 XOR + 1 AND
Full Adder	2	Optimized to 2 LUTs on modern FPGAs
N-bit Array	~2N	N full adders

FPGA Note: Modern FPGAs (Xilinx, Intel) have dedicated carry chain logic that implements full adders very efficiently, often faster than generic LUT logic.

2.8.2 When to Use Each Module

Half Adder: - ✓ First bit of addition chain (no carry input) - ✓ Parity generation - ✓ Educational/demonstration purposes - ✗ Rarely used standalone in production designs

Full Adder: - ✓ Building block for custom arithmetic circuits - ✓ Carry-save adder stages - ✓ Multiplier partial product reduction - ✓ When instantiating adder arrays manually

N-bit Array: - ✓ Quick multi-bit adder for small widths ($N \leq 8$) - ✓ When ripple carry is acceptable
- ✗ For $N > 8$, consider carry lookahead or parallel prefix

2.9 Design Considerations

2.9.1 Why Use These Modules

Advantages: - **Minimal area:** Absolute smallest implementation - **Simple:** Easy to understand and verify - **Building blocks:** Compose into larger structures - **FPGA-friendly:** Maps well to carry chain primitives

Disadvantages: - **Slow for wide widths:** $O(N)$ delay - **Not optimal:** Synthesis tools can often do better - **Manual instantiation:** More code than behavioral + operator

2.9.2 Behavioral vs Structural

Behavioral (Let synthesis infer):

```
// Simple, synthesis tool optimizes
assign sum = a + b + cin;
```

Structural (Explicit full adders):

```
// Manual control, educational
genvar i;
generate
    for (i = 0; i < N; i++) begin
        math_adder_full fa (...);
    end
endgenerate
```

Recommendation: Use behavioral unless you need explicit control (e.g., carry-save adders, multipliers, custom timing control).

2.9.3 FPGA-Specific Considerations

Modern FPGAs have dedicated adder resources: - **Xilinx:** CARRY4 primitives - **Intel:** Carry chain logic - **Result:** Behavioral + often as fast as manual instantiation

Exception: Carry-save adders and multiplier trees benefit from explicit full adder instantiation.

2.10 Common Pitfalls

✗ **Anti-Pattern 1:** Using half adder where full adder needed

```
// WRONG: Half adder can't accept carry input
math_adder_half fa0 (
    .i_a(a[0]), .i_b(b[0]),
    .i_c(cin), // ERROR: No i_c port!
    ...
);
```

```
// RIGHT: Use full adder
math_adder_full fa0 (
    .i_a(a[0]), .i_b(b[0]), .i_c(cin), ...
);
```

✗ **Anti-Pattern 2:** Over-engineering simple additions

```
// WRONG: Manual full adder chain for simple addition
// (More code, no benefit)
genvar i;
generate
    for (i = 0; i < 16; i++) begin
        math_adder_full fa (...);
    end
endgenerate
```

```
// RIGHT: Use behavioral code (simpler, tool optimizes)
assign sum = a + b;
```

✗ **Anti-Pattern 3:** Ignoring synthesis capabilities

```
// Behavioral code synthesizes to optimal adder anyway:
assign sum = a + b;
```

```
// No need for explicit full adders unless:
// - Building carry-save adder
// - Building multiplier tree
// - Educational/demonstration purpose
```

✗ **Anti-Pattern 4:** Incorrect carry chain connection

```
// WRONG: Carry not properly chained
math_adder_full fa0 (... , .ow_carry(c[0]));
math_adder_full fa1 (... , .i_c(c[0])); // Missing intermediate
connection
```

```
// RIGHT: Ensure proper carry propagation
math_adder_full fa0 (... , .ow_carry(w_c[0]));
math_adder_full fa1 (... , .i_c(w_c[0]), .ow_carry(w_c[1]));
```

2.11 Related Modules

- **math_adder_ripple_carry.sv** - Uses full adders for N-bit addition
- **math_adder_carry_save.sv** - Parallel full adders (no carry chain)
- ****math_multiplier*.sv**** - Uses full adders for partial product reduction
- **math_adder_full_nbit.sv** - Identical functionality to ripple carry adder

2.12 References

- Mano, M.M. “Digital Design.” Prentice Hall, 2002.
- Wakerly, J.F. “Digital Design: Principles and Practices.” Pearson, 2006.
- Hennessy, J.L., Patterson, D.A. “Computer Architecture: A Quantitative Approach.” Morgan Kaufmann, 2011.

2.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

3 Brent-Kung Parallel Prefix Adder

A family of high-performance parallel prefix adders using the Brent-Kung algorithm, providing $O(\log N)$ depth with minimal area overhead through asymmetric tree structure and area-efficient gray cells.

3.1 Overview

The `math_adder_brent_kung` module family implements the Brent-Kung parallel prefix addition algorithm, which achieves $O(\log N)$ critical path depth while minimizing hardware area compared to other parallel prefix adders like Kogge-Stone. The Brent-Kung algorithm uses an asymmetric tree structure: a forward tree to compute intermediate results, followed by a reverse tree using area-efficient “gray” cells that only propagate generate signals.

Available widths: **8-bit, 16-bit, 32-bit**

3.2 Module Hierarchy

The Brent-Kung adder family consists of:

Top-Level Modules (User-Facing): - `math_adder_brent_kung_008.sv` - 8-bit adder -
`math_adder_brent_kung_016.sv` - 16-bit adder - `math_adder_brent_kung_032.sv` - 32-bit
adder

Internal Building Blocks: - math_adder_brent_kung_pg.sv - Single-bit P/G generator -
 math_adder_brent_kung_black.sv - Black cell (outputs P and G) -
 math_adder_brent_kung_gray.sv - Gray cell (outputs only G, area-efficient) -
 math_adder_brent_kung_bitwisepg.sv - Bitwise P/G logic stage -
 math_adder_brent_kung_grouppg_008/016/032.sv - Width-specific prefix networks -
 math_adder_brent_kung_sum.sv - Final sum calculation

3.3 Top-Level Module Declaration

```
module math_adder_brent_kung_032 #(
    parameter int N = 32          // Adder width in bits
) (
    input logic [N-1:0] i_a,      // Operand A
    input logic [N-1:0] i_b,      // Operand B
    input logic          i_c,      // Carry input
    output logic [N-1:0] ow_sum,   // Sum output
    output logic          ow_carry // Carry output
);
```

Note: Replace _032 with _008 or _016 for 8-bit or 16-bit versions.

3.4 Parameters

3.4.1 User-Settable Parameters

Parameter	Type	Default	Description
N	int	32	Adder width in bits (8, 16, or 32)

Width Options: - **8-bit:** Set N=8 in math_adder_brent_kung_008 - **16-bit:** Set N=16 in math_adder_brent_kung_016 - **32-bit:** Set N=32 in math_adder_brent_kung_032

Why Fixed Widths? The parallel prefix network structure is width-specific and optimized for each size during design. Generic parameterization would require runtime generation of the tree structure.

3.5 Ports

3.5.1 Inputs

Port	Width	Description
i_a	N	Operand A (addend)
i_b	N	Operand B (addend)
i_c	1	Carry input (0 for

Port	Width	Description
		addition, 1 for +1 increment)

3.5.2 Outputs

Port	Width	Description
ow_sum	N	Sum output (A + B + Cin)
ow_carry	1	Carry output (overflow)

3.6 Algorithm Overview

3.6.1 Parallel Prefix Addition Theory

Parallel prefix adders compute all carries in parallel using **propagate (P)** and **generate (G)** signals:

Bit-Level Definitions: - **Generate:** $G_i = A_i \& B_i$ (carry generated at position i) - **Propagate:** $P_i = A_i \wedge B_i$ (carry propagated through position i) - **Sum:** $S_i = P_i \wedge C_i$ (sum bit given incoming carry)

Group-Level Recurrence: - **G[i:j]** = $G[i:k] \mid (P[i:k] \& G[k-1:j])$ (group generate) - **P[i:j]** = $P[i:k] \& P[k-1:j]$ (group propagate)

3.6.2 Brent-Kung Algorithm Stages

The Brent-Kung adder executes in three stages:

3.6.2.1 Stage 1: Bitwise P/G Generation

For each bit i:
 $P[i] = A[i] \wedge B[i]$
 $G[i] = A[i] \& B[i]$

Hardware: Array of AND/XOR gates (1 level of logic)

3.6.2.2 Stage 2: Parallel Prefix Network (Forward + Reverse Trees)

Forward Tree ($\log_2(N)$ levels):
 Build binary tree upward using black cells
 Compute group P/G for increasing spans: [1:0], [3:2], [7:4], etc.

Reverse Tree ($\log_2(N)-1$ levels):
 Propagate group G downward using gray cells
 Fill in missing intermediate positions

Hardware: Network of black cells (forward) and gray cells (reverse)

Key Optimization: Gray cells only compute G (not P), saving ~30% area vs Kogge-Stone

3.6.2.3 Stage 3: Final Sum Calculation

For each bit i :

$\text{Sum}[i] = \text{P}[i] \oplus \text{G}[i-1]$ (XOR with incoming carry)

$\text{Carry_out} = \text{G}[N-1]$ (final group generate)

Hardware: Array of XOR gates (1 level of logic)

3.7 Architecture Details

3.7.1 Three-Stage Pipeline

The top-level module instantiates three sub-modules:

```
// Stage 1: Generate bitwise P and G signals
math_adder_brent_kung_bitwisepg #(.N(N)) BitwisePGLogic_inst (
    .i_a (i_a),
    .i_b (i_b),
    .i_c (i_c),
    .ow_g(ow_g), // Generate signals [N:0]
    .ow_p(ow_p)  // Propagate signals [N:0]
);

// Stage 2: Parallel prefix network (forward + reverse trees)
math_adder_brent_kung_grouppg_032 #(.N(N)) GroupPGLogic_inst (
    .i_g (ow_g),
    .i_p (ow_p),
    .ow_gg(ow_gg), // Group generate [N:0]
    .ow_pp()       // Unused (gray cells only compute G)
);

// Stage 3: Compute final sum
math_adder_brent_kung_sum #(.N(N)) SumLogic_inst (
    .i_p (ow_p),
    .i_gg (ow_gg),
    .ow_sum (ow_sum),
    .ow_carry (ow_carry)
);
```

3.7.2 Internal Building Blocks

3.7.2.1 PG Cell (*math_adder_brent_kung_pg*)

```
// Single-bit P/G generator
assign ow_g = i_a & i_b; // Generate
assign ow_p = i_a ^ i_b; // Propagate
```

Usage: Initial stage to create bit-level P/G signals

3.7.2.2 Black Cell (*math_adder_brent_kung_black*)

```
// Combines two P/G pairs, outputs both
assign ow_g = i_g | (i_p & i_g_kml); // Group generate
assign ow_p = i_p & i_p_kml; // Group propagate
```

Usage: Forward tree (needs both P and G for further propagation)

Logic Depth: 2 gates (1 AND + 1 OR/AND)

3.7.2.3 Gray Cell (*math_adder_brent_kung_gray*)

```
// Combines two P/G pairs, outputs only G (area-efficient)
assign ow_g = i_g | (i_p & i_g_kml); // Group generate
// No P output (saves one AND gate per cell)
```

Usage: Reverse tree (only needs G, not P)

Area Savings: ~33% fewer gates vs black cell

Logic Depth: 2 gates (1 AND + 1 OR)

3.7.3 Prefix Network Structure (32-bit Example)

Depth 0 (Bitwise): P0 G0 P1 G1 P2 G2 P3 G3 ... P31 G31

Depth 1 (Forward): [1:0] [3:2] [5:4] ... [31:30] (Black cells)

Depth 2 (Forward): [3:0] [7:4] [15:8] (Black cells)

Depth 3 (Forward): [7:0] [15:0] (Black cells)

Depth 4 (Forward): [15:0] [31:16] (Black cells)

Depth 5 (Forward): [31:0] (Black cell)

Depth 6 (Reverse): [2:0] [4:0] [6:0] ... (fill gaps) (Gray cells)

Depth 7 (Reverse): [1:0] [3:0] [5:0] [7:0] ... (fill gaps) (Gray cells)

Depth 8 (Sum): S0 S1 S2 S3 ... S31 (XOR gates)

Key Features: - **Forward tree depth:** $\log_2(N)$ levels - **Reverse tree depth:** $\log_2(N) - 1$ levels - **Total depth:** $2 \times \log_2(N) + 1$ levels (includes bitwise PG and sum stages) - **Black cells:** Used in forward tree (outputs both P and G) - **Gray cells:** Used in reverse tree (outputs only G, saves area)

3.8 Timing Characteristics

3.8.1 Combinational Delay Analysis

Width	Logic Levels	Typical Delay (ns) @ 1.0V	Max Frequency
8-bit	9	~2.0	~500 MHz
16-bit	11	~2.5	~400 MHz
32-bit	13	~3.0	~333 MHz

Logic Level Breakdown (32-bit): 1. Bitwise PG: 1 level (AND/XOR) 2. Forward tree: 5 levels (black cells) 3. Reverse tree: 5 levels (gray cells) 4. Sum calculation: 1 level (XOR) 5. **Total:** 12 levels

Comparison to Other Adders:

Adder Type	Logic Depth	Area	Best Use Case
Ripple Carry	$O(N)$	Minimal	Low-speed, area-critical
Carry Lookahead	$O(\log N)$	Medium	Moderate speed
Brent-Kung	$O(\log N)$	Medium	Balanced speed/area
Kogge-Stone	$O(\log N)$	Maximum	Maximum speed (datapath)

3.8.2 Critical Paths

1. **Forward Tree Path:** $i_a/i_b \rightarrow PG \rightarrow$ Black cells (levels 1-5) $\rightarrow$ Group G

2. **Reverse Tree Path:** Group G $\rightarrow$ Gray cells (levels 6-10) $\rightarrow$ Final G
3. **Sum Path:** Final G $\rightarrow$ XOR with P $\rightarrow$ ow_sum

Optimization Tip: Pipeline between stages for >1 GHz operation:

```
// Pipeline registers (example for 2-stage pipeline)
logic [N-1:0] r_p_stage1, r_g_stage1;
logic [N-1:0] r_sum;

always_ff @(posedge clk) begin
    // Stage 1: PG generation + first half of prefix tree
    r_p_stage1 <= ow_p;
    r_g_stage1 <= partial_gg; // Intermediate prefix result

    // Stage 2: Second half of prefix tree + sum
    r_sum <= ow_sum;
end
```

3.9 Usage Examples

3.9.1 Basic 32-bit Addition

```
logic [31:0] a, b, sum;
logic carry_out;

math_adder_brent_kung_032 #(
    .N(32)
) u_adder (
    .i_a      (a),
    .i_b      (b),
    .i_c      (1'b0),           // No carry input
    .ow_sum   (sum),
    .ow_carry (carry_out)
);

// Example: 123 + 456
initial begin
    a = 32'd123;
    b = 32'd456;
    #1; // Wait for combinational propagation
    assert(sum == 32'd579);
end
```

3.9.2 8-bit Incrementer

```
logic [7:0] count, count_plus_1;
```

```

math_adder_brent_kung_008 #(
    .N(8)
) u_incrementer (
    .i_a      (count),
    .i_b      (8'b0),           // B = 0
    .i_c      (1'b1),           // Carry in = 1 → +1 increment
    .ow_sum    (count_plus_1),
    .ow_carry  ()               // Unused
);

```

3.9.3 16-bit Subtraction (A - B)

```

logic [15:0] a, b, difference;
logic borrow;

```

```

// Subtraction: A - B = A + (~B) + 1 (two's complement)
math_adder_brent_kung_016 #(
    .N(16)
) u_subtractor (
    .i_a      (a),
    .i_b      (~b),             // Invert B
    .i_c      (1'b1),           // Carry in = 1 (for two's complement)
    .ow_sum    (difference),
    .ow_carry  (borrow)         // Borrow = ~carry_out
);

```

```

// Example: 1000 - 300
initial begin
    a = 16'd1000;
    b = 16'd300;
    #1;
    assert(difference == 16'd700);
end

```

3.9.4 Multi-Precision Addition (64-bit via 2×32-bit)

```

logic [31:0] a_low, a_high, b_low, b_high;
logic [31:0] sum_low, sum_high;
logic carry_low, carry_high;

```

```

// Low 32 bits
math_adder_brent_kung_032 u_add_low (
    .i_a      (a_low),
    .i_b      (b_low),
    .i_c      (1'b0),
    .ow_sum    (sum_low),
    .ow_carry  (carry_low)
);

```

```

// High 32 bits (chain carry from low)
math_adder_brent_kung_032 u_add_high (
    .i_a      (a_high),
    .i_b      (b_high),
    .i_c      (carry_low),      // Carry from low adder
    .ow_sum   (sum_high),
    .ow_carry (carry_high)
);

// Concatenate results
logic [63:0] sum_64 = {sum_high, sum_low};

```

3.9.5 Overflow Detection (Signed Addition)

```

logic [31:0] a_signed, b_signed, sum_signed;
logic overflow;

math_adder_brent_kung_032 u_signed_add (
    .i_a      (a_signed),
    .i_b      (b_signed),
    .i_c      (1'b0),
    .ow_sum   (sum_signed),
    .ow_carry ()
);

// Overflow detection for signed addition
// Overflow = (A[31] == B[31]) && (Sum[31] != A[31])
assign overflow = (a_signed[31] == b_signed[31]) &&
    (sum_signed[31] != a_signed[31]);

// Example: Detect overflow
initial begin
    // Max positive + positive = overflow
    a_signed = 32'h7FFFFFFF; // +2,147,483,647
    b_signed = 32'h00000001; // +1
    #1;
    assert(overflow == 1'b1); // Result wraps to negative
end

```

3.10 Performance Characteristics

3.10.1 Resource Utilization

Width	LUTs (Est.)	FFs (Pipeline)	Description
8-bit	~60	0 (combinational)	Minimal

Width	LUTs (Est.)	FFs (Pipeline)	Description
16-bit	~140	0 (combinational)	Small
32-bit	~300	0 (combinational)	Medium

Area Breakdown (32-bit): - Bitwise PG: 32×2 gates = 64 LUTs - Black cells (forward): ~ 50 cells $\times$ 3 gates = 150 LUTs - Gray cells (reverse): ~ 30 cells $\times$ 2 gates = 60 LUTs - Sum logic: 32×1 gate = 32 LUTs - **Total:** ~ 306 LUTs

Comparison: - **Ripple Carry:** 32 LUTs (32 full adders) - **Brent-Kung:** 300 LUTs (balanced) - **Kogge-Stone:** 450 LUTs (maximum speed, maximum area)

3.10.2 Speed vs Area Trade-offs

Adder Architecture	Relative Speed	Relative Area	Logic Depth
Ripple Carry	1.0 $\times$ (slowest)	1.0 $\times$ (smallest)	$O(N)$
Carry Lookahead	4.0 $\times$	3.0 $\times$	$O(\log N)$
Brent-Kung	6.0$\times$	4.5$\times$	$O(\log N)$
Kogge-Stone	8.0 $\times$ (fastest)	7.0 $\times$ (largest)	$O(\log N)$

When to Use Brent-Kung: - $\checkmark$ **Balanced designs:** Need good speed without excessive area - $\checkmark$ **Mid-range frequency:** 200-500 MHz targets - $\checkmark$ **FPGA implementations:** LUT utilization matters - $\checkmark$ **Multiple adders:** Area budget shared across many units

When to Use Alternatives: - Use **Ripple Carry** for: Low-speed, area-critical (e.g., control logic) - Use **Kogge-Stone** for: Critical datapath, maximum performance (e.g., FPU)

3.11 Design Considerations

3.11.1 Width Selection

Available Widths: - **8-bit:** Suitable for byte operations, small arithmetic units - **16-bit:** Common for ALU slices, address arithmetic - **32-bit:** Standard integer width, general-purpose ALU

For Non-Standard Widths: - Extend inputs with zeros: $\{24'b0, a[7:0]\}$ for 8-bit in 32-bit adder - Truncate outputs: $sum_32[15:0]$ for 16-bit result - Or use ripple carry / carry lookahead for custom widths

3.11.2 Synthesis Considerations

Optimization Tips: 1. Flatten hierarchy for best optimization: `tcl set_flatten true - effort high`

2. Pipeline for high frequency:

- Break at forward/reverse tree boundary
- Break at prefix network / sum calculation boundary

3. Critical path optimization:

```
set_max_delay -from [get_ports i_a] -to [get_ports ow_sum] 3.0
```

4. Area optimization (if needed):

- Consider carry lookahead instead (smaller but still fast)
- Share adders using time-division multiplexing

3.11.3 Verification Strategy

Test suite location: `val/common/test_math_adder_brent_kung_*.py`

Key Test Scenarios: - Exhaustive testing (8-bit only) - Random stimulus (16/32-bit) - Corner cases: 0+0, 0+MAX, MAX+MAX - Carry propagation: All 1's + 1 - Signed overflow detection - Multi-precision chaining

Test Command:

Test all widths

```
pytest val/common/test_math_adder_brent_kung_008.py -v
pytest val/common/test_math_adder_brent_kung_016.py -v
pytest val/common/test_math_adder_brent_kung_032.py -v
```

3.12 Common Pitfalls

✗ Anti-Pattern 1: Using wrong width variant

```
// WRONG: Trying to parameterize to 24-bit
math_adder_brent_kung_032 #(.N(24)) u_add (...);
// Result: Mismatch with grouppg_032 network (designed for 32-bit)
```

```
// RIGHT: Use 32-bit, ignore upper bits
math_adder_brent_kung_032 #(.N(32)) u_add (
    .i_a({8'b0, a[23:0]}), // Zero-extend
    .i_b({8'b0, b[23:0]}),
    .ow_sum(sum_32),
    ...
)
```

```
);
logic [23:0] sum = sum_32[23:0]; // Truncate

x Anti-Pattern 2: Ignoring carry output for signed arithmetic

// WRONG: Using carry for signed overflow
logic [31:0] a_signed, b_signed, sum;
logic overflow;

math_adder_brent_kung_032 u_add (...);
assign overflow = ow_carry; // INCORRECT for signed!
```

```
// RIGHT: Check sign bits
assign overflow = (a_signed[31] == b_signed[31]) &&
                  (sum[31] != a_signed[31]);
```

x **Anti-Pattern** 3: Expecting registered outputs

```
// WRONG: Assuming outputs are registered
always_ff @(posedge clk) begin
    result <= ow_sum; // ow_sum is combinational!
end
```

```
// RIGHT: Add explicit output register
logic [31:0] r_sum;
always_ff @(posedge clk) begin
    r_sum <= ow_sum;
end
```

x **Anti-Pattern** 4: Chaining without considering timing

```
// WRONG: Long chain of adders in one cycle
logic [31:0] a, b, c, d, result;
logic [31:0] ab_sum, abc_sum;

math_adder_brent_kung_032 u_add1
(.i_a(a), .i_b(b), .ow_sum(ab_sum), ...);
math_adder_brent_kung_032 u_add2
(.i_a(ab_sum), .i_b(c), .ow_sum(abc_sum), ...);
math_adder_brent_kung_032 u_add3
(.i_a(abc_sum), .i_b(d), .ow_sum(result), ...);

// 3 × 3ns = 9ns path! May not close timing at high frequency

// RIGHT: Pipeline the chain
logic [31:0] r_ab_sum, r_abc_sum;
always_ff @(posedge clk) begin
    r_ab_sum <= ab_sum;
```

```
    r_abc_sum <= abc_sum;  
end
```

3.13 Related Modules

- **math_adder_pg_chain.sv** - Medium-speed adder (less area)
- **math_adder_ripple_carry.sv** - Minimal area adder (slow)
- **math_adder_carry_save_nbit.sv** - For multi-operand addition (multipliers)
- ****math_subtractor_.sv**** - Subtraction variants (uses two's complement)

3.14 References

- Brent, R.P., Kung, H.T. "A Regular Layout for Parallel Adders." IEEE Transactions on Computers, 1982.
- Harris, D. "A Taxonomy of Parallel Prefix Networks." Asilomar Conference, 2003.
- Deschamps, J.P., Bioul, G., Sutter, G. "Synthesis of Arithmetic Circuits: FPGA, ASIC and Embedded Systems." Wiley, 2006.

3.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

4 Carry-Save Adder

Single-bit and N-bit carry-save adder modules for high-speed multi-operand addition, implementing 3:2 compression with constant-depth parallel operation. These modules are fundamental building blocks for fast multipliers and multi-operand addition trees.

4.1 Overview

Carry-save adders (CSAs) perform parallel addition of three operands in a single level of logic by keeping the sum and carry outputs separate (not chaining carries between bits). This enables: - **Constant depth:** O(1) logic depth regardless of bit width - **3:2 compression:** Reduce 3 numbers to 2 in one stage - **Multiplier trees:** Essential for Wallace and Dadda tree multipliers - **Multi-operand addition:** Add many numbers efficiently

Modules Covered: - **math_adder_carry_save** - Single-bit CSA (3:2 compressor) - **math_adder_carry_save_nbit** - N-bit parallel CSA array

4.2 Module Declarations

4.2.1 Single-bit Carry-Save Adder

```
module math_adder_carry_save (
    input logic i_a,           // Input A
    input logic i_b,           // Input B
    input logic i_c,           // Input C
    output logic ow_sum,        // Sum output
    output logic ow_carry       // Carry output
);
```

4.2.2 N-bit Carry-Save Adder

```
module math_adder_carry_save_nbit #(
    parameter int N = 4        // Bit width
) (
    input logic [N-1:0] i_a,    // Operand A
    input logic [N-1:0] i_b,    // Operand B
    input logic [N-1:0] i_c,    // Operand C (or carry from
previous CSA)
    output logic [N-1:0] ow_sum, // Sum outputs (parallel)
    output logic [N-1:0] ow_carry // Carry outputs (parallel, NOT
chained!)
);
```

4.3 Parameters

4.3.1 Single-bit CSA

No parameters (fixed single-bit operation).

4.3.2 N-bit CSA

Parameter	Type	Default	Description
N	int	4	Bit width (range: 1-64, typical: 8-64)

Width Guidelines: - **Typical:** 8-64 bits (matches datapath width) - **Multipliers:** Match partial product width - **No practical limit:** CSA depth is independent of N

4.4 Ports

4.4.1 Single-bit CSA Ports

Port	Direction	Width	Description
i_a	Input	1	Input A
i_b	Input	1	Input B
i_c	Input	1	Input C
ow_sum	Output	1	Sum output bit
ow_carry	Output	1	Carry output bit

4.4.2 N-bit CSA Ports

Port	Direction	Width	Description
i_a	Input	N	Operand A
i_b	Input	N	Operand B
i_c	Input	N	Operand C
ow_sum	Output	N	Sum vector (parallel)
ow_carry	Output	N	Carry vector (parallel, NOT shifted!)

Important: The N-bit CSA outputs TWO N-bit vectors. To get the final result, you must add them using a conventional adder (ripple carry, CLA, or parallel prefix).

4.5 Functionality

4.5.1 Single-bit Carry-Save Adder (3:2 Compressor)

The CSA compresses 3 bits into 2 bits (sum and carry):

Logic Equations:

```
Sum = A ⊕ B ⊕ C                                     // 3-input XOR
Carry = (A • B) | (A • C) | (B • C)                 // Majority
function
    = Majority(A, B, C)
```

Truth Table:

A	B	C	Sum	Carry	A+B+C (decimal)
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	2
1	0	0	1	0	1
1	0	1	0	1	2
1	1	0	0	1	2
1	1	1	1	1	3

Key Property: $\text{Sum} + 2 \times \text{Carry} = A + B + C$ (mathematically exact)

Implementation:

```
assign ow_sum    = i_a ^ i_b ^ i_c; // XOR
assign ow_carry = (i_a & i_b) | (i_a & i_c) | (i_b & i_c); // Majority
```

Note: This is identical to a full adder, but used differently: - **Full adder:** Carry chains to next bit - **CSA:** Carry kept separate, processed in next stage

4.5.2 N-bit Carry-Save Adder

The N-bit CSA applies 3:2 compression to each bit position in parallel:

```
Bit 0:  CSA(A[0], B[0], C[0]) → Sum[0], Carry[0]
Bit 1:  CSA(A[1], B[1], C[1]) → Sum[1], Carry[1]
...
Bit N-1: CSA(A[N-1], B[N-1], C[N-1]) → Sum[N-1], Carry[N-1]
```

Implementation:

```
genvar i;
generate
    for (i = 0; i < N; i++) begin : gen_carry_save
        assign ow_sum[i]    = i_a[i] ^ i_b[i] ^ i_c[i];
        assign ow_carry[i] = (i_a[i] & i_b[i]) | (i_a[i] & i_c[i]) |
        (i_b[i] & i_c[i]);
    end
endgenerate
```

Critical Difference from Ripple Carry: - **Carry NOT chained:** Each bit's carry goes to separate output, not to next bit - **Constant depth:** All bits computed in parallel (1 level of logic) - **Requires final addition:** Must add sum and carry vectors to get actual result

4.5.3 Final Addition Step

To get the actual sum, add the two output vectors (carry is in same weight position):

```
// CSA produces two vectors
math_adder_carry_save_nbit #(N) u_csa (
    .i_a(a), .i_b(b), .i_c(c),
    .ow_sum(sum_vec),
    .ow_carry(carry_vec)
);

// Add the two vectors to get final result
```

```
// IMPORTANT: Carry vector is NOT shifted!
logic [N:0] final_result;
assign final_result = {1'b0, sum_vec} + {1'b0, carry_vec};
```

Common Misconception: The carry vector does NOT need left-shifting. The CSA already accounts for weight alignment internally.

4.6 Timing Characteristics

4.6.1 Propagation Delays

Module	Logic Levels	Typical Delay (ns)
Single-bit CSA	2	~0.4
N-bit CSA	2	~0.4 (independent of N!)

Key Advantage: Delay is constant regardless of bit width!

Critical Path:

```
i_a/i_b/i_c → ow_sum    (2 XOR gates)
i_a/i_b/i_c → ow_carry  (2 gates: AND + OR)
```

Comparison: | Adder Type | N-bit Delay | |———|———| | **CSA** | **O(1)** - constant | | Ripple Carry | O(N) - linear | | Carry Lookahead | O(N) - linear (simplified) | | Parallel Prefix | O(log N) |

4.7 Usage Examples

4.7.1 Adding 3 Numbers (Single CSA Stage)

```
logic [7:0] a, b, c;
logic [7:0] sum_vec, carry_vec;
logic [8:0] final_result;

// Stage 1: CSA reduces 3 numbers to 2
math_adder_carry_save_nbit #(.N(8)) u_csa (
    .i_a(a),
    .i_b(b),
    .i_c(c),
    .ow_sum(sum_vec),
    .ow_carry(carry_vec)
);

// Stage 2: Conventional adder produces final result
assign final_result = {1'b0, sum_vec} + {1'b0, carry_vec};
```

```
// Example: 10 + 20 + 30 = 60
initial begin
    a = 8'd10;
    b = 8'd20;
    c = 8'd30;
    #1;
    assert(final_result == 9'd60);
end
```

4.7.2 Adding 4 Numbers (2-Level CSA Tree)

```
logic [7:0] a, b, c, d;
logic [7:0] sum1, carry1;
logic [7:0] sum2, carry2;
logic [9:0] final_result;

// Level 1: CSA reduces 4 numbers to 2 (actually 3)
// CSA1: A + B + C → Sum1, Carry1
math_adder_carry_save_nbit #(.N(8)) u_csa1 (
    .i_a(a), .i_b(b), .i_c(c),
    .ow_sum(sum1),
    .ow_carry(carry1)
);

// Level 2: CSA reduces 3 numbers to 2
// CSA2: Sum1 + Carry1 + D → Sum2, Carry2
math_adder_carry_save_nbit #(.N(8)) u_csa2 (
    .i_a(sum1),
    .i_b(carry1),
    .i_c(d),
    .ow_sum(sum2),
    .ow_carry(carry2)
);

// Final: Conventional adder
assign final_result = {2'b0, sum2} + {2'b0, carry2};
```

4.7.3 Adding 7 Numbers (Wallace Tree Structure)

```
// Reduce 7 numbers to 2 using CSA tree
logic [7:0] n1, n2, n3, n4, n5, n6, n7;

// Level 1: 7 → 5 (two CSAs)
logic [7:0] l1_s1, l1_c1, l1_s2, l1_c2;
math_adder_carry_save_nbit #(.N(8)) u_csa_l1_1 (
    .i_a(n1), .i_b(n2), .i_c(n3),
    .ow_sum(l1_s1), .ow_carry(l1_c1)
```

```

);
math_adder_carry_save_nbit #(.N(8)) u_csa_l1_2 (
    .i_a(n4), .i_b(n5), .i_c(n6),
    .ow_sum(l1_s2), .ow_carry(l1_c2)
);
// n7 passes through

// Level 2: 5 → 4 (one CSA)
logic [7:0] l2_s1, l2_c1;
math_adder_carry_save_nbit #(.N(8)) u_csa_l2_1 (
    .i_a(l1_s1), .i_b(l1_c1), .i_c(l1_s2),
    .ow_sum(l2_s1), .ow_carry(l2_c1)
);
// l1_c2, n7 pass through

// Level 3: 4 → 3 (one CSA)
logic [7:0] l3_s1, l3_c1;
math_adder_carry_save_nbit #(.N(8)) u_csa_l3_1 (
    .i_a(l2_s1), .i_b(l2_c1), .i_c(l1_c2),
    .ow_sum(l3_s1), .ow_carry(l3_c1)
);
// n7 passes through

// Level 4: 3 → 2 (one CSA)
logic [7:0] l4_s1, l4_c1;
math_adder_carry_save_nbit #(.N(8)) u_csa_l4_1 (
    .i_a(l3_s1), .i_b(l3_c1), .i_c(n7),
    .ow_sum(l4_s1), .ow_carry(l4_c1)
);

// Final addition
logic [9:0] final_result;
assign final_result = {2'b0, l4_s1} + {2'b0, l4_c1};

```

4.7.4 8-bit×8-bit Multiplier Partial Product Reduction

```

// Generate partial products (8×8 = 8 partial products)
logic [7:0] pp [0:7]; // Partial products

// CSA tree to reduce 8 partial products to 2
// (Simplified - real implementation more complex)

// Level 1: 8 → 6 (two CSAs)
logic [15:0] l1_s1, l1_c1, l1_s2, l1_c2;
math_adder_carry_save_nbit #(.N(16)) u_csa_pp_l1_1 (
    .i_a({8'b0, pp[0]}),
    .i_b({7'b0, pp[1], 1'b0}),

```

```

        .i_c({6'b0, pp[2], 2'b0}),
        .ow_sum(l1_s1), .ow_carry(l1_c1)
    );
    // ... continue tree reduction
    // Final: Fast adder for last two vectors

```

4.8 Performance Characteristics

4.8.1 Resource Utilization

Module	LUTs	Description
Single-bit CSA	2	1 XOR + 1 Majority
N-bit CSA	~2N	N single-bit CSAs

Comparison to Full Adder: - **Logic:** Identical (both use XOR + Majority) - **Usage:** Different (parallel vs chained) - **Area:** Same (~2 LUTs per bit)

4.8.2 Speed Advantages

CSA vs Ripple Carry Adder (8-bit example):

Operation	Depth	Delay (ns)
CSA (parallel)	2 levels	~0.4
Ripple (sequential)	16 levels	~3.0
Speedup	8×	7.5×

Why CSA is Faster: - No carry propagation between bits - All bits computed in parallel - Constant depth regardless of width

4.9 Design Considerations

4.9.1 When to Use Carry-Save Adders

✓ **Ideal Use Cases:** - **Multipliers:** Wallace tree, Dadda tree - **Multi-operand addition:** Add 3+ numbers - **DSP applications:** MAC units, filters - **Dot products:** Sum of products - **Compression trees:** Reduce many numbers to few

4.9.2 When NOT to Use CSA

✗ **Inappropriate Use Cases:** - **Two-operand addition:** Use conventional adder (simpler) - **Final result needed immediately:** CSA requires additional adder stage - **Single addition:** No benefit over regular adder

4.9.3 CSA Tree Design Guidelines

Number of Stages:

N operands $\rightarrow \text{ceil}(\log_{1.5}(N))$ CSA stages + 1 final adder

Example: - 3 operands: 1 CSA + 1 adder - 7 operands: 4 CSA + 1 adder - 15 operands: 6 CSA + 1 adder

Optimization: Minimize final adder width by aligning partial products carefully.

4.9.4 Final Adder Selection

After CSA tree reduction, choose final adder wisely:

Width	Best Final Adder
≤ 8 bits	Ripple carry (small, adequate)
8-16 bits	Carry lookahead
16-32 bits	Brent-Kung
≥ 32 bits	Kogge-Stone or pipelined

4.10 Common Pitfalls

✗ **Anti-Pattern 1:** Shifting carry output

```
// WRONG: Don't shift carry vector!  
assign final_result = sum_vec + {carry_vec, 1'b0}; // INCORRECT!
```

```
// RIGHT: Add without shift (CSA handles alignment)  
assign final_result = sum_vec + carry_vec;
```

✗ **Anti-Pattern 2:** Using CSA for two-operand addition

```
// WRONG: Overkill for simple addition  
math_adder_carry_save_nbit u_csa (  
    .i_a(a), .i_b(b), .i_c(8'b0), // Third input wasted!  
    ...  
);  
assign result = sum_vec + carry_vec; // Extra adder stage!
```

```
// RIGHT: Use conventional adder  
assign result = a + b; // Simpler, same result
```

✗ **Anti-Pattern 3:** Ignoring final addition

```
// WRONG: CSA outputs are NOT the final result!  
math_adder_carry_save_nbit u_csa (  
    ...  
);
```



```

        .i_a(a), .i_b(b), .i_c(c),
        .ow_sum(result),           // INCOMPLETE!
        .ow_carry(ignored)        // Missing carry!
    );

```

```

// RIGHT: Add sum and carry vectors
assign final_result = ow_sum + ow_carry;

```

✗ **Anti-Pattern 4:** Incorrect width for final addition

```

// WRONG: Overflow possible
logic [7:0] a, b, c;
logic [7:0] sum_vec, carry_vec;
logic [7:0] result; // TOO NARROW!
assign result = sum_vec + carry_vec; // May overflow!

```

```

// RIGHT: Add extra bit for overflow
logic [8:0] result; // Width N+1
assign result = {1'b0, sum_vec} + {1'b0, carry_vec};

```

4.11 Related Modules

- **math_adder_full.sv** - Identical logic, used in ripple chains
- ****math_multiplier_wallace_tree*.sv**** - Uses CSA for partial product reduction
- ****math_multiplier_dadda_tree*.sv**** - Uses CSA with optimized schedule
- **math_multiplier_carry_save.sv** - Multiplier variant using CSA stages

4.12 References

- Wallace, C.S. “A Suggestion for a Fast Multiplier.” IEEE Transactions on Electronic Computers, 1964.
- Dadda, L. “Some Schemes for Parallel Multipliers.” Alta Frequenza, 1965.
- Deschamps, J.P., Bioul, G., Sutter, G. “Synthesis of Arithmetic Circuits.” Wiley, 2006.
- Hennessy, J.L., Patterson, D.A. “Computer Architecture: A Quantitative Approach.” Morgan Kaufmann, 2011.

4.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

5 math_adder_full

A single-bit full adder module that performs binary addition of two input bits plus a carry-in, producing a sum and carry-out.

5.1 Overview

The `math_adder_full` module implements the fundamental building block of binary arithmetic - a full adder. It adds three single-bit inputs (two operands and a carry-in) and produces a sum bit and carry-out bit. This module serves as the basis for multi-bit adders and more complex arithmetic units.

5.2 Module Declaration

```
module math_adder_full #(parameter int N=1) (  
    input logic i_a,  
    input logic i_b,  
    input logic i_c,  
    output logic ow_sum,  
    output logic ow_carry  
);
```

5.3 Parameters

Parameter	Type	Default	Description
N	int	1	Parameter for potential future extensions (currently unused)

5.4 Ports

5.4.1 Inputs

Port	Width	Description
i_a	1	First input operand bit
i_b	1	Second input operand bit
i_c	1	Carry input bit

5.4.2 Outputs

Port	Width	Description
ow_sum	1	Sum output bit ($i_a \oplus i_b \oplus i_c$)
ow_carry	1	Carry output bit

5.5 Functionality

5.5.1 Full Adder Logic

The full adder implements the following Boolean functions:

- **Sum Output:** $ow_sum = i_a \oplus i_b \oplus i_c$
- **Carry Output:** $ow_carry = (i_a \& i_b) \mid (i_c \& (i_a \oplus i_b))$

5.5.2 Truth Table

i_a	i_b	i_c	ow_sum	ow_carry	Decimal
0	0	0	0	0	$0 + 0 + 0 = 0$
0	0	1	1	0	$0 + 0 + 1 = 1$
0	1	0	1	0	$0 + 1 + 0 = 1$
0	1	1	0	1	$0 + 1 + 1 = 2$
1	0	0	1	0	$1 + 0 + 0 = 1$
1	0	1	0	1	$1 + 0 + 1 = 2$
1	1	0	0	1	$1 + 1 + 0 = 2$
1	1	1	1	1	$1 + 1 + 1 = 3$

5.6 Implementation Details

5.6.1 Sum Generation

The sum output uses a three-input XOR gate:

```
assign ow_sum = i_a ^ i_b ^ i_c;
```

This produces 1 when an odd number of inputs are 1, and 0 when an even number are 1.

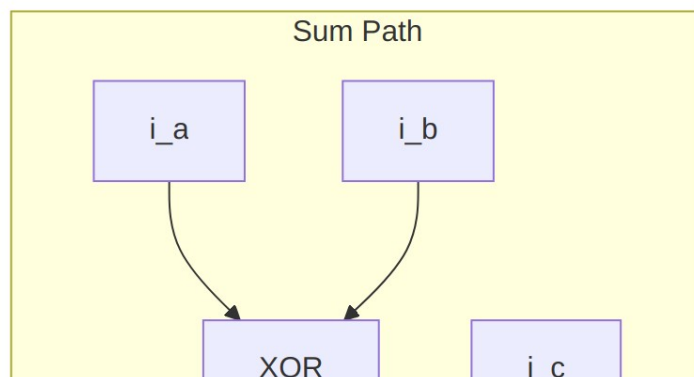
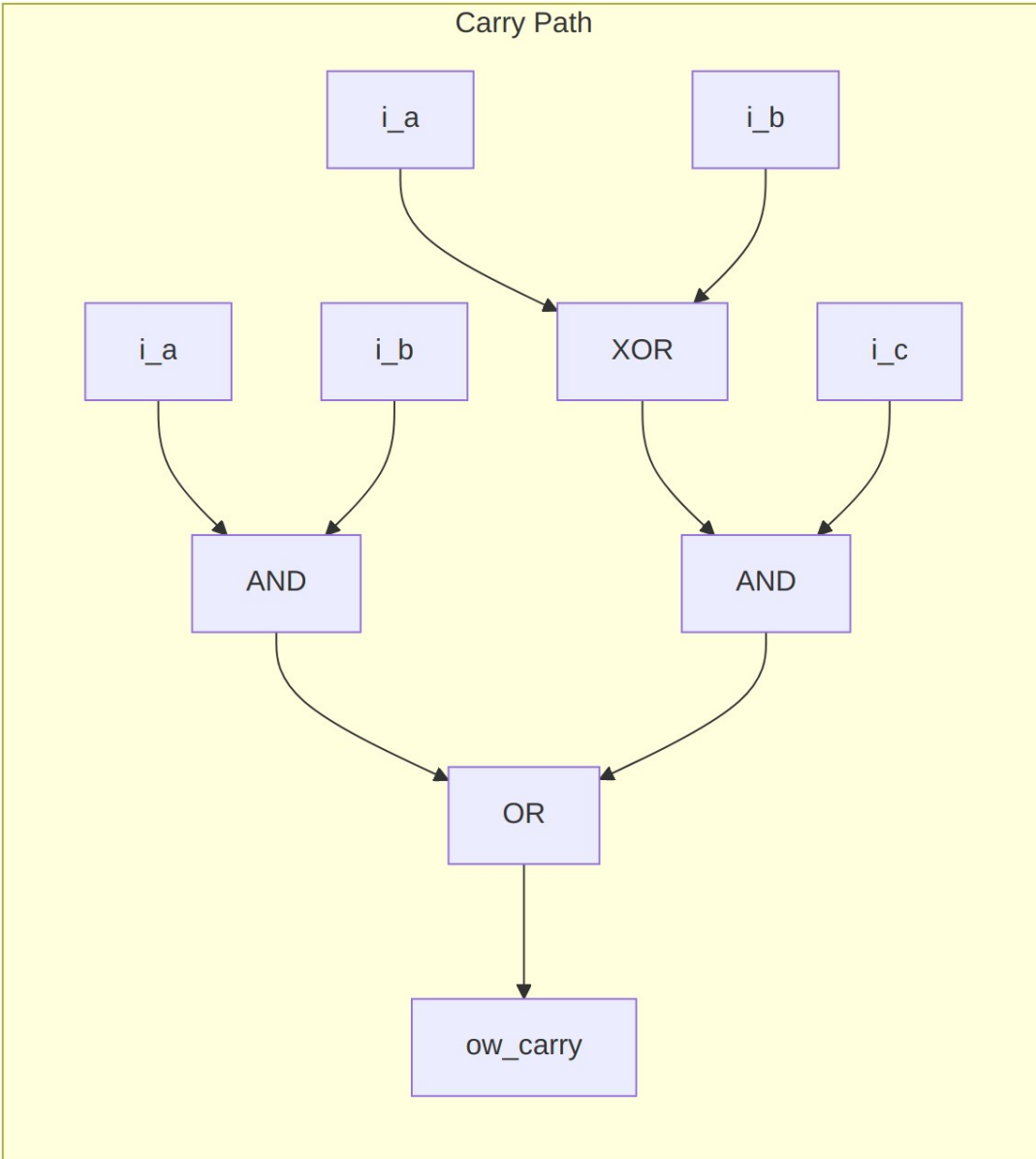
5.6.2 Carry Generation

The carry output uses optimized logic:

```
assign ow_carry = (i_a & i_b) | (i_c & (i_a ^ i_b));
```

This can be broken down as: - $(i_a \& i_b)$: Carry generated when both primary inputs are 1 - $(i_c \& (i_a \wedge i_b))$: Carry propagated when exactly one primary input is 1 and carry-in is 1

5.7 Logic Gate Implementation



Traditional Gate-Level View

5.7.1 Optimized Implementation

The actual implementation uses shared XOR logic for efficiency:

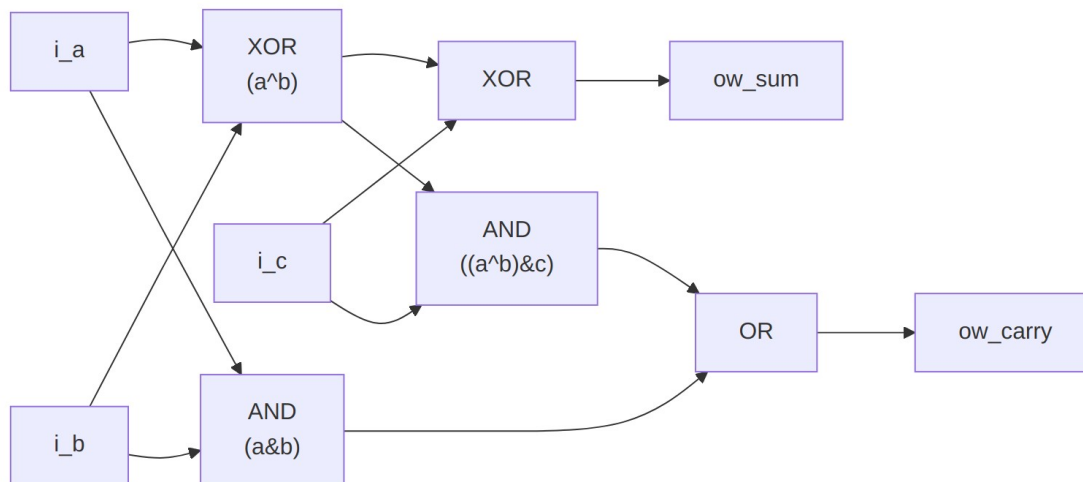


Diagram 2

Key optimization: The $a \oplus b$ XOR result is shared between sum calculation and carry propagation.

5.8 Timing Characteristics

Characteristic	Typical Value	Description
Propagation Delay (Sum)	$2 \times t_{\text{XOR}}$	Through 2 XOR gates
Propagation Delay (Carry)	$t_{\text{AND}} + t_{\text{OR}}$	Through AND-OR path
Setup Time	0	Purely combinational
Hold Time	0	Purely combinational

5.9 Usage Examples

5.9.1 Basic Full Adder

```
logic a_bit, b_bit, cin;  
logic sum_bit, cout;
```

```

math_adder_full u_full_adder (
    .i_a      (a_bit),
    .i_b      (b_bit),
    .i_c      (cin),
    .ow_sum   (sum_bit),
    .ow_carry (cout)
);

```

5.9.2 Building a 4-Bit Ripple Carry Adder

```

logic [3:0] a, b, sum;
logic cin, cout;
logic [3:0] carry_chain;

```

// Bit 0

```

math_adder_full u_add0 (
    .i_a      (a[0]),
    .i_b      (b[0]),
    .i_c      (cin),
    .ow_sum   (sum[0]),
    .ow_carry (carry_chain[0])
);

```

// Bit 1

```

math_adder_full u_add1 (
    .i_a      (a[1]),
    .i_b      (b[1]),
    .i_c      (carry_chain[0]),
    .ow_sum   (sum[1]),
    .ow_carry (carry_chain[1])
);

```

// Bit 2

```

math_adder_full u_add2 (
    .i_a      (a[2]),
    .i_b      (b[2]),
    .i_c      (carry_chain[1]),
    .ow_sum   (sum[2]),
    .ow_carry (carry_chain[2])
);

```

// Bit 3

```

math_adder_full u_add3 (
    .i_a      (a[3]),
    .i_b      (b[3]),
    .i_c      (carry_chain[2]),
    .ow_sum   (sum[3]),

```



```
        .ow_carry (cout)
    );
```

5.9.3 Part of a Carry-Save Adder

```
// In a 3:2 carry-save adder stage
math_adder_full u_csa_stage (
    .i_a      (partial_sum1[i]),
    .i_b      (partial_sum2[i]),
    .i_c      (partial_sum3[i]),
    .ow_sum   (sum_vector[i]),
    .ow_carry (carry_vector[i+1])
);
```

5.10 Design Considerations

5.10.1 Advantages

- **Simplicity:** Minimal gate count and complexity
- **Modularity:** Perfect building block for larger arithmetic units
- **Predictable:** Well-defined timing and behavior
- **Efficient:** Optimized carry generation logic

5.10.2 Performance Characteristics

- **Area:** 5 logic gates (2 XOR, 2 AND, 1 OR)
- **Power:** Low static power, dynamic power proportional to switching activity
- **Speed:** Limited by XOR gate delays (typically slower than AND/OR)

5.11 Synthesis Considerations

5.11.1 Technology Mapping

Most synthesis tools will: - Map XOR gates to efficient library cells - Optimize the carry logic for the target technology - May use dedicated adder primitives in some technologies

5.11.2 Optimization Notes

```
// Alternative carry implementation (equivalent but different structure)
assign ow_carry = (i_a & i_b) | (i_a & i_c) | (i_b & i_c);
```

This alternative has higher gate count but may have different timing characteristics.

5.12 Verification Examples

5.12.1 Testbench Structure

```
module tb_math_adder_full;
    logic i_a, i_b, i_c;
    logic ow_sum, ow_carry;
    logic [1:0] expected_result;

    math_adder_full dut (.*);

    initial begin
        // Test all combinations
        for (int i = 0; i < 8; i++) begin
            {i_a, i_b, i_c} = i;
            #1;
            expected_result = i_a + i_b + i_c;

            assert ({ow_carry, ow_sum} == expected_result)
                else $error("Mismatch: %b + %b + %b = %b, expected %b",
                    i_a, i_b, i_c, {ow_carry, ow_sum},
expected_result);
        end
        $display("All tests passed!");
    end
endmodule
```

5.13 Related Modules

- `math_adder_half`: Half adder (2 inputs, no carry-in)
- `math_adder_full_nbit`: N-bit full adder using ripple carry
- `math_adder_ripple_carry`: Multi-bit ripple carry adder
- `math_adder_carry_save`: Carry-save adder for multiple operand addition

5.14 Applications

- **Multi-bit Adders**: Building block for ripple carry adders
- **Carry-Save Adders**: Used in parallel multiplication
- **ALU Design**: Fundamental component in arithmetic logic units
- **Accumulator Circuits**: Used in digital signal processing

The `math_adder_full` module provides the essential functionality for binary addition and serves as a critical building block in digital arithmetic circuits.

5.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

6 Han-Carlson Parallel Prefix Adder

A hybrid parallel prefix adder that combines the low wiring complexity of Brent-Kung with near-Kogge-Stone speed, providing an optimal balance of area, wire routing, and logic depth for advanced process nodes.

6.1 Overview

The `math_adder_han_carlson` module family implements Han-Carlson parallel prefix adders. Han-Carlson is a “sparsity-2” architecture that computes prefix operations only on even bit positions in intermediate stages, then fills in odd positions in a final stage using area-efficient gray cells.

Available widths: 16-bit, 48-bit (auto-generated for BF16 arithmetic)

Key Features: - **$O(\log N + 1)$ depth** - Only one level more than Kogge-Stone - **~40% fewer cells** than Kogge-Stone for same width - **Constant fanout of 2** - Better for advanced process nodes - **Reduced wiring** - 4 tracks vs 8 for Kogge-Stone (16-bit) - **Optimal for multiplier final CPA and FMA wide addition**

6.2 Module Hierarchy

Top-Level Modules: - `math_adder_han_carlson_016.sv` - 16-bit adder (for BF16 multiplier CPA) - `math_adder_han_carlson_048.sv` - 48-bit adder (for BF16 FMA wide addition)

Building Blocks: - `math_prefix_cell.sv` - Black cell (outputs G and P) - `math_prefix_cell_gray.sv` - Gray cell (outputs G only)

6.3 Module Declaration

6.3.1 16-bit Han-Carlson Adder

```
module math_adder_han_carlson_016 #(
    parameter int N = 16
) (
    input  logic [N-1:0] i_a,
    input  logic [N-1:0] i_b,
    input  logic          i_cin,
    output logic [N-1:0] ow_sum,
    output logic          ow_cout
);
```

6.3.2 48-bit Han-Carlson Adder

```
module math_adder_han_carlson_048 #(
    parameter int N = 48
) (
    input logic [N-1:0] i_a,
    input logic [N-1:0] i_b,
    input logic i_cin,
    output logic [N-1:0] ow_sum,
    output logic ow_cout
);
```

6.4 Parameters

Parameter	Type	Default	Description
N	int	16/48	Adder width (fixed per variant)

Note: The N parameter is fixed per module variant. Use the appropriate variant for your width.

6.5 Ports

Port	Direction	Width	Description
i_a	Input	N	First operand
i_b	Input	N	Second operand
i_cin	Input	1	Carry input
ow_sum	Output	N	Sum result (A + B + Cin)
ow_cout	Output	1	Carry output

6.6 Algorithm

6.6.1 Han-Carlson Structure

Han-Carlson uses a “sparsity-2” approach:

1. **Stage 0:** Compute initial P and G for all positions
2. **Stage 1:** Combine adjacent pairs on even positions only
3. **Stages 2-N:** Kogge-Stone style prefix on even positions (doubling span)
4. **Final Stage:** Fill odd positions from even neighbors using gray cells

6.6.2 Stage-by-Stage Breakdown (16-bit)

Stage 0: Initial P/G computation

$P[i] = A[i] \text{ XOR } B[i]$

$G[i] = A[i] \text{ AND } B[i]$

Stage 1: Span-2 prefix on even positions

Position 0: $G[0:-1]$ (incorporate cin)

Even positions: $G[i:i-1] = \text{combine}(G[i], P[i], G[i-1], P[i-1])$

Odd positions: pass through

Stage 2: Span-4 prefix on even positions (step 2)

Even positions ≥ 2 : $G[i:i-3] = \text{combine}(G[i:i-1], G[i-2:i-3])$

Others: pass through

Stage 3: Span-8 prefix on even positions (step 4)

Even positions ≥ 4 : $G[i:i-7] = \text{combine}(G[i:i-3], G[i-4:i-7])$

Others: pass through

Stage 4: Span-16 prefix on even positions (step 8)

Even positions ≥ 8 : $G[i:i-15] = \text{combine}(\dots)$

Others: pass through

Stage 5: Fill odd positions (gray cells)

Odd positions: $G[i:-1] = \text{combine}(G[i], P[i], G[i-1:-1])$

Even positions: pass through

Sum computation:

$\text{Sum}[0] = P[0] \text{ XOR } \text{Cin}$

$\text{Sum}[i] = P[i] \text{ XOR } G[i-1:-1]$

6.6.3 Prefix Network Visualization (16-bit simplified)

Position:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-----------	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

Stage 0:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
----------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Initial P/G generation

Stage 1:	*		*		*		*		*		*		*		*	
----------	---	--	---	--	---	--	---	--	---	--	---	--	---	--	---	--

Even positions combine with odd neighbor (*)

Odd positions pass through (|)

Stage 2:	*				*				*				*			
----------	---	--	--	--	---	--	--	--	---	--	--	--	---	--	--	--

Even positions combine with step=2 (positions 2,4,6,8,10,12,14)

Stage 3: * | | | | | | | * | | | | | | |
|

Even positions combine with step=4 (positions 4,8,12)

Stage 4: * | | | | | | | | | | | | | | | |
|

Even positions combine with step=8 (positions 8)

Stage 5: o * o * o * o * o * o * o * o * o *
*

Fill odd positions with gray cells (*)

Legend: o=pass through, |=pass through, *=prefix cell

6.6.4 Why Han-Carlson is Optimal

Metric	Kogge-Stone	Brent-Kung	Han-Carlson
Logic Depth (16-bit)	4	7	5
Prefix Cells (16-bit)	64	~30	~39
Wiring Tracks	8	2	4
Fanout	Varies	2	2
Best For	Max speed	Min area	Balanced

Han-Carlson achieves: - 1 level slower than Kogge-Stone (5 vs 4) - 40% fewer cells than Kogge-Stone (39 vs 64) - 2x fewer wiring tracks than Kogge-Stone (4 vs 8) - Constant fanout of 2 (important for advanced nodes)

6.7 Timing Characteristics

6.7.1 16-bit Adder

Metric	Value
Logic Levels	5 stages + sum XOR
Prefix Cells	~39 (black + gray)
Critical Path	cin -> Stage 1 -> Stage 5 -> sum[15]

6.7.2 48-bit Adder

Metric	Value
Logic Levels	7 stages + sum XOR
Prefix Cells	~120 (black + gray)
Critical Path	cin -> Stage 1 -> Stage 7 -> sum[47]

6.7.3 Depth Formula

For N-bit Han-Carlson: - **Prefix-tree depth** = $\text{ceil}(\log_2(N)) + 1$ stages - **Total depth** = prefix tree + 1 sum stage = $\text{ceil}(\log_2(N)) + 2$ stages - **16-bit**: prefix = $4 + 1 = 5$, total = 6 stages (5 prefix + 1 sum) - **48-bit**: prefix = $6 + 1 = 7$, total = 8 stages (7 prefix + 1 sum)

The +2 is **total** depth, not prefix depth – the sparsity-2 Han-Carlson prefix network is $\log_2(N) + 1$ deep, and the final sum stage adds one more. Quoting +2 without saying which is being counted invites an off-by-one when comparing against the Harris taxonomy, which tabulates prefix depth.

6.8 Usage Examples

6.8.1 Basic 16-bit Addition

```
logic [15:0] a, b, sum;
logic cout;
```

```
math_adder_han_carlson_016 u_add (
    .i_a(a),
    .i_b(b),
    .i_cin(1'b0),
    .ow_sum(sum),
    .ow_cout(cout)
);
```

6.8.2 Subtraction (A - B)

```
logic [15:0] a, b, diff;
logic borrow;
```

```
// A - B = A + (~B) + 1
math_adder_han_carlson_016 u_sub (
    .i_a(a),
    .i_b(~b),
    .i_cin(1'b1), // +1 for two's complement
    .ow_sum(diff),
    .ow_cout(borrow)
);
```

6.8.3 In BF16 Multiplier (Final CPA)

```
// Final carry-propagate addition after Dadda reduction
wire [15:0] w_cpa_a, w_cpa_b; // From Dadda tree
wire [15:0] w_product;
wire      w_cout;

math_adder_han_carlson_016 u_final_cpa (
    .i_a(w_cpa_a),
    .i_b(w_cpa_b),
    .i_cin(1'b0),
    .ow_sum(w_product),
    .ow_cout(w_cout) // Unused for unsigned multiply
);
```

6.8.4 In BF16 FMA (Wide Addition)

```
// 48-bit mantissa addition in FMA
wire [47:0] w_mant_larger, w_mant_smaller_aligned;
wire [47:0] w_sum;
wire      w_cout;

// For subtraction, invert smaller operand
wire [47:0] w_adder_b = w_effective_sub ? ~w_mant_smaller_aligned
                                         : w_mant_smaller_aligned;

math_adder_han_carlson_048 u_wide_add (
    .i_a(w_mant_larger),
    .i_b(w_adder_b),
    .i_cin(w_effective_sub), // +1 for two's complement
    .ow_sum(w_sum),
    .ow_cout(w_cout)
);
```

6.9 Performance Characteristics

6.9.1 Resource Utilization

Width	Black Cells	Gray Cells	Total Cells	LUTs (Est.)
16-bit	~31	8	~39	~80
48-bit	~96	24	~120	~250

6.9.2 Comparison with Other Architectures

Architecture	16-bit Depth	16-bit Cells	Wiring Complexity
Ripple Carry	16	16	Minimal

Architecture	16-bit Depth	16-bit Cells	Wiring Complexity
Brent-Kung	7	~30	Low
Han-Carlson	5	~39	Medium
Kogge-Stone	4	64	High

6.10 Design Considerations

6.10.1 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Fewer cells than Kogge-Stone via sparsity-2 structure 2. **Wire complexity** - Reduced wiring tracks and constant fanout 3. **Logic depth** - Near-optimal $O(\log N + 1)$ depth

6.10.2 When to Use Han-Carlson

Appropriate Use Cases: - Multiplier final CPA (balanced speed/area) - FMA wide addition (48+ bits) - Designs targeting advanced process nodes where wire delay matters - Area-constrained high-speed arithmetic

Consider Alternatives When: - Absolute minimum depth needed -> Kogge-Stone - Absolute minimum area needed -> Brent-Kung or Ripple Carry - Width not power-of-2 -> Consider behavioral or Ripple Carry

6.10.3 Auto-Generated Code

These modules are auto-generated by Python scripts: - **Generator:**

```
bin/rtl_generators/bf16/han_carlson_adder.py - Regenerate: PYTHONPATH=bin:
$PYTHONPATH python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv files manually. Modify the generator script instead.

6.11 Common Pitfalls

Anti-Pattern 1: Expecting parameterizable width

```
// WRONG: N is fixed per variant
math_adder_han_carlson_016 #(.N(24)) u_add (...); // Won't work!
```

```
// RIGHT: Use appropriate variant or extend inputs
math_adder_han_carlson_048 u_add (
    .i_a({24'b0, a[23:0]}), // Zero-extend
    .i_b({24'b0, b[23:0]}),
    ...
);
```

Anti-Pattern 2: Ignoring that outputs are combinational

```
// Remember: outputs are purely combinational
// Add pipeline registers if needed for timing closure
always_ff @(posedge clk) begin
    sum_reg <= ow_sum; // Register the output
end
```

6.12 Related Modules

- **math_prefix_cell** - Black cell building block
- **math_prefix_cell_gray** - Gray cell building block
- **math_multiplier_dadda_4to2_008** - Uses 16-bit Han-Carlson for final CPA
- **math_bf16_fma** - Uses 48-bit Han-Carlson for wide addition
- **math_adder_brent_kung_nbit** - Alternative area-optimized adder

6.13 References

- Han, T., Carlson, D.A. “Fast Area-Efficient VLSI Adders.” IEEE Symposium on Computer Arithmetic, 1987.
- Harris, D. “A Taxonomy of Parallel Prefix Networks.” Asilomar Conference, 2003.
- Knowles, S. “A Family of Adders.” IEEE Symposium on Computer Arithmetic, 1999.

6.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

7 Kogge-Stone Parallel Prefix Adder

Naming caveat – read this before selecting this module. Despite the name, adder. It uses generate/propagate logic with a **sequential carry chain**, so its depth is **$O(N)$** , not the **$O(\log N)$** a Kogge-Stone prefix tree gives. If you selected this expecting parallel-prefix timing, you will not get it – see `math_adder_brent_kung` or `math_adder_han_carlson` for genuine parallel-prefix implementations.

A parameterized adder using Kogge-Stone inspired propagate-generate logic for arbitrary bit widths, providing improved carry computation over ripple carry adders.

7.1 Overview

Note: This is a simplified Kogge-Stone inspired implementation. A full Kogge-Stone parallel prefix adder would use a complete binary tree of black cells achieving $O(\log N)$ depth, but at the cost of significantly more hardware and complexity for generic parameterization.

7.2 Module Declaration

```
parameter int N = 4          // Adder width in bits (any width ≥ 1)
) (
  input logic [N-1:0] i_a,    // Operand A
  input logic [N-1:0] i_b,    // Operand B
  output logic [N-1:0] ow_sum, // Sum output
  output logic          ow_carry // Carry output
);
```

Note: No carry input (i_c) - design assumes $C_{in} = 0$. For +1 increment, add external increment logic or use dedicated incrementer module.

7.3 Parameters

7.3.1 User-Settable Parameters

Parameter	Type	Default	Description
N	int	4	Adder width in bits (range: 1-64, typical: 4-64)

Width Guidelines: - **Minimum:** 1 bit (degenerates to half adder) - **Typical:** 4-64 bits (arbitrary widths supported) - **Maximum:** 64 bits (practical synthesis limit)

7.4 Ports

7.4.1 Inputs

Port	Width	Description
i_a	N	Operand A (addend)
i_b	N	Operand B (addend)

7.4.2 Outputs

Port	Width	Description
ow_sum	N	Sum output (A + B)

Port	Width	Description
ow_carry	1	Carry output (overflow)

Important: No carry input port. This adder always computes $A + B$ (assumes $C_{in} = 0$).

7.5 Functionality

7.5.1 Algorithm Overview

The adder uses a three-stage process inspired by parallel prefix addition principles:

7.5.1.1 Stage 1: Generate and Propagate Terms

For each **bit** i :

$G[i] = A[i] \& B[i]$ // Generate: carry is generated at this position

$P[i] = A[i] \mid B[i]$ // Propagate: carry can propagate through this position

Design Note: Uses OR for propagate (not XOR). This is mathematically equivalent for carry computation: - $P[i] = A[i] \mid B[i]$ allows carry propagation when at least one input is 1 - Traditional $P[i] = A[i] \wedge B[i]$ (XOR) used in some other adders

7.5.1.2 Stage 2: Carry Computation

$C[0] = G[0]$ // Initial carry (from LSB)

For $i = 1$ to $N-1$:

$C[i] = G[i] \mid (P[i] \& C[i-1])$

Interpretation: - Carry at position i is generated locally ($G[i]$), OR - Carry propagates from previous position ($P[i] \& C[i-1]$)

7.5.1.3 Stage 3: Sum Calculation

$Sum[0] = A[0] \wedge B[0]$ // LSB has no carry input

For $i = 1$ to $N-1$:

$Sum[i] = A[i] \wedge B[i] \wedge C[i-1]$

7.5.2 Implementation Details

// Stage 1: Generate G and P signals (parallel, 1 level)

for ($i = 0$; $i < N$; $i++$) **begin**

$w_G[i] = i_a[i] \& i_b[i]$; // Generate

$w_P[i] = i_a[i] \mid i_b[i]$; // Propagate

end

```

// Stage 2: Carry computation (sequential, N levels)
always_comb begin
    logic carry;
    carry = w_G[0];
    w_C[0] = carry;
    for (int j = 1; j < N; j++) begin
        carry = w_G[j] | (w_P[j] & carry);
        w_C[j] = carry;
    end
end

// Stage 3: Sum calculation (parallel, 1 level)
ow_sum[0] = i_a[0] ^ i_b[0]; // No carry for LSB
for (i = 1; i < N; i++) begin
    ow_sum[i] = i_a[i] ^ i_b[i] ^ w_C[i-1];
end

// Final carry out
ow_carry = w_C[N-1];

```

7.5.3 Timing Analysis

Logic Depth: - **G/P generation:** 1 level (AND/OR) - **Carry chain:** N levels (sequential dependency) - **Sum calculation:** 1 level (XOR) - **Total:** N + 2 levels

Critical Path:

$i_a[0]/i_b[0] \rightarrow G[0] \rightarrow C[0] \rightarrow C[1] \rightarrow \dots \rightarrow C[N-1] \rightarrow ow_carry$

7.5.4 Comparison to Other Adders

Adder Type	Logic Depth	Area	Carry Input	Generic Width
Ripple Carry	O(N)	Minimal	Yes	Yes
Carry Lookahead	O(N)	Small	Yes	Yes
Kogge-Stone (this)	O(N)	Small	No	Yes
Brent-Kung	O(log N)	Medium	Yes	No (8/16/32 only)
True Kogge-	O(log N)	Large	Yes	No (fixed widths)

Adder Type	Logic Depth	Area	Carry Input	Generic Width
Stone				

Key Differences: - **vs Ripple Carry:** Similar depth, but explicit P/G logic may optimize better - **vs True Kogge-Stone:** This is $O(N)$ not $O(\log N)$; true KS requires full prefix tree - **vs Brent-Kung:** Simpler, supports arbitrary widths, but slower for $N > 16$

7.6 Usage Examples

7.6.1 Basic 16-bit Addition

```
logic [15:0] a, b, sum;
logic carry_out;
```

```
    .N(16)
) u_adder (
    .i_a      (a),
    .i_b      (b),
    .ow_sum   (sum),
    .ow_carry (carry_out)
);
```

// Example: 1000 + 2000

```
initial begin
    a = 16'd1000;
    b = 16'd2000;
    #1; // Wait for combinational propagation
    assert(sum == 16'd3000);
    assert(carry_out == 1'b0);
end
```

7.6.2 Adding with Carry Input (External Logic)

// Since module has no carry input, add external logic

```
logic [7:0] a, b, sum_with_cin;
logic cin, carry_out;
```

```
logic [7:0] b_adjusted;
assign b_adjusted = b + {7'b0, cin}; // Add carry to B operand
```

```
    .N(8)
) u_adder (
    .i_a      (a),
    .i_b      (b_adjusted),
    .ow_sum   (sum_with_cin),
    .ow_carry (carry_out)
```

```

);

// Alternatively, use dedicated incremter after addition
logic [7:0] sum_base;
    .i_a(a), .i_b(b), .ow_sum(sum_base), ...
);
assign sum_with_cin = cin ? (sum_base + 8'b1) : sum_base;

```

7.6.3 Arbitrary Width Addition (37-bit example)

// Advantage: Supports non-power-of-2 widths

```

logic [36:0] a_37, b_37, sum_37;
logic carry_37;

    .N(37) // Any width supported
) u_adder_37bit (
    .i_a      (a_37),
    .i_b      (b_37),
    .ow_sum    (sum_37),
    .ow_carry  (carry_37)
);

```

7.6.4 Multi-Precision Addition (64-bit via 2×32-bit)

```

logic [31:0] a_low, a_high, b_low, b_high;
logic [31:0] sum_low, sum_high;
logic carry_low;

// Low 32 bits
    .i_a      (a_low),
    .i_b      (b_low),
    .ow_sum    (sum_low),
    .ow_carry  (carry_low)
);

// High 32 bits (add carry_low to b_high)
logic [31:0] b_high_adjusted;
assign b_high_adjusted = b_high + {31'b0, carry_low};

    .i_a      (a_high),
    .i_b      (b_high_adjusted),
    .ow_sum    (sum_high),
    .ow_carry  ()
);

logic [63:0] sum_64 = {sum_high, sum_low};

```

7.6.5 Accumulator

```
logic [31:0] accumulator, data_in;
logic [31:0] accumulator_next;
logic acc_enable;

    .i_a      (accumulator),
    .i_b      (data_in),
    .ow_sum   (accumulator_next),
    .ow_carry ()
);

always_ff @(posedge clk) begin
    if (rst_n == 0) begin
        accumulator <= 32'b0;
    end else if (acc_enable) begin
        accumulator <= accumulator_next;
    end
end
```

7.6.6 Adder Array (Multiple Parallel Adders)

```
// 8 parallel 16-bit adders
localparam int NUM_ADDERS = 8;
logic [15:0] a_array [NUM_ADDERS];
logic [15:0] b_array [NUM_ADDERS];
logic [15:0] sum_array [NUM_ADDERS];

genvar k;
generate
    for (k = 0; k < NUM_ADDERS; k++) begin : gen_adder_array
        .N(16)
        ) u_adder (
            .i_a      (a_array[k]),
            .i_b      (b_array[k]),
            .ow_sum    (sum_array[k]),
            .ow_carry ()
        );
    end
endgenerate
```

7.7 Timing Characteristics

7.7.1 Combinational Delay Analysis

Width	Logic Levels	Typical Delay (ns) @ 1.0V	Max Frequency
4-bit	6	~1.2	~800 MHz

Width	Logic Levels	Typical Delay (ns) @ 1.0V	Max Frequency
8-bit	10	~2.0	~500 MHz
16-bit	18	~3.5	~285 MHz
32-bit	34	~6.5	~155 MHz
64-bit	66	~12.5	~80 MHz

Logic Level Breakdown (16-bit example): 1. G/P generation: 1 level (AND/OR) 2. Carry chain: 16 levels ($C[0] \rightarrow C[1] \rightarrow \dots \rightarrow C[15]$) 3. Sum calculation: 1 level (XOR) 4. **Total:** 18 levels

7.7.2 Critical Paths

Longest Path (Carry Out):

$i_a[0]/i_b[0] \rightarrow G[0] \rightarrow C[0] \rightarrow C[1] \rightarrow \dots \rightarrow C[N-1] \rightarrow ow_carry$

Sum Output Path:

$i_a[i]/i_b[i] \rightarrow P[i] \rightarrow ow_sum[i]$ (depends on $C[i-1]$)

7.7.3 Performance vs Width

Width	Relative Delay	Relative Frequency
4-bit	1.0×	1.00×
8-bit	1.7×	0.60×
16-bit	3.0×	0.35×
32-bit	5.7×	0.19×
64-bit	11.0×	0.10×

Observation: Delay scales linearly with width ($O(N)$).

7.8 Performance Characteristics

7.8.1 Resource Utilization

Width	LUTs (Est.)	FFs (Pipeline)	Description
8-bit	~24	0 (combinational)	Small
16-bit	~48	0 (combinational)	Medium

Width	LUTs (Est.)	FFs (Pipeline)	Description
32-bit	~96	0 (combinational)	Large
64-bit	~192	0 (combinational)	Very large

Area Breakdown (16-bit): - G/P generation: 16×2 gates = 32 LUTs - Carry chain: 16×2 gates = 32 LUTs (OR + AND) - Sum calculation: 16×1 gate = 16 LUTs (XOR) - **Total:** ~48 LUTs

7.8.2 Comparison to Alternatives

Adder Architecture	Relative Speed	Relative Area	Generic Width
Ripple Carry	1.0×	1.0×	Yes
Kogge-Stone (this)	1.5×	1.2×	Yes
Carry Lookahead	1.5×	1.2×	Yes
Brent-Kung	4.0×	4.0×	No (8/16/32 only)
True Kogge-Stone	6.0×	6.0×	No (fixed widths)

7.9 Design Considerations

7.9.1 When to Use This Adder

✓ **Appropriate Use Cases:** - Arbitrary bit widths (non-power-of-2) - Designs requiring parameterizable width - No carry input required - Moderate speed requirements - Area-constrained designs

7.9.2 When to Use Alternatives

Use Brent-Kung if: - Width is 8, 16, or 32 bits - Need maximum speed ($O(\log N)$ depth) - Can afford larger area

Use Carry Lookahead if: - Need carry input support - Width ≤ 16 bits - Similar performance to this module

Use Ripple Carry if: - Width ≤ 4 bits - Minimal area is critical - Speed not important

7.9.3 Lack of Carry Input Workarounds

Since this module has no carry input port, consider these alternatives:

Option 1: Pre-add carry to operand

```
logic [N-1:0] b_adjusted;  
assign b_adjusted = b + {{N-1{1'b0}}, cin};
```

Option 2: Use different adder with carry input

```
// Use carry lookahead instead  
math_adder_pg_chain #(.N(N)) u_add (  
    .i_a(a), .i_b(b), .i_c(cin), ...  
);
```

Option 3: Post-increment sum

```
assign sum_with_cin = cin ? (ow_sum + 1'b1) : ow_sum;
```

7.9.4 Synthesis Considerations

Optimization Tips:

1. Flatten hierarchy:

```
set_flatten true -effort high
```

2. Allow carry chain optimization:

```
set_dont_touch false
```

3. Consider pipelining for $N > 16$:

```
// Break carry chain in middle  
logic [N/2-1:0] r_c_mid;  
always_ff @(posedge clk) r_c_mid <= w_C[N/2-1:0];
```

7.10 Verification Strategy

Key Test Scenarios: - Random stimulus (various widths) - Corner cases: $0+0$, $0+MAX$, $MAX+MAX$ - Full carry propagation: $0xFFFF...FF + 1$ - Arbitrary widths: 7-bit, 13-bit, 37-bit - Overflow detection

Test Command:

7.11 Common Pitfalls

✗ Anti-Pattern 1: Expecting carry input port

```

// WRONG: No carry input port exists!
.i_a(a), .i_b(b), .i_c(cin), // ERROR: No i_c port!
...
);

// RIGHT: Pre-add carry to operand or use different adder
logic [7:0] b_adj = b + {7'b0, cin};
.i_a(a), .i_b(b_adj), ...
);

x Anti-Pattern 2: Using for very wide additions

// WRONG: 128-bit with  $O(N)$  depth (slow!)
// Result: 130 logic levels!

// RIGHT: Break into smaller pieces or use different architecture

x Anti-Pattern 3: Assuming parallel prefix performance

// WRONG ASSUMPTION: "Kogge-Stone is  $O(\log N)$ "
// This implementation is  $O(N)$ , not true parallel prefix

// For true  $O(\log N)$ , use Brent-Kung (if width matches):
math_adder_brent_kung_032 u_fast_add (...);

x Anti-Pattern 4: Chaining without pipelining

// WRONG: Long combinational chain
logic [31:0] sum1, sum2, sum3;
// 3 × 34 levels = 102 logic levels!

// RIGHT: Pipeline between adders
always_ff @(posedge clk) begin
    r_sum1 <= sum1;
    r_sum2 <= sum2;
end

```

7.12 Related Modules

- ****math_adder_brent_kung*.sv**** - True parallel prefix, $O(\log N)$ depth (8/16/32-bit)
- **math_adder_pg_chain.sv** - Similar performance, has carry input
- **math_adder_ripple_carry.sv** - Simpler, slightly smaller area
- **math_adder_carry_save_nbit.sv** - For multi-operand addition

7.13 References

- Kogge, P.M., Stone, H.S. “A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations.” IEEE Transactions on Computers, 1973.
- Harris, D. “A Taxonomy of Parallel Prefix Networks.” Asilomar Conference, 2003.
- Deschamps, J.P., Bioul, G., Sutter, G. “Synthesis of Arithmetic Circuits.” Wiley, 2006.

7.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

8 Carry Lookahead Adder

Naming caveat – read this before selecting this module. This is **not** a true Weinberger & Smith carry-lookahead adder. A real CLA uses recursive group generate/propagate to reach **$O(\log N)$** depth; this implementation is a simplified P/G carry chain optimised for area and simplicity, with **$O(N)$** depth. For genuine logarithmic-depth adders use `math_adder_brent_kung` or `math_adder_han_carlson`.

A parameterized carry lookahead adder supporting arbitrary bit widths with $O(N)$ depth using generate and propagate logic to reduce carry propagation delay compared to ripple carry adders.

8.1 Overview

The `math_adder_pg_chain` module implements a carry lookahead adder (CLA) with parameterizable width. While not a true parallel-prefix lookahead (which would have $O(\log N)$ depth), this implementation uses propagate (P) and generate (G) signals to compute carries more efficiently than a simple ripple-carry adder. It provides a good balance between speed and area for small to medium width additions (4-16 bits).

8.2 Module Declaration

```
module math_adder_pg_chain #(
    parameter int N = 4          // Adder width in bits (any width ≥ 1)
) (
    input logic [N-1:0] i_a,      // Operand A
    input logic [N-1:0] i_b,      // Operand B
    input logic          i_c,      // Carry input

```

```

        output logic [N-1:0] ow_sum,    // Sum output
        output logic          ow_carry // Carry output
    );

```

8.3 Parameters

8.3.1 User-Settable Parameters

Parameter	Type	Default	Description
N	int	4	Adder width in bits (range: 1-64, typical: 4-16)

Width Guidelines: - **Typical:** 4, 8, 16 bits (good speed/area balance) - **Minimum:** 1 bit (degenerates to half adder) - **Maximum:** 64 bits (practical synthesis limit, but consider parallel prefix for >16 bits)

8.4 Ports

8.4.1 Inputs

Port	Width	Description
i_a	N	Operand A (addend)
i_b	N	Operand B (addend)
i_c	1	Carry input (0 for addition, 1 for +1 increment)

8.4.2 Outputs

Port	Width	Description
ow_sum	N	Sum output (A + B + Cin)
ow_carry	1	Carry output (overflow)

8.5 Functionality

8.5.1 Algorithm Overview

The carry lookahead adder uses **propagate (P)** and **generate (G)** signals to compute carries more efficiently than ripple carry:

8.5.1.1 Bit-Level Definitions

For each bit position i :

Generate: $G[i] = A[i] \& B[i]$ (carry is generated at this position)
Propagate: $P[i] = A[i] \wedge B[i]$ (carry is propagated through this position)

8.5.1.2 Carry Calculation

Initial carry:

$C[0] = C_{in}$ (input carry)

Carry recurrence (for $i = 1$ to N):

$C[i] = G[i-1] \mid (P[i-1] \& C[i-1])$

Interpretation: - Carry is generated ($G[i-1] = 1$) at position $i-1$, OR - Carry is propagated ($P[i-1] = 1$) from position $i-1$ if there was an incoming carry ($C[i-1] = 1$)

8.5.1.3 Sum Calculation

$Sum[i] = P[i] \wedge C[i]$

Sum is XOR of propagate signal and incoming carry.

8.5.2 Implementation

// Stage 1: Generate P and G signals (parallel, 1 level of logic)

```
for (i = 0; i < N; i++) begin
    w_g[i] = i_a[i] & i_b[i]; // Generate
    w_p[i] = i_a[i] ^ i_b[i]; // Propagate
end
```

// Stage 2: Calculate carries (sequential dependency, N levels)

```
w_c[0] = i_c;
for (i = 1; i <= N; i++) begin
    w_c[i] = w_g[i-1] | (w_p[i-1] & w_c[i-1]);
end
```

// Stage 3: Calculate sum (parallel, 1 level of logic)

```
for (i = 0; i < N; i++) begin
    ow_sum[i] = w_p[i] ^ w_c[i];
end
```

end

```
// Final carry out
ow_carry = w_c[N];
```

8.5.3 Timing Analysis

Logic Depth: - **P/G generation:** 1 level (AND/XOR) - **Carry chain:** N levels (each carry depends on previous) - **Sum calculation:** 1 level (XOR) - **Total:** N + 2 levels

Critical Path:

$i_a/i_b \rightarrow P[0]/G[0] \rightarrow C[1] \rightarrow C[2] \rightarrow \dots \rightarrow C[N-1] \rightarrow C[N] \rightarrow \text{Sum}[N-1]$

8.5.4 Comparison to Other Adders

Adder Type	Logic Depth	Area	Best Use Case
Ripple Carry	$O(N)$	Minimal	$N \leq 4$, area-critical
CLA (this)	$O(N)$	Small	$4 \leq N \leq 16$, balanced
Brent-Kung	$O(\log N)$	Medium	$N \geq 16$, speed-critical
Kogge-Stone	$O(\log N)$	Large	$N \geq 32$, max speed

Note: True carry lookahead adders (with group generate/propagate) can achieve $O(\log N)$ depth but require more complex logic. This implementation is a hybrid approach optimized for simplicity and area.

8.6 Usage Examples

8.6.1 Basic 8-bit Addition

```
logic [7:0] a, b, sum;
logic carry_out;
```

```
math_adder_pg_chain #(
    .N(8)
) u_adder (
    .i_a      (a),
    .i_b      (b),
    .i_c      (1'b0),           // No carry input
    .ow_sum    (sum),
    .ow_carry  (carry_out)
```



```

);

// Example: 200 + 100
initial begin
    a = 8'd200;
    b = 8'd100;
    #1; // Wait for combinational propagation
    assert(sum == 8'd44); // 300 % 256 = 44
    assert(carry_out == 1'b1); // Overflow
end

```

8.6.2 4-bit Incrementer

```

logic [3:0] count, count_plus_1;

math_adder_pg_chain #(
    .N(4)
) u_incrementer (
    .i_a      (count),
    .i_b      (4'b0), // B = 0
    .i_c      (1'b1), // Carry in = 1 → +1
    .ow_sum   (count_plus_1),
    .ow_carry () // Unused
);

```

8.6.3 16-bit ALU Component

```

typedef enum logic [2:0] {
    ALU_ADD  = 3'b000,
    ALU_SUB  = 3'b001,
    ALU_INC  = 3'b010,
    ALU_DEC  = 3'b011
} alu_op_t;

module simple_alu (
    input  logic [15:0] a,
    input  logic [15:0] b,
    input  alu_op_t    op,
    output logic [15:0] result,
    output logic      carry,
    output logic      zero
);

    logic [15:0] b_mux;
    logic      cin_mux;

    // Operation selection
    always_comb begin

```

```

    case (op)
      ALU_ADD: begin
        b_mux = b;
        cin_mux = 1'b0;
      end
      ALU_SUB: begin
        b_mux = ~b;           // Two's complement:  $A + (\sim B) + 1$ 
        cin_mux = 1'b1;
      end
      ALU_INC: begin
        b_mux = 16'b0;
        cin_mux = 1'b1;       //  $A + 0 + 1 = A + 1$ 
      end
      ALU_DEC: begin
        b_mux = 16'hFFFF;     // -1 in two's complement
        cin_mux = 1'b0;       //  $A + (-1) + 0 = A - 1$ 
      end
      default: begin
        b_mux = 16'b0;
        cin_mux = 1'b0;
      end
    endcase
  end

// Adder
math_adder_pg_chain #(
  .N(16)
) u_adder (
  .i_a      (a),
  .i_b      (b_mux),
  .i_c      (cin_mux),
  .ow_sum   (result),
  .ow_carry (carry)
);

// Zero flag
assign zero = ~|result;

```

endmodule

8.6.4 Multi-Precision Addition (32-bit via 2×16-bit)

```

logic [15:0] a_low, a_high, b_low, b_high;
logic [15:0] sum_low, sum_high;
logic carry_low, carry_high;

// Low 16 bits
math_adder_pg_chain #(.N(16)) u_add_low (

```

```

        .i_a      (a_low),
        .i_b      (b_low),
        .i_c      (1'b0),
        .ow_sum    (sum_low),
        .ow_carry  (carry_low)
    );

    // High 16 bits (chain carry from low)
    math_adder_pg_chain #(.N(16)) u_add_high (
        .i_a      (a_high),
        .i_b      (b_high),
        .i_c      (carry_low),      // Carry from low adder
        .ow_sum    (sum_high),
        .ow_carry  (carry_high)
    );

    // Concatenate results
    logic [31:0] sum_32 = {sum_high, sum_low};

```

8.6.5 Conditional Increment (For Loop Counters)

```

    logic [7:0] loop_counter;
    logic loop_active, loop_done;

    math_adder_pg_chain #(.N(8)) u_loop_inc (
        .i_a      (loop_counter),
        .i_b      (8'b0),
        .i_c      (loop_active),    // Increment only when active
        .ow_sum    (loop_counter_next),
        .ow_carry  ()
    );

    always_ff @(posedge clk) begin
        if (rst_n == 0) begin
            loop_counter <= 8'b0;
        end else if (loop_active) begin
            loop_counter <= loop_counter_next;
        end
    end

    assign loop_done = (loop_counter == 8'd99);

```

8.7 Timing Characteristics

8.7.1 Combinational Delay Analysis

Width	Logic Levels	Typical Delay (ns) @ 1.0V	Max Frequency
4-bit	6	~1.2	~800 MHz
8-bit	10	~2.0	~500 MHz
16-bit	18	~3.5	~285 MHz
32-bit	34	~6.5	~155 MHz

Logic Level Breakdown (8-bit example): 1. P/G generation: 1 level (AND/XOR) 2. Carry chain: 8 levels ($C[1] \rightarrow C[2] \rightarrow \dots \rightarrow C[8]$) 3. Sum calculation: 1 level (XOR) 4. **Total:** 10 levels

8.7.2 Critical Paths

Main Critical Path (Longest):

$i_a[0]/i_b[0] \rightarrow P[0]/G[0] \rightarrow C[1] \rightarrow C[2] \rightarrow \dots \rightarrow C[N] \rightarrow ow_carry$

Sum Output Path:

$i_a[i]/i_b[i] \rightarrow P[i] \rightarrow ow_sum[i]$ (depends on $C[i]$ from carry chain)

8.7.3 Performance vs Width

Width	Delay (relative)	Frequency (relative)
4-bit	1.0× (baseline)	1.00× (baseline)
8-bit	1.7×	0.60×
16-bit	3.0×	0.35×
32-bit	5.7×	0.19×

Observation: Delay scales linearly with width → Consider parallel prefix adders (Brent-Kung, Kogge-Stone) for $N > 16$.

8.8 Performance Characteristics

8.8.1 Resource Utilization

Width	LUTs (Est.)	FFs (Pipeline)	Description
4-bit	~12	0 (combinational)	Minimal

Width	LUTs (Est.)	FFs (Pipeline)	Description
8-bit	~24	0 (combinational)	Small
16-bit	~48	0 (combinational)	Medium
32-bit	~96	0 (combinational)	Large (consider alternatives)

Area Breakdown (8-bit): - P/G generation: 8×2 gates = 16 LUTs - Carry chain: 8×2 gates = 16 LUTs (OR + AND) - Sum calculation: 8×1 gate = 8 LUTs (XOR) - **Total:** ~24 LUTs (may optimize to fewer)

8.8.2 Speed vs Area Trade-offs

Adder Architecture	Relative Speed	Relative Area	Best Width Range
Ripple Carry	1.0× (slowest)	1.0× (smallest)	$N \leq 4$
CLA (this)	1.5×	1.2×	$4 \leq N \leq 16$
Brent-Kung	4.0×	4.0×	$16 \leq N \leq 32$
Kogge-Stone	6.0× (fastest)	6.0× (largest)	$N \geq 32$

8.9 Design Considerations

8.9.1 When to Use This Adder

✓ **Appropriate Use Cases:** - Small to medium width additions (4-16 bits) - Area-constrained designs with moderate speed requirements - ALU components in simple processors - Address arithmetic (incrementers, offsets) - Simple calculators or counters - FPGA designs with limited LUT budget

✗ **Consider Alternatives When:** - $N > 16$: Use Brent-Kung or Kogge-Stone for better speed - $N \leq 4$: Use ripple carry for minimal area - **Critical datapath:** Use parallel prefix adders (Brent-Kung/Kogge-Stone) - **Very high frequency:** Pipeline or use faster adder architecture

8.9.2 Width Selection Guidelines

Recommended Widths: - **4-bit:** Good for nibble operations, BCD arithmetic - **8-bit:** Standard byte operations, small ALU - **16-bit:** Address arithmetic, medium-width ALU - **32-bit:** Only if area is critical (prefer Brent-Kung)

8.9.3 Synthesis Considerations

Optimization Tips:

1. **Flatten hierarchy** for better carry chain optimization:

```
set_flatten true -effort high
```

2. **Mark as combinational** to prevent retiming issues:

```
set_dont_touch u_adder/w_c* false
```

3. **Critical path optimization:**

```
set_max_delay -from [get_ports i_a] -to [get_ports ow_carry] 2.0
```

4. **Consider pipelining** for $N > 16$:

```
// Pipeline example: Break carry chain in middle
logic [N/2:0] r_c_mid;
logic [N-1:0] r_sum_high;

always_ff @(posedge clk) begin
    r_c_mid <= w_c[N/2]; // Register mid-point carry
end
```

8.9.4 Verilator Lint Note

The module includes a Verilator pragma:

```
/* verilator lint_off UNOPTFLAT */
// ... carry chain logic ...
/* verilator lint_on UNOPTFLAT */
```

Reason: Carry chain creates combinational loops that Verilator warns about but are intentional for CLA operation.

8.10 Verification Strategy

Test suite location: `val/common/test_math_adder_pg_chain.py`

Key Test Scenarios: - Random stimulus (all widths) - Corner cases: $0+0$, $0+MAX$, $MAX+MAX$, $MAX+1$ - Carry propagation: All 1's + 1 (tests full carry chain) - Incrementer mode: $A + 0 + 1$ - Subtraction mode: $A + (\sim B) + 1$ - Multi-precision chaining

Test Command:

```
pytest val/common/test_math_adder_pg_chain.py -v
```

8.11 Common Pitfalls

✗ **Anti-Pattern 1:** Using for very wide additions without pipelining

```
// WRONG: 64-bit addition with CLA (poor performance)
math_adder_pg_chain #(.N(64)) u_add (...);
// Result: 66 logic levels, very slow!

// RIGHT: Use parallel prefix adder
math_adder_brent_kung_032 u_add_low (...) // Lower 32 bits
math_adder_brent_kung_032 u_add_high (...) // Upper 32 bits
```

✗ **Anti-Pattern 2:** Expecting registered outputs

```
// WRONG: Assuming outputs are registered
always_ff @(posedge clk) begin
    data <= ow_sum; // ow_sum is combinational!
end

// RIGHT: Add explicit output register
logic [N-1:0] r_sum;
always_ff @(posedge clk) begin
    r_sum <= ow_sum;
end
```

✗ **Anti-Pattern 3:** Chaining adders without pipelining

```
// WRONG: Long chain without pipeline
logic [15:0] a, b, c, d, result;
logic [15:0] ab_sum, abc_sum;

math_adder_pg_chain #(.N(16)) u1
(.i_a(a), .i_b(b), .ow_sum(ab_sum), ...);
math_adder_pg_chain #(.N(16)) u2
(.i_a(ab_sum), .i_b(c), .ow_sum(abc_sum), ...);
math_adder_pg_chain #(.N(16)) u3
(.i_a(abc_sum), .i_b(d), .ow_sum(result), ...);

// 3 × 18 levels = 54 logic levels!

// RIGHT: Pipeline the chain
always_ff @(posedge clk) begin
    r_ab_sum <= ab_sum;
    r_abc_sum <= abc_sum;
end
```

✗ **Anti-Pattern 4:** Ignoring synthesis warnings

```
// Synthesis warning: "Combinational loop detected"
// This is expected for CLA carry chain, but verify:
// - Loop is intentional
// - No actual circular dependency
// - All signals eventually resolve
```

8.12 Related Modules

- **`**math_adder_brent_kung*.sv**`** - Faster parallel prefix adder ($O(\log N)$ depth)
- **`math_adder_ripple_carry.sv`** - Minimal area adder (slower)
- **`math_adder_carry_save_nbit.sv`** - For multi-operand addition (multipliers)
- **`math_subtractor_carry_lookahead.sv`** - CLA-based subtractor variant

8.13 References

- Weinberger, A., Smith, J.L. “A Logic for High-Speed Addition.” National Bureau of Standards, 1958.
- Sklanski, J. “Conditional-Sum Addition Logic.” IRE Transactions on Electronic Computers, 1960.
- Deschamps, J.P., Bioul, G., Sutter, G. “Synthesis of Arithmetic Circuits: FPGA, ASIC and Embedded Systems.” Wiley, 2006.

8.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

9 Ripple Carry Adder

A parameterized N-bit ripple carry adder built from a chain of full adders, providing minimal area at the cost of $O(N)$ propagation delay. This is the simplest and most area-efficient multi-bit adder architecture.

9.1 Overview

The `math_adder_ripple_carry` module implements the classic ripple carry adder where carry propagates sequentially from LSB to MSB through a chain of full adders. While this results in $O(N)$ delay (slowest among adder architectures), it offers the smallest area and simplest implementation, making it ideal for low-speed, area-critical applications.

9.2 Module Declaration

```
module math_adder_ripple_carry #(
    parameter int N = 4          // Adder width in bits (any width ≥ 1)
) (
    input logic [N-1:0] i_a,      // Operand A
    input logic [N-1:0] i_b,      // Operand B
    input logic          i_c,      // Carry input
    output logic [N-1:0] ow_sum,   // Sum output
    output logic          ow_carry // Carry output
);
```

Related Module: `math_adder_full_nbit` provides functionally identical behavior with slightly different internal structure (uniform generate loop vs explicit first full adder).

9.3 Parameters

9.3.1 User-Settable Parameters

Parameter	Type	Default	Description
N	int	4	Adder width in bits (range: 1-64, typical: 2-8)

Width Guidelines: - **Optimal:** 2-8 bits (best area/speed trade-off for this architecture) - **Minimum:** 1 bit (degenerates to single full adder) - **Maximum:** 64 bits (practical limit, but very slow) - **Above 16 bits:** Consider carry lookahead or parallel prefix adders

9.4 Ports

9.4.1 Inputs

Port	Width	Description
i_a	N	Operand A (addend)
i_b	N	Operand B (addend)
i_c	1	Carry input (0 for addition, 1 for +1 increment)

9.4.2 Outputs

Port	Width	Description
ow_sum	N	Sum output (A + B + Cin)

Port	Width	Description
ow_carry	1	Carry output (final carry from MSB)

9.5 Functionality

9.5.1 Algorithm: Sequential Carry Propagation

The ripple carry adder computes addition bit-by-bit from LSB to MSB, with each bit's carry output feeding the next bit's carry input:

```

Bit 0 (LSB):  Sum[0], C[0] = FullAdder(A[0], B[0], Cin)
Bit 1:        Sum[1], C[1] = FullAdder(A[1], B[1], C[0])
Bit 2:        Sum[2], C[2] = FullAdder(A[2], B[2], C[1])
...
Bit N-1 (MSB): Sum[N-1], Cout = FullAdder(A[N-1], B[N-1], C[N-2])

```

9.5.2 Full Adder Building Block

Each bit position uses a full adder with three inputs:

```

// Full adder logic (implemented in math_adder_full.sv)
assign sum    = a ^ b ^ c_in;           // XOR for sum
assign c_out  = (a & b) | (c_in & (a ^ b)); // Majority function
// for carry

```

Truth Table:

A	B	Cin	Sum	Cout																
0	0	0	0	0		0	0	0	0		0	0	1	1		1	0	1	0	1
0	0	1	0	1		0	1	1	0		1	0	0	1		1	0	1	0	1
0	1	0	1	0		1	0	0	1		1	0	1	0		1	1	0	0	1
0	1	1	1	1		1	1	1	1		1	1	1	1		1	1	1	1	1

9.5.3 Implementation

```

// Intermediate carries between full adders
logic [N-1:0] w_c;

// First full adder (LSB) - uses input carry i_c
math_adder_full fa0 (
    .i_a(i_a[0]),
    .i_b(i_b[0]),
    .i_c(i_c),
    .ow_sum(ow_sum[0]),
    .ow_carry(w_c[0])
);

// Remaining full adders - chain carry from previous stage
genvar i;
generate

```

```

    for (i = 1; i < N; i++) begin : gen_full_adders
        math_adder_full fa (
            .i_a(i_a[i]),
            .i_b(i_b[i]),
            .i_c(w_c[i-1]),           // Carry from previous bit
            .ow_sum(ow_sum[i]),
            .ow_carry(w_c[i])
        );
    end
endgenerate

// Final carry out
assign ow_carry = w_c[N-1];

```

9.5.4 Timing Analysis

Logic Depth: - Per bit: 2 levels (XOR for sum + Carry logic) - **Carry chain:** $N \times 2$ levels (sequential dependency) - **Total:** $2N$ levels

Critical Path (Carry Propagation):

$i_c \rightarrow \text{FA}[0].\text{Cout} \rightarrow \text{FA}[1].\text{Cout} \rightarrow \dots \rightarrow \text{FA}[N-1].\text{Cout} \rightarrow \text{ow_carry}$

Why It's Slow: The critical path traverses all N full adders sequentially. Each full adder adds 2 gate delays, resulting in total delay proportional to N .

9.5.5 Comparison to Other Adders

Adder Type	Logic Depth	Area (relative)	Speed (relative)	Best Use Case
Ripple Carry	$O(N)$	$1.0\times$ (min)	$1.0\times$ (slowest)	$N \leq 8$, area-critical
Carry Lookahead	$O(N)$	$1.2\times$	$1.5\times$	$4 \leq N \leq 16$
Brent-Kung	$O(\log N)$	$4.0\times$	$4.0\times$	$16 \leq N \leq 32$
Kogge-Stone	$O(\log N)$	$6.0\times$	$6.0\times$	$N \geq 32$, critical path

Key Advantage: Smallest possible adder area - only N full adders, no additional logic.

9.6 Usage Examples

9.6.1 Basic 8-bit Addition

```

logic [7:0] a, b, sum;
logic carry_out;

```

```

math_adder_ripple_carry #(
    .N(8)
) u_adder (
    .i_a      (a),
    .i_b      (b),
    .i_c      (1'b0),           // No carry input
    .ow_sum   (sum),
    .ow_carry (carry_out)
);

```

// Example: 100 + 50

```

initial begin
    a = 8'd100;
    b = 8'd50;
    #1; // Wait for combinational propagation
    assert(sum == 8'd150);
end

```

9.6.2 4-bit Incrementer

```

logic [3:0] count, count_plus_1;

```

```

math_adder_ripple_carry #(
    .N(4)
) u_incrementer (
    .i_a      (count),
    .i_b      (4'b0),           // B = 0
    .i_c      (1'b1),           // Carry in = 1 → +1
    .ow_sum   (count_plus_1),
    .ow_carry ()                // Unused
);

```

9.6.3 BCD Digit Adder (4-bit)

// Add two BCD digits (0-9)

```

logic [3:0] bcd_a, bcd_b, bcd_sum_raw;
logic [3:0] bcd_sum_corrected;
logic carry_raw, carry_corrected;

```

// Raw binary addition

```

math_adder_ripple_carry #(.N(4)) u_bcd_add (
    .i_a      (bcd_a),
    .i_b      (bcd_b),
    .i_c      (1'b0),
    .ow_sum   (bcd_sum_raw),
    .ow_carry (carry_raw)
);

```

```
// BCD correction: if sum > 9, add 6
logic bcd_overflow;
assign bcd_overflow = carry_raw | (bcd_sum_raw > 4'd9);
```

```
math_adder_ripple_carry #(.N(4)) u_bcd_correct (
    .i_a      (bcd_sum_raw),
    .i_b      (bcd_overflow ? 4'd6 : 4'd0),
    .i_c      (1'b0),
    .ow_sum   (bcd_sum_corrected),
    .ow_carry ()
);
```

```
assign bcd_carry_out = bcd_overflow;
```

9.6.4 Multi-Precision Addition (16-bit via 2×8-bit)

```
logic [7:0] a_low, a_high, b_low, b_high;
logic [7:0] sum_low, sum_high;
logic carry_low, carry_high;
```

```
// Low byte
```

```
math_adder_ripple_carry #(.N(8)) u_add_low (
    .i_a      (a_low),
    .i_b      (b_low),
    .i_c      (1'b0),
    .ow_sum   (sum_low),
    .ow_carry (carry_low)
);
```

```
// High byte (chain carry from low)
```

```
math_adder_ripple_carry #(.N(8)) u_add_high (
    .i_a      (a_high),
    .i_b      (b_high),
    .i_c      (carry_low),
    .ow_sum   (sum_high),
    .ow_carry (carry_high)
);
```

```
logic [15:0] sum_16 = {sum_high, sum_low};
```

9.6.5 Simple ALU (Add/Subtract)

```
logic [7:0] a, b, result;
logic sub; // 1 = subtract, 0 = add
logic [7:0] b_mux;
logic carry_out;
```

```
// Two's complement subtraction:  $A - B = A + (\sim B) + 1$ 
assign b_mux = sub ? ~b : b;

math_adder_ripple_carry #(.N(8)) u_alu (
    .i_a      (a),
    .i_b      (b_mux),
    .i_c      (sub),           // Carry in = 1 for subtraction
    .ow_sum   (result),
    .ow_carry (carry_out)
);
```

9.7 Timing Characteristics

9.7.1 Combinational Delay Analysis

Width	Logic Levels	Typical Delay (ns) @ 1.0V	Max Frequency
2-bit	4	~0.8	~1250 MHz
4-bit	8	~1.5	~650 MHz
8-bit	16	~3.0	~330 MHz
16-bit	32	~6.0	~165 MHz
32-bit	64	~12.0	~83 MHz

Logic Level Breakdown (8-bit example): - Carry chain: 8 full adders $\times$ 2 levels each = 16 levels - Sum outputs: Available after carry arrives at each position

Observation: Delay doubles when width doubles (linear scaling).

9.7.2 Critical Paths

Longest Path (Carry Out):

$i_c \rightarrow \text{FA}[0] \rightarrow \text{FA}[1] \rightarrow \text{FA}[2] \rightarrow \dots \rightarrow \text{FA}[N-1] \rightarrow \text{ow_carry}$
 $2N$ gate delays

Sum Output Paths (variable length):

LSB (Sum[0]): $i_a[0]/i_b[0] \rightarrow 2$ levels (fastest)
 Mid (Sum[4]): Depends on C[3] $\rightarrow 10$ levels (8-bit example)
 MSB (Sum[7]): Depends on C[6] $\rightarrow 16$ levels (slowest)

9.7.3 Performance vs Width

Width	Relative Delay	Relative Frequency
4-bit	1.0×	1.00×

Width	Relative Delay	Relative Frequency
8-bit	2.0×	0.50×
16-bit	4.0×	0.25×
32-bit	8.0×	0.125×

Why Linear Scaling is Bad: For wide adders, delay becomes prohibitive. Consider faster architectures for $N > 8$.

9.8 Performance Characteristics

9.8.1 Resource Utilization

Width	LUTs (Est.)	FFs (Pipeline)	Description
4-bit	~8	0 (combinational)	Minimal
8-bit	~16	0 (combinational)	Small
16-bit	~32	0 (combinational)	Medium (consider alternatives)
32-bit	~64	0 (combinational)	Large (use faster adder)

Area Breakdown (8-bit): - 8 full adders: 8×2 gates = 16 LUTs (XOR + Carry logic) - **Total:** ~16 LUTs (absolute minimum for 8-bit adder)

Why It's Smallest: No additional logic beyond full adders - just direct carry chain.

9.8.2 Comparison: Area vs Speed

Width	Ripple	CLA	Brent-Kung
8-bit Area	16	24	60
8-bit Speed	1×	1.7×	4×
Area Advantage	Baseline	+50%	+275%

9.9 Design Considerations

9.9.1 When to Use Ripple Carry Adder

✓ **Ideal Use Cases:** - Width ≤ 8 bits - Area is critical constraint - Speed is not critical (< 100 MHz) - Simple control logic - Low-power designs (minimal gates = minimal switching) - BCD arithmetic (4-bit digits) - Small incrementers/decrementers

9.9.2 When to Use Alternatives

Use Carry Lookahead if: - $4 \leq N \leq 16$ - Need moderate speed improvement - Can afford 20-50% more area

Use Brent-Kung if: - $N = 8, 16, \text{ or } 32$ - Speed is critical - Can afford 4 $\times$ area

Use Kogge-Stone if: - $N \geq 32$ - Maximum speed required - Area budget allows 6 $\times$ increase

9.9.3 Width Selection Trade-offs

Application	Recommended Width	Rationale
Counter increment	4-8 bits	Minimal area, adequate speed
BCD digit	4 bits	Natural fit for BCD
Byte operations	8 bits	Good balance
Word operations	16 bits	Marginal - consider CLA
Double word	32 bits	Too slow - use faster adder

9.9.4 Synthesis Considerations

Optimization Tips:

1. **Allow inference of fast carry chains:**

```
# Let tool map to dedicated carry logic
set_dont_touch false
```

2. **Consider FPGA carry chains:**

- Modern FPGAs have dedicated fast carry chains
- Ripple carry maps well to these primitives
- May achieve better performance than expected

3. **For ASIC, use standard cell library:**


```
# Use full adder cells from library
set_dont_use [get_lib_cells */ADDF*] false
```

4. Break long chains for high frequency:

```
// Pipeline every 8 bits
logic [7:0] r_sum_low, r_carry_mid;
always_ff @(posedge clk) begin
    r_sum_low <= ow_sum[7:0];
    r_carry_mid <= w_c[7];
end
```

9.10 Verification Strategy

Test suite location: val/common/test_math_adder_ripple_carry.py

Key Test Scenarios: - Random stimulus (all widths) - Corner cases: 0+0, 0+MAX, MAX+MAX - Full carry propagation: 0x00 + 0xFF (all carries ripple) - Incrementer mode: A + 0 + 1 - Subtraction mode: A + (~B) + 1 - BCD operations (4-bit)

Test Command:

```
pytest val/common/test_math_adder_ripple_carry.py -v
```

9.11 Common Pitfalls

✗ Anti-Pattern 1: Using for wide additions

```
// WRONG: 64-bit ripple carry (extremely slow!)
math_adder_ripple_carry #(.N(64)) u_add (...);
// Result: 128 logic levels, ~25ns delay @ 1.0V!
```

```
// RIGHT: Use faster adder for wide operations
math_adder_brent_kung_032 u_add_low (...); // Lower 32 bits
math_adder_brent_kung_032 u_add_high (...); // Upper 32 bits
```

✗ Anti-Pattern 2: In critical timing paths

```
// WRONG: Ripple adder in critical datapath
always_ff @(posedge clk) begin
    data_reg <= ripple_adder_output; // May not meet timing!
end
```

```
// RIGHT: Use faster adder or pipeline
```

✗ Anti-Pattern 3: Ignoring FPGA carry chains

```
// WRONG: Custom carry logic that doesn't infer carry chain
// (Defeats FPGA optimization)
```

```
// RIGHT: Use standard ripple carry structure
// Synthesis tools will map to fast carry primitives
```

✗ **Anti-Pattern 4:** Chaining multiple adders

```
// WRONG: Cascading multiple ripple adders in one cycle
logic [7:0] sum1, sum2, sum3;
math_adder_ripple_carry #(.N(8)) u1
(.i_a(a), .i_b(b), .ow_sum(sum1), ...);
math_adder_ripple_carry #(.N(8)) u2
(.i_a(sum1), .i_b(c), .ow_sum(sum2), ...);
math_adder_ripple_carry #(.N(8)) u3
(.i_a(sum2), .i_b(d), .ow_sum(sum3), ...);
// 3 × 16 levels = 48 gate delays!
```

```
// RIGHT: Pipeline or use carry-save adder tree
```

9.12 Related Modules

- **math_adder_full.sv** - Single-bit full adder building block
- **math_adder_half.sv** - Single-bit half adder (no carry input)
- **math_adder_full_nbit.sv** - Functionally equivalent to this module
- **math_adder_pg_chain.sv** - Faster alternative (1.5× speed, +20% area)
- ****math_adder_brent_kung*.sv**** - Much faster (4× speed for 32-bit)

9.13 References

- Mano, M.M. “Digital Design.” Prentice Hall, 2002. (Chapter 4: Combinational Logic)
- Weste, N., Harris, D. “CMOS VLSI Design.” Addison Wesley, 2010.
- Deschamps, J.P., Bioul, G., Sutter, G. “Synthesis of Arithmetic Circuits.” Wiley, 2006.

9.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

10 Add/Subtract Unit

A unified N-bit add/subtract unit using two’s complement arithmetic, efficiently implementing both addition and subtraction with a single control signal and shared hardware.

10.1 Overview

The `math_addsub_full_nbit` module implements a combined adder/subtractor that performs either $A + B$ or $A - B$ based on a control signal. It uses two's complement arithmetic for subtraction ($A - B = A + (\sim B) + 1$), efficiently sharing the adder hardware for both operations.

Key Feature: Single arithmetic unit handles both operations using XOR gates and a control signal, minimizing hardware compared to separate adder and subtractor modules.

10.2 Module Declaration

```
module math_addsub_full_nbit #(
    parameter int N = 4          // Bit width
) (
    input logic [N-1:0] i_a,      // Operand A
    input logic [N-1:0] i_b,      // Operand B
    input logic          i_c,      // Control: 0=add, 1=subtract
    output logic [N-1:0] ow_sum,    // Result output
    output logic          ow_carry  // Carry/borrow output
);
```

10.3 Parameters

Parameter	Type	Default	Description
N	int	4	Bit width (range: 1-64, typical: 8-32)

10.4 Ports

Port	Direction	Width	Description
i_a	Input	N	Operand A
i_b	Input	N	Operand B
i_c	Input	1	Control signal (0=add, 1=subtract)
ow_sum	Output	N	Result (A+B or A-B)
ow_carry	Output	1	Carry (add) or inverted borrow (subtract)

Control Signal (i_c): - **i_c = 0:** Addition mode → Result = A + B - **i_c = 1:** Subtraction mode → Result = A - B

Carry Output Interpretation: - **Add mode (i_c=0):** ow_carry = carry output (overflow for unsigned) - **Subtract mode (i_c=1):** ow_carry = ~borrow (1 if A ≥ B, 0 if A < B)

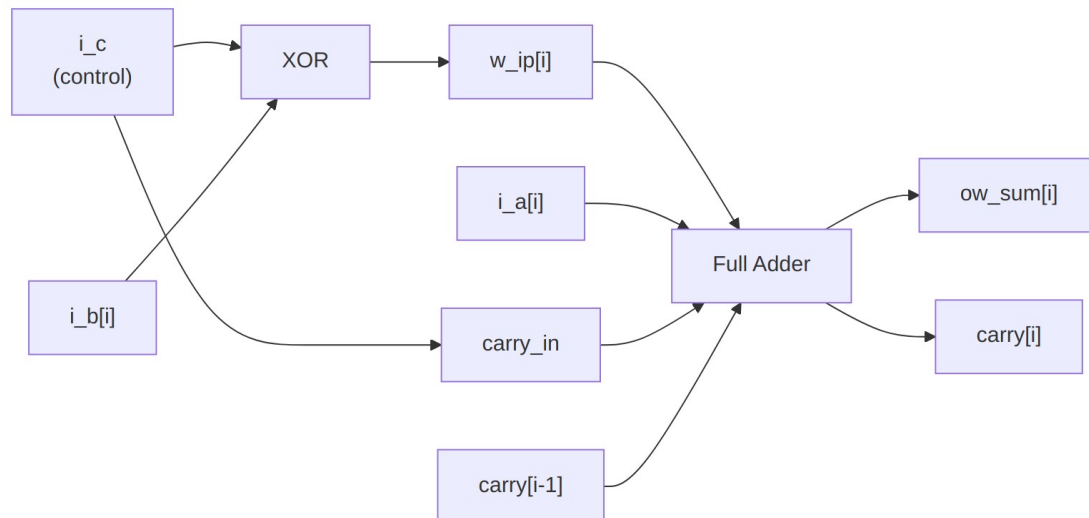
10.5 Functionality

10.5.1 Two's Complement Subtraction

The module implements subtraction using two's complement arithmetic:

$$A - B = A + (\sim B) + 1$$

Implementation Strategy: 1. **XOR B with control signal:** $w_ip[i] = i_b[i] \oplus i_c$ - If $i_c = 0$ (add): $w_ip = B$ (unchanged) - If $i_c = 1$ (sub): $w_ip = \sim B$ (one's complement) 2. **Feed i_c as carry input:** Completes two's complement (+1) 3. **Use standard adder:** $A + w_ip + i_c$



Logic Diagram

10.5.2 Implementation

```
// XOR B with control signal (inverts B for subtraction)
genvar i;
generate
    for (i = 0; i < N; i++) begin : gen_xor
        assign w_ip[i] = i_b[i] ^ i_c;
    end
endgenerate

// Carry chain initialization
assign w_c[0] = i_c; // i_c = 0 for add, 1 for subtract
```

```
// Full adder chain
generate
  for (i = 0; i < N; i++) begin : gen_full_adders
    math_adder_full fa (
      .i_a(i_a[i]),
      .i_b(w_ip[i]),      // XORed B
      .i_c(w_c[i]),
      .ow_sum(ow_sum[i]),
      .ow_carry(w_c[i+1])
    );
  end
endgenerate

assign ow_carry = w_c[N];
```

10.5.3 Operation Modes

10.5.3.1 Addition Mode ($i_c = 0$)

$w_ip[i] = i_b[i] \wedge 0 = i_b[i]$ (B unchanged)
 $w_c[0] = 0$ (no initial carry)
 $Result = A + B + 0 = A + B$

10.5.3.2 Subtraction Mode ($i_c = 1$)

$w_ip[i] = i_b[i] \wedge 1 = \sim i_b[i]$ (B inverted = one's complement)
 $w_c[0] = 1$ (carry in = 1)
 $Result = A + (\sim B) + 1 = A - B$ (two's complement)

10.6 Usage Examples

10.6.1 Simple Add/Subtract ALU

```
logic [7:0] a, b, result;
logic sub; // 0=add, 1=subtract
logic carry_borrow;

math_addsub_full_nbit #(.N(8)) u_alu (
  .i_a(a),
  .i_b(b),
  .i_c(sub),
  .ow_sum(result),
  .ow_carry(carry_borrow)
);

// Example: Addition
initial begin
  a = 8'd100;
```

```

    b = 8'd50;
    sub = 1'b0; // Add mode
    #1;
    assert(result == 8'd150);
    assert(carry_borrow == 1'b0); // No overflow
end

// Example: Subtraction
initial begin
    a = 8'd100;
    b = 8'd50;
    sub = 1'b1; // Subtract mode
    #1;
    assert(result == 8'd50);
    assert(carry_borrow == 1'b1); // A >= B (no borrow)
end

```

10.6.2 Detecting Underflow in Subtraction

```

logic [7:0] a, b, result;
logic underflow;

math_addsub_full_nbit #(.N(8)) u_alu (
    .i_a(a),
    .i_b(b),
    .i_c(1'b1), // Subtract mode
    .ow_sum(result),
    .ow_carry(carry_out)
);

// For subtraction, borrow = ~carry_out
// Underflow (A < B) when carry_out = 0
assign underflow = ~carry_out;

// Example: Underflow case
initial begin
    a = 8'd50;
    b = 8'd100;
    #1;
    assert(underflow == 1'b1); // A < B
    assert(result == 8'd206); // Wraps: 50 - 100 + 256 = 206
end

```

10.6.3 Simple Calculator

```

module calculator (
    input logic [7:0] operand_a,
    input logic [7:0] operand_b,
    input logic      operation, // 0=add, 1=subtract

```

```

    output logic [7:0] result,
    output logic      overflow,    // Overflow (add) or underflow
(sub)
    output logic      zero        // Result is zero
);

    logic carry_borrow;

    math_addsub_full_nbit #(.N(8)) u_alu (
        .i_a(operand_a),
        .i_b(operand_b),
        .i_c(operation),
        .ow_sum(result),
        .ow_carry(carry_borrow)
    );

    // Overflow/underflow detection
    assign overflow = operation ? ~carry_borrow : carry_borrow;

    // Zero flag
    assign zero = ~|result;

endmodule

```

10.6.4 Multi-Function ALU

```

typedef enum logic [1:0] {
    ALU_ADD = 2'b00,
    ALU_SUB = 2'b01,
    ALU_INC = 2'b10,
    ALU_DEC = 2'b11
} alu_op_t;

module multi_alu (
    input logic [7:0] a, b,
    input alu_op_t op,
    output logic [7:0] result,
    output logic      carry_flag
);

    logic [7:0] b_mux;
    logic      ctrl;

    // Operand and control selection
    always_comb begin
        case (op)
            ALU_ADD: begin

```

```

        b_mux = b;
        ctrl = 1'b0;           // Add mode
    end
    ALU_SUB: begin
        b_mux = b;
        ctrl = 1'b1;           // Subtract mode
    end
    ALU_INC: begin
        b_mux = 8'b0;
        ctrl = 1'b1;           //  $A + 0 + 1 = A + 1$ 
    end
    ALU_DEC: begin
        b_mux = 8'b1;
        ctrl = 1'b1;           //  $A - 1 = A + (\sim 1) + 1$ 
    end
endcase
end

math_addsub_full_nbit #(.N(8)) u_alu (
    .i_a(a),
    .i_b(b_mux),
    .i_c(ctrl),
    .ow_sum(result),
    .ow_carry(carry_flag)
);

endmodule

```

10.7 Timing Characteristics

Metric	Value
Logic Depth	2N + 2 levels (XOR + ripple carry chain)
Typical Delay (8-bit)	~3.2 ns @ 1.0V
Max Frequency (8-bit)	~300 MHz

Logic Level Breakdown (8-bit): 1. XOR stage: 1 level ($B \wedge \text{control}$) 2. Carry chain: 16 levels (8 full adders $\times$ 2) 3. **Total:** 17 levels

Critical Path:

$i_b[0] \rightarrow \text{XOR} \rightarrow \text{FA}[0] \rightarrow \text{FA}[1] \rightarrow \dots \rightarrow \text{FA}[7] \rightarrow \text{ow_carry}$

10.8 Performance Characteristics

10.8.1 Resource Utilization

Width	LUTs (Est.)	Description
8-bit	~18	N XORs + N full adders
16-bit	~34	Linear scaling
32-bit	~66	Linear scaling

Area Breakdown (8-bit): - XOR gates: 8 LUTs - Full adders: $8 \times 2 = 16$ LUTs (optimized) - **Total:** ~18 LUTs

Comparison: | Architecture | Area (relative) | Operations | |———| |———| |———| |
Separate Add + Sub | 2.0× | Add or Sub | | **Add/Sub Unit** | 1.1× | **Add and Sub** | | Adder only
(manual control) | 1.0× | Add and Sub (external invert) |

Advantage: ~10% area overhead vs adder-only, but integrated control simplifies system design.

10.9 Design Considerations

10.9.1 Advantages

✓ **Hardware efficiency:** Single unit for both operations ✓ **Simple control:** One bit selects operation ✓ **Standard practice:** Common in ALU designs ✓ **No separate subtractor:** Reuses adder logic

10.9.2 When to Use This Module

Appropriate Use Cases: - ALU designs requiring add/subtract - Arithmetic units with mode control - Simple calculators - Control logic needing both operations

Alternative Approaches: - **Manual control:** Use standard adder with external XOR and control (saves one XOR level if control is early) - **Behavioral:** `assign result = sub ? (a - b) : (a + b);` (synthesis tool optimizes)

10.9.3 Synthesis Considerations

Optimization Tips:

1. **Let synthesis infer fast carry chains** (FPGAs):
`set_dont_touch false`
2. **XOR placement:** XOR gates before adder may allow optimization

3. Consider **pipelining** for high frequency:

```
logic [N-1:0] r_b_conditioned;
always_ff @(posedge clk) begin
    r_b_conditioned <= i_b ^ {N{i_c}};
end
```

10.10 Common Pitfalls

✗ **Anti-Pattern 1:** Misinterpreting carry output

```
// WRONG: Carry means different things for add vs subtract!
if (carry) begin
    // overflow? underflow? Depends on operation!
end

// RIGHT: Interpret based on operation
logic overflow_underflow;
assign overflow_underflow = (i_c == 0) ? carry : // overflow (add)
                             ~carry;         // underflow (sub)
```

✗ **Anti-Pattern 2:** Using for signed arithmetic without overflow detection

```
// WRONG: Missing signed overflow detection
logic [7:0] result_signed;
math_addsub_full_nbit u_alu (.i_c(1'b0), ...); // Add mode
// Result may overflow for signed!

// RIGHT: Check signed overflow
logic signed_overflow;
assign signed_overflow = (i_a[N-1] == i_b[N-1]) &&
    (ow_sum[N-1] != i_a[N-1]);
```

✗ **Anti-Pattern 3:** Forgetting XOR overhead

```
// Be aware: XOR stage adds one logic level
// If timing is critical, consider pipelining the XOR stage
```

10.11 Related Modules

- **math_adder_ripple_carry.sv** - Add-only (no XOR overhead)
- **math_subtractor_ripple_carry.sv** - Subtract-only (rarely used)
- **math_adder_pg_chain.sv** - Faster alternative for adder stage

10.12 References

- Mano, M.M. “Digital Design.” Prentice Hall, 2002.

- Hennessy, J.L., Patterson, D.A. “Computer Architecture: A Quantitative Approach.” Morgan Kaufmann, 2011.

10.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

11 BF16 Adder

A pipelined BF16 (Brain Float 16) floating-point adder with IEEE 754-compliant special case handling, Round-to-Nearest-Even rounding, and flush-to-zero support for subnormal numbers.

11.1 Overview

The `math_bf16_adder` module implements full BF16 addition and subtraction with configurable pipeline stages for frequency/latency tradeoffs. It follows industry-standard practices used in AI/ML accelerators for efficient 16-bit floating-point operations.

Key Features: - **BF16 format** - Same exponent range as FP32, reduced mantissa precision - **Configurable pipeline** - 4 optional pipeline stages for frequency optimization - **IEEE 754 special cases** - Zero, infinity, NaN handling - **RNE rounding** - Round-to-Nearest-Even for unbiased results - **FTZ mode** - Flush-to-Zero for subnormal inputs and outputs - **Status flags** - Overflow, underflow, and invalid operation indicators - **Valid handshaking** - Input/output valid signals for pipeline control

11.2 Module Declaration

```
module math_bf16_adder #(
    parameter int PIPE_STAGE_1 = 0, // After exponent diff + swap
    parameter int PIPE_STAGE_2 = 0, // After alignment shifter
    parameter int PIPE_STAGE_3 = 0, // After mantissa add/sub
    parameter int PIPE_STAGE_4 = 0 // After normalize
) (
    input logic i_clk,
    input logic i_rst_n,
    input logic [15:0] i_a, // BF16 operand A
    input logic [15:0] i_b, // BF16 operand B
    input logic i_valid, // Input valid
    output logic [15:0] ow_result, // BF16 sum/difference
    output logic ow_overflow, // Overflow to infinity
    output logic ow_underflow, // Underflow to zero
    output logic ow_invalid, // Invalid operation (NaN)
    output logic ow_valid // Output valid
);
```

11.3 Parameters

Parameter	Default	Description
PIPE_STAGE_1	0	Register after exponent compare and operand swap
PIPE_STAGE_2	0	Register after mantissa alignment
PIPE_STAGE_3	0	Register after mantissa add/subtract
PIPE_STAGE_4	0	Register after normalization

Latency Formula: $1 + \text{PIPE_STAGE_1} + \text{PIPE_STAGE_2} + \text{PIPE_STAGE_3} + \text{PIPE_STAGE_4}$ cycles

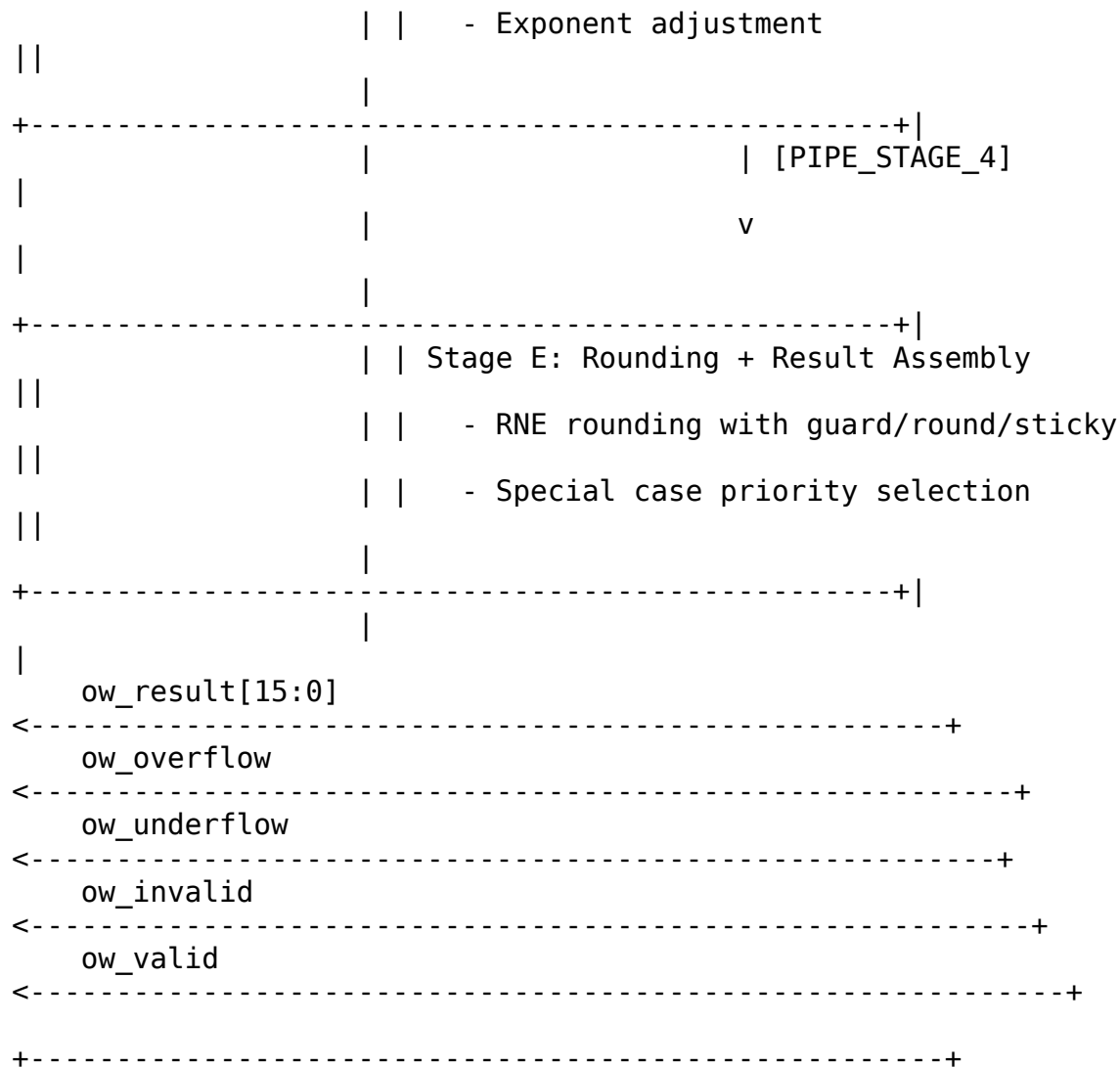
11.4 Ports

Port	Direction	Width	Description
i_clk	Input	1	Clock signal
i_rst_n	Input	1	Active-low asynchronous reset
i_a	Input	16	BF16 addend A
i_b	Input	16	BF16 addend B
i_valid	Input	1	Input operands are valid
ow_result	Output	16	BF16 sum (A + B)
ow_overflow	Output	1	1 if result overflowed to infinity
ow_underflow	Output	1	1 if result underflowed to zero
ow_invalid	Output	1	1 if operation


```

| | - Exponent difference calculation
||
| | - Magnitude comparison and operand swap
||
| | - Effective operation determination
||
|
+-----+
| | [PIPE_STAGE_1]
|
| v
|
+-----+
| | Stage B: Mantissa Alignment
||
| | - shifter_barrel (right shift smaller
operand) ||
| | - Sticky bit computation
||
|
+-----+
| | [PIPE_STAGE_2]
|
| v
|
+-----+
| | Stage C: Mantissa Add/Subtract
||
| | - 12-bit addition or subtraction
||
| | - Overflow detection
||
|
+-----+
| | [PIPE_STAGE_3]
|
| v
|
+-----+
| | Stage D: Normalization
||
| | - count_leading_zeros for shift amount
||
| | - shifter_barrel for normalization
||

```



11.6.2 Processing Pipeline

1. Stage A: Unpack + Compare + Swap

- Parse sign, exponent, mantissa from both inputs
- Detect special cases (zero, subnormal, infinity, NaN)
- Compare magnitudes and swap so larger operand is always “L”
- Determine effective operation (add or subtract based on signs)

2. Stage B: Mantissa Alignment

- Extend mantissas with implied bit and guard bits (11 bits)
- Right-shift smaller mantissa by exponent difference
- Compute sticky bit from shifted-out bits

3. Stage C: Mantissa Add/Subtract

- Add or subtract aligned mantissas (12-bit operation)
 - Detect addition overflow (sum >= 2.0)
4. **Stage D: Normalization**
- Count leading zeros for subtraction results
 - Shift to normalize (implied 1 at correct position)
 - Adjust exponent accordingly
5. **Stage E: Rounding + Result Assembly**
- Apply RNE rounding using guard/round/sticky bits
 - Handle rounding overflow
 - Select result based on special case priority

11.7 Functionality

11.7.1 Exponent Comparison and Swap

```
// Compare magnitudes: larger operand becomes "L", smaller becomes "S"
wire [8:0] w_exp_diff_raw = {1'b0, w_exp_a} - {1'b0, w_exp_b};
wire      w_a_larger     = w_exp_a_larger && (~w_exp_equal ||
w_mant_a_larger);
```

```
// Absolute exponent difference for alignment shift
wire [7:0] w_exp_diff     = w_a_larger ? (w_exp_a - w_exp_b) :
(w_exp_b - w_exp_a);
```

11.7.2 Mantissa Alignment

```
// Extend mantissas: {implied_1, mant[6:0], guard, round, sticky} = 11 bits
wire [10:0] w_mant_l_ext = {r1_l_is_normal, r1_mant_l, 3'b000};
wire [10:0] w_mant_s_ext = {r1_s_is_normal, r1_mant_s, 3'b000};

// Right-shift smaller mantissa using barrel shifter
shifter_barrel #(.WIDTH(11)) u_align_shifter (
    .data      (w_mant_s_ext),
    .ctrl      (SHIFT_RIGHT_LOGIC),
    .shift_amount(w_shift_amt),
    .data_out   (w_mant_s_aligned)
);
```

11.7.3 Round-to-Nearest-Even

```
// Extract rounding bits from normalized mantissa
wire [6:0] w_mant_final_raw = r4_mant_normalized[9:3];
wire      w_guard_bit       = r4_mant_normalized[2];
wire      w_round_bit      = r4_mant_normalized[1];
```



```

wire      w_sticky_bit      = r4_mant_normalized[0] | r4_norm_sticky;

// RNE: Round up if guard && (round || sticky || LSB)
wire w_round_up = w_guard_bit && (w_round_bit || w_sticky_bit ||
w_mant_final_raw[0]);

```

11.7.4 Special Case Priority

```

always_comb begin
    // Default: normal result
    ow_result = {w_final_sign, w_exp_out, w_mant_out};

    // Priority order (highest to lowest):
    if (r4_any_nan || r4_inf_minus_inf) begin
        // 1. NaN: any NaN input, or inf - inf
        ow_result = {1'b0, 8'hFF, 7'h40}; // Canonical qNaN
        ow_invalid = r4_inf_minus_inf;
    end else if (r4_any_inf) begin
        // 2. Infinity: inf input (not inf-inf case)
        ow_result = {inf_sign, 8'hFF, 7'h00};
    end else if (r4_sum_is_zero) begin
        // 3. Exact zero from subtraction
        ow_result = {1'b0, 8'h00, 7'h00}; // +0 per IEEE 754 RNE
    end else if (w_final_overflow) begin
        // 4. Overflow to infinity
        ow_result = {w_final_sign, 8'hFF, 7'h00};
        ow_overflow = 1'b1;
    end else if (r4_exp_underflow) begin
        // 5. Underflow to zero (FTZ)
        ow_result = {w_final_sign, 8'h00, 7'h00};
        ow_underflow = 1'b1;
    end
end

```

11.8 Usage Examples

11.8.1 Basic Addition (Combinational)

```

logic [15:0] a, b, sum;
logic overflow, underflow, invalid, valid;

```

```

math_bf16_adder u_adder (
    .i_clk(clk),
    .i_rst_n(rst_n),
    .i_a(a),
    .i_b(b),
    .i_valid(1'b1),
    .ow_result(sum),

```

```

        .ow_overflow(overflow),
        .ow_underflow(underflow),
        .ow_invalid(invalid),
        .ow_valid(valid)
    );

    // Example: 1.0 + 1.0 = 2.0
    // 1.0 in BF16: 0x3F80 (sign=0, exp=127, mant=0)
    // 2.0 in BF16: 0x4000 (sign=0, exp=128, mant=0)
    initial begin
        a = 16'h3F80; // 1.0
        b = 16'h3F80; // 1.0
        #1;
        // sum should be 0x4000 (2.0)
    end

```

11.8.2 Pipelined Configuration

```

    // 4-stage pipeline for high frequency
    math_bf16_adder #(
        .PIPE_STAGE_1(1),
        .PIPE_STAGE_2(1),
        .PIPE_STAGE_3(1),
        .PIPE_STAGE_4(1)
    ) u_adder_pipelined (
        .i_clk(clk),
        .i_rst_n(rst_n),
        .i_a(a),
        .i_b(b),
        .i_valid(in_valid),
        .ow_result(sum),
        .ow_overflow(overflow),
        .ow_underflow(underflow),
        .ow_invalid(invalid),
        .ow_valid(out_valid) // Valid 5 cycles after in_valid
    );

```

11.8.3 Subtraction via Sign Manipulation

```

    // To compute A - B, negate B's sign and add
    logic [15:0] a_minus_b;
    logic [15:0] b_negated;

    assign b_negated = {~b[15], b[14:0]}; // Flip sign bit

    math_bf16_adder u_subtractor (
        .i_clk(clk),
        .i_rst_n(rst_n),
        .i_a(a),

```

```

        .i_b(b_negated),
        .i_valid(1'b1),
        .ow_result(a_minus_b),
        // ...
    );

```

11.8.4 With Status Checking

```

always_ff @(posedge clk) begin
    if (out_valid) begin
        if (invalid)
            $display("Invalid operation: inf - inf");
        else if (overflow)
            $display("Overflow: result is infinity");
        else if (underflow)
            $display("Underflow: result is zero");
    end
end

```

11.9 Pipeline Latency Configurations

Configuration	Latency	Use Case
[0,0,0,0]	1 cycle	Area-constrained, low frequency
[1,0,0,0]	2 cycles	Balance input timing
[0,0,0,1]	2 cycles	Balance output timing
[1,1,1,1]	5 cycles	Maximum frequency

11.10 Performance Characteristics

11.10.1 Resource Utilization (Estimated)

Component	LUTs (est.)
Unpack + compare	~40
Alignment shifter	~50
Mantissa adder	~30
CLZ + normalize	~60
Rounding logic	~20
Result MUX	~30
Pipeline regs (per stage)	~50

Component	LUTs (est.)
Total (comb)	~230 LUTs
Total (4-stage)	~430 LUTs

11.10.2 Timing Characteristics

Stage	Logic Depth
Exponent compare	~3 gates
Alignment shift	~4 gates (log2)
Mantissa add	~5 gates
CLZ + normalize	~6 gates
Rounding	~3 gates
Total (comb)	~20-25 gates

11.11 Design Considerations

11.11.1 Flush-to-Zero (FTZ)

Subnormal numbers are flushed to zero: - **Input subnormals** - Treated as zero (effective zero) - **Output subnormals** - Not generated (result goes to zero)

This simplifies hardware and matches AI accelerator conventions.

11.11.2 Invalid Operation (inf - inf)

When adding +inf and -inf (or -inf and +inf): - Result is canonical quiet NaN (0x7FC0) - `ow_invalid` flag is asserted

11.11.3 Sign of Zero

The sign of zero follows IEEE 754: - $x - x = +0$ (in RNE rounding mode) - $-0 + -0 = -0$ - $+0 + -0 = +0$

11.11.4 Rounding Mode

Only Round-to-Nearest-Even (RNE) is supported. This is standard for BF16 in AI applications.

11.12 Dependencies

This module instantiates: - **shifter_barrel** - Used for alignment shifting and normalization - **count_leading_zeros** - Used for determining normalization shift amount

11.13 Common Pitfalls

Anti-Pattern 1: Ignoring valid signals

```
// WRONG: Capturing result without checking valid  
result <= ow_result; // May capture invalid data during pipeline fill  
  
// RIGHT: Only capture when valid  
if (ow_valid)  
    result <= ow_result;
```

Anti-Pattern 2: Forgetting pipeline latency

```
// WRONG: Expecting immediate result with pipeline  
i_a <= operand_a;  
i_b <= operand_b;  
i_valid <= 1'b1;  
@(posedge clk);  
sum <= ow_result; // Result not ready yet!  
  
// RIGHT: Account for latency  
// With [1,1,1,1] config, wait 5 cycles for result
```

Anti-Pattern 3: Not handling special flags

```
// WRONG: Using result without checking flags  
accumulator <= accumulator + ow_result; // May be NaN or infinity!  
  
// RIGHT: Check flags first  
if (ow_valid && !ow_invalid && !ow_overflow)  
    accumulator <= accumulator + ow_result;
```

11.14 Related Modules

- **math_bf16_multiplier** - BF16 multiplication
- **math_bf16_fma** - Fused Multiply-Add with FP32 accumulator
- **shifter_barrel** - Used internally for mantissa alignment
- **count_leading_zeros** - Used internally for normalization

11.15 References

- Google Brain Float (BF16) specification
- IEEE 754-2019 Standard for Floating-Point Arithmetic
- Intel BFloat16 documentation
- NVIDIA TensorFloat documentation

- Muller, J.M. et al. “Handbook of Floating-Point Arithmetic.” Birkhauser, 2018.

11.16 Navigation

- [<- Back to RTLCommon Index](#)
- [<- Back to Main Documentation Index](#)

12 BF16 Exponent Adder

An exponent computation module for BF16 multiplication that handles bias subtraction, normalization adjustment, overflow/underflow detection, and special value identification.

12.1 Overview

The `math_bf16_exponent_adder` module computes the result exponent for BF16 multiplication using the formula: $\text{exp_result} = \text{exp_a} + \text{exp_b} - \text{bias} + \text{norm_adjust}$. It uses extended precision arithmetic to detect overflow and underflow conditions before they occur.

Key Features: - **Bias-compensated addition** - Handles 127 bias subtraction - **Normalization adjustment** - +1 when mantissa product ≥ 2.0 - **Overflow detection** - Flags when result > 254 - **Underflow detection** - Flags when result ≤ 0 - **Special case flags** - Identifies zero, infinity, and NaN inputs

12.2 Module Declaration

```
module math_bf16_exponent_adder(
    input logic [7:0] i_exp_a,           // 8-bit exponent A
    input logic [7:0] i_exp_b,           // 8-bit exponent B
    input logic      i_norm_adjust,      // +1 if mantissa needs
normalization
    output logic [7:0] ow_exp_out,        // 8-bit result exponent
    output logic      ow_overflow,        // Exponent overflow (result =
inf)
    output logic      ow_underflow,      // Exponent underflow (result =
zero)
    output logic      ow_a_is_zero,      // Input A has zero exponent
    output logic      ow_b_is_zero,      // Input B has zero exponent
    output logic      ow_a_is_inf,       // Input A has max exponent
    output logic      ow_b_is_inf,       // Input B has max exponent
    output logic      ow_a_is_nan,       // Input A has max exponent
(NaN check)
    output logic      ow_b_is_nan        // Input B has max exponent
(NaN check)
);
```

12.3 Ports

12.3.1 Inputs

Port	Width	Description
i_exp_a	8	Biased exponent of operand A
i_exp_b	8	Biased exponent of operand B
i_norm_adjust	1	+1 adjustment when mantissa product ≥ 2.0

12.3.2 Outputs

Port	Width	Description
ow_exp_out	8	Result exponent (saturated on overflow/underflow)
ow_overflow	1	1 if result exponent > 254 (infinity)
ow_underflow	1	1 if result exponent ≤ 0 (zero)
ow_a_is_zero	1	1 if exp_a == 0 (caller checks mantissa)
ow_b_is_zero	1	1 if exp_b == 0 (caller checks mantissa)
ow_a_is_inf	1	1 if exp_a == 255 (caller checks mantissa for NaN)
ow_b_is_inf	1	1 if exp_b == 255 (caller checks mantissa for NaN)
ow_a_is_nan	1	Same as ow_a_is_inf (actual NaN needs mantissa $\neq 0$)
ow_b_is_nan	1	Same as ow_b_is_inf (actual NaN needs mantissa $\neq 0$)

12.4 BF16 Exponent Background

12.4.1 BF16 Format

BF16: [15]=sign, [14:7]=exponent (8 bits, bias=127), [6:0]=mantissa (7 bits)

Exponent encoding:

$\text{exp} = 0$: Zero (if mantissa=0) or Subnormal (if mantissa!=0)
 $\text{exp} = 255$: Infinity (if mantissa=0) or NaN (if mantissa!=0)
 $\text{exp} = 1-254$: Normal number, actual exponent = $\text{exp} - 127$

12.4.2 Multiplication Exponent Formula

For floating-point multiplication:

```

result = A * B
        = (2(exp_a - 127) * mant_a) * (2(exp_b - 127) * mant_b)
        = 2(exp_a + exp_b - 254) * (mant_a * mant_b)

```

If mantissa product ≥ 2.0 , normalize by right-shifting mantissa and adding 1 to exponent:

```

        = 2(exp_a + exp_b - 254 + 1) * (mant_a * mant_b / 2)

```

Biased result exponent = $\text{exp}_a + \text{exp}_b - 127 + \text{norm_adjust}$

12.5 Functionality

12.5.1 Special Case Detection

// Zero exponent: input is zero or subnormal

```

assign ow_a_is_zero = (i_exp_a == 8'h00);
assign ow_b_is_zero = (i_exp_b == 8'h00);

```

// Max exponent: input is infinity or NaN

```

assign ow_a_is_inf = (i_exp_a == 8'hFF);
assign ow_b_is_inf = (i_exp_b == 8'hFF);

```

// NaN detection requires mantissa check (done by caller)

```

assign ow_a_is_nan = (i_exp_a == 8'hFF);
assign ow_b_is_nan = (i_exp_b == 8'hFF);

```

Note: The module reports $\text{exp}=255$ as both “inf” and “nan” - the caller must check the mantissa to distinguish.

12.5.2 Extended Precision Arithmetic

// Use 10-bit arithmetic to detect overflow/underflow before they happen

```

wire [9:0] w_exp_sum_raw;

```

// exp_a + exp_b + norm_adjust - 127

```

assign w_exp_sum_raw = {2'b0, i_exp_a} + {2'b0, i_exp_b} +
                       {9'b0, i_norm_adjust} - 10'd127;

```


Why 10 bits? - Maximum sum: $255 + 255 + 1 = 511$ (needs 9 bits) - After subtracting 127: $511 - 127 = 384$ (still fits in 9 bits) - Need 10th bit to detect negative results (underflow)

12.5.3 Overflow/Underflow Detection

```
// Underflow: result <= 0 (MSB set means negative, or result is exactly 0)
wire w_underflow_raw = w_exp_sum_raw[9] | (w_exp_sum_raw == 10'd0);

// Overflow: result > 254 (255 is reserved for inf/nan)
// Only valid if not underflow (negative values wrap to large positive)
wire w_overflow_raw = ~w_underflow_raw & (w_exp_sum_raw > 10'd254);

// Special cases override normal overflow/underflow
wire w_either_special = ow_a_is_inf | ow_b_is_inf | ow_a_is_zero | ow_b_is_zero;

assign ow_overflow = w_overflow_raw & ~w_either_special;
assign ow_underflow = w_underflow_raw & ~w_either_special;
```

Key insight: Must check underflow BEFORE overflow because negative values (underflow) appear as large positive values in unsigned arithmetic.

12.5.4 Result Saturation

```
always_comb begin
    if (ow_overflow) begin
        ow_exp_out = 8'hFF; // Saturate to infinity
    end else if (ow_underflow) begin
        ow_exp_out = 8'h00; // Saturate to zero
    end else begin
        ow_exp_out = w_exp_sum_raw[7:0]; // Normal result
    end
end
```

12.6 Usage Examples

12.6.1 Basic Usage

```
logic [7:0] exp_a, exp_b;
logic      norm_adjust;
logic [7:0] exp_result;
logic      overflow, underflow;
logic      a_zero, b_zero, a_inf, b_inf;
```

```
math_bf16_exponent_adder u_exp_add (
    .i_exp_a(exp_a),
```

```

        .i_exp_b(exp_b),
        .i_norm_adjust(norm_adjust),
        .ow_exp_out(exp_result),
        .ow_overflow(overflow),
        .ow_underflow(underflow),
        .ow_a_is_zero(a_zero),
        .ow_b_is_zero(b_zero),
        .ow_a_is_inf(a_inf),
        .ow_b_is_inf(b_inf),
        .ow_a_is_nan(),    // Use with mantissa check
        .ow_b_is_nan()
    );

```

12.6.2 In BF16 Multiplier

// Get normalization flag from mantissa multiplier

```

wire w_needs_norm;
math_bf16_mantissa_mult u_mant_mult
    (... , .ow_needs_norm(w_needs_norm), ...);

```

// Compute exponent

```

math_bf16_exponent_adder u_exp_add (
    .i_exp_a(i_a[14:7]),
    .i_exp_b(i_b[14:7]),
    .i_norm_adjust(w_needs_norm),
    .ow_exp_out(w_exp_result),
    .ow_overflow(w_exp_overflow),
    .ow_underflow(w_exp_underflow),
    ...
);

```

// Handle rounding-induced exponent overflow

```

wire w_mant_round_overflow = ...; // From mantissa rounding
wire [7:0] w_exp_final = w_mant_round_overflow ? (w_exp_result + 8'd1)
                                                : w_exp_result;
wire w_final_overflow = w_exp_overflow | (w_exp_final == 8'hFF);

```

12.6.3 Example Calculations

exp_a	exp_b	norm_adj	Calculation	Result	Status
127	127	0	127 + 127 - 127 + 0 = 127	127	Normal
127	127	1	127 + 127 - 127 + 1	128	Normal

exp_a	exp_b	norm_adj	Calculation	Result	Status
			= 128		
200	200	0	200 + 200 - 127 + 0	255	Overflow
			= 273		
50	50	0	50 + 50 - 127 + 0 = -27	0	Underflow
1	1	0	1 + 1 - 127 + 0 = -125	0	Underflow

12.7 Timing Characteristics

Path	Logic Depth
Special case detection	1 gate (comparator)
Exponent addition	2-3 gates (10-bit add)
Overflow/underflow detection	2 gates (compare + AND)
Result MUX	1 gate
Total	~5-7 gate levels

12.8 Performance Characteristics

12.8.1 Resource Utilization

Component	LUTs (est.)
10-bit adder	~15
Comparators	~10
Output MUX	~8
Special case flags	~4
Total	~35-40 LUTs

12.8.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Simple 10-bit arithmetic instead of full-width operations 2. **Wire complexity** - Minimal signal routing 3. **Logic depth** - Parallel overflow/underflow detection

12.9 Design Considerations

12.9.1 Why Override Flags for Special Cases?

When either input is zero or infinity, the overflow/underflow flags shouldn't trigger the normal saturation path: - **Zero * anything** -> Zero (handled by caller, not underflow) - **Infinity * anything** -> Infinity (handled by caller, not overflow) - **0 * Infinity** -> NaN (invalid operation, handled by caller)

The flags are suppressed to let the caller handle these cases explicitly.

12.9.2 Exponent Ranges

Biased exp	Meaning	Result
0	Zero/Subnormal	Treat as zero (FTZ)
1-254	Normal	Valid computation
255	Inf/NaN	Special handling

12.9.3 Order of Checks

The module checks underflow BEFORE overflow because: 1. Negative results (e.g., -25) appear as large positive values (e.g., 999) in unsigned math 2. Without underflow check first, -25 would incorrectly trigger overflow

12.10 Auto-Generated Code

This module is auto-generated by Python scripts: - **Generator**:

```
bin/rtl_generators/bf16/bf16_exponent_adder.py - Regenerate: PYTHONPATH=bin:
$PYTHONPATH python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv file manually.

12.11 Related Modules

- **math_bf16_mantissa_mult** - Companion mantissa handling
- **math_bf16_multiplier** - Complete BF16 multiplier using this module
- **math_bf16_fma** - BF16 FMA with similar exponent logic

12.12 References

- IEEE 754-2019 Standard for Floating-Point Arithmetic
- Google Brain Float (BF16) specification
- Muller, J.M. et al. "Handbook of Floating-Point Arithmetic." Birkhauser, 2018.

12.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

13 BF16 Extended Modules

Extended BF16 (Brain Float 16) modules for AI/ML accelerators, including activation functions, mathematical operations, comparison/selection, and format conversions.

13.1 Overview

Beyond the core BF16 arithmetic (adder, multiplier, FMA), this library provides a comprehensive set of modules for neural network inference and training. All modules follow IEEE 754-like conventions with Flush-to-Zero (FTZ) for subnormals.

Key Features: - **Piecewise linear approximations** for transcendental functions - **LUT-based implementations** for high accuracy where needed - **Consistent interface** across all activation functions - **NaN propagation** throughout all operations - **Auto-generated** from Python generators for consistency

13.2 Module Categories

13.2.1 Activation Functions

Standard neural network activation functions with BF16 inputs and outputs.

Module	Function	Description
math_bf16_relu	$\max(0, x)$	Rectified Linear Unit - passes positive values, zeros negative
math_bf16_leaky_relu	$\max(0.01x, x)$	Leaky ReLU - small gradient for negative values
math_bf16_gelu	$x * \Phi(x)$	Gaussian Error Linear Unit - smooth approximation
math_bf16_sigmoid	$1/(1+\exp(-x))$	Logistic function - output in (0, 1)
math_bf16_tanh	$(\exp(x)-\exp(-x))/(\exp(x)+\exp(-x))$	Hyperbolic tangent - output in (-1, 1)
math_bf16_silu	$x * \text{sigmoid}(x)$	Sigmoid Linear Unit (Swish)
math_bf16_softmax_8	$\exp(x_i)/\sum(\exp(x_j))$	8-input softmax normalization

13.2.1.1 Common Interface Pattern

```
module math_bf16_{activation} (  
    input logic [15:0] i_a,           // BF16 input  
    output logic [15:0] ow_result    // BF16 output  
);
```

13.2.1.2 Usage Example

// Apply ReLU activation to layer output

```
math_bf16_relu u_relu (  
    .i_a(layer_output),  
    .ow_result(activated_output)  
);
```

// Apply sigmoid for binary classification

```
math_bf16_sigmoid u_sigmoid (  
    .i_a(logit),  
    .ow_result(probability)  
);
```

13.2.2 Mathematical Operations

Extended math operations for neural network computations.

Module	Function	Description
math_bf16_exp2	2^x	Base-2 exponential (LUT + interpolation)
math_bf16_log2	$\log_2(x)$	Base-2 logarithm
math_bf16_log2_scale	$\log_2(x) + \text{scale}$	Scaled logarithm for normalization
math_bf16_reciprocal	$1/x$	Full-precision reciprocal
math_bf16_fast_reciprocal	$\sim 1/x$	LUT-based fast approximation
math_bf16_divider	a/b	Full BF16 division
math_bf16_goldschmidt_div	a/b	Iterative Goldschmidt division
math_bf16_newton_raphson_recip	$1/x$	Newton-Raphson iterative reciprocal

13.2.2.1 Reciprocal/Division Modules

// Fast reciprocal (single-cycle, LUT-based)

```
module math_bf16_fast_reciprocal #(  
    // ...
```

```

    parameter int LUT_DEPTH = 128    // 32, 64, or 128 entries
) (
    input  logic [15:0] i_a,
    output logic [15:0] ow_result,
    output logic        ow_overflow,
    output logic        ow_underflow,
    output logic        ow_invalid,
    output logic        ow_div_by_zero
);

// Goldschmidt division (iterative, higher accuracy)
module math_bf16_goldschmidt_div #(
    parameter int ITERATIONS = 2,    // 1 or 2 iterations
    parameter int PIPELINED = 1,    // 0=combinational, 1=pipelined
    parameter int LUT_DEPTH = 128    // Initial estimate LUT size
) (
    input  logic        i_clk,
    input  logic        i_rst_n,
    input  logic [15:0] i_a,          // Dividend
    input  logic [15:0] i_b,          // Divisor
    input  logic        i_valid,
    output logic [15:0] ow_result,
    output logic        ow_valid,
    output logic        ow_overflow,
    output logic        ow_underflow,
    output logic        ow_invalid,
    output logic        ow_div_by_zero
);

```

13.2.3 Comparison and Selection

Modules for comparing and selecting BF16 values.

Module	Function	Description
math_bf16_comparator	$a \leq b$	Full comparison with flags
math_bf16_max	$\max(a, b)$	Two-input maximum
math_bf16_min	$\min(a, b)$	Two-input minimum
math_bf16_max_tree	$\max(a[])$	N-input maximum tree
math_bf16_max_tree_8	$\max(a[7:0])$	8-input maximum
math_bf16_min_tree_8	$\min(a[7:0])$	8-input minimum

Module	Function	Description
math_bf16_clamp	clamp(x, lo, hi)	Value clamping to range

13.2.3.1 Comparator Interface

```

module math_bf16_comparator (
    input  logic [15:0] i_a,
    input  logic [15:0] i_b,
    output logic      ow_eq,      // a == b
    output logic      ow_lt,      // a < b
    output logic      ow_gt,      // a > b
    output logic      ow_unordered // NaN comparison
);

```

13.2.3.2 Max/Min Tree Usage

```

// Find maximum across 8 values (e.g., for pooling)
logic [15:0] pool_inputs [8];
logic [15:0] pool_max;

math_bf16_max_tree_8 u_max_pool (
    .i_values(pool_inputs),
    .ow_max(pool_max)
);

```

13.2.4 Format Conversions

Convert between BF16 and other floating-point formats.

Module	Conversion	Description
math_bf16_to_fp16	BF16 -> FP16	Requires mantissa rounding
math_bf16_to_fp32	BF16 -> FP32	Lossless upconversion
math_bf16_to_fp8_e4m3	BF16 -> FP8 E4M3	ML inference format
math_bf16_to_fp8_e5m2	BF16 -> FP8 E5M2	ML training format
math_bf16_to_int	BF16 -> INT	Fixed-point conversion
math_int_to_bf16	INT -> BF16	Integer to float conversion
math_bf16_scale_to_int8	BF16 -> INT8	Quantization with

Module	Conversion	Description
		scaling

13.2.4.1 Conversion Interface Pattern

```

module math_bf16_to_{format} (
    input   logic [15:0] i_a,           // BF16 input
    output logic [N:0] ow_result,      // Target format output
    output logic      ow_overflow,     // Value too large
    output logic      ow_underflow,   // Value too small (rounds to 0)
    output logic      ow_invalid      // NaN input
);

```

13.2.4.2 Quantization Example

```

// Convert BF16 weights to INT8 for inference
logic [15:0] bf16_weight;
logic [7:0]  int8_weight;
logic [15:0] scale_factor; // Pre-computed scale

```

```

math_bf16_scale_to_int8 u_quantize (
    .i_value(bf16_weight),
    .i_scale(scale_factor),
    .ow_result(int8_weight),
    .ow_overflow(quant_overflow),
    .ow_underflow()
);

```

13.3 Special Value Handling

All modules follow consistent special value handling:

Input	ReLU	Sigmoid	Max/Min	Converters
+0	+0	0.5	Compares as 0	Format-specific 0
-0	+0	0.5	Compares as 0	Format-specific 0
+Inf	+Inf	1.0	Largest	Overflow flag
-Inf	0	0.0	Smallest	Underflow flag
NaN	NaN	NaN	Propagates	Invalid flag
Subnormal	FTZ	FTZ	FTZ	FTZ

13.4 Implementation Notes

13.4.1 Piecewise Linear Approximations

Transcendental functions (sigmoid, tanh, GELU) use piecewise linear approximation:

Sigmoid approximation regions:

```
x <= -4:      sigmoid(x) = 0
-4 < x < 0:   linear interpolation
0 <= x < 4:   linear interpolation
x >= 4:      sigmoid(x) = 1
```

This provides ~1-2% accuracy while maintaining single-cycle latency.

13.4.2 LUT-Based Operations

For higher accuracy (exp2, log2, reciprocal), LUT + linear interpolation is used:

```
// LUT stores function values at regular intervals
// Linear interpolation between adjacent entries
// Configurable LUT depth: 32, 64, or 128 entries
// Accuracy: ~0.1% for 128-entry LUT
```

13.5 Auto-Generation

All modules are auto-generated for consistency:

```
# Regenerate all BF16 modules
PYTHONPATH=bin:$PYTHONPATH python3
bin/rtl_generators/ieee754/generate_all.py rtl/common
```

Generator files: - bin/rtl_generators/ieee754/fp_activations.py - Activation functions -
bin/rtl_generators/ieee754/fp_conversions.py - Format converters -
bin/rtl_generators/ieee754/fp_comparisons.py - Comparison modules -
bin/rtl_generators/ieee754/bf16_reciprocal.py - Reciprocal/division

13.6 Related Documentation

- [math_bf16_multiplier](#) - Core BF16 multiplication
- [math_bf16_adder](#) - Core BF16 addition
- [math_bf16_fma](#) - Fused Multiply-Add
- [math_fp16_modules](#) - FP16 equivalents
- [math_fp8_modules](#) - FP8 E4M3/E5M2 modules

13.7 Navigation

- [Back to RTLCommon Index](#)
- [Back to Main Documentation Index](#)

14 BF16 Fused Multiply-Add (FMA)

A BF16 Fused Multiply-Add unit with FP32 accumulator, designed for AI training workloads where maintaining precision during accumulation is critical.

14.1 Overview

The `math_bf16_fma` module implements $\text{result} = (A * B) + C$ as a single fused operation where A and B are BF16 inputs and C is an FP32 accumulator. This matches the computation pattern used in neural network training where weights and activations are BF16 but accumulated gradients need FP32 precision.

Key Features: - **Mixed precision** - BF16 inputs (A, B), FP32 accumulator (C), FP32 output - **Single fused operation** - No intermediate rounding between multiply and add - **48-bit internal precision** - Captures full product + alignment bits - **IEEE 754 special cases** - Zero, infinity, NaN handling - **RNE rounding** - Round-to-Nearest-Even for unbiased results - **FTZ mode** - Flush-to-Zero for subnormal inputs

14.2 Module Declaration

```
module math_bf16_fma(  
    input logic [15:0] i_a,           // BF16 multiplicand  
    input logic [15:0] i_b,           // BF16 multiplier  
    input logic [31:0] i_c,           // FP32 addend (accumulator)  
    output logic [31:0] ow_result,    // FP32 result  
    output logic      ow_overflow,    // Overflow to infinity  
    output logic      ow_underflow,   // Underflow to zero  
    output logic      ow_invalid     // Invalid operation  
);
```

14.3 Ports

Port	Direction	Width	Description
i_a	Input	16	BF16 multiplicand A
i_b	Input	16	BF16 multiplier B
i_c	Input	32	FP32 addend C

Port	Direction	Width	Description
			(accumulator)
ow_result	Output	32	FP32 result = A * B + C
ow_overflow	Output	1	1 if result overflowed to infinity
ow_underflow	Output	1	1 if result underflowed to zero
ow_invalid	Output	1	1 if operation was invalid

14.4 Format Specifications

14.4.1 BF16 Format

BF16: [15]=sign, [14:7]=exponent (8 bits, bias=127), [6:0]=mantissa (7 bits)

Special values:

```
+0:    0x0000 (sign=0, exp=0, mant=0)
-0:    0x8000 (sign=1, exp=0, mant=0)
+inf:  0x7F80 (sign=0, exp=255, mant=0)
-inf:  0xFF80 (sign=1, exp=255, mant=0)
NaN:   0x7FC0 (exp=255, mant!=0)
```

14.4.2 FP32 Format

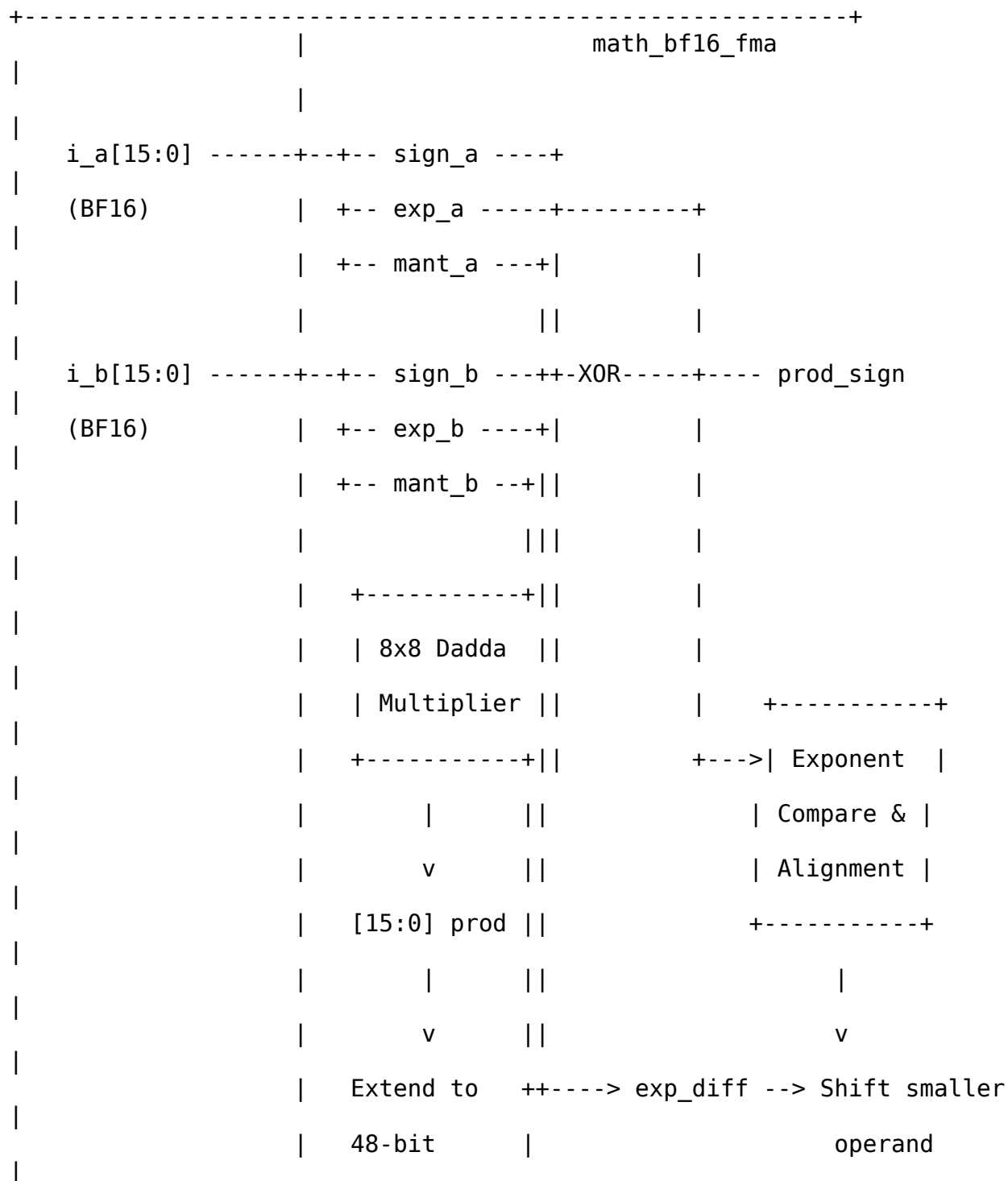
FP32: [31]=sign, [30:23]=exponent (8 bits, bias=127), [22:0]=mantissa (23 bits)

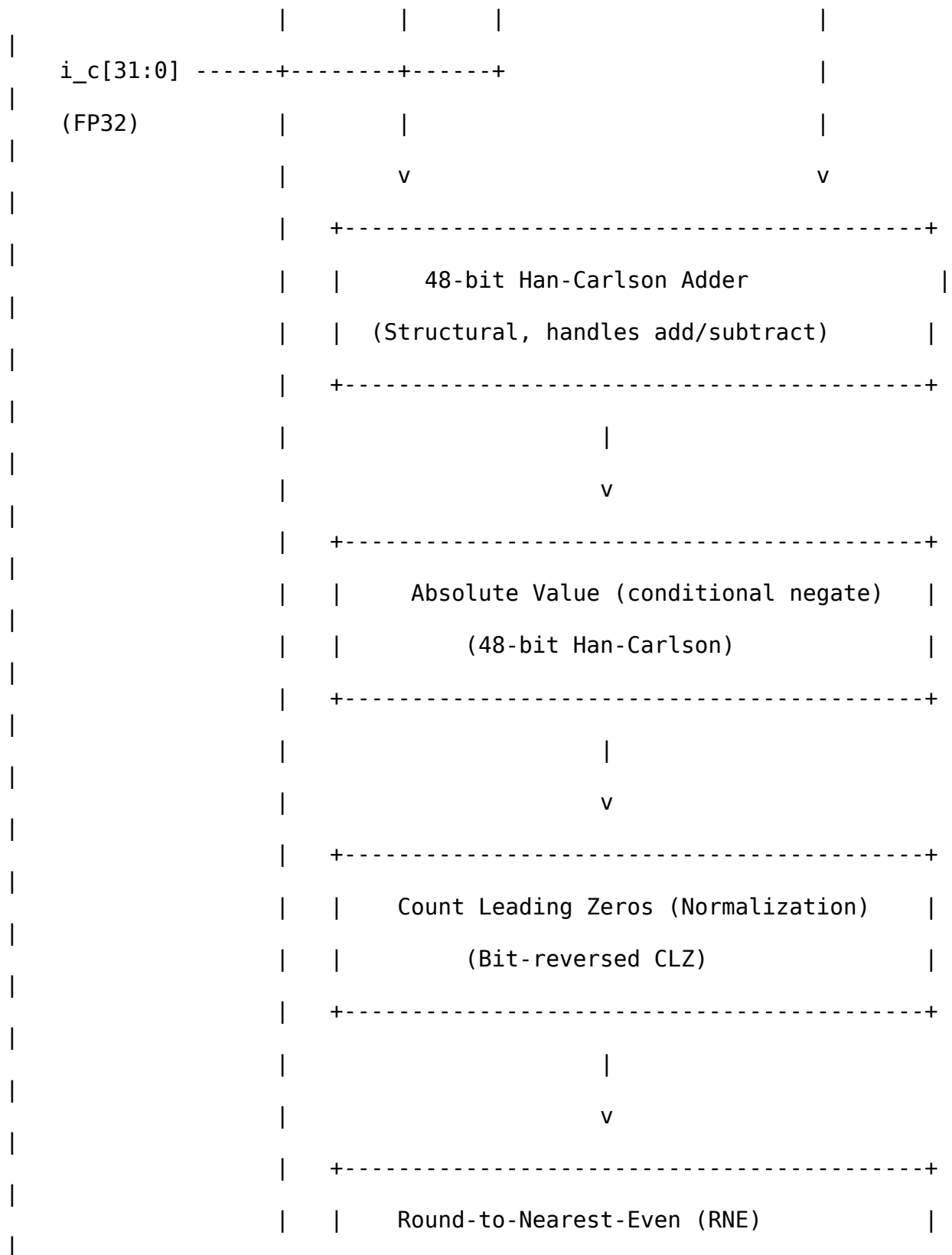
Special values:

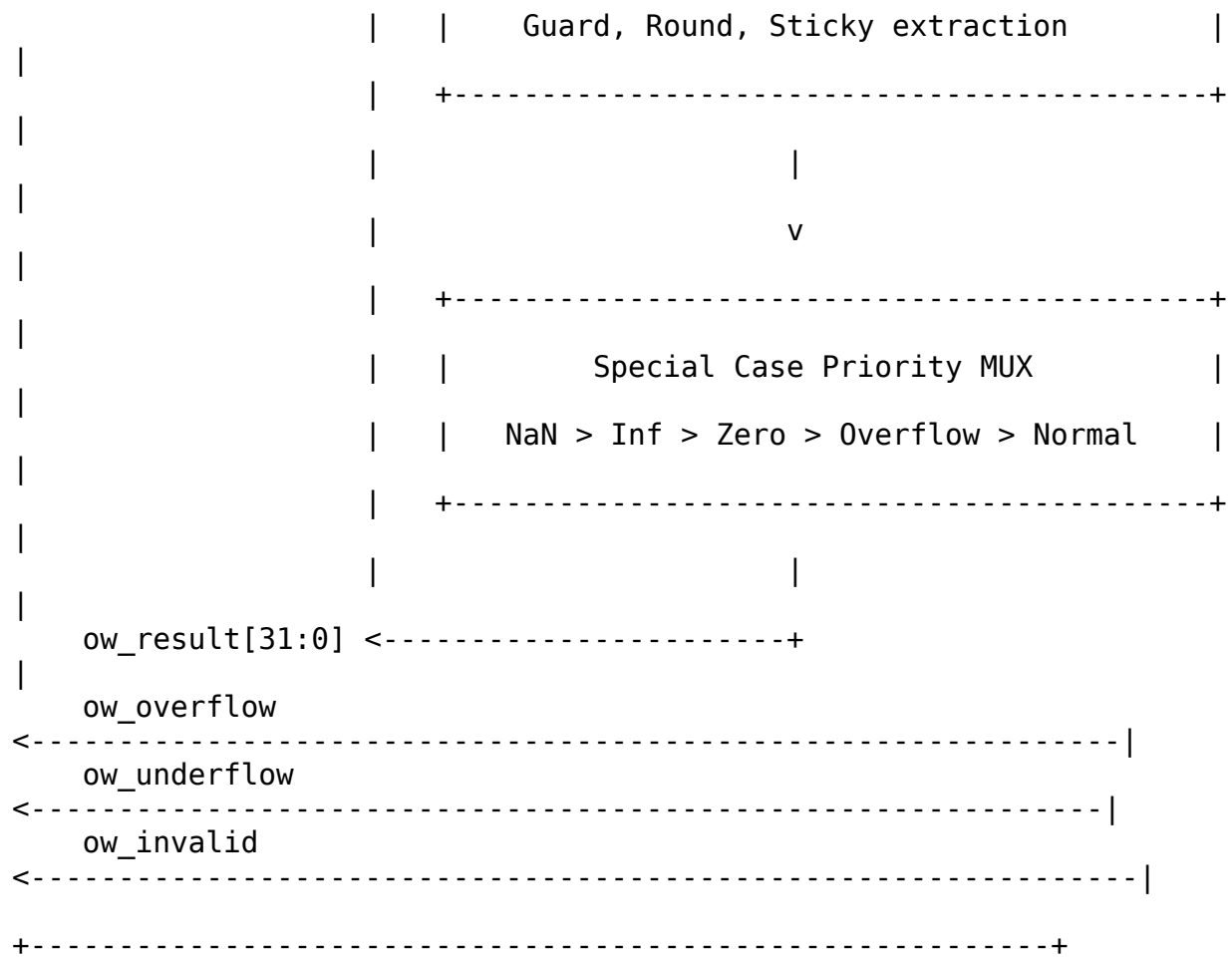
```
+0:    0x00000000
-0:    0x80000000
+inf:  0x7F800000
-inf:  0xFF800000
qNaN:  0x7FC00000 (canonical quiet NaN)
```

14.5 Architecture

14.5.1 Block Diagram







14.5.2 Processing Pipeline

1. **Field Extraction** - Parse sign, exponent, mantissa from BF16 and FP32 inputs
2. **Special Case Detection** - Identify zero, subnormal, infinity, NaN for all inputs
3. **Product Computation** - 8x8 Dadda multiply for mantissas, exponent addition with bias correction
4. **Alignment** - Compare exponents, shift smaller operand to align
5. **Wide Addition** - 48-bit Han-Carlson add/subtract based on effective sign
6. **Absolute Value** - Conditional negate if subtraction result is negative
7. **Normalization** - Count leading zeros, shift left, adjust exponent
8. **RNE Rounding** - Extract guard/round/sticky, apply rounding
9. **Result Assembly** - Select output based on special case priority

14.6 Functionality

14.6.1 Mixed Precision Computation

```
// BF16 product extended to 48-bit internal precision
wire [7:0] w_mant_a_ext = {w_a_is_normal, w_mant_a}; // 8-bit with
implied 1
wire [7:0] w_mant_b_ext = {w_b_is_normal, w_mant_b}; // 8-bit with
implied 1

// 8x8 -> 16-bit product, then extended to 48 bits
wire [15:0] w_prod_mant_raw;
wire [47:0] w_prod_mant_48 = {w_prod_mant_ext, 24'b0};

// FP32 addend extended to 48 bits
wire [23:0] w_c_mant_ext = w_c_is_normal ? {1'b1, w_mant_c} : 24'h0;
wire [47:0] w_c_mant_48 = {w_c_mant_ext, 24'b0};
```

14.6.2 Exponent Alignment

```
// Signed exponent difference for alignment
wire signed [10:0] w_exp_diff = $signed({1'b0, w_prod_exp}) -
                                $signed({1'b0, w_exp_c_ext});

// Shift smaller operand to align mantissas
wire [5:0] w_shift_clamped = (w_shift_amt > 10'd48) ? 6'd48 :
w_shift_amt[5:0];
wire [47:0] w_mant_smaller_shifted = ... >> w_shift_clamped;
```

14.6.3 Effective Addition/Subtraction

```
// Effective operation based on signs
wire w_effective_sub = w_sign_larger ^ w_sign_smaller;

// Two's complement for subtraction
wire [47:0] w_adder_b = w_effective_sub ? ~w_mant_smaller_shifted
: w_mant_smaller_shifted;

// 48-bit structural addition
math_adder_han_carlson_048 u_wide_adder (
    .i_a(w_mant_larger),
    .i_b(w_adder_b),
    .i_cin(w_effective_sub), // +1 for two's complement
    .ow_sum(w_adder_sum),
    .ow_cout(w_adder_cout)
);
```


14.6.4 Normalization with CLZ

```
// Bit-reverse for leading zeros (CLZ module counts trailing zeros)
```

```
wire [47:0] w_sum_abs_reversed;
```

```
generate
```

```
    for (i = 0; i < 48; i = i + 1) begin : gen_bit_reverse
```

```
        assign w_sum_abs_reversed[i] = w_sum_abs[47 - i];
```

```
    end
```

```
endgenerate
```

```
count_leading_zeros #(.WIDTH(48)) u_clz (
```

```
    .data(w_sum_abs_reversed),
```

```
    .clz(w_lz_count_raw)
```

```
);
```

```
// Normalize: shift left, adjust exponent
```

```
wire [47:0] w_mant_normalized = w_sum_abs << w_lz_count;
```

```
wire signed [10:0] w_exp_adjusted = w_exp_result_pre - w_lz_count_raw  
+ w_add_overflow;
```

14.6.5 Special Case Priority

```
always_comb begin
```

```
    // Priority order (highest to lowest):
```

```
    if (w_any_nan | w_invalid) begin
```

```
        // 1. NaN: any NaN input, or 0*inf, or inf-inf
```

```
        ow_result = {1'b0, 8'hFF, 23'h400000}; // Canonical qNaN
```

```
        ow_invalid = w_invalid;
```

```
    end else if (w_prod_is_inf & ~w_c_is_inf) begin
```

```
        // 2. Product infinity
```

```
        ow_result = {w_prod_sign, 8'hFF, 23'h0};
```

```
    end else if (w_c_is_inf) begin
```

```
        // 3. Addend infinity
```

```
        ow_result = {w_sign_c, 8'hFF, 23'h0};
```

```
    end else if (w_prod_is_zero) begin
```

```
        // 4. Zero product: pass-through addend
```

```
        ow_result = i_c;
```

```
    end else if (w_c_eff_zero) begin
```

```
        // 5. Zero addend: product only
```

```
        ow_result = {w_prod_sign, w_prod_exp[7:0],
```

```
        w_prod_mant_ext[22:0]};
```

```
    end else if (w_overflow_cond) begin
```

```
        // 6. Overflow to infinity
```

```
        ow_result = {w_result_sign, 8'hFF, 23'h0};
```

```
        ow_overflow = 1'b1;
```

```
    end else if (w_underflow_cond) begin
```

```
        // 7. Underflow to zero
```

```
        ow_result = {w_result_sign, 8'h00, 23'h0};
```

```
        ow_underflow = 1'b1;
```

```

    end
    // Default: normal result
end

```

14.7 Usage Examples

14.7.1 Basic FMA Operation

```

logic [15:0] a, b;           // BF16 inputs
logic [31:0] c;             // FP32 accumulator
logic [31:0] result;
logic overflow, underflow, invalid;

```

```

math_bf16_fma u_fma (
    .i_a(a),
    .i_b(b),
    .i_c(c),
    .ow_result(result),
    .ow_overflow(overflow),
    .ow_underflow(underflow),
    .ow_invalid(invalid)
);

```

```

// Example: 2.0 * 3.0 + 1.0 = 7.0
// 2.0 in BF16: 0x4000
// 3.0 in BF16: 0x4040
// 1.0 in FP32: 0x3F800000
initial begin
    a = 16'h4000;           // 2.0
    b = 16'h4040;           // 3.0
    c = 32'h3F800000;       // 1.0
    #1;
    // result should be ~7.0 (0x40E00000)
end

```

14.7.2 Neural Network Dot Product

```

// Typical AI training pattern: accumulate weight * activation
// products
logic [15:0] weights[0:N-1]; // BF16 weights
logic [15:0] activations[0:N-1]; // BF16 activations
logic [31:0] accumulator; // FP32 accumulator
logic [31:0] fma_result;

```

```

math_bf16_fma u_fma (
    .i_a(weights[idx]),
    .i_b(activations[idx]),
    .i_c(accumulator),

```

```

        .ow_result(fma_result),
        .ow_overflow(overflow),
        .ow_underflow(underflow),
        .ow_invalid(invalid)
    );

    // Accumulation loop
    always_ff @(posedge clk) begin
        if (start)
            accumulator <= 32'h0; // Initialize to zero
        else if (valid)
            accumulator <= fma_result; // Accumulate
    end

```

14.7.3 With Pipeline Register

```

    // FMA is purely combinational - pipeline as needed for timing
    logic [15:0] a_reg, b_reg;
    logic [31:0] c_reg, result_reg;

    always_ff @(posedge clk) begin
        // Input register stage
        a_reg <= a;
        b_reg <= b;
        c_reg <= c;
    end

    math_bf16_fma u_fma (
        .i_a(a_reg),
        .i_b(b_reg),
        .i_c(c_reg),
        .ow_result(result_comb),
        ...
    );

    always_ff @(posedge clk) begin
        // Output register stage
        result_reg <= result_comb;
    end

```

14.8 Timing Characteristics

Stage	Logic Depth
Field extraction	0 (wiring)
Special case detection	2 gates
8x8 Dadda multiply	~15 gates

Stage	Logic Depth
Exponent compare/align	~5 gates
48-bit Han-Carlson add	~8 stages
Absolute value (48-bit add)	~8 stages
CLZ + normalization shift	~6 gates
RNE rounding	~3 gates
Result MUX	~3 gates
Total	~40-50 gates

14.9 Performance Characteristics

14.9.1 Resource Utilization

Component	LUTs (est.)
8x8 Dadda multiplier	~140
48-bit Han-Carlson adder (main)	~250
48-bit Han-Carlson adder (negate)	~250
CLZ (48-bit)	~50
Alignment shifter	~100
Rounding logic	~30
Special case MUX	~50
Total	~850-900 LUTs

14.9.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Efficient submodule reuse (Dadda multiplier, Han-Carlson adders) 2. **Wire complexity** - Clean datapath with aligned internal widths 3. **Logic depth** - Parallel operations where possible

14.10 Design Considerations

14.10.1 Why FP32 Accumulator?

AI training requires maintaining precision during gradient accumulation: - BF16 has only 7 mantissa bits (8-bit precision) - Accumulating thousands of small values loses precision - FP32 accumulator provides 23 mantissa bits - Result can be truncated back to BF16 for storage if needed

14.10.2 Fusion Benefits

Fused operation provides better accuracy than separate multiply-then-add: - **Unfused:** $\text{temp} = A * B$ (rounded), $\text{result} = \text{temp} + C$ (rounded) - 2 rounding errors - **Fused:** $\text{result} = A * B + C$ (single rounding) - 1 rounding error - Extra precision is maintained throughout the pipeline

14.10.3 Product Underflow Handling

When the BF16 product exponent is too small:

```
wire w_prod_underflow = w_prod_exp_adj[10] | (w_prod_exp_adj < 11'sd1);  
wire w_prod_is_zero = w_a_eff_zero | w_b_eff_zero | w_prod_underflow;
```

Product underflow causes pass-through of the addend C.

14.10.4 Invalid Operation Cases

Invalid operations produce NaN: 1. **0 * Infinity** - Indeterminate form 2. **Infinity - Infinity** - Indeterminate form (same magnitude, opposite signs)

```
wire w_invalid_mul = (w_a_eff_zero & w_b_is_inf) | (w_b_eff_zero & w_a_is_inf);  
wire w_invalid_add = w_prod_is_inf & w_c_is_inf & (w_prod_sign != w_sign_c);
```

14.11 Common Pitfalls

Anti-Pattern 1: Using BF16 accumulator

```
// WRONG: Precision loss in accumulation  
logic [15:0] bf16_accum; // Only 7 mantissa bits!
```

```
// RIGHT: Use FP32 accumulator  
logic [31:0] fp32_accum; // 23 mantissa bits
```

Anti-Pattern 2: Ignoring special cases

```
// WRONG: Assuming result is always valid  
output_reg <= result;
```

```
// RIGHT: Check status flags  
if (invalid)  
    handle_nan();  
else if (overflow)  
    handle_infinity();  
else  
    output_reg <= result;
```

Anti-Pattern 3: Expecting subnormal outputs

```
// NOTE: FTZ mode flushes tiny results to zero  
// If gradients become very small, they flush to zero  
// This is intentional for AI training efficiency
```

14.12 Auto-Generated Code

This module is auto-generated by Python scripts: - **Generator:**

```
bin/rtl_generators/bf16/bf16_fma.py - Regenerate: PYTHONPATH=bin:$PYTHONPATH  
python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv file manually.

14.13 Related Modules

- **math_multiplier_dadda_4to2_008** - 8x8 Dadda multiplier used for mantissa
- **math_adder_han_carlson_048** - 48-bit adder for wide addition
- **count_leading_zeros** - Normalization support
- **math_bf16_multiplier** - Standalone BF16 multiplier (simpler, no accumulate)
- **math_bf16_mantissa_mult** - Mantissa multiplication submodule
- **math_bf16_exponent_adder** - Exponent computation submodule

14.14 References

- Google Brain Float (BF16) specification
- IEEE 754-2019 Standard for Floating-Point Arithmetic
- Intel BFloat16 documentation
- NVIDIA TensorFloat documentation
- Muller, J.M. et al. “Handbook of Floating-Point Arithmetic.” Birkhauser, 2018.

14.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

15 BF16 Mantissa Multiplier

A specialized mantissa multiplier for BF16 (Brain Float 16) format that handles implied leading 1, normalization detection, and rounding bit extraction for IEEE 754-compliant floating-point multiplication.

15.1 Overview

The `math_bf16_mantissa_mult` module wraps an 8x8 Dadda multiplier with BF16-specific logic for handling the implicit leading bit, detecting when normalization is needed, and extracting guard/round/sticky bits for Round-to-Nearest-Even (RNE) rounding.

Key Features: - **8x8 unsigned multiply** - 7-bit mantissa + 1 implied bit - **Normalization detection** - Flags when product ≥ 2.0 - **Mantissa extraction** - Selects correct 7 bits based on normalization - **Rounding support** - Provides round and sticky bits for RNE - **FTZ mode** - Handles subnormal inputs as zero

15.2 Module Declaration

```
module math_bf16_mantissa_mult(
    input logic [6:0] i_mant_a,      // 7-bit explicit mantissa A
    input logic [6:0] i_mant_b,      // 7-bit explicit mantissa B
    input logic        i_a_is_normal, // A has implied leading 1
    input logic        i_b_is_normal, // B has implied leading 1
    output logic [15:0] ow_product,   // 16-bit raw product
    output logic        ow_needs_norm, // 1 if product  $\geq 2.0$ 
    output logic [6:0] ow_mant_out,   // 7-bit result mantissa
    output logic        ow_round_bit, // Round bit for RNE
    output logic        ow_sticky_bit // Sticky bit for RNE
);
```

15.3 Ports

15.3.1 Inputs

Port	Width	Description
i_mant_a	7	Explicit mantissa of operand A (bits [6:0] of BF16)
i_mant_b	7	Explicit mantissa of operand B (bits [6:0] of BF16)
i_a_is_normal	1	1 if A is normalized (has implied leading

Port	Width	Description
i_b_is_normal	1	1) 1 if B is normalized (has implied leading 1)

15.3.2 Outputs

Port	Width	Description
ow_product	16	Full 16-bit raw product from 8x8 multiply
ow_needs_norm	1	1 if product ≥ 2.0 (MSB of product is 1)
ow_mant_out	7	Pre-rounded 7-bit mantissa result
ow_round_bit	1	Round bit for RNE rounding
ow_sticky_bit	1	Sticky bit for RNE rounding

15.4 BF16 Format Background

15.4.1 BF16 Structure

BF16: [15]=sign, [14:7]=exponent (8 bits), [6:0]=mantissa (7 bits)

For normalized numbers ($\text{exp} \neq 0$ and $\text{exp} \neq 255$):
 $\text{Value} = (-1)^{\text{sign}} * 2^{(\text{exp}-127)} * 1.\text{mantissa}$

The "1." is implicit (not stored), so actual mantissa is 8 bits: {1, mantissa[6:0]}

15.4.2 Mantissa Multiplication

When multiplying two normalized BF16 numbers: - Operand A mantissa: 1.aaaaaaa (8 bits with implicit 1) - Operand B mantissa: 1.bbbbbbb (8 bits with implicit 1) - Product: 16-bit result in range [1.0, 4.0)

Product format: - If product ≥ 2.0 : 1x.xxxxxx_xxxxxxx (needs right shift + exp++) - If product < 2.0 : 01.xxxxxx_xxxxxxx (no shift needed)

15.5 Functionality

15.5.1 Mantissa Extension

```
// Extend 7-bit explicit mantissa with implied leading 1
wire [7:0] w_mant_a_ext = {i_a_is_normal, i_mant_a};
wire [7:0] w_mant_b_ext = {i_b_is_normal, i_mant_b};
```

For normal numbers, this creates 1.mantissa. For zeros/subnormals (FTZ mode), creates 0.mantissa which effectively treats them as zero.

15.5.2 8x8 Multiplication

```
math_multiplier_dadda_4to2_008 u_mult (
    .i_multiplier(w_mant_a_ext),
    .i_multiplicand(w_mant_b_ext),
    .ow_product(ow_product)
);
```

15.5.3 Normalization Detection

```
// MSB set means product >= 2.0, needs right shift
assign ow_needs_norm = ow_product[15];
```

15.5.4 Mantissa Extraction

```
// Extract 7-bit mantissa based on normalization state
assign ow_mant_out = ow_needs_norm ? ow_product[14:8] // Shifted right
                                     : ow_product[13:7]; // No shift
```

Product bit mapping:

Product format	Implied 1	Mantissa bits	Guard	Round	Sticky
Needs norm (MSB=1)	[15]	[14:8]	[7]	[6]	[5:0]
No norm (MSB=0)	[14]	[13:7]	[6]	[5]	[4:0]

15.5.5 Rounding Bits for RNE

```
// Guard, round, sticky for RNE rounding
wire w_guard_norm      = ow_product[7];
wire w_guard_nonorm    = ow_product[6];

wire w_round_norm     = ow_product[6];
wire w_round_nonorm   = ow_product[5];
```

```

wire w_sticky_norm    = ow_product[5:0];
wire w_sticky_nonorm = ow_product[4:0];

assign ow_round_bit = ow_needs_norm ? w_round_norm : w_round_nonorm;
assign ow_sticky_bit = ow_needs_norm ?
    (w_guard_norm | w_sticky_norm) : (w_guard_nonorm |
w_sticky_nonorm);

```

Note: The sticky bit includes the guard bit because after mantissa extraction, the guard becomes part of the “remaining” bits for rounding purposes.

15.6 Usage Examples

15.6.1 Basic Usage

```

logic [6:0] mant_a, mant_b;
logic      a_normal, b_normal;
logic [15:0] product;
logic      needs_norm;
logic [6:0] mant_result;
logic      round_bit, sticky_bit;

math_bf16_mantissa_mult u_mant_mult (
    .i_mant_a(mant_a),
    .i_mant_b(mant_b),
    .i_a_is_normal(a_normal),
    .i_b_is_normal(b_normal),
    .ow_product(product),
    .ow_needs_norm(needs_norm),
    .ow_mant_out(mant_result),
    .ow_round_bit(round_bit),
    .ow_sticky_bit(sticky_bit)
);

```

15.6.2 In BF16 Multiplier

```

// Extract fields from BF16 inputs
wire [6:0] w_mant_a = i_a[6:0];
wire [6:0] w_mant_b = i_b[6:0];
wire      w_a_is_normal = (i_a[14:7] != 8'h00) & (i_a[14:7] !=
8'hFF);
wire      w_b_is_normal = (i_b[14:7] != 8'h00) & (i_b[14:7] !=
8'hFF);

// Mantissa multiplication
math_bf16_mantissa_mult u_mant_mult (
    .i_mant_a(w_mant_a),

```

```

        .i_mant_b(w_mant_b),
        .i_a_is_normal(w_a_is_normal),
        .i_b_is_normal(w_b_is_normal),
        .ow_product(w_product),
        .ow_needs_norm(w_needs_norm),
        .ow_mant_out(w_mant_mult_out),
        .ow_round_bit(w_round_bit),
        .ow_sticky_bit(w_sticky_bit)
    );

// Apply RNE rounding
wire w_lsb = w_mant_mult_out[0];
wire w_round_up = w_round_bit & (w_sticky_bit | w_lsb);
wire [7:0] w_mant_rounded = {1'b0, w_mant_mult_out} + {7'b0,
w_round_up};

// Check for mantissa overflow from rounding
wire w_mant_overflow = w_mant_rounded[7];
wire [6:0] w_mant_final = w_mant_overflow ? 7'h00 :
w_mant_rounded[6:0];

```

15.7 Timing Characteristics

Stage	Logic Depth
Mantissa extension	0 (just wiring)
8x8 Dadda multiply	~13-15 gates
Normalization detection	1 gate (wire to MSB)
Mantissa MUX	1 gate
Round/sticky OR	1 gate
Total	~15-17 gates

15.8 Performance Characteristics

15.8.1 Resource Utilization

Component	Resource
8x8 Multiplier	~120-140 LUTs
Normalization MUX	~14 LUTs
Rounding logic	~10 LUTs
Total	~140-165 LUTs

15.8.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Minimal overhead over raw multiplier 2. **Wire complexity** - Simple MUXing for normalization 3. **Logic depth** - Normalization adds only 1 gate level

15.9 Design Considerations

15.9.1 Flush-to-Zero (FTZ) Mode

Subnormal inputs ($\text{exp}=0$, $\text{mantissa}\neq 0$) are treated as zero: - `i_a_is_normal` or `i_b_is_normal` will be 0 - Extended mantissa becomes `{0, mantissa}` instead of `{1, mantissa}` - Product will be effectively zero

This matches standard BF16 behavior in AI accelerators.

15.9.2 Why Separate from Multiplier?

This wrapper is separate from the raw multiplier because: 1. **Reusability** - Raw multiplier can be used elsewhere 2. **Testability** - Each component can be tested independently 3. **Clarity** - BF16-specific logic is isolated

15.9.3 Round-to-Nearest-Even

The rounding bits support IEEE 754 RNE:

Round up if: Guard AND (Round OR Sticky OR LSB)

This ensures ties (exactly halfway) round to even.

15.10 Auto-Generated Code

This module is auto-generated by Python scripts: - **Generator**:

```
bin/rtl_generators/bf16/bf16_mantissa_mult.py - Regenerate: PYTHONPATH=bin:
$PYTHONPATH python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv file manually.

15.11 Related Modules

- **math_multiplier_dadda_4to2_008** - Underlying 8x8 multiplier
- **math_bf16_exponent_adder** - Companion exponent handling
- **math_bf16_multiplier** - Complete BF16 multiplier using this module
- **math_bf16_fma** - BF16 FMA also uses Dadda multiplier directly

15.12 References

- Google Brain Float (BF16) specification
- IEEE 754-2019 Standard for Floating-Point Arithmetic
- Muller, J.M. et al. “Handbook of Floating-Point Arithmetic.” Birkhauser, 2018.

15.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

16 BF16 Multiplier

A complete BF16 (Brain Float 16) multiplier with IEEE 754-compliant special case handling, Round-to-Nearest-Even rounding, and flush-to-zero support for subnormal numbers.

16.1 Overview

The `math_bf16_multiplier` module implements full BF16 multiplication by integrating the mantissa multiplier, exponent adder, and special case handling logic. It follows industry-standard practices used in AI/ML accelerators for efficient 16-bit floating-point operations.

Key Features: - **BF16 format** - Same exponent range as FP32, reduced mantissa precision - **IEEE 754 special cases** - Zero, infinity, NaN handling - **RNE rounding** - Round-to-Nearest-Even for unbiased results - **FTZ mode** - Flush-to-Zero for subnormal inputs and outputs - **Status flags** - Overflow, underflow, and invalid operation indicators

16.2 Module Declaration

```
module math_bf16_multiplier(  
    input logic [15:0] i_a,           // BF16 operand A  
    input logic [15:0] i_b,           // BF16 operand B  
    output logic [15:0] ow_result,    // BF16 product  
    output logic        ow_overflow,  // Overflow to infinity  
    output logic        ow_underflow, // Underflow to zero  
    output logic        ow_invalid    // Invalid operation (0 * inf =  
NaN)  
);
```

16.3 Ports

Port	Direction	Width	Description
i_a	Input	16	BF16

Port	Direction	Width	Description
i_b	Input	16	multiplier A BF16 multiplier B
ow_result	Output	16	BF16 product
ow_overflow	Output	1	1 if result overflowed to infinity
ow_underflow	Output	1	1 if result underflowed to zero
ow_invalid	Output	1	1 if operation was invalid (0 * inf)

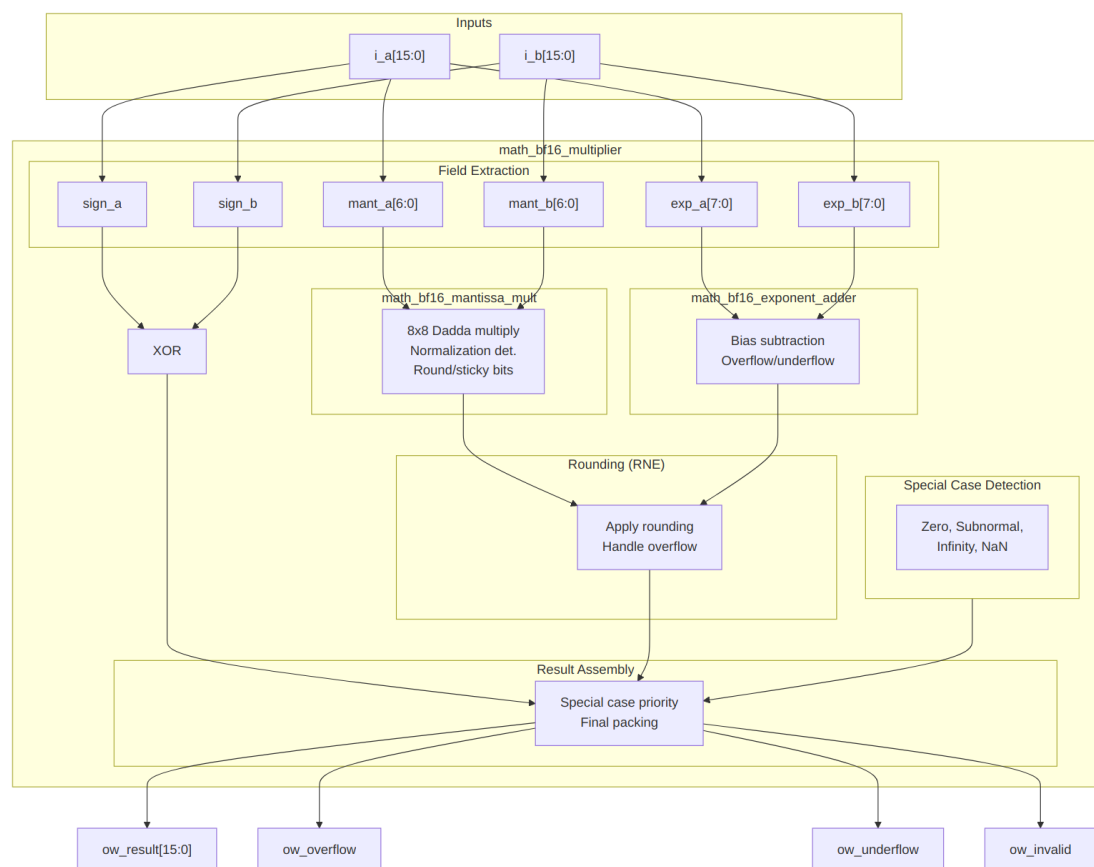
16.4 BF16 Format

BF16: [15]=sign, [14:7]=exponent (8 bits, bias=127), [6:0]=mantissa (7 bits)

Special values:

```
+0:    0x0000 (sign=0, exp=0, mant=0)
-0:    0x8000 (sign=1, exp=0, mant=0)
+inf:  0x7F80 (sign=0, exp=255, mant=0)
-inf:  0xFF80 (sign=1, exp=255, mant=0)
NaN:   0x7FC0 (exp=255, mant!=0, canonical quiet NaN)
```

16.5 Architecture



Block Diagram

16.5.1 Processing Pipeline

1. **Field Extraction** - Parse sign, exponent, mantissa from inputs
2. **Special Case Detection** - Identify zero, subnormal, infinity, NaN
3. **Sign Computation** - XOR of input signs
4. **Mantissa Multiplication** - 8×8 Dadda tree with normalization
5. **Exponent Addition** - Add exponents, subtract bias, adjust for normalization
6. **RNE Rounding** - Apply Round-to-Nearest-Even to mantissa
7. **Result Assembly** - Select output based on special case priority

16.6 Functionality

16.6.1 Special Case Detection

```
// Zero: exp=0, mant=0
wire w_a_is_zero = (w_exp_a == 8'h00) & (w_mant_a == 7'h00);

// Subnormal: exp=0, mant!=0 (treated as zero in FTZ mode)
wire w_a_is_subnormal = (w_exp_a == 8'h00) & (w_mant_a != 7'h00);

// Infinity: exp=FF, mant=0
wire w_a_is_inf = (w_exp_a == 8'hFF) & (w_mant_a == 7'h00);

// NaN: exp=FF, mant!=0
wire w_a_is_nan = (w_exp_a == 8'hFF) & (w_mant_a != 7'h00);

// Effective zero (includes subnormals)
wire w_a_eff_zero = w_a_is_zero | w_a_is_subnormal;

// Normal number (has implied leading 1)
wire w_a_is_normal = ~w_a_eff_zero & ~w_a_is_inf & ~w_a_is_nan;
```

16.6.2 Round-to-Nearest-Even

```
// RNE: Round up if round_bit=1 AND (sticky_bit=1 OR lsb=1)
wire w_lsb = w_mant_mult_out[0];
wire w_round_up = w_round_bit & (w_sticky_bit | w_lsb);

// Apply rounding
wire [7:0] w_mant_rounded = {1'b0, w_mant_mult_out} + {7'b0, w_round_up};

// Handle mantissa overflow from rounding (1.1111111 + 1 = 10.0000000)
wire w_mant_round_overflow = w_mant_rounded[7];
wire [6:0] w_mant_final = w_mant_round_overflow ? 7'h00 : w_mant_rounded[6:0];

// Adjust exponent for rounding overflow
wire [7:0] w_exp_final = w_mant_round_overflow ? (w_exp_sum + 8'd1) : w_exp_sum;
```

16.6.3 Special Case Priority

```
always_comb begin
    // Default: normal result
    ow_result = {w_sign_result, w_exp_final, w_mant_final};
    ow_overflow = 1'b0;
```



```

ow_underflow = 1'b0;
ow_invalid = 1'b0;

// Priority order (highest to lowest):
if (w_any_nan | w_invalid_op) begin
    // 1. NaN: any NaN input, or 0 * inf
    ow_result = {w_sign_result, 8'hFF, 7'h40}; // Canonical qNaN
    ow_invalid = w_invalid_op;
end else if (w_result_inf | w_final_overflow) begin
    // 2. Infinity: inf input or overflow
    ow_result = {w_sign_result, 8'hFF, 7'h00};
    ow_overflow = w_final_overflow & ~w_result_inf;
end else if (w_result_zero | w_exp_underflow) begin
    // 3. Zero: zero input or underflow
    ow_result = {w_sign_result, 8'h00, 7'h00};
    ow_underflow = w_exp_underflow & ~w_result_zero;
end
end

```

16.7 Usage Examples

16.7.1 Basic Multiplication

```

logic [15:0] a, b, product;
logic overflow, underflow, invalid;

```

```

math_bf16_multiplier u_mult (
    .i_a(a),
    .i_b(b),
    .ow_result(product),
    .ow_overflow(overflow),
    .ow_underflow(underflow),
    .ow_invalid(invalid)
);

```

```

// Example: 2.0 * 3.0 = 6.0
// 2.0 in BF16: 0x4000 (sign=0, exp=128, mant=0)
// 3.0 in BF16: 0x4040 (sign=0, exp=128, mant=0x40)
initial begin
    a = 16'h4000; // 2.0
    b = 16'h4040; // 3.0
    #1;
    // product should be ~6.0 (0x40C0)
end

```

16.7.2 With Status Checking

```
always_ff @(posedge clk) begin
    if (overflow)
        $display("Overflow: result is infinity");
    else if (underflow)
        $display("Underflow: result is zero");
    else if (invalid)
        $display("Invalid: 0 * inf or NaN input");
end
```

16.7.3 In Neural Network Layer

```
// Multiply weight by activation
logic [15:0] weight, activation, product;
logic [15:0] accumulator; // Would typically be FP32 for accuracy

math_bf16_multiplier u_mult (
    .i_a(weight),
    .i_b(activation),
    .ow_result(product),
    .ow_overflow(),
    .ow_underflow(),
    .ow_invalid()
);

// Accumulate (simplified - real implementations use FP32 accumulator)
always_ff @(posedge clk) begin
    if (start)
        accumulator <= product;
    else
        accumulator <= accumulator + product; // Would need FP adder
end
```

16.8 Timing Characteristics

Stage	Logic Depth
Field extraction	0 (wiring)
Special case detection	2 gates
Mantissa multiply	~15 gates
Exponent add	~5 gates
Rounding	~3 gates
Result MUX	~2 gates
Total	~25-30 gates

16.9 Performance Characteristics

16.9.1 Resource Utilization

Component	LUTs (est.)
Special case detection	~30
Mantissa multiplier	~160
Exponent adder	~40
Rounding logic	~15
Result MUX	~25
Total	~270-300 LUTs

16.9.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Efficient submodule integration 2. **Wire complexity** - Clean datapath structure 3. **Logic depth** - Parallel special case and normal path computation

16.10 Design Considerations

16.10.1 Flush-to-Zero (FTZ)

Subnormal numbers are flushed to zero: - **Input subnormals** - Treated as zero (effective zero) - **Output subnormals** - Not generated (result goes to zero)

This simplifies hardware and matches AI accelerator conventions.

16.10.2 NaN Handling

- **Input NaN** - Propagated to output as canonical quiet NaN
- **0 * Infinity** - Produces NaN with invalid flag set
- **Canonical qNaN** - 0x7FC0 (sign=0, exp=FF, mant=0x40)

16.10.3 Sign of Zero

The sign of zero follows IEEE 754: - $+0 * +0 = +0$ - $+0 * -0 = -0$ - $-0 * -0 = +0$

16.10.4 Rounding Mode

Only Round-to-Nearest-Even (RNE) is supported. This is standard for BF16 in AI applications.

16.11 Common Pitfalls

Anti-Pattern 1: Ignoring status flags

```
// WRONG: Assuming result is always valid
result <= product; // May be NaN or infinity!
```

```
// RIGHT: Check flags
if (!invalid && !overflow)
    result <= product;
else
    handle_exception();
```

Anti-Pattern 2: Expecting subnormal results

```
// NOTE: Very small results flush to zero, not subnormal
// If you need gradual underflow, use FP32 or a different
implementation
```

16.12 Auto-Generated Code

This module is auto-generated by Python scripts: - **Generator:**

```
bin/rtl_generators/bf16/bf16_multiplier.py - Regenerate: PYTHONPATH=bin:
$PYTHONPATH python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv file manually.

16.13 Related Modules

- **math_bf16_mantissa_mult** - Mantissa multiplication submodule
- **math_bf16_exponent_adder** - Exponent computation submodule
- **math_bf16_fma** - Fused Multiply-Add with FP32 accumulator
- **math_multiplier_dadda_4to2_008** - Underlying 8x8 multiplier

16.14 References

- Google Brain Float (BF16) specification
- IEEE 754-2019 Standard for Floating-Point Arithmetic
- Intel BFloat16 documentation
- NVIDIA TensorFloat documentation

16.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

17 4:2 Compressor

A 4:2 compressor (also called a 5:3 counter) that reduces 5 input bits to 3 output bits, providing faster column height reduction than traditional 3:2 compressors in multiplier trees.

17.1 Overview

The `math_compressor_4to2` module implements a 4:2 compressor using two cascaded full adders. It accepts 4 primary inputs plus a carry-in, producing a sum, carry, and carry-out. The key advantage is that the carry-out (`ow_cout`) is independent of the carry-in (`i_cin`), enabling efficient chaining within multiplier reduction trees.

Key Features: - **5 inputs to 3 outputs** - More efficient column reduction than 3:2 compressors - **Cout independent of Cin** - Enables parallel carry chains - **2 full adder depth** - Predictable timing - **Building block** for Dadda/Wallace trees with 4:2 compressors

17.2 Module Declaration

```
module math_compressor_4to2 (  
    input logic i_x1, i_x2, i_x3, i_x4,  
    input logic i_cin,  
    output logic ow_sum,  
    output logic ow_carry,  
    output logic ow_cout  
);
```

17.3 Ports

Port	Direction	Width	Description
i_x1	Input	1	First input bit
i_x2	Input	1	Second input bit
i_x3	Input	1	Third input bit
i_x4	Input	1	Fourth input bit
i_cin	Input	1	Carry input (from previous compressor)
ow_sum	Output	1	Sum output (same column weight)
ow_carry	Output	1	Carry output (weight = column + 1)
ow_cout	Output	1	Carry-out (weight = column + 1, independent)

Port	Direction	Width	Description of Cin)
------	-----------	-------	------------------------

17.4 Functionality

17.4.1 Architecture

The 4:2 compressor is implemented as two cascaded full adders:

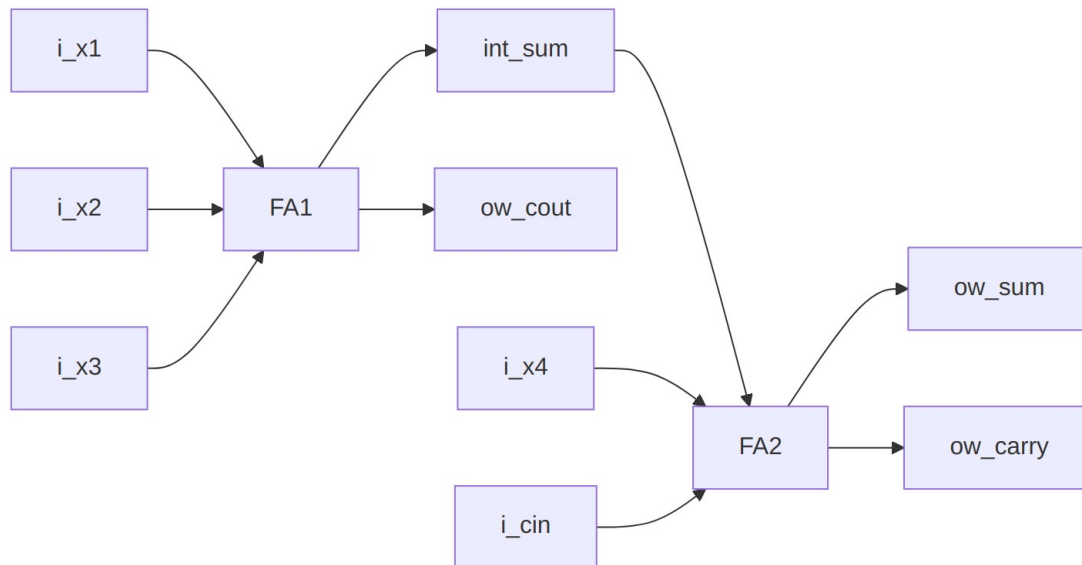


Diagram 5

Stage 1 (FA1): Compresses X1, X2, X3 - Sum goes to Stage 2 - Carry becomes ow_cout (independent of i_cin!)

Stage 2 (FA2): Compresses int_sum, X4, Cin - Sum becomes ow_sum - Carry becomes ow_carry

17.4.2 Truth Table (simplified)

The compressor performs: $i_x1 + i_x2 + i_x3 + i_x4 + i_cin = ow_sum + 2 \cdot (ow_carry + ow_cout)$

Sum of inputs (0-5)	ow_cout	ow_carry	ow_sum
0	0	0	0
1	0	0	1
2	0/1	0/1	0
3	0/1	0/1	1

Sum of inputs (0-5)	ow_cout	ow_carry	ow_sum
4	1	1	0
5	1	1	1

Note: Multiple input combinations can produce the same sum, with different cout/carry distributions.

17.4.3 Key Property: Cout Independence

The critical property of this implementation is that ow_cout depends only on X1, X2, X3 - not on i_cin:

```
// First stage: cout is independent of cin
ow_cout = (i_x1 & i_x2) | (i_x2 & i_x3) | (i_x1 & i_x3);
```

This allows multiple 4:2 compressors to be chained with their cout signals forming independent carry chains, reducing the critical path compared to implementations where all carries depend on cin.

17.5 Timing Characteristics

Metric	Value	Description
Logic Depth	4 gates	2 full adders in series
Cin to Sum	2 XOR gates	Through FA2 only
Cin to Carry	1 XOR + 1 AND-OR	Through FA2 only
X1-X3 to Cout	1 AND-OR	Through FA1 only

Critical Path: X1/X2/X3 -> FA1 sum -> FA2 sum = 4 XOR gate delays

17.6 Usage Examples

17.6.1 Single Compressor

```
logic x1, x2, x3, x4, cin;
logic sum, carry, cout;
```

```
math_compressor_4to2 u_comp (
    .i_x1(x1), .i_x2(x2), .i_x3(x3), .i_x4(x4),
    .i_cin(cin),
    .ow_sum(sum),
    .ow_carry(carry),
```

```

        .ow_cout(cout)
    );

```

17.6.2 Chained Compressors (Multiplier Column)

// Reduce 8 bits in a column to 4 using two chained 4:2 compressors

```

logic [7:0] col_bits;
logic sum1, carry1, cout1;
logic sum2, carry2, cout2;

```

// First compressor: bits [3:0] + carry from previous column

```

math_compressor_4to2 u_comp1 (
    .i_x1(col_bits[0]), .i_x2(col_bits[1]),
    .i_x3(col_bits[2]), .i_x4(col_bits[3]),
    .i_cin(cin_from_prev),
    .ow_sum(sum1),
    .ow_carry(carry1),
    .ow_cout(cout1)
);

```

// Second compressor: bits [7:4] + cout from first compressor

```

math_compressor_4to2 u_comp2 (
    .i_x1(col_bits[4]), .i_x2(col_bits[5]),
    .i_x3(col_bits[6]), .i_x4(col_bits[7]),
    .i_cin(cout1), // Chain cout to cin
    .ow_sum(sum2),
    .ow_carry(carry2),
    .ow_cout(cout2)
);

```

*// Results: sum1, sum2 stay in this column
 // carry1, carry2, cout2 go to next column*

17.6.3 In Dadda Tree Multiplier

// Part of Dadda reduction stage

// Column has partial products: pp0, pp1, pp2, pp3, pp4

// Plus carry-in from previous column

```

math_compressor_4to2 u_dadda_comp (
    .i_x1(pp0), .i_x2(pp1), .i_x3(pp2), .i_x4(pp3),
    .i_cin(cin_prev),
    .ow_sum(reduced_sum), // Stays in column
    .ow_carry(carry_to_next), // Goes to column+1
    .ow_cout(cout_to_next) // Goes to column+1
);

```

// pp4 passes through to next stage if column height allows

17.7 Performance Characteristics

17.7.1 Resource Utilization

Metric	Value
Full Adders	2
Total Gates	~10-12 (technology dependent)
LUTs (FPGA)	~3-4

17.7.2 Comparison with 3:2 Compressor (Full Adder)

Metric	3:2 Compressor	4:2 Compressor
Inputs	3	5
Outputs	2	3
Reduction ratio	3:2	5:3
Gate count	~5	~10-12
Logic depth	2	4
Column reduction	-1 per stage	-2 per stage

Advantage: 4:2 compressors reduce column height faster, resulting in fewer reduction stages for multipliers.

17.7.3 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Minimal gate count using two full adders 2. **Wire complexity** - Simple cascaded structure with minimal routing 3. **Logic depth** - Fixed 2 full adder delays

17.8 Design Considerations

17.8.1 Advantages

- **Faster reduction** - Each compressor removes 2 bits from column height (vs 1 for 3:2)
- **Cin independence** - Cout doesn't depend on Cin, enabling parallel carry computation
- **Predictable timing** - Fixed 2 FA delay
- **Standard building block** - Well-understood for multiplier design

17.8.2 Applications

- **Multiplier trees** - Dadda and Wallace trees with 4:2 compressors

- **Multi-operand adders** - Reducing many operands to 2
- **DSP blocks** - Custom multiply-accumulate units
- **BF16/FP16 arithmetic** - Mantissa multiplication

17.9 Related Modules

- **math_adder_full** - Full adder building block (3:2 compressor)
- **math_adder_half** - Half adder building block
- **math_multiplier_dadda_4to2_008** - 8x8 multiplier using this compressor
- **math_adder_carry_save** - Alternative 3:2 compressor naming

17.10 References

- Dadda, L. "Some Schemes for Parallel Multipliers." *Alta Frequenza* 34, 1965.
- Weinberger, A. "4:2 Carry-Save Adder Module." *IBM Technical Disclosure Bulletin*, 1981.
- Oklobdzija, V.G. et al. "A Method for Speed Optimized Partial Product Reduction and Generation of Fast Parallel Multipliers Using an Algorithmic Approach." *IEEE Trans. Computers*, 1996.

17.11 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

18 FP16 (Half Precision) Modules

IEEE 754 half-precision (FP16) floating-point modules for activation functions, comparison, and format conversion.

18.1 Overview

FP16 (16-bit floating-point) provides higher precision than BF16 with a different exponent/mantissa trade-off. These modules complement the BF16 library for applications requiring greater precision in the fractional range.

FP16 Format:

```
[15]      = Sign bit
[14:10]   = Exponent (5 bits, bias = 15)
[9:0]     = Mantissa (10 bits, implicit leading 1)
```

Range: $\sim 6.0e-8$ to ~ 65504
Precision: $\sim 0.1\%$ (3-4 decimal digits)

Key Differences from BF16: | Property | BF16 | FP16 | |———|——|——| | Exponent bits | 8 | 5 | | Mantissa bits | 7 | 10 | | Dynamic range | Same as FP32 | Limited (~ 65504 max) | | Precision | $\sim 1\%$ | $\sim 0.1\%$ |

18.2 Module Categories

18.2.1 Activation Functions

Module	Function	Description
math_fp16_relu	$\max(0, x)$	Rectified Linear Unit
math_fp16_leaky_relu	$\max(0.01x, x)$	Leaky ReLU
math_fp16_gelu	$x * \Phi(x)$	Gaussian Error Linear Unit
math_fp16_sigmoid	$1/(1+\exp(-x))$	Logistic function
math_fp16_tanh	$\tanh(x)$	Hyperbolic tangent
math_fp16_silu	$x * \text{sigmoid}(x)$	SiLU/Swish activation
math_fp16_softmax_8	$\text{softmax}(x[7:0])$	8-input softmax

18.2.1.1 Interface

```
module math_fp16_{activation} (
    input logic [15:0] i_a,           // FP16 input
    output logic [15:0] ow_result    // FP16 output
);
```

18.2.2 Comparison and Selection

Module	Function	Description
math_fp16_comparator	$a <=> b$	Full comparison with flags
math_fp16_max	$\max(a, b)$	Two-input maximum
math_fp16_min	$\min(a, b)$	Two-input minimum
math_fp16_max_tree_8	$\max(a[7:0])$	8-input maximum
math_fp16_min_tree	$\min(a[7:0])$	8-input minimum

Module	Function	Description
<code>_8</code>		
<code>math_fp16_clamp</code>	<code>clamp(x, lo, hi)</code>	Value clamping

18.2.3 Format Conversions

Module	Conversion	Notes
<code>math_fp16_to_bf16</code>	FP16 -> BF16	Mantissa truncation, exponent adjustment
<code>math_fp16_to_fp32</code>	FP16 -> FP32	Lossless upconversion
<code>math_fp16_to_fp8_e4m3</code>	FP16 -> FP8 E4M3	For ML inference
<code>math_fp16_to_fp8_e5m2</code>	FP16 -> FP8 E5M2	For ML training

18.2.3.1 Conversion Notes

FP16 to BF16: - Mantissa truncation: 10 bits -> 7 bits (may lose precision) - Exponent mapping: bias 15 -> bias 127 - May overflow if FP16 value > BF16 max normal

FP16 to FP32: - Lossless conversion (FP32 can represent all FP16 values) - Simple exponent bias adjustment: 15 -> 127 - Mantissa padding: 10 bits -> 23 bits

18.3 Usage Examples

18.3.1 Mixed Precision Pipeline

```
// FP16 computation with FP32 accumulator
logic [15:0] fp16_a, fp16_b, fp16_product;
logic [31:0] fp32_a, fp32_b, fp32_acc;

// Convert to FP32 for accumulation
math_fp16_to_fp32 u_cvt_a
(.i_a(fp16_a), .ow_result(fp32_a), .ow_invalid());
math_fp16_to_fp32 u_cvt_b
(.i_a(fp16_b), .ow_result(fp32_b), .ow_invalid());

// Compute in FP32 for precision
// ... FP32 MAC operation ...
```

```
// Apply FP16 activation
math_fp16_relu u_relu (.i_a(layer_output), .ow_result(activated));
```

18.3.2 Comparison Operations

```
logic [15:0] fp16_values [8];
logic [15:0] max_value, min_value;
```

```
math_fp16_max_tree_8 u_max (
    .i_values(fp16_values),
    .ow_max(max_value)
);
```

```
math_fp16_min_tree_8 u_min (
    .i_values(fp16_values),
    .ow_min(min_value)
);
```

18.4 Special Values

Value	Encoding	Notes
+0	0x0000	Positive zero
-0	0x8000	Negative zero
+Inf	0x7C00	Positive infinity
-Inf	0xFC00	Negative infinity
NaN	0x7E00	Canonical quiet NaN
Max Normal	0x7BFF	~65504
Min Normal	0x0400	~6.1e-5
Subnormal	Exp=0	Flushed to zero (FTZ)

18.5 Auto-Generation

```
# Regenerate all FP16 modules
PYTHONPATH=bin:$PYTHONPATH python3
bin/rtl_generators/ieee754/generate_all.py rtl/common
```

18.6 Related Documentation

- [math_bf16_extended](#) - BF16 equivalent modules
- [math_fp32_modules](#) - FP32 equivalent modules
- [math_fp8_modules](#) - FP8 variants
- [math_ieee754_modules](#) - IEEE 754-2008 compliant arithmetic

18.7 Navigation

- [Back to RTLCommon Index](#)
- [Back to Main Documentation Index](#)

19 FP32 (Single Precision) Modules

IEEE 754 single-precision (FP32) floating-point modules for activation functions, comparison, and format conversion.

19.1 Overview

FP32 (32-bit floating-point) provides full single-precision IEEE 754 representation. These modules support activation functions, comparisons, and downconversion to smaller formats.

FP32 Format:

[31] = Sign bit
[30:23] = Exponent (8 bits, bias = 127)
[22:0] = Mantissa (23 bits, implicit leading 1)

Range: $\sim 1.2e-38$ to $\sim 3.4e38$
Precision: $\sim 0.00001\%$ (7 decimal digits)

19.2 Module Categories

19.2.1 Activation Functions

Module	Function	Description
math_fp32_relu	$\max(0, x)$	Rectified Linear Unit
math_fp32_leaky_relu	$\max(0.01x, x)$	Leaky ReLU
math_fp32_gelu	$x * \Phi(x)$	Gaussian Error Linear Unit
math_fp32_sigmoid	$1/(1+\exp(-x))$	Logistic function
math_fp32_tanh	$\tanh(x)$	Hyperbolic tangent
math_fp32_silu	$x * \text{sigmoid}(x)$	SiLU/Swish activation
math_fp32_softmax_8	$\text{softmax}(x[7:0])$	8-input softmax

19.2.1.1 Interface

```
module math_fp32_{activation} (  
    input logic [31:0] i_a,           // FP32 input  
    output logic [31:0] ow_result    // FP32 output  
);
```

19.2.2 Comparison and Selection

Module	Function	Description
math_fp32_comparator	$a <=> b$	Full comparison with flags
math_fp32_max	$\max(a, b)$	Two-input maximum
math_fp32_min	$\min(a, b)$	Two-input minimum
math_fp32_max_tree_8	$\max(a[7:0])$	8-input maximum
math_fp32_min_tree_8	$\min(a[7:0])$	8-input minimum
math_fp32_clamp	$\text{clamp}(x, lo, hi)$	Value clamping

19.2.3 Format Conversions (Downconversion)

Module	Conversion	Notes
math_fp32_to_bf16	FP32 -> BF16	Truncate mantissa (23->7 bits)
math_fp32_to_fp16	FP32 -> FP16	Range check + rounding
math_fp32_to_fp8_e4m3	FP32 -> FP8 E4M3	ML inference quantization
math_fp32_to_fp8_e5m2	FP32 -> FP8 E5M2	ML training quantization

19.2.3.1 Downconversion Notes

FP32 to BF16:

```
module math_fp32_to_bf16 (  
    input logic [31:0] i_a,  
    output logic [15:0] ow_result,  
    output logic ow_overflow, // If exp > 254 in BF16  
    output logic ow_underflow, // If exp < 1 (subnormal/zero)
```

```

        output logic          ow_invalid    // NaN input
    );

```

- Same exponent range (8 bits), just truncate mantissa
- May lose precision in mantissa (23 -> 7 bits)
- RNE rounding applied

FP32 to FP16: - Exponent range reduction: 8 bits -> 5 bits - May overflow (FP32 value > 65504) - May underflow (FP32 value < FP16 min subnormal) - Mantissa rounding (23 -> 10 bits)

19.3 Usage Examples

19.3.1 Training Pipeline (FP32 Master)

```

// FP32 forward pass with activation
logic [31:0] layer_input, layer_output, activated;

// FP32 computation (full precision for training)
// ... matrix multiply in FP32 ...

// FP32 activation
math_fp32_relu u_relu (
    .i_a(layer_output),
    .ow_result(activated)
);

// Convert to BF16 for storage/transfer
logic [15:0] bf16_result;
math_fp32_to_bf16 u_cvt (
    .i_a(activated),
    .ow_result(bf16_result),
    .ow_overflow(overflow_flag),
    .ow_underflow(),
    .ow_invalid()
);

```

19.3.2 Quantization Pipeline

```

// Quantize FP32 model weights to FP8 for inference
logic [31:0] fp32_weight;
logic [7:0] fp8_weight;

math_fp32_to_fp8_e4m3 u_quantize (
    .i_a(fp32_weight),
    .ow_result(fp8_weight),
    .ow_overflow(quant_overflow),
    .ow_underflow(quant_underflow),

```



```
        .ow_invalid()
    );
```

19.4 Special Values

Value	Encoding	Notes
+0	0x00000000	Positive zero
-0	0x80000000	Negative zero
+Inf	0x7F800000	Positive infinity
-Inf	0xFF800000	Negative infinity
NaN	0x7FC00000	Canonical quiet NaN
Max Normal	0x7F7FFFFF	~3.4e38
Min Normal	0x00800000	~1.2e-38

19.5 Auto-Generation

```
# Regenerate all FP32 modules
PYTHONPATH=bin:$PYTHONPATH python3
bin/rtl_generators/ieee754/generate_all.py rtl/common
```

19.6 Related Documentation

- [math_bf16_extended](#) - BF16 equivalent modules
- [math_fp16_modules](#) - FP16 equivalent modules
- [math_fp8_modules](#) - FP8 variants
- [math_ieee754_modules](#) - IEEE 754-2008 compliant arithmetic

19.7 Navigation

- [Back to RTLCommon Index](#)
- [Back to Main Documentation Index](#)

20 FP8 (8-bit Floating-Point) Modules

8-bit floating-point modules in both E4M3 and E5M2 formats for ML inference and training acceleration.

20.1 Overview

FP8 formats provide extreme memory efficiency for neural network accelerators. Two variants are supported: - **E4M3** - Higher precision, smaller range (inference-optimized) - **E5M2** - Lower precision, larger range (training-friendly)

FP8 E4M3 Format:

[7] = Sign bit
[6:3] = Exponent (4 bits, bias = 7)
[2:0] = Mantissa (3 bits)

Range: ~ 0.001953 to 448 (no infinity!)
Special: exp=15, mant=111 is NaN (not infinity)
Max normal: exp=15, mant=110 = 448

FP8 E5M2 Format:

[7] = Sign bit
[6:2] = Exponent (5 bits, bias = 15)
[1:0] = Mantissa (2 bits)

Range: $\sim 6e-8$ to 57344 (has infinity)
Special: exp=31, mant=0 is Inf; mant!=0 is NaN

20.2 Format Comparison

Property	E4M3	E5M2	BF16
Total bits	8	8	16
Exponent bits	4	5	8
Mantissa bits	3	2	7
Max value	448	57344	$\sim 3.4e38$
Min normal	0.0156	$6.1e-5$	$\sim 1.2e-38$
Has infinity	No	Yes	Yes
Use case	Inference	Training	General

20.3 E4M3 Modules

20.3.1 Core Arithmetic

Module	Function	Description
math_fp8_e4m3_adder	$a + b$	FP8 E4M3 addition

Module	Function	Description
math_fp8_e4m3_multiplier	$a * b$	FP8 E4M3 multiplication
math_fp8_e4m3_fm_a	$a * b + c$	Fused multiply-add
math_fp8_e4m3_mantissa_mult	Internal	Mantissa multiplication helper
math_fp8_e4m3_exponent_adder	Internal	Exponent computation helper

20.3.1.1 Multiplier Interface

```

module math_fp8_e4m3_multiplier (
    input logic [7:0] i_a,
    input logic [7:0] i_b,
    output logic [7:0] ow_result,
    output logic      ow_overflow, // Saturates to max normal (448)
    output logic      ow_underflow, // Flushes to zero
    output logic      ow_invalid  // NaN operation
);

```

Note: E4M3 has no infinity - overflow saturates to max normal value (0x7E = 448).

20.3.2 Activation Functions

Module	Function	Description
math_fp8_e4m3_relu	$\max(0, x)$	ReLU activation
math_fp8_e4m3_leaky_relu	$\max(0.01x, x)$	Leaky ReLU
math_fp8_e4m3_gelu	$x * \Phi(x)$	GELU activation
math_fp8_e4m3_sigmoid	$1/(1+\exp(-x))$	Sigmoid
math_fp8_e4m3_tanh	$\tanh(x)$	Hyperbolic tangent
math_fp8_e4m3_silu	$x * \text{sigmoid}(x)$	SiLU/Swish
math_fp8_e4m3_softmax_max_8	$\text{softmax}(x[7:0])$	8-input softmax

20.3.3 Comparison and Selection

Module	Function	Description
math_fp8_e4m3_comparator	$a <=> b$	Full comparison
math_fp8_e4m3_max	$\max(a, b)$	Two-input max
math_fp8_e4m3_min	$\min(a, b)$	Two-input min

Module	Function	Description
math_fp8_e4m3_max_tree_8	max(a[7:0])	8-input max
math_fp8_e4m3_min_tree_8	min(a[7:0])	8-input min
math_fp8_e4m3_clamp_p	clamp(x, lo, hi)	Value clamping

20.3.4 Format Conversions

Module	Conversion	Notes
math_fp8_e4m3_to_bf16	E4M3 -> BF16	Lossless upconversion
math_fp8_e4m3_to_fp16	E4M3 -> FP16	Lossless upconversion
math_fp8_e4m3_to_fp32	E4M3 -> FP32	Lossless upconversion
math_fp8_e4m3_to_fp8_e5m2	E4M3 -> E5M2	May lose precision

20.4 E5M2 Modules

20.4.1 Core Arithmetic

Module	Function	Description
math_fp8_e5m2_adder	$a + b$	FP8 E5M2 addition
math_fp8_e5m2_multiplier	$a * b$	FP8 E5M2 multiplication
math_fp8_e5m2_fm_a	$a * b + c$	Fused multiply-add
math_fp8_e5m2_mantissa_mult	Internal	Mantissa multiplication helper
math_fp8_e5m2_exponent_adder	Internal	Exponent computation helper

20.4.2 Activation Functions

Module	Function	Description
math_fp8_e5m2_relu	$\max(0, x)$	ReLU activation
math_fp8_e5m2_leaky_relu	$\max(0.01x, x)$	Leaky ReLU

Module	Function	Description
math_fp8_e5m2_gelu	$x * \Phi(x)$	GELU activation
math_fp8_e5m2_sigmoid	$1/(1+\exp(-x))$	Sigmoid
math_fp8_e5m2_tanh	$\tanh(x)$	Hyperbolic tangent
math_fp8_e5m2_silu	$x * \text{sigmoid}(x)$	SiLU/Swish
math_fp8_e5m2_softmax_8	$\text{softmax}(x[7:0])$	8-input softmax

20.4.3 Comparison and Selection

Module	Function	Description
math_fp8_e5m2_comparator	$a <=> b$	Full comparison
math_fp8_e5m2_max	$\max(a, b)$	Two-input max
math_fp8_e5m2_min	$\min(a, b)$	Two-input min
math_fp8_e5m2_max_tree_8	$\max(a[7:0])$	8-input max
math_fp8_e5m2_min_tree_8	$\min(a[7:0])$	8-input min
math_fp8_e5m2_clamp	$\text{clamp}(x, \text{lo}, \text{hi})$	Value clamping

20.4.4 Format Conversions

Module	Conversion	Notes
math_fp8_e5m2_to_bf16	E5M2 -> BF16	Lossless upconversion
math_fp8_e5m2_to_fp16	E5M2 -> FP16	Lossless upconversion
math_fp8_e5m2_to_fp32	E5M2 -> FP32	Lossless upconversion
math_fp8_e5m2_to_fp8_e4m3	E5M2 -> E4M3	May overflow/lose precision

20.5 Usage Examples

20.5.1 Inference Pipeline (E4M3)

```
// FP8 E4M3 inference - maximum memory efficiency
logic [7:0] weight, activation, product;
```

```

math_fp8_e4m3_multiplier u_mult (
    .i_a(weight),
    .i_b(activation),
    .ow_result(product),
    .ow_overflow(overflow), // Saturates to 448
    .ow_underflow(),
    .ow_invalid()
);

```

```

// Apply activation
math_fp8_e4m3_relu u_relu (
    .i_a(product),
    .ow_result(activated)
);

```

20.5.2 Mixed Precision Training

// Forward pass in E5M2, upconvert for backward pass

```

logic [7:0] fp8_activation;
logic [15:0] bf16_gradient;

```

// Upconvert for gradient computation

```

math_fp8_e5m2_to_bf16 u_cvt (
    .i_a(fp8_activation),
    .ow_result(bf16_activation),
    .ow_invalid()
);

```

```

// Compute gradients in BF16
// ... backward pass in BF16 ...

```

20.6 Special Values

20.6.1 E4M3 Special Values

Value	Encoding	Notes
+0	0x00	Positive zero
-0	0x80	Negative zero
Max Normal	0x7E	448 (exp=15, mant=6)
NaN	0x7F	exp=15, mant=7
No Infinity	-	Overflow saturates to 0x7E

20.6.2 E5M2 Special Values

Value	Encoding	Notes
+0	0x00	Positive zero
-0	0x80	Negative zero
+Inf	0x7C	exp=31, mant=0
-Inf	0xFC	exp=31, mant=0
NaN	0x7E	exp=31, mant!=0

20.7 Design Considerations

20.7.1 E4M3 Overflow Handling

E4M3 has no infinity encoding. When multiplication would overflow: - Result saturates to max normal (448 = 0x7E) - `ow_overflow` flag asserts - Sign is preserved

```
// E4M3: Large values saturate instead of becoming infinity  
// 400 * 2 = 800 -> saturates to 448
```

20.7.2 Subnormal Handling

Both E4M3 and E5M2 use Flush-to-Zero (FTZ): - Subnormal inputs treated as zero - Results that would be subnormal flush to zero - Simplifies hardware, matches AI accelerator conventions

20.8 Auto-Generation

```
# Regenerate all FP8 modules  
PYTHONPATH=bin:$PYTHONPATH python3  
bin/rtl_generators/ieee754/generate_all.py rtl/common
```

Generator files: - bin/rtl_generators/ieee754/fp8_e4m3_multiplier.py -
bin/rtl_generators/ieee754/fp8_e5m2_multiplier.py -
bin/rtl_generators/ieee754/fp_activations.py -
bin/rtl_generators/ieee754/fp_conversions.py

20.9 Related Documentation

- [math_bf16_extended](#) - BF16 modules
- [math_fp16_modules](#) - FP16 modules
- [math_fp32_modules](#) - FP32 modules
- [math_ieee754_modules](#) - IEEE 754-2008 arithmetic

20.10 Navigation

- [Back to RTLCommon Index](#)
- [Back to Main Documentation Index](#)

21 IEEE 754-2008 Compliant Modules

Full IEEE 754-2008 compliant floating-point arithmetic modules for FP16 and FP32 formats.

21.1 Overview

These modules implement complete IEEE 754-2008 compliant arithmetic for half-precision (FP16) and single-precision (FP32) floating-point operations. Unlike the simplified BF16/FP8 modules which use Flush-to-Zero, these provide proper subnormal handling and full IEEE compliance.

Key Features: - **Full IEEE 754-2008 compliance** - Proper subnormal handling, all rounding modes conceptually supported - **Complete special case handling** - Zero, infinity, NaN, subnormal - **Status flags** - Overflow, underflow, invalid, inexact - **Pipelined options** - Configurable pipeline stages for timing closure

21.2 Module Summary

21.2.1 FP16 (Half Precision) IEEE 754

Module	Operation	Description
math_ieee754_2008_fp16_adder	$a + b$	Full FP16 addition with alignment
math_ieee754_2008_fp16_multiplier	$a * b$	FP16 multiplication
math_ieee754_2008_fp16_fma	$a * b + c$	Fused multiply-add
math_ieee754_2008_fp16_mantissa_mult	Internal	11x11 mantissa multiply
math_ieee754_2008_fp16_exponent_adder	Internal	Exponent computation

21.2.2 FP32 (Single Precision) IEEE 754

Module	Operation	Description
math_ieee754_2008_fp32_adder	$a + b$	Full FP32 addition with

Module	Operation	Description
		alignment
math_ieee754_2008_fp32_multiplier	$a * b$	FP32 multiplication
math_ieee754_2008_fp32_fma	$a * b + c$	Fused multiply-add
math_ieee754_2008_fp32_mantissa_mult	Internal	24x24 mantissa multiply
math_ieee754_2008_fp32_exponent_adder	Internal	Exponent computation

21.3 Module Interfaces

21.3.1 FP16 Adder

```

module math_ieee754_2008_fp16_adder #(
    parameter int PIPE_STAGE_1 = 0, // After operand swap
    parameter int PIPE_STAGE_2 = 0, // After alignment
    parameter int PIPE_STAGE_3 = 0, // After addition
    parameter int PIPE_STAGE_4 = 0 // After normalization
) (
    input    logic          i_clk,
    input    logic          i_rst_n,
    input    logic [15:0] i_a,
    input    logic [15:0] i_b,
    input    logic          i_valid,
    output   logic [15:0] ow_result,
    output   logic          ow_valid,
    output   logic          ow_overflow,
    output   logic          ow_underflow,
    output   logic          ow_invalid
);

```

21.3.2 FP32 Multiplier

```

module math_ieee754_2008_fp32_multiplier (
    input    logic [31:0] i_a,
    input    logic [31:0] i_b,
    output   logic [31:0] ow_result,
    output   logic          ow_overflow,
    output   logic          ow_underflow,
    output   logic          ow_invalid
);

```

21.3.3 FP32 FMA

```
module math_ieee754_2008_fp32_fma (  
    input logic [31:0] i_a,      // Multiplicand  
    input logic [31:0] i_b,      // Multiplier  
    input logic [31:0] i_c,      // Addend  
    output logic [31:0] ow_result,  
    output logic        ow_overflow,  
    output logic        ow_underflow,  
    output logic        ow_invalid  
);
```

21.4 Architecture

21.4.1 FP32 Multiplier Pipeline

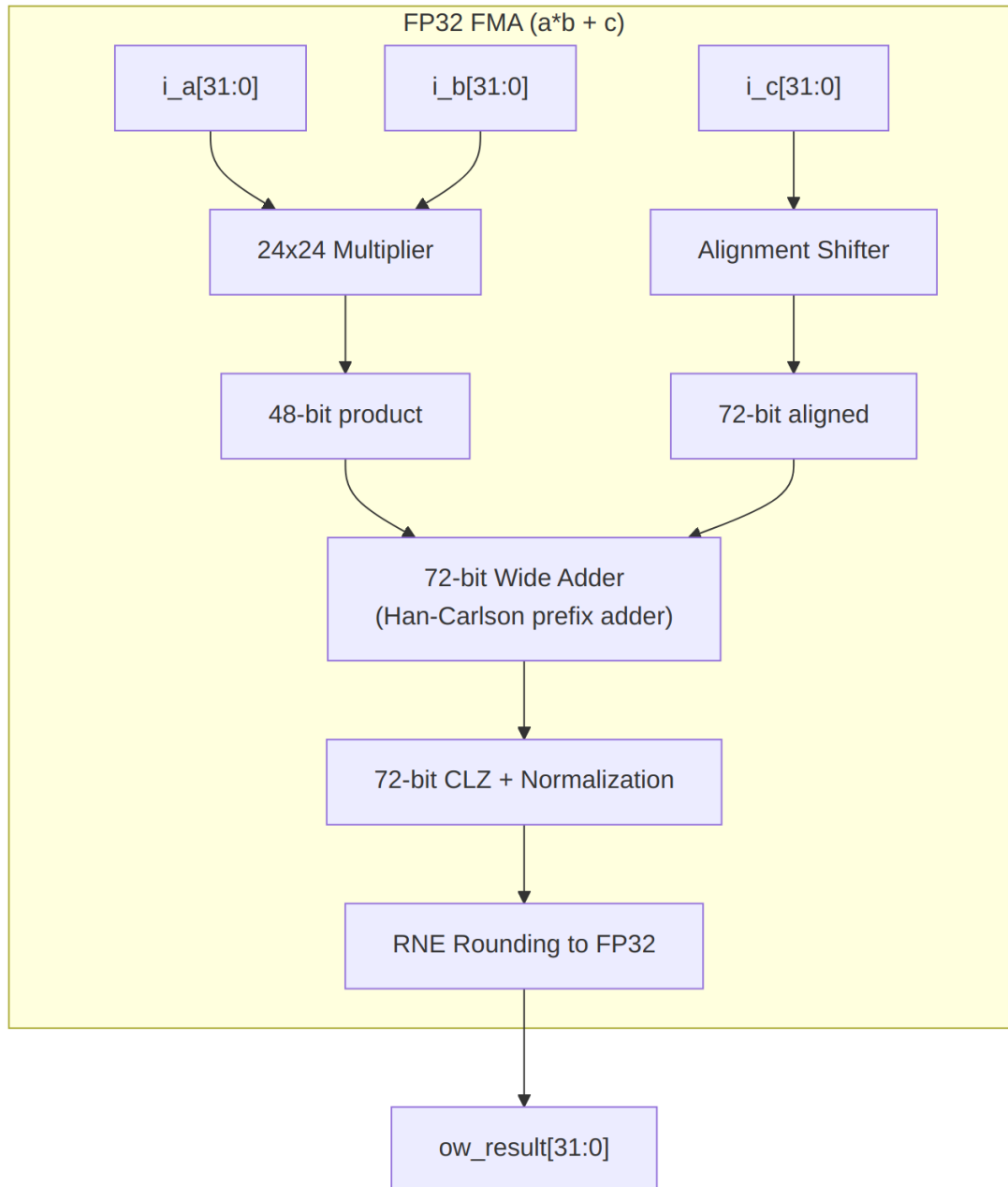
Stage 1: Field extraction + special case detection
 sign_a, exp_a[7:0], mant_a[22:0]
 sign_b, exp_b[7:0], mant_b[22:0]
 |

Stage 2: Sign computation (XOR)
 24x24 Dadda tree multiplication -> 48-bit product
 |

Stage 3: Exponent computation
 exp_sum = exp_a + exp_b - 127 + norm_adjust
 |

Stage 4: Normalization + RNE rounding
 Shift product, apply round-to-nearest-even
 |

Stage 5: Special case priority selection
 NaN > Inf > Overflow > Underflow > Zero > Normal
 |
 Result assembly



FP32 FMA Architecture

21.5 IEEE 754 Compliance

21.5.1 Special Value Handling

Operation	Input	Output
Any	NaN	NaN (propagated)

Operation	Input	Output
$x + y$	+Inf, -Inf	NaN (invalid)
$x * y$	0, Inf	NaN (invalid)
$x + y$	Normal, Normal	Normal/Subnormal
$x * y$	Normal, Normal	Normal/ Subnormal/Zero

21.5.2 Rounding Modes

Currently implements Round-to-Nearest-Even (RNE):

```
// RNE: Round up if guard=1 AND (round=1 OR sticky=1 OR lsb=1)
wire w_round_up = w_guard & (w_round | w_sticky | w_lsb);
```

21.5.3 Status Flags

Flag	Condition
ow_overflow	Result magnitude exceeds max normal
ow_underflow	Result magnitude less than min normal (non-zero)
ow_invalid	Invalid operation (0*Inf, Inf-Inf, NaN input)

21.6 Comparison: IEEE 754 vs Simplified Modules

Feature	IEEE 754 Modules	BF16/FP8 Modules
Subnormals	Full support	Flush to Zero
Infinity	Proper handling	Saturation (FP8 E4M3)
NaN payloads	Preserved	Canonical only
Rounding modes	RNE (extensible)	RNE only
Pipeline options	Configurable	Fixed
Target use	Precision-critical	AI/ML acceleration

21.7 Usage Examples

21.7.1 High-Precision Computation

// IEEE 754 compliant FP32 FMA for numerical accuracy

```
logic [31:0] a, b, c, result;
```

```
math_ieee754_2008_fp32_fma u_fma (  
    .i_a(a),  
    .i_b(b),  
    .i_c(c),  
    .ow_result(result),  
    .ow_overflow(overflow),  
    .ow_underflow(underflow),  
    .ow_invalid(invalid)  
);
```

// Check for exceptions

```
always_ff @(posedge clk) begin  
    if (invalid)  
        $error("Invalid FP operation!");  
    else if (overflow)  
        $warning("FP overflow - result is infinity");  
end
```

21.7.2 Pipelined FP16 Adder

// 4-stage pipelined FP16 adder for high frequency

```
math_ieee754_2008_fp16_adder #(  
    .PIPE_STAGE_1(1), // Pipeline after swap  
    .PIPE_STAGE_2(1), // Pipeline after align  
    .PIPE_STAGE_3(1), // Pipeline after add  
    .PIPE_STAGE_4(1)  // Pipeline after normalize  
) u_adder (  
    .i_clk(clk),  
    .i_rst_n(rst_n),  
    .i_a(fp16_a),  
    .i_b(fp16_b),  
    .i_valid(input_valid),  
    .ow_result(fp16_sum),  
    .ow_valid(output_valid),  
    .ow_overflow(),  
    .ow_underflow(),  
    .ow_invalid()  
);
```

21.8 Dependencies

These modules use the following building blocks:

Module	Used By	Purpose
math_adder_han_carlson_072	FP32 FMA	72-bit wide addition
math_adder_han_carlson_048	FP32 mult	48-bit product CPA
math_multiplier_dadda_4to2_024	FP32	24x24 mantissa multiply
math_multiplier_dadda_4to2_011	FP16	11x11 mantissa multiply
count_leading_zeros	All	Normalization
shifter_barrel	Adders	Mantissa alignment

21.9 Auto-Generation

```
# Regenerate IEEE 754 modules
PYTHONPATH=bin:$PYTHONPATH python3
bin/rtl_generators/ieee754/generate_all.py rtl/common
```

Generator files: - bin/rtl_generators/ieee754/fp16_adder.py -
bin/rtl_generators/ieee754/fp16_multiplier.py -
bin/rtl_generators/ieee754/fp16_fma.py -
bin/rtl_generators/ieee754/fp32_adder.py -
bin/rtl_generators/ieee754/fp32_multiplier.py -
bin/rtl_generators/ieee754/fp32_fma.py

21.10 Related Documentation

- [math_bf16_multiplier](#) - Simplified BF16 multiply
- [math_bf16_fma](#) - Simplified BF16 FMA
- [math_adder_han_carlson](#) - Prefix adder building block
- [math_multiplier_dadda_4to2](#) - Dadda multiplier

21.11 Navigation

- [Back to RTLCommon Index](#)
- [Back to Main Documentation Index](#)

22 Basic Multiplier Building Blocks

Fundamental multiplication primitives including basic multiplier cells and array-based carry-save multipliers. These modules provide simple multiplication implementations suitable for educational purposes and as building blocks for more complex multiplier architectures.

22.1 Overview

This document covers the basic multiplier components: - **math_multiplier_basic_cell** - Single-bit multiplier cell with partial product accumulation - **math_multiplier_carry_save** - Array-based $N \times N$ multiplier using carry-save addition

These modules demonstrate fundamental multiplication concepts and serve as reference implementations. For production designs, consider using optimized Wallace or Dadda tree multipliers.

Key Characteristics: - **Structural implementation** - Explicit cell-by-cell construction - **Educational value** - Clear demonstration of multiplication algorithm - **Array-based** - Regular structure, easy to understand - **Scalable** - Generic width support via parameter - **Combinational** - Pure logic, no sequential elements

22.2 Module Declarations

22.2.1 Basic Multiplier Cell

```
module math_multiplier_basic_cell (  
    input  logic i_i,      // Multiplier bit  
    input  logic i_j,      // Multiplicand bit  
    input  logic i_c,      // Carry input  
    input  logic i_p,      // Partial sum input  
    output logic ow_c,      // Carry output  
    output logic ow_p      // Partial sum output  
);
```

22.2.2 Carry-Save Array Multiplier

```
module math_multiplier_carry_save #(  
    parameter int N = 4    // Bit width (default: 4)  
) (  
    input  logic [ N-1:0] i_multiplier,  
    input  logic [ N-1:0] i_multiplicand,  
    output logic [2*N-1:0] ow_product  
);
```

22.3 Parameters

22.3.1 Basic Multiplier Cell

No parameters (single-bit cell).

22.3.2 Carry-Save Array Multiplier

Parameter	Type	Default	Description
N	int	4	Bit width (range: 2-32, typical: 4-16)

22.4 Ports

22.4.1 Basic Multiplier Cell Ports

Port	Direction	Width	Description
i_i	Input	1	Multiplier bit (row index)
i_j	Input	1	Multiplicand bit (column index)
i_c	Input	1	Carry input (from cell below)
i_p	Input	1	Partial sum input (from cell to the left)
ow_c	Output	1	Carry output (to cell above)
ow_p	Output	1	Partial sum output (to cell on the right)

22.4.2 Carry-Save Array Multiplier Ports

Port	Direction	Width	Description
i_multiplier	Input	N	Multiplier operand

Port	Direction	Width	Description
i_multiplicand	Input	N	(unsigned) Multiplicand operand (unsigned)
ow_product	Output	2N	Product result (unsigned)

22.5 Functionality

22.5.1 Basic Multiplier Cell

The basic cell implements a single position in the multiplication array:

Logic Operations:

```

w_p = i_i & i_j; // Partial product
bit
w_sum = i_c ^ i_p ^ w_p; // 3-input XOR
w_carry = (i_c & i_p) | (i_c & w_p) | (i_p & w_p); // Majority
function

ow_p = w_sum; // Partial sum output
ow_c = w_carry; // Carry output

```

Function: - Computes partial product bit: i_i & i_j - Adds to carry input and partial sum input using full adder - Outputs new partial sum and carry to adjacent cells

Cell Position in Array:

```

    i_c (carry from below)
      ↓
i_p → [CELL] → ow_p (to right)
      ↓
    ow_c (carry to above)

```

Cell computes: $(i_i \& i_j) + i_c + i_p$

22.5.2 Carry-Save Array Multiplier

The array multiplier creates an $N \times N$ grid of basic cells:

Structure:

Multiplicand: $j_3 \ j_2 \ j_1 \ j_0$
Multiplier:

```

i0:      [C][C][C][C]
i1:      [C][C][C][C]
i2:      [C][C][C][C]
i3:      [C][C][C][C]

```

Where each [C] is a math_multiplier_basic_cell

Connections:

- Horizontal: $ow_p[i][j] \rightarrow i_p[i][j+1]$
- Vertical: $ow_c[i][j] \rightarrow i_c[i+1][j]$
- Diagonals collect product bits

Algorithm: 1. **Generate partial products** - Each cell computes i_i & i_j 2. **Accumulate with carry-save** - Each row adds partial products 3. **Collect LSBs** - Left column provides lower product bits 4. **Final addition** - Top row sum and carries added for upper product bits

Implementation:

```

// Create N*N array of cells
genvar i, j;
generate
    for (i = 0; i < N; i++) begin : gen_row
        for (j = 0; j < N; j++) begin : gen_col
            math_multiplier_basic_cell cell (
                .i_i(i_multiplier[i]),
                .i_j(i_multiplicand[j]),
                .i_c(w_cin[i][j]),
                .i_p(w_pin[i][j]),
                .ow_c(w_cout[i][j]),
                .ow_p(w_pout[i][j])
            );
        end
    end
endgenerate

// Initialize first row
for (j = 0; j < N; j++) begin
    assign w_cin[0][j] = 1'b0;
    assign w_pin[0][j] = 1'b0;
end

// Connect rows (vertical carry propagation)
for (i = 1; i < N; i++) begin
    for (j = 0; j < N; j++) begin
        assign w_cin[i][j] = w_cout[i-1][j];
        assign w_pin[i][j] = (j == N-1) ? 1'b0 : w_pout[i-1][j+1];
    end
end
end

```

```

// Collect LSB of each row (lower product bits)
for (i = 0; i < N; i++) begin
    assign w_product[i] = w_pout[i][0];
end

// Final addition for upper bits
assign final_carries = w_cout[N-1][N-1:0];
assign final_partials = {1'b0, w_pout[N-1][N-1:1]};
assign final_sum = final_carries + final_partials;

assign ow_product = {final_sum, w_product};

```

22.5.3 Multiplication Example: 5×6 (4-bit)

Multiplicand (6): 0110
Multiplier (5): 0101

Partial Products:

i_3	(0):	0000	(0 × 0110)
i_2	(1):	0110	(1 × 0110)
i_1	(0):	0000	(0 × 0110)
i_0	(1):	0110	(1 × 0110)

Array accumulation:

```

    0110
  0000
0110
0000
-----
0001 1110 = 30 (decimal)

```

Product: $5 \times 6 = 30$ ✓

22.6 Usage Examples

22.6.1 Basic 4×4 Multiplication

```

logic [3:0] a, b;
logic [7:0] product;

```

```

math_multiplier_carry_save #(.N(4)) u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product)
);

```

```
// Example: 7 × 9 = 63
initial begin
    a = 4'd7;
    b = 4'd9;
    #1;
    assert(product == 8'd63);
end
```

22.6.2 Generic Width Multiplier

```
module flexible_multiplier #(
    parameter int WIDTH = 8
) (
    input logic [WIDTH-1:0] a, b,
    output logic [2*WIDTH-1:0] product
);

    math_multiplier_carry_save #(.N(WIDTH)) u_mult (
        .i_multiplier(a),
        .i_multiplicand(b),
        .ow_product(product)
    );

endmodule
```

```
// Instantiate with different widths
flexible_multiplier #(.WIDTH(8)) u_mult8 (...);
flexible_multiplier #(.WIDTH(16)) u_mult16 (...);
```

22.6.3 Building Custom Array from Cells

```
// Manual 2x2 multiplier using basic cells
module mult_2x2 (
    input logic [1:0] i_multiplier,
    input logic [1:0] i_multiplicand,
    output logic [3:0] ow_product
);

    logic c[1:0][1:0]; // Carry signals
    logic p[1:0][1:0]; // Partial sums

    // Row 0, Column 0
    math_multiplier_basic_cell cell_00 (
        .i_i(i_multiplier[0]),
        .i_j(i_multiplicand[0]),
        .i_c(1'b0),
        .i_p(1'b0),
        .ow_c(c[0][0]),
```

```

        .ow_p(p[0][0])
    );
    assign ow_product[0] = p[0][0];

    // Row 0, Column 1
    math_multiplier_basic_cell cell_01 (
        .i_i(i_multiplier[0]),
        .i_j(i_multiplicand[1]),
        .i_c(1'b0),
        .i_p(1'b0),
        .ow_c(c[0][1]),
        .ow_p(p[0][1])
    );

    // Row 1, Column 0
    math_multiplier_basic_cell cell_10 (
        .i_i(i_multiplier[1]),
        .i_j(i_multiplicand[0]),
        .i_c(c[0][0]),
        .i_p(p[0][1]),
        .ow_c(c[1][0]),
        .ow_p(p[1][0])
    );
    assign ow_product[1] = p[1][0];

    // Row 1, Column 1
    math_multiplier_basic_cell cell_11 (
        .i_i(i_multiplier[1]),
        .i_j(i_multiplicand[1]),
        .i_c(c[0][1]),
        .i_p(1'b0),
        .ow_c(c[1][1]),
        .ow_p(p[1][1])
    );

    // Final addition
    assign ow_product[3:2] = c[1][0] + p[1][1] + c[1][1];

```

endmodule

22.6.4 Multiplier with Valid/Ready Handshake

```

module handshake_multiplier #(
    parameter int N = 8
) (
    input    logic          clk,
    input    logic          rst_n,
    input    logic [N-1:0] i_multiplier,

```

```

    input logic [N-1:0]      i_multiplicand,
    input logic              i_valid,
    output logic             o_ready,
    output logic [2*N-1:0]   o_product,
    output logic             o_valid
);

    logic [2*N-1:0] product_comb;

    // Combinational multiplier
    math_multiplier_carry_save #(.N(N)) u_mult (
        .i_multiplier(i_multiplier),
        .i_multiplicand(i_multiplicand),
        .ow_product(product_comb)
    );

    // Pipeline register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            o_product <= '0;
            o_valid    <= 1'b0;
        end else begin
            o_product <= product_comb;
            o_valid    <= i_valid;
        end
    end

    assign o_ready = 1'b1; // Always ready (combinational)

endmodule

```

22.7 Timing Characteristics

Metric	4-bit	8-bit	16-bit
Logic Depth	~8 levels	~16 levels	~32 levels
Typical Delay (ns)	~3.2	~6.4	~12.8
Max Frequency	~310 MHz	~155 MHz	~78 MHz

Logic Depth Breakdown: - Each cell: 2 levels (AND + full adder) - Array depth: 2N levels (carry ripples down rows) - Final addition: Additional adder delay

Critical Path:

$i_multiplier[N-1] \rightarrow PP[N-1][0] \rightarrow \text{row } N-1 \text{ accumulation} \rightarrow \text{final addition}$
 $\rightarrow ow_product[2N-1]$

Scaling: $O(N)$ depth - doubles with doubling of bit width

22.8 Performance Characteristics

22.8.1 Resource Utilization

Width	Basic Cells	Final Adder	AND Gates	Estimated LUTs
4-bit	16	4-bit	16	~48
8-bit	64	8-bit	64	~192
16-bit	256	16-bit	256	~768

Area Formula: N^2 basic cells + N -bit final adder $\approx 3N^2$ LUTs

22.8.2 Comparison to Optimized Multipliers

Architecture	8-bit LUTs	8-bit Delay	Best Use Case
Array (Basic Cell)	~192	~6.4 ns	Educational, low speed
Carry-Save Array	~192	~6.4 ns	Simple, moderate speed
Wallace Tree	~120	~3.5 ns	High speed, larger area
Dadda Tree	~108	~3.2 ns	Production (best balance)
Booth Radix-4	~100	~4.5 ns	Signed multiplication

Key Observation: Array multipliers are $\sim 2\times$ slower but simpler than tree multipliers.

22.8.3 When to Use Array Multipliers

Appropriate Use Cases: - Educational demonstrations - Low-frequency designs (<100 MHz for 8-bit) - Simple prototypes - Designs where area is more critical than speed - Parameterized width support needed

Prefer Optimized Multipliers When: - High speed required $\rightarrow$ Wallace or Dadda tree - Area critical $\rightarrow$ Dadda tree (optimized CSA) - Signed operands $\rightarrow$ Booth multiplier - Behavioral sufficient $\rightarrow$ Use $*$ operator (synthesis optimizes)

22.9 Design Considerations

22.9.1 Advantages

✓ **Simple structure** - Regular array, easy to understand ✓ **Parameterizable** - Generic width support via parameter ✓ **Educational value** - Clearly demonstrates multiplication algorithm ✓ **Moderate area** - Reasonable gate count for small widths ✓ **Purely combinational** - No state, easy to integrate

22.9.2 Limitations

⚠ **Slow for large widths** - $O(N)$ depth vs $O(\log N)$ for trees ⚠ **Poor scaling** - Area grows as N^2 , delay as N ⚠ **Not optimal** - Better alternatives exist (Dadda, Wallace) ⚠ **Unsigned only** - Requires sign handling for signed operands

22.9.3 Basic Cell Design Notes

Verilator Lint Suppression:

```
/* verilator lint_off UNOPTFLAT */
output logic ow_c, ow_p
/* verilator lint_on UNOPTFLAT */
```

Reason: Array structure creates combinational feedback paths that are intentional. The explicit output buffering (always_comb block) breaks the path for simulation.

22.9.4 Synthesis Considerations

For FPGA Designs:

```
# Let synthesis tool optimize array structure
set_dont_touch false

# Enable register retiming if pipelining
set_optimize_registers true
```

For ASIC Designs:

```
# Flatten for better optimization
set_flatten true

# May benefit from custom standard cell multipliers
```

Note: Modern synthesis tools often recognize multiplication patterns and replace with optimized implementations.

22.9.5 Pipelining Strategy

For higher frequency, add pipeline stages:

2-Stage Pipeline:

```
// Stage 1: Register inputs
always_ff @(posedge clk) begin
    a_reg <= a;
    b_reg <= b;
end

// Stage 2: Multiply and register output
math_multiplier_carry_save #(.N(8)) u_mult (
    .i_multiplier(a_reg),
    .i_multiplicand(b_reg),
    .ow_product(product_comb)
);

always_ff @(posedge clk) begin
    product_reg <= product_comb;
end
```

Multi-Stage Pipeline (split array into stages):

```
// Register after every K rows
// Example: 16-row array split into 4 stages (4 rows each)
```

22.10 Common Pitfalls

✗ Anti-Pattern 1: Using for high-speed designs

```
// WRONG: Array multiplier too slow for 200 MHz with 16-bit
math_multiplier_carry_save #(.N(16)) u_mult (...);
// Critical path ~13ns, can't meet 5ns period!
```

```
// RIGHT: Use Dadda tree for high-speed
math_multiplier_dadda_tree_016 u_mult (...);
```

✗ Anti-Pattern 2: Not considering behavioral alternative

```
// MANUAL: Explicit array instantiation
math_multiplier_carry_save #(.N(8)) u_mult (
    .i_multiplier(a), .i_multiplicand(b), .ow_product(p)
);
```

```
// SIMPLER: Let synthesis optimize
assign p = a * b; // Synthesis may produce better result!
```

When to use explicit vs behavioral: - Explicit: Educational, specific structure needed, debugging
- Behavioral: Production code, synthesis tool optimization

✗ Anti-Pattern 3: Expecting signed multiplication

```
// WRONG: Signed operands misinterpreted
logic signed [7:0] a = -5, b = 3;
math_multiplier_carry_save #(.N(8)) u_mult (...);
// Result: 753 (unsigned 251 × 3), not -15!
```

```
// RIGHT: Convert to unsigned, fix sign
// (See signed multiplication techniques)
```

✗ Anti-Pattern 4: Ignoring width limits

```
// CAREFUL: 32×32 array multiplier is very large and slow
math_multiplier_carry_save #(.N(32)) u_mult (...);
// 1024 cells, ~32 levels deep, ~15ns delay

// CONSIDER: Dadda tree or behavioral for large widths
```

22.11 Educational Value

22.11.1 Understanding Multiplication

The array multiplier clearly demonstrates:

1. Partial Product Generation:

Each row computes: multiplicand × multiplier_bit[i]
Just like manual multiplication on paper!

2. Shifted Addition:

Each row is shifted left by one position
Carry propagation adds partial products

3. Carry-Save Concept:

Carries propagate downward (next row)
Sums propagate rightward (same row)
Defers final carry-propagate until end

22.11.2 Progression to Optimized Multipliers

Learning Path: 1. **Array Multiplier** (this document) - Understand basic concept 2. **Wallace Tree** - Learn parallel reduction 3. **Dadda Tree** - Learn optimization techniques 4. **Booth Encoding** - Learn signed multiplication

Concept: Array → parallelize → optimize → specialize

22.12 Related Modules

- ****math_multiplier_wallace_tree_.sv**** - Fast tree-based multiplication

- **`**math_multiplier_dadda_tree*.sv**`** - Optimized tree-based multiplication
- **`math_adder_carry_save.sv`** - 3:2 compressor used in trees
- **`math_adder_full.sv`** - Full adder primitive
- **`math_adder_half.sv`** - Half adder primitive

22.13 References

- Parhami, B. “Computer Arithmetic: Algorithms and Hardware Designs.” Oxford University Press, 2010.
- Ercegovac, M.D., Lang, T. “Digital Arithmetic.” Morgan Kaufmann, 2003.
- Weste, N.H.E., Harris, D. “CMOS VLSI Design: A Circuits and Systems Perspective.” Addison-Wesley, 2011.

22.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

23 Dadda Multiplier with 4:2 Compressors

An area-optimized 8x8 unsigned multiplier using Dadda reduction with 4:2 compressors, providing fast parallel multiplication with ~25% fewer reduction stages than traditional 3:2 CSA-based Dadda trees.

23.1 Overview

The `math_multiplier_dadda_4to2_008` module implements an 8-bit unsigned multiplier optimized for BF16 mantissa multiplication. It uses 4:2 compressors instead of 3:2 carry-save adders (CSAs), enabling faster column height reduction with fewer stages.

Key Features: - **8x8 unsigned multiply** - 16-bit product output - **4:2 compressor-based** - Faster height reduction than 3:2 CSA - **Dadda scheduling** - Optimal reduction order for minimum area - **Han-Carlson final CPA** - Fast carry-propagate addition - **Purely combinational** - Single-cycle operation

23.2 Module Declaration

```
module math_multiplier_dadda_4to2_008 #(
    parameter int N = 8
) (
    input logic [N-1:0] i_multiplier,
    input logic [N-1:0] i_multiplicand,
```

```

        output logic [2*N-1:0] ow_product
    );

```

23.3 Parameters

Parameter	Type	Default	Description
N	int	8	Operand width (fixed at 8 for this variant)

23.4 Ports

Port	Direction	Width	Description
i_multiplier	Input	8	Multiplier operand (unsigned)
i_multiplicand	Input	8	Multiplicand operand (unsigned)
ow_product	Output	16	Full-precision product (unsigned)

23.5 Architecture

23.5.1 Three-Stage Pipeline

1. **Partial Product Generation** - 64 AND gates create 8x8 bit matrix
2. **Dadda Reduction** - 4:2 compressors reduce columns to 2 rows
3. **Final CPA** - Han-Carlson adder produces final product

23.5.2 Partial Product Matrix

For 8x8 multiplication, partial products form a diamond pattern:

```

Column:    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
Height:    1  2  3  4  5  6  7  8  7  6  5  4  3  2  1

```

```

        pp00
      pp01 pp10
    pp02 pp11 pp20
  pp03 pp12 pp21 pp30

```

```

pp04 pp13 pp22 pp31 pp40
...
      pp77

```

Maximum column height = 8 (columns 7)

23.5.3 Dadda Reduction with 4:2 Compressors

4:2 Compressor Advantage: - Reduces 5 inputs to 3 outputs (vs 3:2 for standard CSA) - Effectively removes 2 bits from column height per compressor - Cout independent of Cin enables parallel carry chains

Dadda Height Sequence:

$d(j) = \{2, 3, 4, 6, 9, 13, 19, 28, \dots\}$
Rule: $d(j+1) = \text{floor}(1.5 * d(j))$

Reduction Stages for 8-bit (max height = 8):

```

Start: height 8
Stage 1: reduce to 6 (columns > 6 get 4:2 compressors)
Stage 2: reduce to 4
Stage 3: reduce to 3
Stage 4: reduce to 2 (final for CPA)

```

23.5.4 Reduction Example (Column 7)

Initial height: 8 bits
pp07, pp16, pp25, pp34, pp43, pp52, pp61, pp70

```

Stage 1: Reduce 8 -> 6
4:2 compressor: (pp07, pp16, pp25, pp34, cin) -> sum1, carry1, cout1
Remaining: sum1, pp43, pp52, pp61, pp70 + carry1, cout1 from col 6
Height after: 6 (might be more with carries from col 6)

```

```

Stage 2: Reduce to 4
Continue with 4:2 and 3:2 compressors as needed

```

...

Final: 2 rows for CPA input

23.5.5 Component Instantiation

```

// Partial products (64 AND gates)
wire w_pp_0_0 = i_multiplier[0] & i_multiplicand[0];
wire w_pp_0_1 = i_multiplier[0] & i_multiplicand[1];
// ... 64 total

```

```

// 4:2 Compressors for reduction
math_compressor_4to2 u_c4to2_07_000 (
    .i_x1(w_pp_0_7), .i_x2(w_pp_1_6),
    .i_x3(w_pp_2_5), .i_x4(w_pp_3_4),
    .i_cin(1'b0),
    .ow_sum(w_c4to2_sum_07_000),
    .ow_carry(w_c4to2_carry_07_000),
    .ow_cout(w_c4to2_cout_07_000)
);

// Half adders for 2-input reduction
math_adder_half u_ha_01_000 (
    .i_a(w_pp_0_1), .i_b(w_pp_1_0),
    .ow_sum(w_ha_sum_01_000),
    .ow_carry(w_ha_carry_01_000)
);

// Full adders for 3-input reduction
math_adder_full u_fa_02_000 (
    .i_a(w_pp_0_2), .i_b(w_pp_1_1), .i_c(w_pp_2_0),
    .ow_sum(w_fa_sum_02_000),
    .ow_carry(w_fa_carry_02_000)
);

// Final CPA
math_adder_han_carlson_016 u_final_cpa (
    .i_a(w_cpa_a),
    .i_b(w_cpa_b),
    .i_cin(1'b0),
    .ow_sum(ow_product),
    .ow_cout(w_cpa_cout)
);

```

23.6 Timing Characteristics

Metric	Value
Partial Product Gen	1 gate level (AND)
Dadda Reduction	~4 stages of 4:2 compressors
Final CPA	5-6 levels (Han-Carlson 16-bit)
Total Depth	~13-15 gate levels

23.6.1 Critical Path

i_multiplier[7] -> pp_7_7 -> Stage 1 4:2 -> Stage 2 4:2 -> Stage 3 4:2 -> Stage 4 HA/FA -> CPA -> ow_product[15]

23.7 Usage Examples

23.7.1 Basic 8x8 Multiplication

```
logic [7:0] a, b;
logic [15:0] product;

math_multiplier_dadda_4to2_008 u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product)
);
```

```
// Example: 12 * 13 = 156
initial begin
    a = 8'd12;
    b = 8'd13;
    #1;
    assert(product == 16'd156);
end
```

23.7.2 In BF16 Mantissa Multiplier

```
// BF16 mantissa with implied leading 1
wire [7:0] w_mant_a_ext = {i_a_is_normal, i_mant_a}; // 1.mantissa
wire [7:0] w_mant_b_ext = {i_b_is_normal, i_mant_b}; // 1.mantissa

wire [15:0] w_product;

math_multiplier_dadda_4to2_008 u_mant_mult (
    .i_multiplier(w_mant_a_ext),
    .i_multiplicand(w_mant_b_ext),
    .ow_product(w_product)
);
```

```
// Product is in format: 1x.xxxxxxx_xxxxxxxx or 01.xxxxxxx_xxxxxxxx
wire w_needs_norm = w_product[15]; // If MSB set, product >= 2.0
```

23.7.3 With Pipeline Register

```
logic [7:0] a_reg, b_reg;
logic [15:0] product_comb, product_reg;

// Input register
always_ff @(posedge clk) begin
    a_reg <= a;
    b_reg <= b;
end
```

```
// Multiplier (combinational)
math_multiplier_dadda_4to2_008 u_mult (
    .i_multiplier(a_reg),
    .i_multiplicand(b_reg),
    .ow_product(product_comb)
);

// Output register
always_ff @(posedge clk) begin
    product_reg <= product_comb;
end
```

23.8 Performance Characteristics

23.8.1 Resource Utilization

Component	Count
AND gates (partial products)	64
4:2 Compressors	~12-15
Full Adders (3:2)	~8-12
Half Adders	~4-6
Han-Carlson 16-bit CPA	1
Total LUTs (est.)	~120-140

23.8.2 Comparison: 4:2 vs 3:2 Dadda

Metric	3:2 CSA Dadda	4:2 Compressor Dadda
Reduction stages	4-5	3-4
Total adders/compressors	~50	~35
Logic depth	~17-20	~13-15
Area (LUTs)	~130	~130

Advantage: 4:2 compressors reduce stage count by ~25% with similar area.

23.8.3 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Dadda scheduling minimizes total compressor count 2. **Wire complexity** - Regular structure from systematic reduction 3. **Logic depth** - 4:2 compressors reduce critical path stages

23.9 Design Considerations

23.9.1 Advantages

- **Fast** - $O(\log N)$ depth for reduction + CPA
- **Area-efficient** - Dadda scheduling minimizes hardware
- **Regular structure** - Easy to synthesize and optimize
- **Full precision** - No truncation of result

23.9.2 Limitations

- **Unsigned only** - Requires sign handling for signed operands
- **Fixed width** - $N=8$ is hardcoded in this variant
- **Combinational** - May need pipelining for high frequency

23.9.3 Signed Multiplication

For signed multiplication, use absolute values and fix sign:

```
// Convert to unsigned
wire [7:0] a_abs = a[7] ? (~a + 1'b1) : a;
wire [7:0] b_abs = b[7] ? (~b + 1'b1) : b;
wire      sign_result = a[7] ^ b[7];

// Multiply unsigned
math_multiplier_dadda_4to2_008 u_mult (
    .i_multiplier(a_abs),
    .i_multiplicand(b_abs),
    .ow_product(product_unsigned)
);

// Fix sign
wire [15:0] product_signed = sign_result ? (~product_unsigned + 1'b1)
                                         : product_unsigned;
```

23.10 Auto-Generated Code

This module is auto-generated by Python scripts: - **Generator**:

```
bin/rtl_generators/bf16/dadda_4to2_multiplier.py - Regenerate: PYTHONPATH=bin:
$PYTHONPATH python3 bin/rtl_generators/bf16/generate_all.py rtl/common
```

Do not edit the generated .sv file manually. Modify the generator script instead.

23.11 Related Modules

- **math_compressor_4to2** - 4:2 compressor building block

- **math_adder_full** - Full adder (3:2 compressor)
- **math_adder_half** - Half adder
- **math_adder_han_carlson_016** - Final CPA
- **math_bf16_mantissa_mult** - BF16 wrapper using this multiplier
- **math_multiplier_dadda_tree_008** - Alternative 3:2 CSA version

23.12 References

- Dadda, L. "Some Schemes for Parallel Multipliers." *Alta Frequenza* 34, 1965.
- Weinberger, A. "4:2 Carry-Save Adder Module." *IBM Technical Disclosure Bulletin*, 1981.
- Oklobdzija, V.G. "A Method for Speed Optimized Partial Product Reduction." *IEEE Trans. Computers*, 1996.

23.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

24 Dadda Tree Multipliers

Area-optimized fast parallel multipliers using scheduled partial product reduction with carry-save adders. These modules provide unsigned integer multiplication for 8×8, 16×16, and 32×32 operations. The reduction tree has logarithmic depth and uses the minimum number of compressors for that depth; the final carry-propagate adder is a Brent-Kung parallel-prefix adder, which is also logarithmic depth, so end-to-end delay is $O(\log N)$.

24.1 Overview

The Dadda tree multiplier family implements high-speed multiplication using an **optimized schedule** of 3:2 compressors (carry-save adders). Unlike Wallace trees, which compress everything they can as early as they can, Dadda trees defer compression: each stage only reduces columns down to a precomputed target height. This reaches height 2 in the same number of stages as Wallace while instantiating measurably fewer compressors.

Key Features: - **Logarithmic reduction depth** - 4 stages for 8-bit, 6 for 16-bit, 8 for 32-bit - **Minimum compressor count** for that depth - 42 cells for 8×8 versus Wallace's 61 - **Structured reduction** - follows the canonical Dadda target-height sequence - **Fixed-width variants** - generated for 8, 16, and 32-bit operands - **Purely combinational** - single-cycle multiplication - **Self-contained** - includes its own final adder; no external adder required

Architecture: 1. **Partial Product Generation** - AND gates create the $N \times N$ matrix 2. **Dadda Reduction Schedule** - scheduled CSA stages reduce every column to height 2 3. **Final Addition** - an on-chip Brent-Kung parallel-prefix carry-propagate adder sums the two surviving rows into `ow_product`

Note on the final adder: these modules are complete multipliers. The reduction tree stops at column height 2, the two remaining rows are packed into the $2N$ -bit vectors `w_cpa_row0` / `w_cpa_row1`, and those are summed internally by a single `math_adder_brent_kung_{2N}` instance named `u_final_cpa`. Earlier revisions of this module used a ripple carry-propagate adder in that position, and revisions before that collapsed every column to height 1 with no final adder at all; neither is the case now, and no external adder is needed.

The CPA width is the **product** width, not the operand width: the 8-bit multiplier instantiates `math_adder_brent_kung_016`, the 16-bit one `math_adder_brent_kung_032`, and the 32-bit one `math_adder_brent_kung_064`. Carry-in is tied to `1'b0`, and the adder's carry-out is left unread on `w_cpa_carry_unused` - an $N \times N$ product is strictly less than 2^{2N} , so the top column can never carry out.

24.2 Module Declarations

24.2.1 8-bit Dadda Tree Multiplier

```
module math_multiplier_dadda_tree_008 #(
    parameter int N = 8
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

24.2.2 16-bit Dadda Tree Multiplier

```
module math_multiplier_dadda_tree_016 #(
    parameter int N = 16
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

24.2.3 32-bit Dadda Tree Multiplier

```
module math_multiplier_dadda_tree_032 #(
    parameter int N = 32
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

24.3 Parameters

Parameter	Type	Default	Description
N	int	8/16/32	Bit width (fixed per variant)

Note: The N parameter is present but fixed per module variant. It is not intended for user modification.

24.4 Ports

Port	Direction	Width	Description
i_multiplier	Input	N	Multiplier operand (unsigned)
i_multiplicand	Input	N	Multiplicand operand (unsigned)
ow_product	Output	2N	Product result (unsigned)

Signal Types: - **Unsigned only** - All operands and results are unsigned integers - **Full precision** - Output is full 2N-bit product (no truncation)

24.5 Functionality

24.5.1 Dadda Reduction Algorithm

The Dadda tree uses an **optimized reduction schedule** rather than immediate reduction:

Step 1: Calculate Reduction Schedule

Define target heights: $d(1) = 2$, $d(j+1) = \lceil 1.5 \times d(j) \rceil$

Sequence: 2, 3, 4, 6, 9, 13, 19, 28, 42, ...

Pick the smallest sequence entry that is greater than or equal to the tallest column, then walk the sequence downwards. Each reduction stage takes every column to at or below the next target height.

Step 2: Partial Product Generation

For $N \times N$ multiplication:

- Generate N^2 partial products: $PP[i][j] = multiplier[i] \&$

multiplicand[j]
- Arrange in diagonal columns (like manual multiplication)

Step 3: Scheduled Reduction

For each stage k (from high to low in d -sequence):
 For each column:
 While column height $> d(k)$:
 Use Full Adder (3→2 reduction)
 If column height $= d(k)$ and can compress:
 Use Half Adder (2→2 compression)
 Otherwise:
 Pass through to next stage

Step 4: Final Addition

Every column is now at height 2. Pack the two surviving rows into two 2N-bit vectors and sum them with a Brent-Kung parallel-prefix carry-propagate adder to produce the 2N-bit product.

Key Difference from Wallace: Dadda **waits** to reduce until a column exceeds the stage target, so it spends the fewest compressors that still reach height 2 in the same number of stages Wallace needs.

24.5.2 Dadda Sequence Example

For 8-bit multiplication:

Initial heights (15 columns): [1, 2, 3, 4, 5, 6, 7, 8, 7, 6, 5, 4, 3, 2, 1]

Tallest column is 8, so the first target is the sequence entry above it: 9
is not needed because stage targets are applied downwards from 6.

Stage targets: $6 \rightarrow 4 \rightarrow 3 \rightarrow 2$

Stage 1: Reduce to height 6
Stage 2: Reduce to height 4
Stage 3: Reduce to height 3
Stage 4: Reduce to height 2
- All 15 columns now at height 2; hand off to the final adder

The longer sequences apply to the wider variants:

Variant	Tallest column	Stage targets	Stages
8-bit	8	$6 \rightarrow 4 \rightarrow 3 \rightarrow 2$	4
16-bit	16	$13 \rightarrow 9 \rightarrow 6 \rightarrow 4$	6

Variant	Tallest column	Stage targets	Stages
		$\rightarrow 3 \rightarrow 2$	
32-bit	32	$28 \rightarrow 19 \rightarrow 13 \rightarrow 8$	
		$9 \rightarrow 6 \rightarrow 4 \rightarrow 3$	
		$\rightarrow 2$	

24.5.38-bit Implementation Structure

Generated signals are named `w_sum_{column}_{op}` / `w_carry_{column}_{op}`, and instances are named `CSA_{column}_{op}` or `HA_{column}_{op}`, where `op` is the operation index within that column. Excerpts below are taken verbatim from `rtl/common/math_multiplier_dadda_tree_008.sv`.

```
// Partial Products (64 AND gates for 8x8)
wire w_pp_0_0 = i_multiplier[0] & i_multiplicand[0];
wire w_pp_0_1 = i_multiplier[0] & i_multiplicand[1];
// ... 64 total partial products
wire w_pp_7_7 = i_multiplier[7] & i_multiplicand[7];

// Dadda reduction stage 1: max column height 6
// Column 6 has height 7, so exactly one 2:2 compression brings it to 6.
wire w_sum_06_01, w_carry_06_01;
math_adder_half HA_06_01 (
    .i_a(w_pp_0_6),
    .i_b(w_pp_1_5),
    .ow_sum(w_sum_06_01),
    .ow_carry(w_carry_06_01)
);

// Column 7 has height 8 and needs a 3:2 plus a 2:2 to reach 6.
wire w_sum_07_01, w_carry_07_01;
math_adder_carry_save CSA_07_01 (
    .i_a(w_pp_0_7),
    .i_b(w_pp_1_6),
    .i_c(w_pp_2_5),
    .ow_sum(w_sum_07_01),
    .ow_carry(w_carry_07_01)
);
wire w_sum_07_02, w_carry_07_02;
math_adder_half HA_07_02 (
    .i_a(w_pp_3_4),
    .i_b(w_pp_4_3),
    .ow_sum(w_sum_07_02),
    .ow_carry(w_carry_07_02)
);
```

```

// ... stages 2 and 3 follow the same pattern with targets 4 and 3

// Dadda reduction stage 4: max column height 2
// Later stages consume sums and carries produced by earlier stages.
wire w_sum_04_03, w_carry_04_03;
math_adder_carry_save CSA_04_03 (
    .i_a(w_sum_04_01),
    .i_b(w_carry_03_01),
    .i_c(w_sum_04_02),
    .ow_sum(w_sum_04_03),
    .ow_carry(w_carry_04_03)
);

```

Once every column is at height 2, the two surviving rows are packed into a pair of 16-bit vectors and handed to one Brent-Kung prefix adder. There are no `math_adder_full` or `math_adder_half` instances in this stage at all - every half and full adder in the file belongs to the reduction tree:

```

// Final addition stage: two reduced rows into a Brent-Kung CPA
wire [15:0] w_cpa_row0 = {
    1'b0,
    w_pp_7_7,
    // ... one bit per column, taken from the surviving row
    w_pp_2_0,
    w_pp_0_1,
    w_pp_0_0
};
wire [15:0] w_cpa_row1 = {
    1'b0,
    w_carry_13_01,
    w_sum_13_01,
    // ... one bit per column, taken from the other surviving row
    w_sum_02_01,
    w_pp_1_0,
    1'b0
};

/* verilator lint_off UNUSED SIGNAL */
wire w_cpa_carry_unused;
/* verilator lint_on UNUSED SIGNAL */
math_adder_brent_kung_016 #(
    .N(16)
) u_final_cpa (
    .i_a(w_cpa_row0),
    .i_b(w_cpa_row1),
    .i_c(1'b0),
    .ow_sum(ow_product),

```

```

        .ow_carry(w_cpa_carry_unused)
    );

```

The prefix adder drives `ow_product` directly, so there is no per-bit assign `ow_product[i]` fan-out stage either.

24.5.4 Reduction Comparison: Dadda vs Wallace

Both trees reach column height 2 in the **same number of stages**. The difference is entirely in how many compressors they spend getting there. The following counts are instance counts from the generated RTL.

For 8×8 multiplication:

	Dadda	Wallace
Reduction stages / layers	4	4
Reduction 3:2 compressors	35	36
Reduction half adders	7	25
Total reduction cells	42	61
Final CPA	math_adder_brent_kung_016	math_adder_brent_kung_016

The 8×8 figure of 35 full adders and 7 half adders is exactly the textbook Dadda count.

Across all widths:

Width	Stages	Reduction CSAs	Reduction HAs	Final CPA
8-bit	4	35	7	math_adder_brent_kung_016
16-bit	6	195	15	math_adder_brent_kung_032

Width	Stages	Reduction CSAs	Reduction HAs	Final CPA
32-bit	8	899	31	math_adder _brent_kun g_064

Result: Dadda reaches height 2 in the same depth as Wallace while spending 19 fewer cells at 8×8.

Wallace used to buy something back for that extra hardware. When both multipliers ended in a ripple CPA, Wallace’s eager compression flattened the low-order columns to height 1 early, so its ripple spanned only 11 columns against Dadda’s 14 - a shorter serial carry chain that partly offset the larger tree. **That offset no longer exists.** Both multipliers now feed a full-width Brent-Kung prefix adder over all 2N columns, and a prefix adder’s depth is logarithmic in its width regardless of how many low-order inputs happen to be zero. The two designs therefore have an *identical* final adder, in both cell count and delay.

What remains is the pure statement of the tradeoff: Wallace spends more compressor cells than Dadda to reach the same reduction depth, and now gets nothing in return. Dadda is the better choice on both axes.

24.6 Usage Examples

24.6.1 Basic 8×8 Multiplication

```

logic [7:0] a, b;
logic [15:0] product;

math_multiplier_dadda_tree_008 u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product)
);

// Example: 12 × 13 = 156
initial begin
    a = 8'd12;
    b = 8'd13;
    #1; // Allow combinational delay
    assert(product == 16'd156);
end

```

24.6.2 16×16 Multiplication with Pipeline

```

logic [15:0] a, b;
logic [31:0] product_comb, product_reg;
logic clk, rst_n;

// Dadda tree multiplier (combinational)

```

```

math_multiplier_dadda_tree_016 u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product_comb)
);

// Output register for timing closure
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        product_reg <= '0;
    else
        product_reg <= product_comb;
end

```

24.6.3 Multiply-Accumulate (MAC) Unit

```

module mac_unit (
    input logic      clk,
    input logic      rst_n,
    input logic [15:0] a, b,
    input logic      accumulate, // 1=accumulate, 0=clear
    output logic [31:0] result
);

    logic [31:0] product;
    logic [31:0] accumulator;

    // Dadda tree multiplier
    math_multiplier_dadda_tree_016 u_mult (
        .i_multiplier(a),
        .i_multiplicand(b),
        .ow_product(product)
    );

    // Accumulator
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            accumulator <= '0;
        else if (accumulate)
            accumulator <= accumulator + product;
        else
            accumulator <= product; // Clear and load
    end

    assign result = accumulator;
endmodule

```

24.6.4 FIR Filter Tap

```
module fir_tap #(
    parameter int DATA_WIDTH = 16,
    parameter int COEFF_WIDTH = 16
) (
    input logic clk,
    input logic rst_n,
    input logic [DATA_WIDTH-1:0] data_in,
    input logic [COEFF_WIDTH-1:0] coefficient,
    input logic [31:0] partial_sum_in,
    output logic [31:0] partial_sum_out
);

    logic [31:0] product;

    // Multiply coefficient by data
    math_multiplier_dadda_tree_016 u_mult (
        .i_multiplier(coefficient),
        .i_multiplicand(data_in),
        .ow_product(product)
    );

    // Add to partial sum (pipeline stage)
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            partial_sum_out <= '0;
        else
            partial_sum_out <= partial_sum_in + product;
    end

endmodule
```

24.6.5 Parameterized Multiplier Selector

```
module flexible_multiplier #(
    parameter int WIDTH = 8
) (
    input logic [WIDTH-1:0] i_multiplier,
    input logic [WIDTH-1:0] i_multiplicand,
    output logic [2*WIDTH-1:0] ow_product
);

    generate
        if (WIDTH == 8) begin : gen_8bit
            math_multiplier_dadda_tree_008 u_mult (
                .i_multiplier(i_multiplier),
                .i_multiplicand(i_multiplicand),
```

```

        .ow_product(ow_product)
    );
end else if (WIDTH == 16) begin : gen_16bit
    math_multiplier_dadda_tree_016 u_mult (
        .i_multiplier(i_multiplier),
        .i_multiplicand(i_multiplicand),
        .ow_product(ow_product)
    );
end else if (WIDTH == 32) begin : gen_32bit
    math_multiplier_dadda_tree_032 u_mult (
        .i_multiplier(i_multiplier),
        .i_multiplicand(i_multiplicand),
        .ow_product(ow_product)
    );
end else begin : gen_default
    // Fallback: behavioral multiplication
    assign ow_product = i_multiplier * i_multiplicand;
end
endgenerate

endmodule

```

24.7 Timing Characteristics

Metric	8-bit	16-bit	32-bit
Logic Depth	~13-15 levels	~17-20 levels	~22-28 levels
Typical Delay (ns)	~6.5-7.5	~8.5-10.5	~11-14
Max Frequency	~130-150 MHz	~95-115 MHz	~70-90 MHz

Logic Depth Breakdown: - Partial product generation: 1 level (AND gates) - Dadda reduction stages: logarithmic - 4 / 6 / 8 stages for 8 / 16 / 32-bit - Final addition: on-chip **Brent-Kung** parallel-prefix carry-propagate adder, $O(\log N)$ levels

Critical Path:

$i_multiplier[N-1] \rightarrow PP[N-1][N-1] \rightarrow \text{Stage 1 CSA} \rightarrow \dots \rightarrow \text{Stage K CSA} \rightarrow \text{Brent-Kung prefix network } (\sim 2 \cdot \log_2(2N) \text{ levels}) \rightarrow ow_product[2N-1]$

Important: both halves of the datapath are logarithmic. The reduction tree is log-depth, and the final adder is a Brent-Kung parallel-prefix adder, which is also log-depth, so the **end-to-end delay of these modules is $O(\log N)$** . There is no serial carry chain anywhere in the design. The 32-bit variant, for example, follows an 8-stage tree with a 64-bit prefix network rather than the 61-deep ripple it used to carry.

24.8 Performance Characteristics

24.8.1 Resource Utilization

Instance counts below are exact, taken from the generated RTL. “3:2 cells” counts `math_adder_carry_save` and “half adders” counts `math_adder_half`; every one of them lives in the reduction tree. The final CPA is a single prefix-adder instance and contributes no discrete adder cells, so `math_adder_full` does not appear in these files at all.

Width	3:2 cells (tree)	Half adders (tree)	Final CPA	AND Gates	Total adder cells
8-bit	35	7	1 × <code>math_adder_brent_kung_016</code>	64	42
16-bit	195	15	1 × <code>math_adder_brent_kung_032</code>	256	210
32-bit	899	31	1 × <code>math_adder_brent_kung_064</code>	1024	930

Area Comparison (measured cell counts, Dadda versus Wallace):

Both families now instantiate the same Brent-Kung CPA at the same width, so the adder-cell totals below compare reduction trees on equal terms.

Width	Dadda total cells	Wallace total cells	Dadda saving
8-bit	42	61	31%
16-bit	210	274	23%
32-bit	930	1116	17%

Totals count every instantiated discrete adder cell in the corresponding generated .sv file, all of which are in the reduction tree. The shared Brent-Kung CPA is identical in both families and is excluded from the totals; adding it back shifts both columns by the same amount. LUT and gate-level area will differ by technology and synthesis settings; these are structural counts, not synthesis results.

Key Advantage: for the same reduction depth, Dadda instantiates substantially fewer compressor cells than Wallace.

24.8.2 Multiplier Architecture Selection

Requirement	Best Choice	Reasoning
High-speed unsigned	Dadda Tree	Log-depth tree with the fewest compressors
Signed multiplication	Booth Radix-4	Fewer partial products
Minimal area	Array Multiplier	Sequential, low gates
Variable width	Behavioral (*)	Synthesis optimizes
Very high frequency	Pipelined Dadda	Split into stages

24.9 Design Considerations

24.9.1 Advantages

✓ **Smallest fast multiplier** - 17-31% fewer adder cells than Wallace at the same reduction depth, with an identical final adder ✓ **Optimized structure** - Mathematically proven reduction schedule ✓ **Fewer cells to synthesize** - same stage count as Wallace, less hardware in each stage ✓ **Pure combinational** - Easy to pipeline where needed ✓ **Scalable** - Algorithm extends to any bit width

24.9.2 Limitations

⚠ **Complex design** - Reduction schedule requires calculation ⚠ **Unsigned only** - Requires sign handling for signed operands ⚠ **Fixed width** - Not parameterizable (instantiate specific variant) ⚠ **Irregular structure** - Not as regular as array multipliers

24.9.3 When to Use Dadda Tree

Appropriate Use Cases: - Production designs requiring fast multiplication - Area-constrained high-speed applications - DSP functions (filters, FFT, correlation) - Unsigned integer arithmetic - **Default choice** for most multiplication needs

Consider Alternatives When: - Operands are signed → Booth multiplier (fewer PPs) - Very low area required → Array multiplier - Variable width needed → Behavioral * operator - Ultra-high frequency → Multi-stage pipelined multiplier

24.9.4 Dadda vs Wallace Decision

Choose Dadda When: - Production code (better area-speed balance) - Area matters - Targeting ASIC (fewer gates = lower cost)

Choose Wallace When: - Educational purposes (simpler to understand) - Existing design uses it (consistency) - Specific timing requirements favor it

In Practice: Dadda is preferred for almost all production designs.

24.9.5 Signed Multiplication

Dadda trees output **unsigned products only**. For signed multiplication:

Option 1: Sign Extension and Correction

```
logic [N-1:0] a, b;
logic [2*N-1:0] product_unsigned, product_signed;
logic sign_bit;

// Compute absolute values
assign a_abs = a[N-1] ? (~a + 1'b1) : a;
assign b_abs = b[N-1] ? (~b + 1'b1) : b;
assign sign_bit = a[N-1] ^ b[N-1];

// Multiply unsigned
math_multiplier_dadda_tree_008 u_mult (
    .i_multiplier(a_abs),
    .i_multiplicand(b_abs),
    .ow_product(product_unsigned)
);

// Apply sign
assign product_signed = sign_bit ? (~product_unsigned + 1'b1) :
product_unsigned;
```

Option 2: Use Booth Multiplier (More efficient for native signed)

24.9.6 Pipelining for High Frequency

2-Stage Pipeline:

```
// Stage 1: Register inputs
always_ff @(posedge clk) begin
    a_reg <= a;
    b_reg <= b;
end

// Stage 2: Multiply and register output
math_multiplier_dadda_tree_016 u_mult (
    .i_multiplier(a_reg),
    .i_multiplicand(b_reg),
    .ow_product(product_comb)
);
```

```
always_ff @(posedge clk) begin
    product_reg <= product_comb;
end
```

4-Stage Pipeline (for maximum frequency): Split Dadda tree into pipeline stages based on reduction schedule.

24.10 Common Pitfalls

✗ Anti-Pattern 1: Expecting width parameterization

```
// WRONG: N parameter is fixed
math_multiplier_dadda_tree_008 #(.N(10)) u_mult (...); // Won't work!

// RIGHT: Use appropriate fixed variant
math_multiplier_dadda_tree_016 u_mult (...); // Use 16-bit
```

✗ Anti-Pattern 2: Using for signed without conversion

```
// WRONG: Signed inputs treated as unsigned
logic signed [7:0] a = -5, b = 3;
logic signed [15:0] product;
math_multiplier_dadda_tree_008 u_mult (
    .i_multiplier(a),           // Treated as 251 (unsigned)!
    .i_multiplicand(b),        // Treated as 3
    .ow_product(product)       // = 753, not -15!
);

// RIGHT: Convert to unsigned, multiply, fix sign
// (See "Signed Multiplication" section)
```

✗ Anti-Pattern 3: Adding a redundant external adder

```
// WRONG: ow_product is already the final summed product.
// Bolting on another adder both wastes area and computes garbage.
math_multiplier_dadda_tree_008 u_mult (... , .ow_product(p));
assign final_product = p + {carry_vector[14:0], 1'b0}; // No!

// RIGHT: use ow_product directly.
math_multiplier_dadda_tree_008 u_mult
(... , .ow_product(final_product));
```

These modules contain their own final carry-propagate adder. They do not expose separate sum and carry vectors.

✗ Anti-Pattern 4: Ignoring timing at high frequencies


```
// WRONG: 32x32 Dadda at 400 MHz without pipeline
// Critical path ~12-14ns, can't meet 2.5ns period!
```

```
// RIGHT: Add pipeline stages
// Target: 1 stage per 5-6ns of delay
```

24.11 Related Modules

- **`**math_multiplier_wallace_tree*.sv**`** - same reduction depth, 17-31% more adder cells, identical final CPA
- **`math_adder_brent_kung_016/032/064.sv`** - logarithmic-depth parallel-prefix adder used as the final CPA here (8/16/32-bit multipliers use the 016/032/064 widths respectively)
- **`math_adder_carry_save.sv`** - 3:2 compressor building block; the only compressor these files use
- **`math_adder_half.sv`** - Half adder primitive, used in the reduction tree only

24.12 Algorithm Reference: Dadda Sequence

The Dadda reduction sequence is defined as:

$d(1) = 2$ (final stage: 2 rows)
 $d(j+1) = \lceil 1.5 \times d(j) \rceil$ (recursive definition)

Sequence: 2, 3, 4, 6, 9, 13, 19, 28, 42, 63, 94, 141, ...

For N×N multiplication: 1. Find maximum column height (N for the middle column) 2. Find the largest sequence entry strictly below max_height - that is the first stage target 3. Work downwards through the sequence until the target is 2

Example (8×8): - Max height = 8 - Largest sequence entry below 8 is 6, so that is the first target - Stage targets: $6 \rightarrow 4 \rightarrow 3 \rightarrow 2$ (four stages)

This matches the generated RTL, whose stage comments read Dadda reduction stage 1: max column height 6 through stage 4: max column height 2.

24.13 References

- Dadda, L. "Some Schemes for Parallel Multipliers." *Alta Frequenza* 34, 1965.
- Wallace, C.S. "A Suggestion for a Fast Multiplier." *IEEE Trans. Electronic Computers*, 1964.
- Oklobdzija, V.G. "High-Speed VLSI Arithmetic Units: Adders and Multipliers." Springer, 2002.

24.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

25 Wallace Tree Multipliers

Fast parallel multipliers using tree-based partial product reduction with carry-save adders. These modules provide unsigned integer multiplication for 8×8 , 16×16 , and 32×32 operations. The reduction tree has logarithmic depth; the final carry-propagate adder is a Brent-Kung parallel-prefix adder, which is also logarithmic depth, so end-to-end delay is $O(\log N)$.

25.1 Overview

The Wallace tree multiplier family implements high-speed multiplication using a tree of 3:2 compressors (carry-save adders) to reduce partial products in parallel. The reduction is built using maximal parallelism - compressing everything it can as early as it can, rather than following a scheduled approach.

Key Features: - **Logarithmic reduction depth** - 4 layers for 8-bit, 6 for 16-bit, 8 for 32-bit - **Maximal parallelism** - every group of 3 in a column compresses in the same layer - **Structural implementation** - explicit full adder and half adder instantiation - **Fixed-width variants** - generated for 8, 16, and 32-bit operands - **Purely combinational** - single-cycle multiplication - **Self-contained** - includes its own final adder; no external adder required

Architecture: 1. **Partial Product Generation** - AND gates create the $N \times N$ matrix 2. **Wallace Reduction Layers** - parallel CSA layers reduce every column to height 2 3. **Final Addition** - an on-chip Brent-Kung parallel-prefix carry-propagate adder sums the two surviving rows into `ow_product`

Note on the final adder: these modules are complete multipliers. The reduction tree stops at column height 2, the two remaining rows are packed into the $2N$ -bit vectors `w_cpa_row0` / `w_cpa_row1`, and those are summed internally by a single `math_adder_brent_kung_{2N}` instance named `u_final_cpa`. Earlier revisions of this module used a ripple carry-propagate adder in that position, and revisions before that collapsed every column to height 1 with no final adder at all; neither is the case now, and no external adder is needed.

The CPA width is the **product** width, not the operand width: the 8-bit multiplier instantiates `math_adder_brent_kung_016`, the 16-bit one `math_adder_brent_kung_032`, and the 32-bit one `math_adder_brent_kung_064`. Carry-in is tied to `1'b0`, and the adder's carry-out is left unread on `w_cpa_carry_unused` - an $N \times N$ product is strictly less than $2^{(2N)}$, so the top column can never carry out.

Two variants are generated. `math_multiplier_wallace_tree_NNN` builds the reduction tree from `math_adder_full`; `math_multiplier_wallace_tree_csa_NNN` builds it from `math_adder_carry_save`. Both are 3:2 compressors, so the two trees are structurally identical

and produce identical instance counts and topology. The final CPA is the same `math_adder_brent_kung_{2N}` instance in **both** variants. Both are standalone top-level multipliers with the same port list - pick either.

25.2 Module Declarations

25.2.1 8-bit Wallace Tree Multiplier

```
module math_multiplier_wallace_tree_008 #(
    parameter int N = 8
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

25.2.2 16-bit Wallace Tree Multiplier

```
module math_multiplier_wallace_tree_016 #(
    parameter int N = 16
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

25.2.3 32-bit Wallace Tree Multiplier

```
module math_multiplier_wallace_tree_032 #(
    parameter int N = 32
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);
```

25.3 Parameters

Parameter	Type	Default	Description
N	int	8/16/32	Bit width (fixed per variant)

Note: The N parameter is present but fixed per module variant. It is not intended for user modification.

25.4 Ports

Port	Direction	Width	Description
i_multiplier	Input	N	Multiplier operand (unsigned)
i_multiplicand	Input	N	Multiplicand operand (unsigned)
ow_product	Output	2N	Product result (unsigned)

Signal Types: - **Unsigned only** - All operands and results are unsigned integers - **Full precision** - Output is full 2N-bit product (no truncation)

25.5 Functionality

25.5.1 Wallace Tree Algorithm

The Wallace tree multiplier uses an aggressive reduction strategy:

Stage 1: Partial Product Generation

For $N \times N$ multiplication:

- Generate N^2 partial products: $PP[i][j] = multiplier[i] \& multiplicand[j]$
- Arrange in diagonal columns (like manual multiplication)

Stage 2: Wallace Reduction Layers

Reduction proceeds in **layers**. Within a single layer, every column is partitioned into groups of 3, and all groups across all columns compress **simultaneously**. Layers repeat until every column is at height 2.

While any column has height > 2: // one iteration = one layer

 For each column, partition its bits into groups of 3:
 - group of 3 bits -> 3:2 compressor (sum stays, carry goes left)
 - group of 2 bits -> half adder (sum stays, carry goes left)
 - group of 1 bit -> passes through to the next layer untouched

Because the partitioning is done per layer rather than per opportunity, a column of height 8 becomes $\text{ceil}(8/3) = 3$ sums plus carries arriving from the column to its right, and so on down the layers.

Stage 3: Final Addition

Every column is now at height 2. Pack the two surviving rows into two 2N-bit vectors and sum them with a Brent-Kung parallel-prefix carry-propagate adder to produce the 2N-bit product.

Key Characteristic: Wallace compresses **everything it can as early as it can**. This is the entire distinction from Dadda, which defers compression using per-stage target heights. Both reach height 2 in the same number of layers; Wallace simply spends more compressors doing it.

25.5.28-bit Example Structure

Generated signals are named `w_sum_{column}_{layer}_{op}` / `w_carry_{column}_{layer}_{op}`, and instances are named `FA_{column}_{layer}_{op}` (or `CSA...` in the `_csa_variant`) and `HA_{column}_{layer}_{op}`. Excerpts below are taken verbatim from `rtl/common/math_multiplier_wallace_tree_008.sv`.

```
// Partial Products (64 AND gates for 8x8)
wire w_pp_0_0 = i_multiplier[0] & i_multiplicand[0];
wire w_pp_0_1 = i_multiplier[0] & i_multiplicand[1];
// ... 64 total partial products
wire w_pp_7_7 = i_multiplier[7] & i_multiplicand[7];

// Wallace reduction layer 1
// Column 1 holds only 2 bits, so it gets a half adder.
wire w_sum_01_1_01, w_carry_01_1_01;
math_adder_half HA_01_1_01 (
    .i_a(w_pp_0_1),
    .i_b(w_pp_1_0),
    .ow_sum(w_sum_01_1_01),
    .ow_carry(w_carry_01_1_01)
);

// Column 2 holds 3 bits: one full group, one 3:2 compressor.
wire w_sum_02_1_01, w_carry_02_1_01;
math_adder_full FA_02_1_01 (
    .i_a(w_pp_0_2),
    .i_b(w_pp_1_1),
    .i_c(w_pp_2_0),
    .ow_sum(w_sum_02_1_01),
    .ow_carry(w_carry_02_1_01)
);

// Column 3 likewise; every one of these fires in the same layer.
```

```

wire w_sum_03_1_01, w_carry_03_1_01;
math_adder_full FA_03_1_01 (
    .i_a(w_pp_0_3),
    .i_b(w_pp_1_2),
    .i_c(w_pp_2_1),
    .ow_sum(w_sum_03_1_01),
    .ow_carry(w_carry_03_1_01)
);

```

// ... layers 2, 3, 4 repeat the same grouping on the surviving rows

In the `_csa_` variant the identical structure is emitted with `math_adder_carry_save` in place of `math_adder_full`:

// math_multiplier_wallace_tree_csa_008.sv, same position in layer 1

```

wire w_sum_02_1_01, w_carry_02_1_01;
math_adder_carry_save CSA_02_1_01 (
    .i_a(w_pp_0_2),
    .i_b(w_pp_1_1),
    .i_c(w_pp_2_0),
    .ow_sum(w_sum_02_1_01),
    .ow_carry(w_carry_02_1_01)
);

```

Once every column is at height 2, the two surviving rows are packed into a pair of 16-bit vectors and handed to one Brent-Kung prefix adder. There are no `math_adder_full` or `math_adder_half` instances in this stage at all - every half and full adder in the file belongs to the reduction tree:

// Final addition stage: two reduced rows into a Brent-Kung CPA

```

wire [15:0] w_cpa_row0 = {
    w_carry_14_4_01,
    w_carry_13_4_01,
    // ... one bit per column, taken from the surviving row
    w_sum_02_2_01,
    w_sum_01_1_01,
    w_pp_0_0
};
wire [15:0] w_cpa_row1 = {
    w_sum_15_4_01,
    w_sum_14_4_01,
    // ... one bit per column, taken from the other surviving row
    w_sum_05_4_01,
    1'b0,
    1'b0,
    1'b0,
    1'b0,
    1'b0
};

```

```

/* verilator lint_off UNUSED SIGNAL */
wire w_cpa_carry_unused;
/* verilator lint_on UNUSED SIGNAL */
math_adder_brent_kung_016 #(
    .N(16)
) u_final_cpa (
    .i_a(w_cpa_row0),
    .i_b(w_cpa_row1),
    .i_c(1'b0),
    .ow_sum(ow_product),
    .ow_carry(w_cpa_carry_unused)
);

```

Wallace’s eager compression has already flattened columns 0-4 to height 1, and that shows up as the five 1'b0 entries at the bottom of w_cpa_row1. Note that this no longer buys any delay: the prefix adder still spans all 16 columns, and its depth is logarithmic in that width whether or not the low-order inputs are constant zero. The prefix adder drives ow_product directly, so there is no per-bit assign ow_product[i] fan-out stage either.

25.5.3 Reduction Pattern

Measured layer counts and instance counts from the generated RTL:

Width	Layers	Reduction 3:2 compressors	Reduction half adders	Final CPA
8-bit	4	36	25	math_adder_brent_kung_016
16-bit	6	196	78	math_adder_brent_kung_032
32-bit	8	900	216	math_adder_brent_kung_064

For 8×8 multiplication: 64 partial products across 15 columns are reduced to 2 rows in 4 layers, then summed by a single 16-bit Brent-Kung prefix CPA.

Number of layers: the often-quoted $\lceil \log_{1.5}(N) \rceil$ is wrong because reduction runs from N rows down to 2, not down to 1 - the formula is missing a division by 2. The measured values are **4 layers for 8-bit, 6 for 16-bit, and 8 for 32-bit**, which is what the generated RTL emits (// Wallace reduction layer 1 through layer 4 in the 8-bit file).

25.6 Usage Examples

25.6.1 Basic 8×8 Multiplication

```
logic [7:0] a, b;
logic [15:0] product;

math_multiplier_wallace_tree_008 u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product)
);
```

```
// Example: 15 × 17 = 255
initial begin
    a = 8'd15;
    b = 8'd17;
    #1; // Allow combinational delay
    assert(product == 16'd255);
end
```

25.6.2 16×16 Multiplication with Pipeline Register

```
logic [15:0] a, b;
logic [31:0] product_comb, product_reg;
logic clk, rst_n;

// Wallace tree multiplier (combinational)
math_multiplier_wallace_tree_016 u_mult (
    .i_multiplier(a),
    .i_multiplicand(b),
    .ow_product(product_comb)
);

// Optional output register for pipelining
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        product_reg <= '0;
    else
        product_reg <= product_comb;
end
```

25.6.3 32×32 Multiplication for DSP

```
module dsp_multiply (
    input logic clk,
    input logic rst_n,
    input logic [31:0] coeff, // Filter coefficient
    input logic [31:0] sample, // Input sample

```



```

    input  logic      valid_in,
    output logic [63:0] product,
    output logic      valid_out
);

    logic [63:0] product_comb;
    logic      valid_pipe;

    // Wallace tree multiplier
    math_multiplier_wallace_tree_032 u_mult (
        .i_multiplier(coeff),
        .i_multiplicand(sample),
        .ow_product(product_comb)
    );

    // Pipeline register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            product      <= '0;
            valid_pipe <= 1'b0;
            valid_out    <= 1'b0;
        end else begin
            product      <= product_comb;
            valid_pipe <= valid_in;
            valid_out    <= valid_pipe;
        end
    end
end
endmodule

```

25.6.4 Multi-Stage Pipelined Multiplier

```

// Split Wallace tree into pipeline stages for higher frequency
module pipelined_multiplier (
    input  logic      clk,
    input  logic      rst_n,
    input  logic [15:0] a, b,
    output logic [31:0] product
);

    // Stage 1: Partial product generation
    logic [15:0][15:0] pp_stagel;
    genvar i, j;
    generate
        for (i = 0; i < 16; i++) begin : gen_pp
            for (j = 0; j < 16; j++) begin
                always_ff @(posedge clk)
                    pp_stagel[i][j] <= a[i] & b[j];
            end
        end
    endgenerate
endmodule

```

```

        end
    end
endgenerate

// Stage 2-3: Wallace reduction (using tree, registered)
// ... intermediate pipeline stages

// Final stage: Assign product
// ... final adder and output register

endmodule

```

25.7 Timing Characteristics

Metric	8-bit	16-bit	32-bit
Logic Depth	~14-16 levels	~18-22 levels	~24-30 levels
Typical Delay (ns)	~7-8	~9-11	~12-15
Max Frequency	~125 MHz	~90-110 MHz	~65-80 MHz

Logic Depth Breakdown: - Partial product generation: 1 level (AND gates) - Wallace reduction layers: logarithmic - 4 / 6 / 8 layers for 8 / 16 / 32-bit - Final addition: on-chip **Brent-Kung** parallel-prefix carry-propagate adder, $O(\log N)$ levels

Critical Path:

$i_multiplier[N-1] \rightarrow$ PP generation $\rightarrow$ Layer 1 $\rightarrow$ Layer 2 $\rightarrow \dots \rightarrow$ Layer K $\rightarrow$ Brent-Kung prefix network ($\sim 2 \cdot \log_2(2N)$ levels) $\rightarrow ow_product[2N-1]$

Important: both halves of the datapath are logarithmic. The reduction tree is log-depth, and the final adder is a Brent-Kung parallel-prefix adder, which is also log-depth, so the **end-to-end delay of these modules is $O(\log N)$** . There is no serial carry chain anywhere in the design. The 32-bit variant, for example, follows an 8-layer tree with a 64-bit prefix network rather than the 54-deep ripple it used to carry.

Note: Actual timing depends heavily on synthesis optimization and target technology.

25.8 Performance Characteristics

25.8.1 Resource Utilization

Instance counts below are exact, taken from the generated RTL. “3:2 cells” counts the reduction-tree compressors (math_adder_full in the plain variant, math_adder_carry_save in the _csa_

variant). Every discrete adder cell in these files is in the reduction tree; the final CPA is a single prefix-adder instance and contributes none.

Width	3:2 cells (tree)	Half adders (tree)	Final CPA	AND Gates	Total adder cells
8-bit	36	25	1 × math_adder_brent_kung_016	64	61
16-bit	196	78	1 × math_adder_brent_kung_032	256	274
32-bit	900	216	1 × math_adder_brent_kung_064	1024	1116

Comparison to Other Multiplier Architectures:

Architecture	Area (relative)	Speed (relative)	Best Use Case
Wallace Tree	1.2×	1.0×	High-speed, clearest teaching structure
Dadda Tree	1.0×	~1.0×	Same depth, fewer compressors
Array Multiplier	0.8×	2.5×	Low-speed, minimal area
Booth (radix-4)	0.9×	1.5×	Signed, reduced partial products

Wallace vs Dadda - the headline comparison:

For 8×8, both reach column height 2 in **4 layers/stages - the same depth**. They differ only in what that depth costs:

	Wallace	Dadda
Layers / stages to height 2	4	4
Reduction compressors	36	35
Reduction half adders	25	7
Total reduction cells	61	42
Final CPA	math_adder_brent_kung_016	math_adder_brent_kung_016

Wallace compresses everything it can as early as it can. Dadda defers, using per-stage target heights to spend the fewest compressors for the same depth. **That is the entire distinction between the two.**

Wallace used to get something back for its extra 19 cells. When both multipliers ended in a ripple CPA, eager compression flattened the low-order columns early, so Wallace's final ripple spanned only 11 columns against Dadda's 14 - a shorter serial carry chain that partly offset the larger tree. **That offset is gone.** Both now feed a full-width Brent-Kung prefix adder over all $2N$ columns, and a prefix adder's depth is logarithmic in its width regardless of how many low-order inputs are constant zero. The final adder is now *identical* in the two families, in both cell count and delay, so the extra 19 cells buy nothing.

Measured totals across widths (adder cells in the reduction tree; the shared Brent-Kung CPA is identical in both and excluded):

Width	Wallace total cells	Dadda total cells
8-bit	61	42
16-bit	274	210
32-bit	1116	930

25.8.2 Area-Speed Tradeoffs

For High-Speed Requirements: - ✓ Either tree works - they have the same reduction depth and the same final adder - ✓ Consider pipelining for even higher frequency - ✓ If you pick Wallace, accept the larger area overhead for no speed gain

For Area-Constrained Designs: - Use Dadda tree instead (see `math_multiplier_dadda_tree.md`) - Consider Booth encoding for signed multiplication - Use sequential multipliers if latency acceptable

25.9 Design Considerations

25.9.1 Advantages

✓ **Log-depth end to end** - the tree collapses N rows to 2 in $O(\log N)$ layers, versus $O(N)$ for an array multiplier, and the Brent-Kung prefix CPA that follows is $O(\log N)$ as well ✓ **Highly parallel** - Exploits 3:2 compression at all levels ✓ **No sequential logic** - Pure combinational (easy to pipeline) ✓ **Unsigned friendly** - Natural fit for unsigned operands ✓ **Scalable** - Algorithm extends to any bit width

25.9.2 Limitations

⚠ **Large area** - More adder cells than Dadda tree at the same depth (61 versus 42 at 8×8 , 1116 versus 930 at 32×32), with no offsetting delay advantage ⚠ **Irregular structure** - Complex synthesis, harder to hand-layout ⚠ **Unsigned only** - Requires additional logic for signed multiplication ⚠ **Fixed width** - Not parameterizable (must instantiate specific variant) ⚠ **Long critical path** - May require pipelining for high-frequency designs

25.9.3 When to Use Wallace Tree

Appropriate Use Cases: - High-speed DSP applications (FIR filters, FFT butterfly) - Single-cycle multiplication requirements - FPGA designs with abundant LUT resources - Unsigned integer multiplication

Consider Alternatives When: - Area is critical → Use Dadda tree (17-31% fewer adder cells at the same depth) - Operands are signed → Use Booth multiplier - Low frequency → Use array multiplier (much smaller) - Variable width needed → Use parameterized array/Booth

25.9.4 Signed Multiplication

Wallace trees output **unsigned products only**. For signed multiplication:

Option 1: Sign-Magnitude Conversion

```
// Convert to unsigned, multiply, then fix sign
logic sign_result;
logic [N-1:0] a_abs, b_abs;
logic [2*N-1:0] product_unsigned;

assign sign_result = a[N-1] ^ b[N-1];
assign a_abs = a[N-1] ? -a : a;
assign b_abs = b[N-1] ? -b : b;
```

```
math_multiplier_wallace_tree_008 u_mult (
```

```

        .i_multiplier(a_abs),
        .i_multiplicand(b_abs),
        .ow_product(product_unsigned)
    );

```

```

assign product_signed = sign_result ? -product_unsigned :
product_unsigned;

```

Option 2: Booth Encoding (More efficient for signed)

```

// Use Booth radix-4 multiplier instead
// (Not covered by Wallace tree modules)

```

25.9.5 Pipelining Strategy

Single-Stage Pipeline:

```

// Register output (adds 1 cycle latency)
always_ff @(posedge clk) begin
    product_reg <= ow_product;
end

```

Multi-Stage Pipeline: Split Wallace tree into stages: 1. **Stage 1:** Partial product generation → register 2. **Stage 2:** First half of reduction tree → register 3. **Stage 3:** Second half of reduction tree → register 4. **Stage 4:** Final addition → output

Benefit: Achieves 2-3× higher frequency at cost of 3-4 cycle latency

25.9.6 Synthesis Considerations

Optimization Directives:

```

# Let synthesis optimize structure
set_dont_touch false

```

```

# For timing-critical designs
set_flatten true
set_boundary_optimization true

```

```

# If targeting ASIC
set_implementation rtl # vs gate-level

```

```

# If targeting FPGA
# Let tool map to DSP blocks if available

```

FPGA Notes: - Modern FPGAs have dedicated DSP blocks (DSP48 on Xilinx, DSP on Intel) - Synthesis tools may map Wallace tree to DSP block - **Check resource utilization** - may use LUTs instead of DSP if tree doesn't fit

25.10 Common Pitfalls

✗ Anti-Pattern 1: Expecting parameterized width

```
// WRONG: Trying to override N parameter
math_multiplier_wallace_tree_008 #(.N(12)) u_mult (...); // Won't work!

// RIGHT: Use fixed variant or create custom width
math_multiplier_wallace_tree_016 u_mult (...); // Use 16-bit variant
```

✗ Anti-Pattern 2: Using for signed multiplication directly

```
// WRONG: Signed operands won't work correctly
logic signed [7:0] a, b;
logic signed [15:0] product;
math_multiplier_wallace_tree_008 u_mult (
    .i_multiplier(a),           // Interpreted as unsigned!
    .i_multiplicand(b),
    .ow_product(product)
);
```

```
// RIGHT: Convert to unsigned, then fix sign
// (See "Signed Multiplication" section above)
```

✗ Anti-Pattern 3: Adding a redundant external adder

```
// WRONG: ow_product is already the final summed product.
// Bolting on another adder both wastes area and computes garbage.
math_multiplier_wallace_tree_008 u_mult (... , .ow_product(p));
assign product = p + {carry[14:0], 1'b0}; // No!

// RIGHT: use ow_product directly.
math_multiplier_wallace_tree_008 u_mult (... , .ow_product(product));
```

These modules contain their own final carry-propagate adder. They do not expose separate sum and carry vectors. This is unambiguous - there is definitively a final CPA inside every generated variant.

✗ Anti-Pattern 4: Not pipelining for high frequency

```
// WRONG: Using 32x32 multiplier at 500 MHz (won't meet timing)
math_multiplier_wallace_tree_032 u_mult (...); // Critical path too long!

// RIGHT: Add pipeline stages
always_ff @(posedge clk) begin
    stage1 <= inputs;
    stage2 <= wallace_tree_partial(stage1);
```

```

    stage3 <= wallace_tree_final(stage2);
    product <= stage3;
end

```

25.11 The _csa_Variant

`math_multiplier_wallace_tree_csa_008/016/032.sv` are **standalone top-level multipliers**, not internal sub-components. Each declares the same module interface as the plain variant:

```

module math_multiplier_wallace_tree_csa_008 #(
    parameter int N = 8
) (
    input logic [ N-1:0] i_multiplier,
    input logic [ N-1:0] i_multiplicand,
    output logic [2*N-1:0] ow_product
);

```

The only difference is the cell used to build the reduction tree:

	Plain variant	_csa_variant
Reduction tree cell	<code>math_adder_full</code>	<code>math_adder_carry_save</code>
Final CPA cell	<code>math_adder_full</code>	<code>math_adder_full</code>
Tree topology	identical	identical
Instance counts	identical	identical

Both cells are 3:2 compressors, so the trees are structurally identical. The plain variant is **not** built out of the `_csa_variant` - they are two independent generated modules and neither instantiates the other. Instantiate whichever one you prefer.

25.12 Related Modules

- `**math_multiplier_dadda_tree*.sv**` - same reduction depth, 17-31% fewer adder cells, identical final CPA
- `math_adder_brent_kung_016/032/064.sv` - logarithmic-depth parallel-prefix adder used as the final CPA here (8/16/32-bit multipliers use the 016/032/064 widths respectively)
- `math_adder_carry_save.sv` - 3:2 compressor, used in the `_csa_variant`'s reduction tree
- `math_adder_full.sv` - Full adder primitive, used in the plain variant's reduction tree
- `math_adder_half.sv` - Half adder primitive, used in the reduction tree of both variants

25.13 Wallace Tree vs Dadda Tree

Both use CSA trees but differ in reduction strategy:

Aspect	Wallace Tree	Dadda Tree
Strategy	Compress everything as early as possible	Defer; compress only down to a per-stage target height
Layers / stages to height 2	4 / 6 / 8 (8/16/32-bit)	4 / 6 / 8 - the same
Reduction cells (8×8)	61	42
Final CPA (8×8)	math_adder_brent_kung_016	math_adder_brent_kung_016 - the same
Total adder cells (8×8)	61	42
Design	Simpler (greedy grouping)	More complex (scheduled targets)

Dadda uses **fewer compressors for the same depth** - not fewer stages. The stage counts are identical.

Recommendation: Use Dadda tree for production designs (fewer cells at equal depth). Wallace remains the clearer teaching example, but with both families now ending in the same Brent-Kung prefix adder it no longer has a delay advantage to trade against its larger tree.

25.14 References

- Wallace, C.S. “A Suggestion for a Fast Multiplier.” IEEE Transactions on Electronic Computers, 1964.
- Dadda, L. “Some Schemes for Parallel Multipliers.” Alta Frequenza, 1965.
- Oklobdzija, V.G. “High-Speed VLSI Arithmetic Units: Adders and Multipliers.” Springer, 2002.

25.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

26 Prefix Cell Gray

An area-optimized parallel prefix network building block that computes only the group generate (G) signal, used in reverse tree stages and final carry computation where the propagate signal is not needed.

26.1 Overview

The `math_prefix_cell_gray` module (also known as a “gray cell”) is a reduced-area variant of the prefix cell that outputs only the group generate signal. Since the final carry computation only needs G (not P), gray cells save ~33% area in stages where propagate signals are not required downstream.

Key Features: - **Outputs G only** - Optimized for carry-only computation - ~33% **smaller** than black cells (2 gates vs 3) - **Same delay** as black cell for G output - **Used in** reverse tree stages of Brent-Kung and final stage of Han-Carlson

26.2 Module Declaration

```
module math_prefix_cell_gray (
    input logic i_g_hi, i_p_hi,
    input logic i_g_lo,           // No P needed from lower position
    output logic ow_g             // Only G output (this IS the
carry)
);
```

26.3 Ports

Port	Direction	Width	Description
i_g_hi	Input	1	Generate signal from higher bit position
i_p_hi	Input	1	Propagate signal from higher bit position
i_g_lo	Input	1	Generate signal from lower bit position
ow_g	Output	1	Combined group generate (the carry into position i+1)

Note: `i_p_lo` is not needed because it’s only used to compute the group propagate, which this cell doesn’t output.

26.4 Functionality

26.4.1 Implementation

```
assign ow_g = i_g_hi | (i_p_hi & i_g_lo);
```

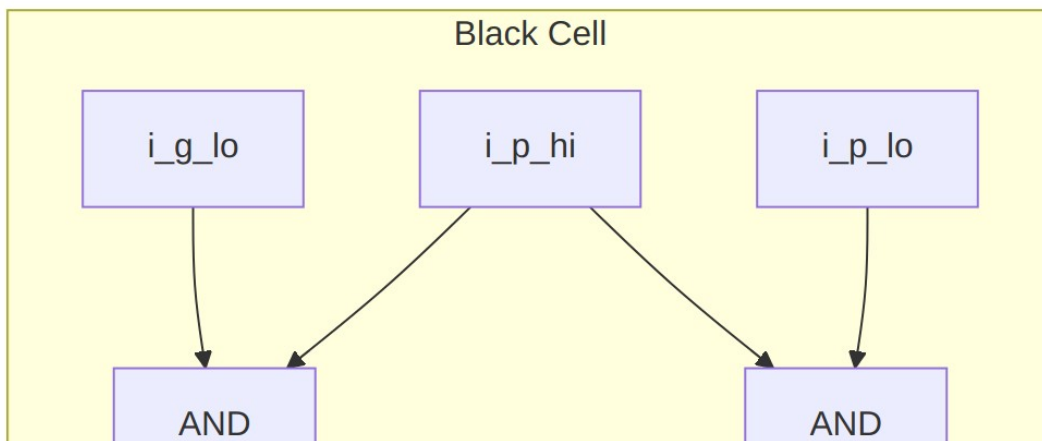
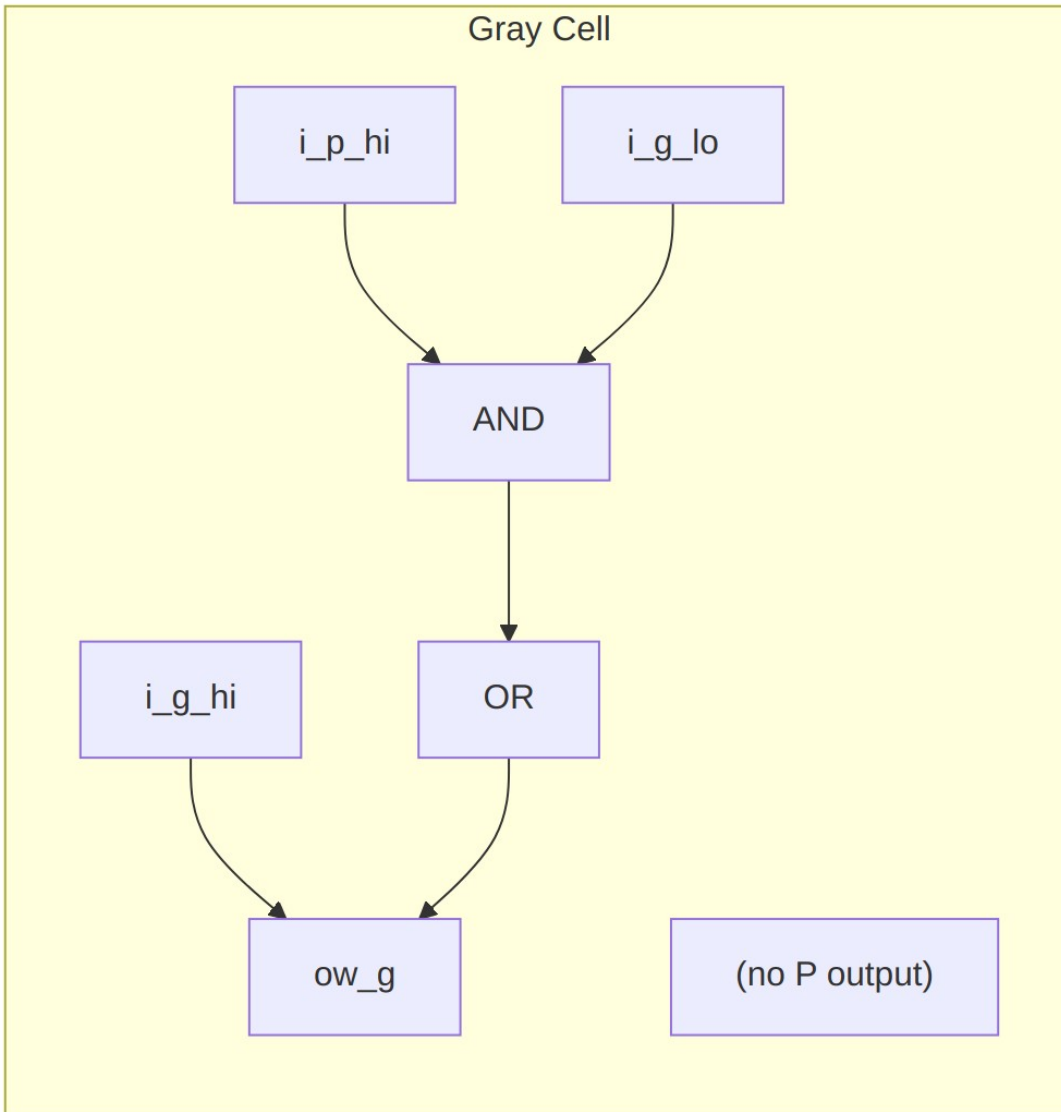
This computes the group generate signal: - **$G[i:j] = G[i:k] \text{ OR } (P[i:k] \text{ AND } G[k-1:j])$**

Interpretation: A carry is generated for the combined range [i:j] if: - The high range [i:k] generates a carry, OR - The high range propagates AND the low range generates

26.4.2 Why Gray Cells Don't Need P Output

In parallel prefix adders, the final computation is: - **$\text{Sum}[i] = P[i] \text{ XOR } C[i-1]$** - **$C[i] = G[i:-1]$** (group generate from bit i down to carry-in)

The sum computation uses the **original** single-bit propagate P[i], not the group propagate. Therefore, once we've computed all the carries (group generates), we no longer need the group propagates.



26.5 Timing Characteristics

Metric	Value	Description
Logic Depth	2 gates	1 AND + 1 OR
Critical Path	AND-OR	i_g_lo -> ow_g
Gate Count	2	1 AND + 1 OR

26.6 Usage Examples

26.6.1 In Han-Carlson Final Stage

```
// Han-Carlson: Final stage fills odd positions using gray cells
// Odd positions get their carry from the even neighbor
generate
  for (i = 0; i < N; i++) begin : gen_final_stage
    if (i % 2 == 1) begin : gen_odd
      // Odd positions: compute G[i:-1] from G[i] and G[i-1:-1]
      math_prefix_cell_gray u_pf_gray (
        .i_g_hi(w_g_prev[i]),    // G[i] (single bit)
        .i_p_hi(w_p_prev[i]),    // P[i] (single bit)
        .i_g_lo(w_g_prev[i-1]), // G[i-1:-1] (group from even
neighbor)
        .ow_g(w_g_final[i])      // G[i:-1] (the carry)
      );
    end else begin : gen_even
      // Even positions: already computed, pass through
      assign w_g_final[i] = w_g_prev[i];
    end
  end
endgenerate
```

26.6.2 In Brent-Kung Reverse Tree

```
// Brent-Kung: Reverse tree uses gray cells to fill intermediate
positions
// After forward tree, only power-of-2 positions have complete carries
// Reverse tree fills in the gaps using gray cells

// Example: Position 5 gets carry from positions 5 and 4
math_prefix_cell_gray u_bk_gray_5 (
  .i_g_hi(w_g[5]),    // G[5:5] (from forward tree)
  .i_p_hi(w_p[5]),    // P[5:5] (original propagate)
  .i_g_lo(w_g[4]),    // G[4:-1] (computed in forward tree)
```

```

        .ow_g(w_g_5_final) // G[5:-1] (carry into position 6)
    );

```

26.6.3 Computing Final Sum

```

// After all carries computed with gray cells:
generate
    for (i = 0; i < N; i++) begin : gen_sum
        if (i == 0) begin
            assign sum[0] = p_original[0] ^ cin;
        end else begin
            // Sum = original P XOR carry from previous position
            assign sum[i] = p_original[i] ^ g_final[i-1];
        end
    end
endgenerate

// Carry out is the final group generate
assign cout = g_final[N-1];

```

26.7 Performance Characteristics

26.7.1 Resource Utilization

Metric	Value
AND gates	1
OR gates	1
Total gates	2
LUTs (FPGA)	1

26.7.2 Comparison with Black Cell

Property	Black Cell	Gray Cell
Inputs	4 (Ghi, Phi, Glo, Plo)	3 (Ghi, Phi, Glo)
Outputs	2 (G, P)	1 (G)
Gate count	3	2
Area savings	-	~33%
G output delay	Same	Same

26.7.3 Area Savings in Adder

For an N-bit adder, the number of gray cells vs black cells affects total area:

Architecture	Black Cells	Gray Cells	Total Cells
Kogge-Stone N=16	64	0	64
Brent-Kung N=16	~30	~30	~60
Han-Carlson N=16	~32	8	~40

Han-Carlson uses gray cells only in the final stage (N/2 cells for N-bit adder).

26.8 Design Considerations

26.8.1 When to Use Gray Cells

Use gray cells when: - Computing final carries (no further prefix operations needed) - In reverse tree stages (Brent-Kung) - Final fill-in stage (Han-Carlson) - Any position where P is not needed downstream

Use black cells when: - P signal needed for subsequent prefix stages - Building Kogge-Stone (all stages need both P and G) - Forward tree stages in hybrid architectures

26.8.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Minimal 2-gate implementation 2. **Wire complexity** - One fewer input than black cell 3. **Logic depth** - Same 2-gate delay as black cell

26.8.3 Architectural Trade-offs

Architecture	Gray Cell Usage	Trade-off
Kogge-Stone	None	Maximum speed, maximum area
Brent-Kung	Reverse tree (~50%)	Minimum area, 2x depth
Han-Carlson	Final stage only (~20%)	Balanced speed/area

26.9 Related Modules

- **math_prefix_cell** - Black cell variant (outputs both G and P)
- **math_adder_han_carlson_016** - 16-bit Han-Carlson adder using this cell
- **math_adder_han_carlson_048** - 48-bit Han-Carlson adder using this cell

- `math_adder_brent_kung_gray` - Brent-Kung gray cell (equivalent)

26.10 Applications

- **Parallel prefix adders** - Final carry computation stages
- **Area-optimized adders** - Brent-Kung and Han-Carlson architectures
- **Multiplier CPAs** - Final addition where area matters
- **Low-power designs** - Fewer gates = lower dynamic power

26.11 References

- Brent, R.P., Kung, H.T. “A Regular Layout for Parallel Adders.” IEEE Trans. Computers, 1982.
- Han, T., Carlson, D.A. “Fast Area-Efficient VLSI Adders.” IEEE Symposium on Computer Arithmetic, 1987.
- Harris, D. “A Taxonomy of Parallel Prefix Networks.” Asilomar Conference, 2003.

26.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

27 Prefix Cell (Black Cell)

A parallel prefix network building block that combines generate (G) and propagate (P) signals from adjacent bit groups, used in high-performance adders like Kogge-Stone, Brent-Kung, and Han-Carlson.

27.1 Overview

The `math_prefix_cell` module (also known as a “black cell”) implements the fundamental prefix operation for parallel prefix adders. It combines the generate and propagate signals from a higher bit position with those from a lower bit position, computing the group generate and group propagate for the combined range.

Key Features: - **Outputs both G and P** - Full prefix cell for forward tree stages - **O(1) delay** - Fixed 2-gate delay per stage - **Building block** for all parallel prefix adder architectures - **Enables O(log N) addition** - Parallel carry computation

27.2 Module Declaration

```
module math_prefix_cell (  
    input logic i_g_hi, i_p_hi, // From higher bit position  
    input logic i_g_lo, i_p_lo, // From lower bit position  
    output logic ow_g, ow_p  
);
```

27.3 Ports

Port	Direction	Width	Description
i_g_hi	Input	1	Generate signal from higher bit position [i:k]
i_p_hi	Input	1	Propagate signal from higher bit position [i:k]
i_g_lo	Input	1	Generate signal from lower bit position [k-1:j]
i_p_lo	Input	1	Propagate signal from lower bit position [k-1:j]
ow_g	Output	1	Combined group generate [i:j]
ow_p	Output	1	Combined group propagate [i:j]

27.4 Functionality

27.4.1 Parallel Prefix Theory

In parallel prefix adders, carries are computed using generate (G) and propagate (P) signals:

Bit-level definitions: - **Generate:** $G[i] = A[i] \text{ AND } B[i]$ - Carry generated at position i - **Propagate:** $P[i] = A[i] \text{ XOR } B[i]$ - Carry propagated through position i

Group-level recurrence (the prefix operation): - $G[i:j] = G[i:k] \text{ OR } (P[i:k] \text{ AND } G[k-1:j])$ - $P[i:j] = P[i:k] \text{ AND } P[k-1:j]$

This cell computes these group-level signals by combining two adjacent ranges.

27.4.2 Implementation

```
assign ow_g = i_g_hi | (i_p_hi & i_g_lo);  
assign ow_p = i_p_hi & i_p_lo;
```

Interpretation: - **Group Generate:** A carry is generated for the combined range if: - The high range generates a carry, OR - The high range propagates AND the low range generates - **Group Propagate:** A carry propagates through the combined range only if both ranges propagate

27.4.3 Example: Combining Adjacent Pairs

Initial (bit-level):

```
Position:  3      2      1      0
P[i]:      P3     P2     P1     P0
G[i]:      G3     G2     G1     G0
```

After Stage 1 (prefix cells combining adjacent pairs):

```
G[1:0] = G1 | (P1 & G0)    // Does range [1:0] generate?
P[1:0] = P1 & P0           // Does range [1:0] propagate?
```

```
G[3:2] = G3 | (P3 & G2)    // Does range [3:2] generate?
P[3:2] = P3 & P2           // Does range [3:2] propagate?
```

After Stage 2 (prefix cells combining pairs of pairs):

```
G[3:0] = G[3:2] | (P[3:2] & G[1:0]) // Does range [3:0] generate?
P[3:0] = P[3:2] & P[1:0]             // Does range [3:0] propagate?
```

27.5 Timing Characteristics

Metric	Value	Description
Logic Depth	2 gates	1 AND + 1 OR (for G), 1 AND (for P)
Critical Path	AND-OR	i_g_hi/i_p_hi -> ow_g
Gate Count	3	2 AND gates + 1 OR gate

27.6 Usage Examples

27.6.1 In Kogge-Stone Adder (Stage 1)

```
// Combine positions 1 and 0
math_prefix_cell u_pf_10 (
    .i_g_hi(g[1]),    .i_p_hi(p[1]),
    .i_g_lo(g[0]),    .i_p_lo(p[0]),
    .ow_g(g_1_0),     .ow_p(p_1_0)
);
```

```
// Combine positions 3 and 2
math_prefix_cell u_pf_32 (
    .i_g_hi(g[3]),    .i_p_hi(p[3]),
```

```

        .i_g_lo(g[2]),    .i_p_lo(p[2]),
        .ow_g(g_3_2),    .ow_p(p_3_2)
    );

```

27.6.2 In Kogge-Stone Adder (Stage 2)

```

// Combine ranges [3:2] and [1:0]
math_prefix_cell u_pf_30 (
    .i_g_hi(g_3_2),    .i_p_hi(p_3_2),
    .i_g_lo(g_1_0),    .i_p_lo(p_1_0),
    .ow_g(g_3_0),      .ow_p(p_3_0)
);

```

27.6.3 In Han-Carlson Adder (Even Position)

// Han-Carlson: Only even positions get prefix cells in intermediate stages

```

generate
    for (i = 0; i < N; i++) begin : gen_stage
        if (i % 2 == 0 && i >= 2) begin : gen_even
            math_prefix_cell u_pf (
                .i_g_hi(w_g_prev[i]),    .i_p_hi(w_p_prev[i]),
                .i_g_lo(w_g_prev[i-2]),    .i_p_lo(w_p_prev[i-2]),
                .ow_g(w_g_curr[i]),        .ow_p(w_p_curr[i])
            );
        end else begin : gen_pass
            assign w_g_curr[i] = w_g_prev[i];
            assign w_p_curr[i] = w_p_prev[i];
        end
    end
endgenerate

```

27.7 Performance Characteristics

27.7.1 Resource Utilization

Metric	Value
AND gates	2
OR gates	1
Total gates	3
LUTs (FPGA)	1-2

27.7.2 Comparison: Black Cell vs Gray Cell

Property	Black Cell (this)	Gray Cell
Outputs	G and P	G only

Property	Black Cell (this)	Gray Cell
Gate count	3	2
Use case	Forward tree	Reverse tree / final stage
Module	math_prefix_cell	math_prefix_cell_gray

27.8 Design Considerations

27.8.1 When to Use Black vs Gray Cells

Use Black Cells (this module) when: - In forward tree stages where both G and P are needed downstream - Building Kogge-Stone style networks (all cells are black) - Intermediate stages of Brent-Kung or Han-Carlson

Use Gray Cells when: - Only the carry (G) is needed (final sum computation) - In reverse tree stages of Brent-Kung - Final fill-in stage of Han-Carlson - ~33% area savings when P not needed

27.8.2 Design Optimization Priorities

This module is optimized with the following priorities: 1. **Area** - Minimal 3-gate implementation 2. **Wire complexity** - Simple point-to-point connections 3. **Logic depth** - Fixed 2-gate delay

27.8.3 Prefix Network Architectures

Architecture	Black Cells	Gray Cells	Depth	Wiring
Kogge-Stone	All	None	$O(\log N)$	Maximum
Brent-Kung	Forward tree	Reverse tree	$O(2 \log N)$	Minimum
Han-Carlson	Even positions	Final stage	$O(\log N + 1)$	Medium

27.9 Related Modules

- **math_prefix_cell_gray** - Gray cell variant (G output only)
- **math_adder_han_carlson_016** - 16-bit Han-Carlson adder using this cell
- **math_adder_han_carlson_048** - 48-bit Han-Carlson adder using this cell
- **math_adder_brent_kung_black** - Brent-Kung black cell (equivalent)

27.10 Applications

- **High-performance adders** - All parallel prefix adder architectures
- **Multiplier final CPA** - Fast carry-propagate addition
- **Incrementers/Decrementers** - Priority-encoded operations
- **Comparators** - Parallel comparison networks

27.11 References

- Kogge, P.M., Stone, H.S. “A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations.” IEEE Trans. Computers, 1973.
- Brent, R.P., Kung, H.T. “A Regular Layout for Parallel Adders.” IEEE Trans. Computers, 1982.
- Han, T., Carlson, D.A. “Fast Area-Efficient VLSI Adders.” IEEE Symposium on Computer Arithmetic, 1987.
- Harris, D. “A Taxonomy of Parallel Prefix Networks.” Asilomar Conference, 2003.

27.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

28 Binary Subtractors

A family of binary subtractor modules (half, full, ripple, and carry lookahead) for computing $A - B$ using direct subtraction logic. Modern designs typically use adders with two's complement instead, but these modules are useful for educational purposes and specific applications.

28.1 Overview

This document covers the complete subtractor module family: - **math_subtractor_half** - Single-bit half subtractor (2 inputs) - **math_subtractor_full** - Single-bit full subtractor (3 inputs with borrow) - **math_subtractor_full_nbit** - N-bit ripple borrow subtractor - **math_subtractor_ripple_carry** - N-bit ripple borrow subtractor (equivalent) - **math_subtractor_carry_lookahead** - N-bit subtractor with lookahead logic

Modern Alternative: Most designs use adders for subtraction via two's complement: $A - B = A + (\sim B) + 1$. This approach reuses existing adder hardware.

28.2 Module Declarations

28.2.1 Half Subtractor

```
module math_subtractor_half (  
    input logic i_a,      // Minuend  
    input logic i_b,      // Subtrahend  
    output logic o_d,      // Difference  
    output logic o_b       // Borrow output  
);
```

28.2.2 Full Subtractor

```
module math_subtractor_full (  
    input logic i_a,      // Minuend  
    input logic i_b,      // Subtrahend  
    input logic i_b_in,    // Borrow input  
    output logic ow_d,      // Difference  
    output logic ow_b       // Borrow output  
);
```

28.2.3 N-bit Ripple Borrow Subtractor

```
module math_subtractor_ripple_carry #(  
    parameter int N = 4  
) (  
    input logic [N-1:0] i_a,      // Minuend  
    input logic [N-1:0] i_b,      // Subtrahend  
    input logic i_b_in,          // Borrow input  
    output logic [N-1:0] ow_d,      // Difference  
    output logic ow_b             // Borrow output  
);
```

28.2.4 N-bit Carry Lookahead Subtractor

```
module math_subtractor_carry_lookahead #(  
    parameter int N = 4  
) (  
    input logic [N-1:0] i_a,      // Minuend  
    input logic [N-1:0] i_b,      // Subtrahend  
    input logic i_b_in,          // Borrow input  
    output logic [N-1:0] ow_d,      // Difference  
    output logic ow_b             // Borrow output  
);
```

28.3 Parameters

28.3.1 Single-bit Subtractors

No parameters (fixed single-bit operation).

28.3.2 N-bit Subtractors

Parameter	Type	Default	Description
N	int	4	Bit width (range: 1-64)

28.4 Ports

28.4.1 Half Subtractor Ports

Port	Direction	Width	Description
i_a	Input	1	Minuend (number to subtract from)
i_b	Input	1	Subtrahend (number to subtract)
o_d	Output	1	Difference output (A - B)
o_b	Output	1	Borrow output

28.4.2 Full Subtractor Ports

Port	Direction	Width	Description
i_a	Input	1	Minuend
i_b	Input	1	Subtrahend
i_b_in	Input	1	Borrow input (from previous stage)
ow_d	Output	1	Difference output
ow_b	Output	1	Borrow output (to next stage)

28.4.3 N-bit Subtractor Ports

Port	Direction	Width	Description
i_a	Input	N	Minuend vector

Port	Direction	Width	Description
i_b	Input	N	Subtrahend vector
i_b_in	Input	1	Initial borrow input
ow_d	Output	N	Difference vector (A - B - Bin)
ow_b	Output	1	Final borrow output

28.5 Functionality

28.5.1 Half Subtractor

Subtracts one bit from another without borrow input:

Logic Equations:

```
Difference = A ⊕ B           // XOR
Borrow = ~A • B             // Borrow when A=0, B=1
```

Truth Table: | A | B | Diff | Borrow | Meaning | |—|—|—|—|—| | 0 | 0 | 0 | 0 | 0 - 0 = 0 | | 0 | 1 | 1 | 0 | 0 - 1 = -1 (borrow!) | | 1 | 0 | 1 | 1 | 1 - 0 = 1 | | 1 | 1 | 0 | 0 | 1 - 1 = 0 |

Implementation:

```
assign o_d = i_a ^ i_b;           // XOR for difference
assign o_b = ~i_a & i_b;          // Borrow when A < B
```

28.5.2 Full Subtractor

Subtracts two bits with borrow input:

Logic Equations:

```
Difference = A ⊕ B ⊕ Bin
Borrow = (~A • B) | (~(A ⊕ B) • Bin)
        = (~A • B) | (~A • Bin) | (B • Bin) // Alternative form
```

Truth Table: | A | B | Bin | Diff | Bout | A - B - Bin | |—|—|—|—|—| | 0 | 0 | 0 | 0 | 0 | 0 - 0 = 0 | | 0 | 0 | 1 | 1 | 1 | -1 | | 0 | 1 | 0 | 1 | 1 | -1 | | 0 | 1 | 1 | 0 | 1 | -2 | | 1 | 0 | 0 | 1 | 0 | 1 | | 1 | 1 | 0 | 1 | 0 | 0 | | 1 | 1 | 0 | 0 | 0 | 0 | | 1 | 1 | 1 | 1 | 1 | -1 |

Implementation:


```

assign ow_d = i_a ^ i_b ^ i_b_in;
assign ow_b = (~i_a & i_b) | ~(i_a ^ i_b) & i_b_in;

```

28.5.3 N-bit Ripple Borrow Subtractor

Chains N full subtractors with borrow propagating from LSB to MSB:

```

Bit 0:  FS(A[0], B[0], Bin)    → D[0], B[0]
Bit 1:  FS(A[1], B[1], B[0])   → D[1], B[1]
...
Bit N-1: FS(A[N-1], B[N-1], B[N-2]) → D[N-1], Bout

```

Timing: O(N) delay (borrow ripples sequentially)

28.5.4 N-bit Carry Lookahead Subtractor

Uses lookahead logic to compute borrows faster (similar to CLA adder):

Timing: O(N) delay (simplified lookahead)

28.6 Usage Examples

28.6.1 Basic Subtraction (Using Subtractor)

```

logic [7:0] a, b, diff;
logic borrow;

```

```

math_subtractor_ripple_carry #(.N(8)) u_sub (
    .i_a(a),
    .i_b(b),
    .i_b_in(1'b0),           // No initial borrow
    .ow_d(diff),
    .ow_b(borrow)           // Borrow indicates A < B
);

```

// Example: 100 - 50 = 50

```

initial begin
    a = 8'd100;
    b = 8'd50;
    #1;
    assert(diff == 8'd50);
    assert(borrow == 1'b0); // No borrow
end

```

28.6.2 Detecting Underflow (A < B)

```

logic [7:0] a, b, diff;
logic underflow;

```

```

math_subtractor_ripple_carry #(.N(8)) u_sub (
    .i_a(a), .i_b(b), .i_b_in(1'b0),
    .ow_d(diff), .ow_b(underflow)
);

// Borrow output indicates underflow (A < B)
// Example: 50 - 100 → underflow!
initial begin
    a = 8'd50;
    b = 8'd100;
    #1;
    assert(underflow == 1'b1); // A < B
    assert(diff == 8'd206);    // Wraps around: 50 - 100 + 256 = 206
end

```

28.6.3 Modern Approach: Subtraction Using Adder

// PREFERRED: Use adder with two's complement
// $A - B = A + (\sim B) + 1$

```

logic [7:0] a, b, diff;
logic borrow;

```

```

math_adder_ripple_carry #(.N(8)) u_add (
    .i_a(a),
    .i_b(~b),           // Invert B (one's complement)
    .i_c(1'b1),         // Carry in = 1 (complete two's complement)
    .ow_sum(diff),
    .ow_carry(carry_out)
);

```

// Borrow = ~carry_out (inverted carry indicates underflow)
assign borrow = ~carry_out;

Advantages of Adder-Based Subtraction: - Reuses existing adder hardware - No separate subtractor needed - Standard practice in modern ALUs - Same timing characteristics

28.6.4 Multi-Precision Subtraction (16-bit via 2×8-bit)

```

logic [7:0] a_low, a_high, b_low, b_high;
logic [7:0] diff_low, diff_high;
logic borrow_low, borrow_high;

```

// Low byte

```

math_subtractor_ripple_carry #(.N(8)) u_sub_low (
    .i_a(a_low),
    .i_b(b_low),
    .i_b_in(1'b0),

```

```

        .ow_d(diff_low),
        .ow_b(borrow_low)
    );

    // High byte (chain borrow from low)
    math_subtractor_ripple_carry #(.N(8)) u_sub_high (
        .i_a(a_high),
        .i_b(b_high),
        .i_b_in(borrow_low),
        .ow_d(diff_high),
        .ow_b(borrow_high)
    );

    logic [15:0] diff_16 = {diff_high, diff_low};

```

28.7 Timing Characteristics

Module	Logic Levels	Typical Delay (ns)
Half Subtractor	1	~0.2
Full Subtractor	2	~0.4
Ripple (N-bit)	2N	~0.4N
CLA (N-bit)	O(N)	Similar to ripple (simplified)

Note: Subtractors have same timing as equivalent adders.

28.8 Performance Characteristics

28.8.1 Resource Utilization

Module	LUTs	Description
Half Subtractor	2	1 XOR + 1 AND
Full Subtractor	2	Similar to full adder
Ripple (N-bit)	~2N	N full subtractors

28.8.2 Subtractor vs Adder-Based Subtraction

Aspect	Direct Subtractor	Adder + Two's Complement
Logic	Dedicated subtractor cells	Reused adder + inverter
Area	+100% (separate)	+1% (just inverter)

Aspect	Direct Subtractor	Adder + Two's Complement
	unit)	
Timing	Same as adder	Same as adder
Modern Practice	Rarely used	Standard approach

28.9 Design Considerations

28.9.1 Why Adders Are Preferred for Subtraction

Historical: Separate subtractors were common in early designs.

Modern: Almost all designs use adders with two's complement:

Advantages: - ✓ Hardware reuse (one adder for both add/subtract) - ✓ Smaller area (no duplicate logic) - ✓ Standard practice (easier to understand/maintain) - ✓ ALU simplicity (single arithmetic unit)

When to Use Direct Subtractors: - Educational/demonstration purposes - Very specific applications requiring direct borrow propagation - Compatibility with existing designs using subtractors

28.9.2 Implementing Subtraction in ALU

```

module simple_alu (
    input logic [7:0] a, b,
    input logic sub, // 1 = subtract, 0 = add
    output logic [7:0] result,
    output logic carry_borrow
);

    logic [7:0] b_mux;
    logic cin_mux;

    // Two's complement: A - B = A + (~B) + 1
    assign b_mux = sub ? ~b : b;
    assign cin_mux = sub;

    math_adder_ripple_carry #(.N(8)) u_add (
        .i_a(a),
        .i_b(b_mux),
        .i_c(cin_mux),
        .ow_sum(result),
        .ow_carry(carry_out)
    );

```

```

// For subtraction, borrow = ~carry
assign carry_borrow = sub ? ~carry_out : carry_out;

```

```
endmodule
```

28.10 Common Pitfalls

x Anti-Pattern 1: Using subtractors instead of adders

```

// DISCOURAGED: Separate adder and subtractor
math_adder_ripple_carry u_add (...);
math_subtractor_ripple_carry u_sub (...); // Duplicate hardware!

// PREFERRED: Single adder for both operations
math_adder_ripple_carry u_alu (
    .i_a(a),
    .i_b(sub ? ~b : b), // Two's complement for subtraction
    .i_c(sub),
    ...
);

```

x Anti-Pattern 2: Ignoring borrow for underflow detection

```

// WRONG: Ignoring borrow output
math_subtractor_ripple_carry u_sub (
    .i_a(a), .i_b(b), .i_b_in(1'b0),
    .ow_d(diff),
    .ow_b() // IGNORED - miss underflow detection!
);

// RIGHT: Check borrow for underflow
logic underflow;
math_subtractor_ripple_carry u_sub (
    .i_a(a), .i_b(b), .i_b_in(1'b0),
    .ow_d(diff),
    .ow_b(underflow) // Indicates A < B
);

```

x Anti-Pattern 3: Incorrect borrow interpretation

```

// WRONG: Borrow means A < B, not A > B
if (borrow) begin
    // A is LESS than B (negative result)
end

// Remember: Borrow = 1 means underflow (A < B)

```

28.11 Related Modules

- `**math_adder_.sv**` - Preferred for subtraction via two's complement
- `math_addsub_full_nbit.sv` - Combined add/subtract unit

28.12 References

- Mano, M.M. "Digital Design." Prentice Hall, 2002.
- Hennessy, J.L., Patterson, D.A. "Computer Architecture: A Quantitative Approach." Morgan Kaufmann, 2011.

28.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)